U-Boot (v5) 开发指南

文件标识：RK-KF-YF-C65

发布版本：V1.0.0

日期：2026-04-17

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2026 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

本文主要指导读者如何在 U-Boot (v5) 版本进行项目开发。

已支持平台

芯片平台
RK3576, RK3588, RK3538, RK3572

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	陈健洪	2026-04-17	初始版本

目录

U-Boot (v5) 开发指南

1. Chapter-1 基础简介
    - 1.1 Feature
    - 1.2 DM
    - 1.3 Security
    - 1.4 Boot-order
    - 1.5 Driver-probe
    - 1.6 Shell
    - 1.7 Boot-Command
    - 1.8 SPL/U-Boot-proper
    - 1.9 Build-Output
    - 1.10 Environment-Variables
    - 1.11 U-Boot DTS
    - 1.12 Relocation
2. Chapter-2 RK架构
    - 2.1 前言
    - 2.2 平台文件
    - 2.3 平台配置
    - 2.4 启动流程
    - 2.5 内存布局
    - 2.6 存储布局
    - 2.7 Aliases
    - 2.8 AMP
    - 2.9 Atags
    - 2.10 Bootdev
    - 2.11 Bidram/sysmem
    - 2.12 Fuse/OTP
    - 2.13 Hotkey
    - 2.14 Image Decompress
    - 2.15 Image kernel
    - 2.16 Image U-Boot
    - 2.17 Interrupt
    - 2.18 Kernel-DTB
    - 2.19 MMU Cache
    - 2.20 Make.sh
    - 2.21 HW-ID DTB
    - 2.22 Partition Table
    - 2.23 Relocation
    - 2.24 Reset
    - 2.25 Sd/Udisk
    - 2.26 Stacktrace
    - 2.27 TimeCost
    - 2.28 Timestamp
    - 2.29 Vendor storage
3. Chapter-3 编译烧写
    - 3.1 前期准备
    - 3.2 固件编译
    - 3.3 固件烧写
    - 3.4 固件大小修改
4. Chapter-4 系统模块
    - 4.1 AArch32
    - 4.2 ANDROID AB
        - 4.2.1 配置项
        - 4.2.2 分区表
    - 4.3 ANDROID BCB

4.4 AVB 安全启动
4.4.1 Feature
4.4.2 配置
4.4.3 参考
4.5 Cmdline
4.5.1 数据来源
4.5.2 数据含义
4.6 DTBO/DTO
4.6.1 原理介绍
4.6.2 DTO 启用
4.6.3 DTO 结果
4.7 ENV
4.7.1 框架支持
4.7.2 相关接口
4.7.3 高级接口
4.7.4 存储位置
4.7.5 通用选项
4.7.6 fw_printenv 工具
4.8 Fastboot
4.8.1 配置选项
4.8.2 触发方式
4.8.3 fastboot 命令
4.8.3.1 命令速览
4.8.3.2 命令使用
4.9 FileSystem
4.9.1 框架支持
4.9.2 相关接口
4.9.3 命令示例
4.10 HW-ID DTB
4.10.1 设计原理
4.10.2 硬件参考
4.10.3 DTB 命名
4.10.4 DTB 打包
4.10.5 功能启用
4.10.6 加载结果
4.11 SD 和 U 盘
4.11.1 机制原理
4.11.2 固件制作
4.11.3 SD 配置
4.11.4 USB 配置
4.11.5 功能生效
4.11.6 注意事项
5. Chapter-5 驱动模块
5.1 AMP
5.1.1 实现思路
5.1.2 框架支持
5.1.3 功能启用
5.2 Button
5.2.1 框架支持
5.2.2 相关接口
5.3 Charge
5.3.1 框架支持
5.3.2 打包图片
5.3.3 DTS 配置
5.3.4 系统休眠
5.3.5 更换图片
5.3.6 充电灯
5.4 Clock

5.4.1 框架支持
5.4.2 相关接口
5.4.3 时钟初始化
5.4.4 CPU提频
5.4.5 时钟树
5.5 Crypto
5.5.1 框架支持
5.5.2 相关接口
5.6 Display
5.6.1 框架支持
5.6.2 相关接口
5.6.3 DTS 配置
5.6.4 defconfig
5.6.5 LOGO 分区
5.6.6 常见问题分析
5.7 Efuse/Otp
5.7.1 框架支持
5.7.2 相关接口
5.7.3 DTS 配置
5.7.4 调用示例
5.7.5 其他
5.8 Ethernet
5.8.1 框架支持
5.8.2 相关接口
5.8.3 DTS 配置
5.8.4 使用示例
5.8.5 网络故障排查
5.9 Gpio
5.9.1 框架支持
5.9.2 DM接口
5.9.3 Legacy接口
5.10 Interrupt
5.10.1 框架支持
5.10.2 相关接口
5.11 I2C
5.11.1 框架支持
5.11.2 相关接口
5.12 Led
5.12.1 框架支持
5.12.2 相关接口
5.12.3 DTS 节点
5.13 Mtd
5.13.1 框架支持
5.13.2 相关接口
5.13.3 使用示例
5.14 Mtd_blk
5.14.1 框架支持
5.14.2 相关接口
5.14.3 相关接口
5.15 Optee Client
5.15.1 框架支持
5.15.2 固件说明
5.15.3 接口说明
5.15.3.1 适用性
5.15.3.2 返回值
5.15.3.3 optee_base_write_security_data
5.15.3.4 optee_base_read_security_data
5.15.3.5 optee_read_vbootkey_hash

5.15.3.6 optee_write_vbootkey_hash
5.15.3.7 optee_read_vbootkey_enable_flag
5.15.3.8 optee_read_attribute_hash
5.15.3.9 optee_write_attribute_hash
5.15.3.10 optee_write_oem_otp_key
5.15.3.11 optee_oem_otp_key_is_written
5.15.3.12 optee_set_oem_hr_otp_read_lock
5.15.3.13 optee_oem_otp_key_cipher
5.15.3.14 optee_read_oem_ns_otp
5.15.3.15 optee_write_oem_ns_otp
5.15.3.16 optee_write_ta_encryption_key
5.15.3.17 optee_ta_encryption_key_is_written
5.15.3.18 optee_write_oem_encrypt_data
5.15.3.19 optee_oem_encrypt_data_is_written
5.15.3.20 optee_check_security_level_flag
5.15.3.21 optee_write_oem_huk
5.15.4 共享内存
5.15.5 测试命令
5.15.6 常见错误打印
5.16 PCIe
5.16.1 开发须知
5.16.2 框架支持
5.16.3 DTS 配置
5.16.4 使用示例
5.16.4.1 PCIe CMD
5.16.4.2 NVMe
5.16.4.3 RK3588 RC dma
5.16.4.4 RK3568 RC dma
5.16.5 常见问题分析
5.17 Pinctrl
5.17.1 框架支持
5.17.2 相关接口
5.18 Pmic/Regulator
5.18.1 框架支持
5.18.2 相关接口
5.18.3 init 电压
5.18.4 跳过初始化
5.19 Reset
5.19.1 框架支持
5.19.2 相关接口
5.19.3 DTS 配置
5.20 Rng
5.20.1 框架支持
5.20.2 相关接口
5.20.3 DTS 配置
5.21 Spi
5.21.1 框架支持
5.21.2 相关接口
5.21.3 DTS 配置
5.21.4 调用示例
5.21.5 测试命令
5.21.6 常见问题分析
5.22 Storage
5.22.1 框架支持
5.22.2 相关接口
5.22.3 类型区分
5.22.4 DTS 配置
5.22.5 双存储扩展

5.22.6 常见问题分析
5.23 Thermal
5.23.1 框架支持
5.23.2 相关接口
5.23.3 DTS配置
5.24 Uart
5.24.1 单独更换
5.24.2 全局替换
5.24.3 关闭串口
5.24.4 相关接口
5.25 USB
5.25.1 框架支持
5.25.2 板级配置
5.25.3 DTS 配置
5.25.4 相关命令
5.26 Vendor Storage
5.26.1 原理概述
5.26.2 框架支持
5.26.3 相关接口
5.26.4 功能自测
5.27 Watchdog
5.27.1 框架支持
5.27.2 相关接口
6. Chapter-6 进阶原理
6.1 kernel-DTB
6.1.1 设计背景
6.1.2 Live DT
6.1.3 机制实现
6.1.4 U-Boot
6.2 内核传参
6.2.1 cmdline
6.2.2 内存容量
6.2.3 其它方式
6.3 AB系统
6.3.1 AB 数据格式
6.3.2 AB 启动模式
6.3.2.1 successful-boot
6.3.2.2 reset-retry
6.3.2.3 模式对比
6.3.3 启动流程
6.3.4 升级和异常
6.3.5 验证方法
6.3.5.1 successful-boot
6.3.5.2 reset-retry
6.3.6 引用参考
6.4 AVB安全启动
6.4.1 引用参考
6.4.2 术语
6.4.3 简介
6.4.4 加密示例
6.4.5 AVB
6.4.5.1 AVB 特性
6.4.5.2 生成key，签名和证书
6.4.5.3 AVB lock
6.4.5.4 AVB unlock
6.4.5.5 Cmdline 新增内容
6.4.6 固件烧写
6.4.7 Pre-loader verified

6.4.8 U-boot verified
6.4.9 系统校验启动
6.4.10 Linux AVB
6.4.10.1 操作流程
6.4.10.2 验证流程
6.5 SD启动和升级
6.5.1 简介
6.5.2 分类
6.5.2.1 常规卡
6.5.2.2 升级卡
6.5.2.3 启动卡
6.5.2.4 修复卡
6.5.3 固件标志
6.5.4 启动流程
6.5.4.1 pre-loader 启动
6.5.4.2 U-Boot 启动
6.5.4.3 Recovery 和 PCBA
6.5.5 注意事项
7. Chapter-7 配置裁剪
8. Chapter-8 调试手段
8.1 DEBUG
8.2 Initcall
8.3 io 命令
8.4 iomem 命令
8.5 i2c 命令
8.6 gpio 命令
8.7 fdt 命令
8.8 mmc 命令
8.9 时间戳
8.10 dm tree
8.11 dm uclass
8.12 stacktrace.sh
8.13 系统卡死
8.14 修改 DDR 容量
8.15 跳转信息
8.16 启动信息
8.16.1 Android 固件
9. Chapter-9 测试用例
10. Chapter-10 SPL
10.1 固件引导
10.1.1 FIT 固件
10.1.2 RKFW 固件
10.1.3 存储优先级
10.2 编译打包
10.2.1 代码编译
10.3 系统模块
10.3.1 分区表
10.3.2 A/B system
10.3.3 启动优先级
10.3.4 ATAGS
10.3.5 kernel boot
10.3.6 pinctrl
10.4 驱动模块
10.4.1 MMC
10.4.2 MTD block
10.4.3 OTP
11. Chapter-11 TPL
12. Chapter-12 FIT

12.1 前言
12.2 简介
12.2.1 基础介绍
12.2.2 范例介绍
12.2.3 itb结构
12.3 平台配置
12.3.1 芯片支持
12.3.2 代码配置
12.3.3 镜像文件
12.3.4 its 文件
12.3.5 相关工具
12.4 非安全启动
12.4.1 uboot.img
12.4.2 boot.img
12.5 安全启动
12.5.1 原理
12.5.1.1 校验流程
12.5.1.2 key存放
12.5.1.3 key使用
12.5.1.4 签名存放
12.5.1.5 防回滚
12.5.2 前期准备
12.5.2.1 Key
12.5.2.2 配置
12.5.2.3 固件
12.5.3 编译打包
12.5.4 校验原则
12.5.5 启动信息
12.6 独立签名
12.6.1 签名流程
12.6.2 签名脚本
12.6.3 签名配置
12.7 远程签名
12.7.1 实现思路
12.7.2 被签名数据
12.7.3 具体步骤
12.8 固件解包
12.9 固件替换
12.10 安全校验Step-by-Step
13. Chapter-13 快速开机
14. Chapter-14 平台定义
14.1 ATF/OPTEE
14.2 Clock
14.3 Defconfig
14.4 DFU
14.5 OP-TEE
15. Chapter-15 注意事项
15.1 配置修改
15.2 文档使用
15.3 调试与发布
16. Chapter-16 工具
16.1 boot_merger
16.2 resource_tool
16.3 mkimage
16.4 stacktrace.sh
16.5 mkbootimg
16.6 unpack_bootimg
16.7 repack-bootimg

16.8 pack_resource.sh
17. Chapter-17 附录
17.1 术语

1. Chapter-1 基础简介

1.1 Feature

U-Boot v5 目前支持的功能主要有：

- 支持 Android AOSP 固件启动
- 支持 Linux Distro 固件启动
- 支持 DDR + SPL 的 Pre-loader 引导
- 支持 LVDS、EDP、MIPI、HDMI、CVBS、RGB 等显示设备
- 支持 eMMC、NAND Flash、SPI NAND Flash、SPI NOR Flash、SD 卡、U 盘等存储设备启动
- 支持 FAT、EXT2、EXT4 文件系统
- 支持 GPT 分区表
- 支持开机 LOGO、充电动画、低电管理、电源管理
- 支持 I2C、PMIC、CHARGE、FUEL GAUGE、USB、GPIO、PWM、GMAC、eMMC、NAND、Interrupt 等
- 支持 Vendor Storage 保存用户数据和配置
- 支持 RockUSB 和 Google Fastboot 两种 USB gadget 烧写 eMMC
- 支持 Mass storage、ethernet、HID 等 USB 设备
- 支持通过硬件状态动态选择 Kernel DTB

1.2 DM

DM (Driver Model) 是 U-Boot 标准的 device-driver 开发模型，与 kernel 的 device-driver 模型非常类似。v5 版本也遵循 DM 框架开发各功能模块。建议读者先阅读 DM 文档，了解 DM 架构原理和实现。

README：

```
./doc/develop/driver-model/design.rst
```

```
Terminology
-----------

Uclass - a group of devices which operate in the same way. A uclass provides
         a way of accessing individual devices within the group, but always
         using the same interface. For example a GPIO uclass provides
         operations for get/set value. An I2C uclass may have 10 I2C ports,
         4 with one driver, and 6 with another.

Driver - some code which talks to a peripheral and presents a higher-level
         interface to it.

Device - an instance of a driver, tied to a particular port or peripheral.
```

简要概括：

- uclass：设备类
- driver：驱动

- device：设备

核心代码：

```
./drivers/core/
```

1.3 Security

U-Boot 在 ARM TrustZone 的安全体系中属于 Non-Secure World，无法直接访问任何安全资源（如：安全 memory、安全 OTP（efuse）等），需要借助 Trust 间接访问。RK 平台上 U-Boot 的 CPU 运行模式如下：

```
32位平台： Non-Secure PL1
64位平台：EL2（Non-Secure）
```

1.4 Boot-order

```
// ddr/ATF/OPTEE: closed-source; SPL/U-Boot: open-source
BOOTROM => ddr bin => SPL => Trust(ATF/OPTEE) => U-Boot => KERNEL
```

1.5 Driver-probe

U-Boot 虽然引入了 device-driver 开发模型，但初始化阶段不会像 kernel 那样自动发起已注册 device-driver 的 probe。driver 的 probe 必须由用户主动调用发起。接口如下：

```
int uclass_get_device(enum uclass_id id, int index, struct udevice **devp);
int uclass_get_device_by_name(enum uclass_id id, const char *name,
                              struct udevice **devp);
int uclass_get_device_by_seq(enum uclass_id id, int seq, struct udevice **devp);
int uclass_get_device_by_of_offset(enum uclass_id id, int node, struct udevice
**devp);
int uclass_get_device_by_ofnode(enum uclass_id id, ofnode node, struct udevice
**devp);
int uclass_get_device_by_phandle_id(enum uclass_id id,
                                    int phandle_id, struct udevice **devp);
int uclass_get_device_by_phandle(enum uclass_id id,
                                 struct udevice *parent, struct udevice **devp);
int uclass_get_device_by_driver(enum uclass_id id,
                                const struct driver *drv, struct udevice
**devp);
int uclass_get_device_tail(struct udevice *dev, int ret, struct udevice **devp);
......
```

上述接口的核心调用：

```
int device_probe(struct udevice *dev);  /* 建议用户重点理解其内部实现 */
```

1.6 Shell

U-Boot 的 Shell 叫 CLI (command line interface)，即命令行模式，用户可以根据自身需求自定义 CMD。CMD 除了通过 Shell 调用，还能通过 `run_command()` 和 `run_command_list()` 以代码形式调用。

```
int run_command(const char *cmd, int flag);
int run_command_list(const char *cmd, int len, int flag);
```

1.7 Boot-Command

U-Boot 最终通过 `CONFIG_BOOTCOMMAND` 定义的启动命令引导 kernel。在执行 `CONFIG_BOOTCOMMAND` 之前还会执行 `CONFIG_PREBOOT` 预启动命令，通常这个命令定义为空。

1.8 SPL/U-Boot-proper

U-Boot 通过使用**不同的编译条件**可以用同一套代码获取两种不同功能的 Loader：SPL/U-Boot-proper。

SPL (Secondary Program Loader) 是比 U-Boot 更早阶段的 Loader：

- SPL：运行在 DDR，负责完成系统的 lowlevel 初始化和后级固件加载；
- U-Boot proper：运行在 DDR，即我们通常所说的"U-Boot"，它负责引导 kernel；

> 说明：U-Boot proper 这一说法主要是为了和 SPL 区分开。出于习惯，后续章节提到的 U-Boot proper 都简称为 U-Boot。

启动流程：

```
BOOTROM => ddr bin => SPL => Trust(ATF/OPTEE) => U-Boot => KERNEL
```

1.9 Build-Output

U-Boot 编译成功后会在根目录下生成一些重要文件（支持 TPL/SPL 编译时才会生成对应文件）：

```
// U-Boot阶段
./u-boot.map            // MAP表文件
./u-boot.sym            // SYMBOL表文件
./u-boot                // ELF文件，类同内核的vmlinux（重要！）
./u-boot.dtb            // u-boot自己的dtb文件
./u-boot.bin            // 可执行二进制文件，会被打包成uboot.img用于烧写

// SPL阶段
./spl/u-boot-spl.map    // MAP表文件
./spl/u-boot-spl.sym    // SYMBOL表文件
./spl/u-boot-spl        // ELF文件，类同内核的vmlinux（重要！）
./spl/u-boot-spl.dtb    // spl自己的dtb文件
./spl/u-boot-spl.bin    // 可执行二进制文件，会被打包成loader用于烧写
```

如果平台启用了 TPL，还会额外生成 `./tpl/u-boot-tpl`、`./tpl/u-boot-tpl.bin`、`./tpl/u-boot-tpl.dtb` 等产物。

1.10 Environment-Variables

ENV（Environment Variables）是 U-Boot 支持的一种全局数据管理和传递方式，其原理是构建一张 HASH 映射表，把用户的数据以“键值-数据”作为表项进行管理。

ENV 通常用于定义平台配置参数：固件加载地址、网络配置（ipaddr、serverip）、bootcmd、bootargs 等，用户可以在命令行下使用 `printenv` 命令打印出来。

- 用户可选择是否把 ENV 数据保存到本地存储上
- ENV 数据仅限于 U-Boot 使用，无法直接传递给内核、内核也无法直接解析
- 用户层可以通过 U-Boot 提供的 fw_printenv 工具访问 ENV 数据

RK 平台上 ENV 数据的存储地址和大小定义如下（单位：字节）：

```
config ENV_OFFSET
    hex "Environment offset"
    depends on ENV_IS_IN_EEPROM || ENV_IS_IN_MMC || ENV_IS_IN_NAND || \
            ENV_IS_IN_SPI_FLASH || ENV_IS_IN_BLK || ENV_IS_IN_FRAGMENT
    default 0x0 if ENV_IS_IN_FRAGMENT
    default 0x3f8000
    help
      Offset from the start of the device (or partition).

config ENV_SIZE
    hex "Environment Size"
    default 0x8000
    help
      Size of the environment storage area
```

其中 `ENV_IS_IN_FRAGMENT` 为 Rockchip 平台扩展场景使用的配置项，使用前请结合实际 defconfig 和 Kconfig 说明确认。

1.11 U-Boot DTS

U-Boot 有自己的 DTS 文件，编译时会自动生成相应的 DTB 文件，并追加到 `u-boot.bin` 末尾。文件目录：

```
arch/arm/dts/                           // rkxxx-u-boot.dtsi
dts/upstream/src/arm64/rockchip/        // 基础dtsi/dts文件
```

各平台具体使用哪个 DTS 文件，通过 defconfig 中的 `CONFIG_DEFAULT_DEVICE_TREE` 指定。

1.12 Relocation

通常在开机阶段，U-Boot 由前一级 bootloader 加载到 DRAM 的低地址。U-Boot 完成 `board_f.c` 的流程后，会把自己重定向到内存末尾某个预留地址上（称为 relocation，该地址由 U-Boot 内存布局决定），完成 relocation 后再继续执行 `board_r.c` 流程。可以通过开机信息识别：

```
U-Boot v5-gabfd1c5e3d-210202 #cjh (Mar 08 2021 - 16:57:31 +0800)

Model: Rockchip RK3588 Evaluation Board
PreSerial: 2, raw, 0xfe660000
DRAM:  2 GiB
Sysmem: init
/* relocate 到 DDR 首地址偏移 0x7d304000 的地址。如果为 0，则没有做 relocation。 */
Relocation Offset: 7d304000, fdt: 7b9f8ed8
Using default environment
......
```

2. Chapter-2 RK架构

本章主要介绍 RK 平台上一些重要的基础信息和特性。

2.1 前言

全文所有章节中只要提到开启/关闭CONFIG_配置项，均是指通过`make menuconfig`的方式开启/关闭。

> 除非是一些特殊的、直接定义在.h文件中的CONFIG_配置项。

请勿直接在defconfig中开启/关闭CONFIG_配置项，避免因为Kconfig中定义的配置依赖关系而导致.config不生效。

更新defconfig时请使用`make savedefconfig`的方式。

2.2 平台文件

平台目录：

```
./arch/arm/include/asm/arch-rockchip/
./arch/arm/mach-rockchip/
./board/rockchip/
./include/configs/
```

defconfig目录：

```
./configs/
```

核心公共板级文件！

```
./arch/arm/mach-rockchip/board.c
```

2.3 平台配置

配置文件

各平台的配置选项、参数通常位于如下几个位置：

```
// 各平台公共文件（开发者通常不需要修改）
./arch/arm/mach-rockchip/Kconfig
./include/configs/rockchip-common.h

// 各平台独有，以RK3588为例
./include/configs/rk3588_common.h
./include/configs/evb_rk3588.h
./configs/rk3588_defconfig
```

2.4 启动流程

RK平台的U-Boot 启动流程如下，仅列出一些重要步骤：

```
start.S
    // 汇编环境
    => IRQ/FIQ/lowlevel/vbar/errata/cp15/gic    // ARM架构相关的lowlevel初始化
    => _main
        => stack                                // 准备好C环境需要的栈
        // 【第一阶段】C环境初始化，发起一系列的函数调用
        => board_init_f: init_sequence_f[]
            initf_malloc
            arch_cpu_init                       // 【SoC的lowlevel初始化】
            serial_init                         // 串口初始化
            dram_init                           // 【获取ddr容量信息】
            reserve_mmu                         // 从ddr末尾开始往低地址reserve内存
            reserve_video
            reserve_uboot
            reserve_malloc
            reserve_global_data
            reserve_fdt
            reserve_stacks
            dram_init_banksize
            sysmem_init
            setup_reloc                         // 确定U-Boot自身要reloc的地址
        // 汇编环境
        => relocate_code                        // 汇编实现U-Boot代码的relocation
        // 【第二阶段】C环境初始化，发起一系列的函数调用
        => board_init_r: init_sequence_r[]
            initr_caches                        // 使能MMU和I/Dcache
            initr_malloc
            bidram_initr
            sysmem_initr
            initr_of_live                       // 初始化of_live
            initr_dm                            // 初始化dm框架
            board_init                          // 【平台初始化，最核心部分】
                board_debug_uart_init           // 串口iomux、clk配置
                init_kernel_dtb                 // 【切换到 Kernel DTB】！
                clks_probe                      // 初始化系统频率
                regulators_enable_boot_on       // 初始化系统电源
                io_domain_init                  // io-domain初始化
                set_armclk_rate                 // __weak，ARM提频(平台有需求才实现)
                dvfs_init                       // 宽温芯片的调频调压
                rk_board_init                   // __weak，由各具体平台实现
            console_init_r
```

```
            board_late_init                 // 【平台late初始化】
                rockchip_set_ethaddr        // 设置 MAC 地址
                rockchip_set_serialno       // 设置serialno
                setup_boot_mode             // 解析"reboot xxx"命令、
                                            // 识别按键和loader烧写模式、recovery
                charge_display              // U-Boot充电
                rockchip_show_logo          // 显示开机logo
                soc_clk_dump                // 打印clk tree
                rk_board_late_init          // __weak，由各具体平台实现
            run_main_loop                   // 【进入命令行模式，或执行启动命令】
```

2.5 内存布局

U-Boot 由前级 Loader 加载到 `CONFIG_TEXT_BASE` 地址，初始化时会探明当前系统的总内存容量，32位平台上认为最大4GB可用（但是不影响内核对容量的识别），64位平台上认为所有内存都可用。然后通过一系列reserve_xxx() 接口从内存末尾往前预留需要的内存，最后把自己relocate到某段reserve的空间上。内存整体使用布局如下，以ARM64为例（常规情况）：

Name	Start Addr Offset	Size	Usage	Secure
ATF	0x00000000	1M	ARM Trusted Firmware	Yes
SHM	0x00100000	1M	SHM, Pstore	No
OP-TEE	0x08400000	2M~30M	参考 TEE 开发手册	Yes
FDT	fdt_addr_r	-	kernel dtb	No
KERNEL	kernel_addr_r	-	kernel 镜像	No
RAMDISK	ramdisk_addr_r	-	ramdisk 镜像	No
……	-	-	-	-
FASTBOOT	-	-	Fastboot buffer	No

Name	Start Addr Offset	Size	Usage	Secure
……	-	-	-	
SP	-	-	stack	No
FDT	-	sizeof(dtb)	U-Boot dtb	No
GD	-	sizeof(gd)	-	No
Board	-	sizeof(bd_t)	-	No
MALLOC	-	CONFIG_SYS_MALLOC_LEN	系统的堆空间	No
U-Boot	-	sizeof(mon)	u-boot 镜像	No
Video FB	-	fb size	32M	No
TLB Table	RAM_TOP-64K	32K	MMU 页表	No

> 上表中的`Start Addr Offset`一栏表示基于 DDR base 的地址偏移；
>
> Fastboot 地址和大小由配置决定：`CONFIG_FASTBOOT_BUF_ADDR`，`CONFIG_FASTBOOT_BUF_SIZE`。

- Video FB/U-Boot/Malloc/Board/Gd/Fdt/Sp 由顶向下根据实际需求大小来分配；
- 64 位平台：ATF 是 ARMv8 必需的，OP-TEE 是可选项；32 位平台：只有 OP-TEE；
- kernel fdt/kernel/ramdisk 是 U-Boot 需要加载的固件地址，由 `ENV_MEM_LAYOUT_SETTINGS` 定义；
- Fastboot 功能需要的 buffer 地址和大小在 defconfig 中定义；
- OP-TEE 占据的空间需要根据实际需求而定，最大为 30M。

2.6 存储布局

RK Linux 方案的存储布局如下，Android 方案除了 boot/rootfs 的定义跟 Linux 平台有差异，其它基本一致，可借鉴参考。

Partition	Start Sector		Number of Sectors		Partition Size		Requirements
MBR	0	00000000	1	00000001	512	0.5KB	
Primary GPT	1	00000001	63	0000003F	32256	31.5KB	
loader1	**64**	**00000040**	**7104**	**00001bc0**	**4096000**	**2.5MB**	**preloader (miniloader or U-Boot SPL)**
Vendor Storage	7168	00001c00	512	00000200	262144	256KB	SN, MAC and etc.
Reserved Space	7680	00001e00	384	00000180	196608	192KB	Not used
reserved1	8064	00001f80	128	00000080	65536	64KB	legacy DRM key
U-Boot ENV	8128	00001fc0	64	00000040	32768	32KB	
reserved2	8192	00002000	8192	00002000	4194304	4MB	legacy parameter
loader2	**16384**	**00004000**	**8192**	**00002000**	**4194304**	**4MB**	**U-Boot or UEFI**
trust	**24576**	**00006000**	**8192**	**00002000**	**4194304**	**4MB**	**trusted-os like ATF, OP-TEE**
boot (bootable must be set)	**32768**	**00008000**	**229376**	**00038000**	**117440512**	**112MB**	**kernel, dtb, extlinux.conf, ramdisk**
rootfs	**262144**	**00040000**	**-**	**-**	**-**	**-MB**	**Linux system**
Secondary GPT	16777183	00FFFFDF	33	00000021	16896	16.5KB	

> 图片引用：http://opensource.rock-chips.com/wiki_Partitions

2.7 Aliases

U-Boot 中有些特殊的 aliases 有别于 kernel DTS 中的定义。

eMMC/SD在U-Boot中统称为 mmc 设备，使用编号0、1作区分；SD的启动优先级高于eMMC。

```
mmc0: 表示 eMMC
mmc1: 表示 SD
```

2.8 AMP

RK平台的U-Boot支持AMP(Asymmetric Multi-Processing) 固件引导。

> 更多参考：驱动模块章节。

2.9 Atags

RK平台的启动流程：

```
BOOTROM => ddr-bin => SPL => TRUST(ATF/OPTEE) => U-Boot => KERNEL
```

RK平台的各级固件之间可以通过ATAGS机制传递一些配置信息。

- 适用范围：ddr-bin、SPL、ATF、OP-TEE、U-Boot，不包含Kernel。
- 传递内容：串口配置、存储类型、ATF 和 OP-TEE 占用的内存、DDR 容量等。

代码实现：

```
./arch/arm/include/asm/arch-rockchip/atags.h
./arch/arm/mach-rockchip/atags.c
```

2.10 Bootdev

获取当前启动存储的句柄：

```
struct blk_desc *plat_bootdev(void);
```

使用方式请搜索现有代码。

2.11 Bidram/sysmem

U-Boot可以使用系统的所有内存，且从高地址往低地址预留系统所需的内存，预留完后通常还剩余较大的内存空间。U-Boot没有机制去管理这块空间，因此RK平台引入bidram、sysmem内存块机制对这块内存进行管理。

由此，加上U-Boot已有的malloc管理机制，RK平台就把系统所有内存通过sysmem + bidram + malloc 管理起来了，防止出现内存冲突等问题。

```
low-addr
 high-addr
|-------------------------------------------------------------|-----------------|
|              no management                                  | system used     |
|-------------------------------------------------------------|-----------------|
0x0                                                                            N
GB
```

- bidram：管理 U-Boot、kernel 阶段不可用、需要剔除的内存块，例如：ATF、OP-TEE 占用的空间；
- sysmem：管理 kernel 可见、可用的内存块。例如：fdt、ramdisk、kernel、fastboot 占用的空间。

相关代码：

```
./lib/sysmem.c
./lib/bidram.c
./include/memblk.h
./arch/arm/mach-rockchip/memblk.c
```

如下是 bidram 和 sysmem 的内存管理信息表，当出现内存块初始化或分配异常时会被 dump 出来。如下做出简单介绍。

bidram 内存信息表：

```
bidram_dump_all:
    ------------------------------------------------------------------
    // <1> 这里显示了U-Boot从前级loader获取的ddr的总容量信息，一共有2GB
    memory.rgn[0].addr      = 0x00000000 - 0x80000000 (size: 0x80000000)

    memory.total            = 0x80000000 (2048 MiB. 0 KiB)
    ------------------------------------------------------------------
    // <2> 这里显示了被预留起来的各固件内存信息，这些空间对kernel不可见
    reserved.rgn[0].name    = "ATF"
                    .addr   = 0x00000000 - 0x00100000 (size: 0x00100000)
    reserved.rgn[1].name    = "SHM"
                    .addr   = 0x00100000 - 0x00200000 (size: 0x00100000)
    reserved.rgn[2].name    = "OP-TEE"
                    .addr   = 0x08400000 - 0x0a200000 (size: 0x01e00000)

    reserved.total          = 0x02000000 (32 MiB. 0 KiB)
    ------------------------------------------------------------------
    // <3> 这里是核心算法对上述<2>进行的预留信息整理，例如：会对相邻块进行合并
    LMB.reserved[0].addr    = 0x00000000 - 0x00200000 (size: 0x00200000)
    LMB.reserved[1].addr    = 0x08400000 - 0x0a200000 (size: 0x01e00000)

    reserved.core.total     = 0x02000000 (32 MiB. 0 KiB)
    ------------------------------------------------------------------
```

sysmem 内存信息表：

```
sysmem_dump_all:
    ------------------------------------------------------------------
    // <1> 这里是sysmem可管理的总内存容量，即bidram<3>之外的可用ddr容量，对kernel可见。
    memory.rgn[0].addr      = 0x00200000 - 0x08400000 (size: 0x08200000)
    memory.rgn[1].addr      = 0x0a200000 - 0x80000000 (size: 0x75e00000)
```

```
    memory.total            = 0x7e000000 (2016 MiB. 0 KiB)
    ------------------------------------------------------------------
    // <2> 这里显示了各个固件alloc走的内存块信息
    allocated.rgn[0].name   = "U-Boot"
                    .addr   = 0x71dd6140 - 0x80000000 (size: 0x0e229ec0)
    allocated.rgn[1].name   = "STACK"       <Overflow!> // 表明栈溢出
                    .addr   = 0x71bd6140 - 0x71dd6140 (size: 0x00200000)
    allocated.rgn[2].name   = "FDT"
                    .addr   = 0x08300000 - 0x08316204 (size: 0x00016204)
    allocated.rgn[3].name   = "KERNEL"      <Overflow!> // 表明内存块溢出
                    .addr   = 0x00280000 - 0x014ce204 (size: 0x0124e204)
    allocated.rgn[4].name   = "RAMDISK"
                    .addr   = 0x0a200000 - 0x0a3e6804 (size: 0x001e6804)
    // <3> malloc_r/f的大小
    malloc_r: 192 MiB, malloc_f: 16 KiB

    allocated.total         = 0x0f874acc (248 MiB. 466 KiB)
    ------------------------------------------------------------------
    // <4> 这里是核心算法对上述<2>进行的信息整理，显示被占用走的内存块信息
    LMB.reserved[0].addr    = 0x00280000 - 0x014ce204 (size: 0x0124e204)
    LMB.reserved[1].addr    = 0x08300000 - 0x08316204 (size: 0x00016204)
    LMB.reserved[2].addr    = 0x0a200000 - 0x0a3e6804 (size: 0x001e6804)
    LMB.reserved[3].addr    = 0x71bd6140 - 0x80000000 (size: 0x0e429ec0)

    reserved.core.total     = 0x0f874acc (248 MiB. 466 KiB)
    ------------------------------------------------------------------
```

如下是一些常见的错误打印，当出现这些异常时，请结合上述 bidram 和 sysmem dump 内存信息进行分析。

```
// 期望申请的内存已经被其他固件占用了，存在内存重叠。这说明当前系统的内存块使用规划不合理
Sysmem Error: "KERNEL" (0x00200000 - 0x02200000) alloc is overlap with existence
"RAMDISK" (0x00100000 - 0x01200000)

// 期望申请的内存因为一些特殊原因无法申请到（分析sysmem和bidram信息）
Sysmem Error: Failed to alloc "KERNEL" expect at 0x00200000 - 0x02200000 but at
0x00400000 - 0x04200000

// sysmem管理的空间起始地址为0x200000，所以根本申请不到0x100000起始的空间
Sysmem Error: Failed to alloc "KERNEL" at 0x00100000 - 0x02200000

// 重复申请"RAMDISK"内存块
Sysmem Error: Failed to double alloc for existence "RAMDISK"
```

2.12 Fuse/OTP

RK平台为了方便调试secure-boot功能，只需要对固件签名就能开启secure-boot模式（可不烧写efuse/otp）。Miniloader 会通过 U-Boot 向 kernel 追加 cmdline 表明当前的 efuse/otp 使能是否被烧写：

- `"fuse.programmed=1"`：开启了secure-boot，efuse/otp已经被烧写。
- `"fuse.programmed=0"`：开启了secure-boot，efuse/otp没有被烧写。
- cmdline中没有`fuse.programmed`：没有开启secure-boot（SPL不传递）。

2.13 Hotkey

RK平台提供串口组合键触发一些事件用于调试、烧写（如果无法触发，请多尝试几次；启用secure-boot时无效）。**开机时长按：**

- ctrl+c：进入 U-Boot 命令行模式；
- ctrl+d：进入 loader 烧写模式；
- ctrl+b：进入 maskrom 烧写模式；
- ctrl+f：进入 fastboot 模式；
- ctrl+m：打印 bidram/system 信息；
- ctrl+i：使能内核 initcall_debug；
- ctrl+p：打印 cmdline 信息；
- ctrl+s: "Starting kernel..." 之后进入 U-Boot 命令行；

2.14 Image Decompress

- 64 位平台的机器通常烧写Image，由U-Boot 加载到目标运行地址。但是 RK平台的 U-Boot 还可支持对64位 LZ4格式的压缩内核进行解压。但是用户必须使能：

  ```
  CONFIG_LZ4=y
  ```

  64位LZ4压缩内核的解压前、后地址必须定义在各平台的`rkxxx_common.h`文件中：

  ```
  #define ENV_MEM_LAYOUT_SETTINGS \
      "scriptaddr=0x00500000\0" \
      "pxefile_addr_r=0x00600000\0" \
      "fdt_addr_r=0x01f00000\0" \
      "kernel_addr_no_bl32_r=0x00280000\0" \
      "kernel_addr_r=0x00680000\0" \        // LZ4压缩内核的地址
      "kernel_addr_c=0x02480000\0" \        // LZ4解压内核的地址
      "ramdisk_addr_r=0x04000000\0"
  ```

- 32 位平台的机器通常烧写zImage，由U-Boot加载到`kernel_addr_r`地址上，再由内核完成自解压。但是RK平台的U-Boot还可支持Image格式，由U-Boot加载到目标运行地址。

  目前各平台的`rkxxx_common.h`文件只定义了`kernel_addr_r`而没有定义`kernel_addr_c`地址。但是用户不需要更改配置，U-Boot会判断当前是zImage还是Image，动态处理这2个地址。但是用户必须关闭：

  ```
  CONFIG_SKIP_RELOCATE_UBOOT
  ```

  32位内核的加载地址定义:

  ```
  #define ENV_MEM_LAYOUT_SETTINGS \
      "scriptaddr=0x60000000\0" \
      "pxefile_addr_r=0x60100000\0" \
      "fdt_addr_r=0x68300000\0" \
      "kernel_addr_r=0x62008000\0" \       // zImage 内核的地址
      "ramdisk_addr_r=0x6a200000\0"
  ```

2.15 Image kernel

RK平台的U-Boot支持三种格式的内核固件引导：

- Android格式

  镜像文件的 magic 为”ANDROID!”：

  ```
  00000000   41 4E 44 52  4F 49 44 21  24 10 74 00  00 80 40 60
  ANDROID!$.t...@`
  00000010   F9 31 CD 00  00 00 00 62  00 00 00 00  00 00 F0 60
  .1.....b.......`
  ```

  boot.img = kernel + ramdisk+ resource + <dtb>；

  recovery.img = kernel + ramdisk(for recovery) + resource + <recovery_dtbo> + <dtb>；

  > 说明：recovery_dtbo：从Android-9.0才开始新增的镜像；dtb：从Android-10.0才开始新增的镜像；

- FIT格式

  U-Boot mainline支持的一种灵活性极高的固件格式。U-Boot、trust以及mcu等固件一起打包为uboot.img。

  uboot.img 的镜像文件的magic 为"d0 0d fe ed"，用命令`fdtdump uboot.img` 可以查看固件头。

  ```
  00000000  d0 0d fe ed 00 00 06 00  00 00 00 58 00 00 04 c4
   |...........X....|
  00000010  00 00 00 28 00 00 00 11  00 00 00 10 00 00 00 00  |...
  (............|
  ```

  > 更多FIT介绍请参考FIT章节。

- Distro格式

  目前开源 Linux 的一种通用固件打包格式：将 ramdisk、dtb、kernel 打包成一个镜像。这个镜像通常以某种文件系统格式存在，例如 ext2、ext4、fat 等，U-Boot需要通过文件系统才能访问到它的内容。更多参考：

  ```
  boot/bootstd-uclass.c
  cmd/bootstd.c
  boot/bootflow.c
  boot/bootflow_internal.h
  boot/bootflow_menu.c
  cmd/bootflow.c
  ......
  ```

- 引导优先级：android > FIT > distro，每一类固件都有对应的启动命令，三个命令会按优先级逐个执行，直到把固件引导起来。如果所有命令都失败，则停在U-Boot命令行模式。

  启动优先级定义：

  ```
  #define CONFIG_BOOTCOMMAND          \
      "boot_android ${devtype} ${devnum};"    \
      "boot_fit;"             \
      "bootflow scan -bl;"
  ```

2.16 Image U-Boot

- FIT 格式

  U-Boot mainline支持的一种灵活性极高的固件格式。U-Boot、trust以及mcu等固件一起打包为uboot.img。

  uboot.img 的镜像文件的magic 为"d0 0d fe ed"，用命令`fdtdump uboot.img` 可以查看固件头。

```
00000000  d0 0d fe ed 00 00 06 00  00 00 00 58 00 00 04 c4
 |...........X....|
00000010  00 00 00 28 00 00 00 11  00 00 00 10 00 00 00 00  |...
(............|
```

  > 更多FIT介绍请参考FIT章节。

- 备份打包

  通常为了应对OTA升级过程断电等可能导致固件损坏的情况，uboot.img 做了多备份打包。

固件	单份大小	打包份数
FIT uboot.img	3MB	2

  - CONFIG_SPL_FIT_IMAGE_KB：单份大小（KB）
  - CONFIG_SPL_FIT_IMAGE_MULTIPLE：打包份数

  注意事项：

  - SPL是根据CONFIG_SPL_FIT_IMAGE_KB或CONFIG_SPL_FIT_IMAGE_MULTIPLE来确定uboot.img中备份项的位置关系和数量。
  - 如果开发者对这两个选项进行了修改，必须对 `make.sh` 追加参数 `--spl-new`，这样生成的 Loader 才会包含实时编译生成的 `u-boot-spl.bin`，否则默认打包 rkbin 中提交的 `u-boot-spl.bin`（默认按照每份 3MB * 2 查找 `uboot.img` 中的备份项）。

2.17 Interrupt

U-Boot的原生代码没有完整支持中断，RK平台完善了该功能，支持GIC-V2、GIC-V3。

> 更多参考：驱动模块章节。

2.18 Kernel-DTB

原生的U-Boot只支持使用U-Boot自己的DTB，RK平台增加了kernel DTB机制的支持，即使用kernel DTB去初始化外设。主要目的是为了兼容外设板级差异，如：power、clock、display 等。

二者的作用：

- U-Boot DTB：负责初始化存储、打印串口等设备；
- Kernel DTB：负责初始化存储、打印串口以外的设备；

U-Boot 初始化时先用 U-Boot DTB 完成存储、打印串口初始化，然后从存储上加载 Kernel DTB 并转而使用这份 DTB 继续初始化其余外设。Kernel DTB 的代码实现在函数：`init_kernel_dtb()`。

开发者一般不需要修改 U-Boot DTB（除非更换打印串口），各平台发布的SDK里使用的 defconfig 都已启用kernel DTB机制。所以通常对于外设的DTS修改，用户应该修改kernel DTB。

关于U-Boot DTB：

DTS目录：

```
./arch/arm/dts/rkxxx-u-boot.dtsi
```

启用kernel DTB机制后：编译阶段会把U-Boot DTS里带`bootph-all`属性的节点过滤出来，在此基础上再剔除defconfig中`CONFIG_OF_REMOVE_PROPS`指定的property，最终生成u-boot.dtb文件并且追加在u-boot.bin的末尾。

用户编译完U-Boot后可以通过`fdtdump`命令检查DTB内容：

```
fdtdump ./u-boot.dtb | less
```

> 更多参考：进阶原理章节。

2.19 MMU Cache

RK平台默认使能MMU、Dcache、Icache，MMU采用1:1线性映射，Dcache采用write-back策略。相关接口：

```
// Icache接口：
void icache_enable (void);
void icache_disable (void);
void invalidate_icache_all(void);

// Dcache接口：
void dcache_disable (void);
void dcache_enable(void);
void flush_dcache_range(unsigned long start, unsigned long stop);
void flush_cache(unsigned long start, unsigned long size);
void flush_dcache_all(void);
void invalidate_dcache_range(unsigned long start, unsigned long stop);
void invalidate_dcache_all(void);
// 重新映射某块内存区间的Dcache属性
void mmu_set_region_dcache_behaviour(phys_addr_t start, size_t size,
                                     enum dcache_option option);
```

2.20 Make.sh

make.sh既是一个编译脚本，也是一个打包、调试工具。可用于反汇编、打包固件。

```
// 帮助命令
./make.sh --help

// 打包固件的功能
./make.sh itb                 // 打包u-boot.itb（64位平台只支持打包ATF和U-Boot，OP-TEE不打包）
./make.sh env                 // 生成fw_printenv工具
```

```
// 反汇编的功能
./make.sh elf-[x] [type]      // 反汇编：使用-[x]参数，[type]可选择是否反汇编SPL或TPL
./make.sh elf                 // 反汇编u-boot文件，默认使用-D参数
./make.sh elf-S               // 反汇编u-boot文件，使用-S参数
./make.sh elf-d               // 反汇编u-boot文件，使用-d参数
./make.sh elf spl             // 反汇编tpl/u-boot-tpl文件，默认使用-D参数
./make.sh elf tpl             // 反汇编spl/u-boot-tpl文件，默认使用-D参数
./make.sh <addr>              // 需要addr对应的函数名和代码位置
./make.sh map                 // 打开u-boot.map
./make.sh sym                 // 打开u-boot.sym
```

2.21 HW-ID DTB

RK平台的U-Boot可以根据GPIO或者ADC的硬件状态，从多份DTB文件中筛选与硬件状态匹配的DTB进行加载。

> 更多参考：系统模块章节。

2.22 Partition Table

RK平台的U-Boot支持标准GPT格式，烧写用的分区表文件为parameter.txt：

```
FIRMWARE_VER:8.1
MACHINE_MODEL:RK3399
MACHINE_ID:007
MANUFACTURER: RK3399
MAGIC: 0x5041524B
ATAG: 0x00200800
MACHINE: 3399
CHECK_MASK: 0x80
PWR_HLD: 0,0,A,0,1
TYPE: GPT
CMDLINE:mtdparts=rk29xxnand:0x00002000@0x00004000(uboot),0x00002000@0x00006000(t
rust),0
x00002000@0x00008000(misc),0x00008000@0x0000a000(resource),0x00010000@0x00012000
(kernel
),0x00010000@0x00022000(boot),0x00020000@0x00032000(recovery),0x00038000@0x00052
000(bac
kup),0x00002000@0x0008a000(security),0x00100000@0x0008c000(cache),0x00500000@0x0
018c000
(system),0x00008000@0x0068c000(metadata),0x00100000@0x00694000(vendor),0x0010000
0@0x007
96000(oem),0x00000400@0x00896000(frp),-@0x00896400(userdata:grow)
```

2.23 Relocation

U-Boot会在完成board_f.c的流程后把自己relocate到内存末尾的某个地址上，具体地址视U-Boot内存布局而定。RK的U-Boot默认：

- 32位平台：`CONFIG_SKIP_RELOCATE_UBOOT=y`时不做relocation，否则有做。

- 64位平台默认有做relocation。

2.24 Reset

- U-Boot的复位和kernel一样，最终需要陷入trust里完成。
- U-Boot 命令行模式可以支持跟kernel一样的reboot xxx命令（依赖于kernel dts中的定义）。

2.25 Sd/Udisk

RK平台的U-Boot支持SD/U盘的固件启动或升级。其中：

- SD启动/升级是从bootrom这一级开始支持
- U盘启动/升级是从U-Boot这一级开始支持

> 更多参考：系统模块章节。

2.26 Stacktrace

原生的U-Boot不支持调用栈回溯机制，RK平台增加了该功能。目前一共有3种方式触发调用栈打印：

- 系统崩溃时自动触发；
- 用户主动调用`dump_stack()`；
- 使能`CONFIG_IRQ_TIMER_DUMP`；

例如系统abort：

```
"Synchronous Abort" handler, esr 0x96000010

// abort的原因、pc、lr、sp
* Reason:        Exception from a Data abort, from current exception level
* PC          =   000000000028f430
* LR          =   00000000002608d0
* SP          =   00000000f3dceb30

...

// 重点突出PC和LR
Call trace:
  PC:    [< 0028f430 >]
  LR:    [< 002608d0 >]

// 函数调用关系
Stack:
         [< 0028f430 >]
         [< 0028da24 >]
         [< 00211600 >]
         [< 002117b0 >]
         [< 00202910 >]
         [< 00202aa8 >]
         [< 0027698c >]
         [< 002151ec >]
         [< 00201b2c >]
```

```
// 指导用户转换上述调用栈信息
Copy info from "Call trace..." to a file(eg. dump.txt), and run
command in your U-Boot project: ./scripts/stacktrace.sh dump.txt
```

用户根据上述说明，把调用栈信息复制到任意txt文件（比如dump.txt）后执行如下命令：

```
cjh@Ubuntu:~/u-boot$ ./scripts/stacktrace.sh dump.txt

// 符号表来源
SYMBOL File: ./u-boot.sym

// 重点列出PC和LR对应的代码位置
Call trace:
 PC:   [< 0028f430 >]  strncpy+0xc/0x20       ./lib/string.c:98
 LR:   [< 002608d0 >]  on_serialno+0x10/0x1c ./drivers/usb/gadget/g_dnl.c:217

// 转换后得到真实函数名
Stack:
       [< 0028f430 >]  strncpy+0xc/0x20
       [< 0028da24 >]  hdelete_r+0xcc/0xf0
       [< 00211600 >]  _do_env_set.isra.0+0x70/0x1b8
       [< 002117b0 >]  env_set+0x3c/0x58
       [< 00202910 >]  rockchip_set_serialno+0x54/0x140
       [< 00202aa8 >]  board_late_init+0x5c/0xa0
       [< 0027698c >]  initcall_run_list+0x58/0x94
       [< 002151ec >]  board_init_r+0x20/0x24
       [< 00201b2c >]  relocation_return+0x4/0x0
```

注意事项：

- 转换命令有三种，具体用哪种请根据调用栈打印之后的指导说明。

  ```
  ./scripts/stacktrace.sh ./dump.txt        // 解析来自U-Boot的调用栈信息
  ./scripts/stacktrace.sh ./dump.txt tpl    // 解析来自tpl的调用栈信息
  ./scripts/stacktrace.sh ./dump.txt spl    // 解析来自spl的调用栈信息
  ```

  > 执行该命令时，当前机器上的固件必须和当前代码环境匹配才有意义！否则会得到错误的转换。

2.27 TimeCost

U-Boot 初始化结束默认会打印本阶段的总耗时：

```
### Booting Android Image at 0x02007800 ...
Kernel load addr 0x02008000 size 8062 KiB
### Flattened Device Tree blob at 08300000
   Booting using the fdt blob at 0x8300000
   XIP Kernel Image ... OK
  'reserved-memory' dma-unusable@fe000000: addr=fe000000 size=1000000
  'reserved-memory' ramoops@00000000: addr=8000000 size=f0000
   Using Device Tree in place at 08300000, end 08316ed1
Adding bank: 0x00000000 - 0x08400000 (size: 0x08400000)
Adding bank: 0x09200000 - 0x80000000 (size: 0x76e00000)
Total: 812.613 ms    // U-Boot阶段的总耗时

Starting kernel ...
```

用户可以打开`lib/initcall.c`的`debug()`和`DEBUG`获得如下的流程耗时，函数地址可借助./make.sh进行反汇编获得。

```
U-Boot v5-00019-g9b55ed0-dirty (Dec 26 2019 - 14:45:33 +0800)

                                        #    5212 us # 137.868 ms
initcall: 0020de1f
                                        #       1 us # 142.636 ms
initcall: 0020e015
Model: Evb-RK3288
                                        #    1646 us # 149.48 ms
initcall: 0020dd61
PreSerial: 2
                                        #    1213 us # 155.28 ms
initcall: 0020ddcd
DRAM:                                   #     606 us # 160.401 ms
initcall: 00203719
              // 如下 187 us     是 initcall: 00203719 调用的耗时
              // 如下 165.355 ms 是 initcall: 00203719 为止的U-Boot启动耗时
                                        #     187 us # 165.355 ms
initcall: 0020de81
                                        #       2 us # 169.938 ms
initcall: 0020dc29
                                        #       1 us # 174.703 ms
initcall: 0020dc3d
                                        #       1 us # 179.469 ms
initcall: 0020ddad
                                        #       2 us # 184.237 ms
initcall: 0020de27
                                        #       1 us # 189.2 ms
......
```

2.28 Timestamp

Kernel的打印信息默认带有时间戳，方便用户关注时间。U-Boot的打印信息默认没有带时间戳，用户有需要的话可以使能配置`CONFIG_BOOTSTAGE_PRINTF_TIMESTAMP`。如下：

```
[    0.324987] U-Boot v5-00019-g9b55ed0-dirty (Dec 26 2019 - 14:31:44 +0800)

[    0.327215] Model: Evb-RK3288
```

```
[    0.330039] PreSerial: 2
[    0.332526] DRAM:  2 GiB
[    0.336454] Relocation Offset: 00000000, fdt: 7be22c38
[    0.346981] Using default environment

[    0.351075] dwmmc@ff0c0000: 1, dwmmc@ff0f0000: 0
[    0.394136] Bootdev(atags): mmc 0
[    0.394272] MMC0: High Speed, 52Mhz
[    0.395276] PartType: EFI
[    0.400347] Android 9.0, Build 2019.6
[    0.402070] boot mode: None
[    0.405213] Found DTB in boot part
[    0.407833] DTB: rk-kernel.dtb
[    0.418211] ANDROID: fdt overlay OK
[    0.432128] I2c0 speed: 400000Hz
[    0.435916] PMIC:  RK808
[    0.439113] vdd_arm 1100000 uV
[    0.444148] vdd_gpu 1100000 uV
......

[    1.005018] ## Booting Android Image at 0x02007800 ...
[    1.009917] Kernel load addr 0x02008000 size 8062 KiB
[    1.014981] ## Flattened Device Tree blob at 08300000
[    1.019970]    Booting using the fdt blob at 0x8300000
[    1.025185]    XIP Kernel Image ... OK
[    1.035469]   'reserved-memory' dma-unusable@fe000000: addr=fe000000
size=1000000
[    1.037448]   'reserved-memory' ramoops@00000000: addr=8000000 size=f0000
[    1.044412]    Using Device Tree in place at 08300000, end 08316ed1
[    1.064363] Adding bank: 0x00000000 - 0x08400000 (size: 0x08400000)
[    1.064976] Adding bank: 0x09200000 - 0x80000000 (size: 0x76e00000)
[    1.075259] Total: 812.613 ms

[    1.075279] Starting kernel ...
……
```

> 注意：时间戳打印的是相对时间，而非绝对时间。

2.29 Vendor storage

RK平台的U-Boot提供了Vendor storage区域给用户保存SN、MAC等信息。存储偏移如下（详见vendor.c）：

```
#define EMMC_VENDOR_PART_OFFSET     (1024 * 7)
/* --- Spi Nand/SLC/MLC large capacity case define --- */
#define NAND_VENDOR_PART_OFFSET     0
/* --- Spi/Spi Nand/SLC/MLC small capacity case define --- */
#define FLASH_VENDOR_PART_OFFSET    8
......
```

用户一般不需要关注和修改存储偏移，只需要关注读写接口：

```
int vendor_storage_read(u16 id, void *pbuf, u16 size)
int vendor_storage_write(u16 id, void *pbuf, u16 size)
```

3. Chapter-3 编译烧写

3.1 前期准备

- 下载rkbin

  这是一个工具包仓库，用于存放RK不开源的bin、脚本、打包工具。U-Boot 编译时会从该仓库索引相关文件，打包生成loader、trust、uboot固件。rkbin和U-Boot工程必须保持同级目录关系。

- 下载GCC

  GCC编译器使用gcc-arm-10.3-2021.07，放置于prebuilts目录之内。prebuilts和U-Boot保持同级目录关系。如下：

  ```
  // 32位：
  prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabihf
  // 64位：
  prebuilts/gcc/linux-x86/aarch64/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu
  ```

- 选择defconfig：请参考平台定义章节。

- config fragment介绍

  由于单个平台上的产品存在差异化需求，一个 defconfig 已经无法满足要求。因此从 rk3588 开始支持 config fragment，即对 defconfig 进行 overlay。

  例如：`rk3588-emmc-tb.config` 里指定了 `CONFIG_BASE_DEFCONFIG="rk3588_defconfig"`，当执行 `./make.sh rk3588-emmc` 命令时，会先用 `rk3588_defconfig` 生成 `.config`，然后再使用 `rk3588-emmc-tb.config` 里的配置对 `.config` 进行 overlay。若仓库中使用的是 `rk3588-emmc.config`，请以实际文件名为准。该命令等效为：

  ```
  make rk3588_defconfig rk3588-emmc.config && make
  ```

  如果要对config fragment文件进行更新，只需要借助`./scripts/sync-fragment.sh`。例如：

  ```
  ./scripts/sync-fragment.sh configs/rk3588-emmc.config
  ```

  命令效果：将当前 `.config` 与 `rk3588_defconfig` 的配置差异项同步到目标 fragment 文件中。

3.2 固件编译

编译命令：

```
./make.sh [board]           // [board]：configs/[board]_defconfig文件。
```

首次编译：无论32位或64位平台，第一次或想重新指定defconfig，则编译命令必须指定[board]。例如：

```
./make.sh rk3588            // build for rk3588_defconfig
./make.sh rk3572            // build for rk3572_defconfig
./make.sh rk3572 --spl-new  // build for rk3572_defconfig with u-boot-spl.bin in
Loader
```

二次编译：无论32位或64位平台，如果要基于当前".config"二次编译，则编译命令不用指定[board]：

```
./make.sh
```

自编译 SPL：如果希望编译完成后得到的 Loader 文件中包含实时编译出来的 `u-boot-spl.bin`，则需要指定 `--spl-new`。

```
./make.sh --spl-new
```

开源USBPLUG编译：指定usbplug_defconfig编译后拷贝usbplug.bin至rkbin仓库即可。例如：

```
./make.sh rk3576-usbplug                                        // 编译出 usbplug
cp usbplug.bin ../rkbin/bin/rk35/rk3576_usbplug_vX.XX.bin // 更新 usbplug.bin 到
rkbin
./make.sh rk3576                                                // 再次编译得到包含
usbplug的loader
```

> 注意：
>
> 1. 如果编译时出现奇怪的问题导致编译失败，请尝试 `make distclean` 后重新编译。
> 2. 想了解已适配开源usbplug的平台，请参考CH14-平台定义_CN.md，暂不支持的平台，请参考rockchip-usbplug_defconfig实现或联系RK工程师。

固件生成：编译完成后会在U-Boot根目录下打包生成 `uboot.img`、`loader`。最终在根目录下生成可烧写的固件：

```
./uboot.img
./rk3572_loader_v1.01.101.bin
```

> 固件打包工具：请参考工具章节。

3.3 固件烧写

烧写工具：

Windows/Linux的固件烧写工具建议使用SDK发布的工具版本或最新版本。

烧写模式：

RK 平台共有两种烧写模式：Maskrom 模式和 Loader 模式（U-Boot）。

1. 进入Loader烧写模式的方法：

- 开机时，机器长按音量+
- 开机时，上位机长按`ctrl+d`组合键
- U-Boot命令行输入：`download` 或者 `rockusb 0 $devtype $devnum`

2. 进入Maskrom烧写模式的方法：

- 开机时，上位机长按`ctrl+b`组合键
- U-Boot命令行输入：`rbrom`

3.4 固件大小修改

请参考 CH02-RK架构_CN.md 中的 `Image U-Boot` 章节。

4. Chapter-4 系统模块

4.1 AArch32

ARMv8 的 64 位芯片支持从 AArch64 切换到 AArch32 模式运行 (与 ARMv7 兼容)，代码必须使用 32 位编译。

用户可以通过这个宏来确认当前是否为ARMv8的 AArch32 模式：

```
CONFIG_ARM64_BOOT_AARCH32=y
```

4.2 ANDROID AB

所谓的 A/B System，即把系统固件分为两份，分别称为 slot-a 和 slot-b。系统可以从任意一个 slot 启动；当一个 slot 启动失败后，还可以从另一个 slot 启动。同时，升级时可以直接将固件拷贝到另一个 slot 上，而无需进入系统升级模式。详细原理和流程请参考进阶原理章节。

目前 RK 平台的 pre-loader 和 U-Boot 都支持 A/B 系统。

4.2.1 配置项

A/B System 需要依赖 LIBAVB，如下：

```
// A/B依赖的库
CONFIG_LIBAVB=y
CONFIG_LIBAVB_AB=y
CONFIG_LIBAVB_ATX=y
CONFIG_LIBAVB_USER=y
// 使能A/B功能
CONFIG_ANDROID_AB=y
```

4.2.2 分区表

A/B System 对分区表有要求：需要支持 A/B 的分区必须增加后缀 `_a` 和 `_b`。`parameter.txt` 参考如下：

```
FIRMWARE_VER:8.1
MACHINE_MODEL:RK3326
MACHINE_ID:007
MANUFACTURER: RK3326
MAGIC: 0x5041524B
ATAG: 0x00200800
MACHINE: 3326
CHECK_MASK: 0x80
PWR_HLD: 0,0,A,0,1
TYPE: GPT
CMDLINE:
mtdparts=rk29xxnand:0x00002000@0x00004000(uboot_a),0x00002000@0x00006000(uboot_b),0x00002000@0x00008000(trust_a),0x00002000@0x0000a000(trust_b),0x00001000@0x00000c000(misc),0x00001000@0x0000d000(vbmeta_a),0x00001000@0x0000e000(vbmeta_b),0x00020000@0x0000f000(boot_a),0x00020000@0x0002f000(boot_b),0x00100000@0x0004f000(system_a),0x00300000@0x0032f000(system_b),0x00100000@0x0062f000(vendor_a),0x00100000@0x0072f000(vendor_b),0x00002000@0x0082f000(oem_a),0x00002000@0x00831000(oem_b),0x0010000@0x00833000(factory),0x00008000@0x00843000(factory_bootloader),0x00080000@0x008cb000(oem),-@0x0094b000(userdata)
```

4.3 ANDROID BCB

BCB（Bootloader Control Block）是 Android 为控制系统启动流程而设计的一种与 bootloader 交互的机制。数据结构定义在 misc 分区偏移 16 KB 或 0 的位置。

数据结构：

```
struct android_bootloader_message {
    char command[32];
    char status[32];
    char recovery[768];

    /* The 'recovery' field used to be 1024 bytes.  It has only ever
     * been used to store the recovery command line, so 768 bytes
     * should be plenty.  We carve off the last 256 bytes to store the
     * stage string (for multistage packages) and possible future
     * expansion. */
    char stage[32];

    /* The 'reserved' field used to be 224 bytes when it was initially
     * carved off from the 1024-byte recovery field. Bump it up to
     * 1184-byte so that the entire bootloader_message struct rounds up
     * to 2048-byte. */
    char slot_suffix[32];
    char reserved[1184-32];
};
```

command：启动命令，目前支持以下三个：

参数	功能
bootonce-bootloader	启动进入 U-Boot fastboot
boot-recovery	启动进入 recovery
boot-fastboot	启动进入 recovery 的 fastboot 模式（简称 fastbootd）

4.4 AVB 安全启动

AVB 为 Android Verified Boot，谷歌设计的一套固件校验流程，主要用于校验 boot system 等固件。Rockchip Secure Boot 参考前文加密校验思路及 AVB，实现一套完整的 Secure Boot 校验方案。

4.4.1 Feature

- 安全校验
- 完整性校验
- 防回滚保护
- persistent partition 支持
- chained partitions 支持，可以与 boot、system 签名私钥一致，也可以由 OEM 自己保存私钥，但必须由 PRK 签名

4.4.2 配置

开启 AVB 需要 OP-TEE 支持，以 rk3572 为例：

```
CONFIG_TEE=y
CONFIG_OPTEE=y
CONFIG_ROCKCHIP_OPTEE_V3=y
CONFIG_OPTEE_ALWAYS_USE_SECURITY_PARTITION=y // 安全数据默认存储到security分区，也可以选择存放在RPMB
```

OP-TEE 一共有两个版本，不同芯片具体使用哪个版本请参考 defconfig：

- `CONFIG_ROCKCHIP_OPTEE_V2`
- `CONFIG_ROCKCHIP_OPTEE_V3`

开启 AVB 相关配置：

```
// 编译 AVB 和 A/B system 库
CONFIG_LIBAVB=y
CONFIG_LIBAVB_AB=y
CONFIG_LIBAVB_ATX=y
CONFIG_LIBAVB_USER=y
// 开启 A/B system 和 AVB verify
CONFIG_ANDROID_AB=y
CONFIG_AVB_VERIFY=y
CONFIG_LIBAVB_VBMETA_PUBLIC_KEY_VALIDATE=y
// 开启严格 unlock 校验，需要证书才能将设备解锁
CONFIG_LIBAVB_ATH_UNLOCK_SUPPORT=y
// 如果需要cpuid作为challenge number，开启以下宏
```

```
CONFIG_MISC=y
CONFIG_ROCKCHIP_EFUSE=y
CONFIG_ROCKCHIP_OTP=y
```

4.4.3 参考

因为AVB涉及的内容比较多，其余原理、配置请参考进阶原理章节。

4.5 Cmdline

cmdline 是 U-Boot 向 kernel 传递参数的重要手段之一，例如传递启动存储、设备状态等信息。目前 cmdline 参数有多个来源，U-Boot 会对其进行拼接并过滤重复项，然后再传给 kernel。U-Boot 阶段的 cmdline 被保存在 `bootargs` 环境变量中。

U-Boot 最终通过修改 kernel DTB 中的 `/chosen/bootargs` 来实现 cmdline 传递。

4.5.1 数据来源

- kernel DTS 的 `/chosen/bootargs`，例如：

  ```
  chosen {
      bootargs = "earlyprintk=uart8250,mmio32,0xff30000 swiotlb=1
  console=ttyFIQ0
                  androidboot.baseband=N/A androidboot.veritymode=enforcing
                  androidboot.hardware=rk30board androidboot.console=ttyFIQ0
                  init=/init kpti=0";
  };
  ```

- U-Boot：根据当前运行的状态，U-Boot 会动态追加一些内容到 cmdline。比如：

  ```
  storagemedia=emmc androidboot.mode=emmc ......
  ```

- boot/recovery.img 固件头里通常也会有 cmdline 字段信息。

4.5.2 数据含义

下面列出 RK 平台常用的 cmdline 参数含义。更多内容可参考内核文档：`Documentation/admin-guide/kernel-parameters.txt`。

- sdfwupdate：SD 升级卡标志，recovery 程序需要；
- root=PARTUUID：指定 rootfs(system) 分区的UUID，仅 GPT 表支持
- skip_initramfs：kernel 不使用 U-Boot 加载的 ramdisk，而使用 rootfs(system) 中的 ramdisk
- storagemedia：存储启动类型；
- console：kernel 打印口的配置信息；
- earlycon：在串口节点未建立之前，指定串口及其配置
- loop.max_part：max_part 用来设定每个 loop 的设备所能支持的分区数目

- rootwait：用于文件系统暂时不可用的情况，例如 eMMC 尚未初始化完成。如果不设置 `rootwait`，可能会出现 `mount rootfs failed`；加上该参数后，可以等待驱动加载完成，再从存储设备中读取 rootfs 并挂载，从而避免启动失败
- ro/rw：加载 rootfs 的属性，只读/读写
- firmware_class.path：指定固件路径，如 wifi、bt、gpu 等
- dm="lroot none 0, 0 4096 linear 98:3 0, 4096 4096 linear 98:32" root=/dev/dm-0：Will boot to a rw dm-linear target of 8192 sectors split across two block devices identified by their major:minor numbers. After boot, udev will rename this target to /dev/mapper/lroot (depending on the rules). No uuid was assigned. 参考链接：https://android.googlesource.com/kernel/common/+/android-3.18/Documentation/device-mapper/boot.txt
- androidboot.slot_suffix：A/B 系统时为 kernel 指定从哪个 slot 启动
- androidboot.serialno：为 kernel 及上层提供序列号，例如 adb 的序列号等
- androidboot.verifiedbootstate：安卓需求，用于向上层提供 U-Boot 校验固件的状态，共有三种：
  1. green：如果处于 LOCKED 状态且用于验证的密钥不是由最终用户设置
  2. yellow：如果处于 LOCKED 状态且用于验证的密钥是由最终用户设置
  3. orange：如果处于 UNLOCKED 状态
- androidboot.hardware：启动设备，如 rk30board
- androidboot.verifymode：指定验证分区的真实模式/状态（即验证固件的完整性）
- androidboot.selinux：SELinux 是一种基于域-类型模型（domain-type）的强制访问控制（MAC）安全系统。有三种模式：
  1. enforcing：强制模式，表示 SELinux 正在运行，且已经正确开始限制 domain/type
  2. permissive：宽容模式，表示 SELinux 正在运行，但只会输出警告信息，不会实际限制 domain/type 的访问。这种模式可用作 SELinux 的调试用途
  3. disabled：关闭，SELinux 并没有实际运作
- androidboot.mode：安卓启动方式，有 normal 与 charger：
  1. normal：正常开机启动
  2. charger：关机后接入电源开机时，`androidboot.mode` 会被设置为 `charger`，该值由 U-Boot 检测到充电状态后写入 `bootargs` 环境变量
- androidboot.wificountrycode：设置 WiFi 国家码，如 US、CN
- androidboot.baseband：配置基带，RK 无此功能，设置为 N/A；
- androidboot.console：android 信息输出口配置
- androidboot.vbmeta.device=PARTUUID：指定 vbmeta 在存储中的位置
- androidboot.vbmeta.hash_alg：设置 vbmeta hash 算法，如 sha512
- androidboot.vbmeta.size：指定 vbmeta 的 size
- androidboot.vbmeta.digest：给 kernel 上传 vbmeta 的 digest，kernel 加载 vbmeta 后计算 digest，并与此 digest 对比
- androidboot.vbmeta.device_state：avb2.0 指定系统 lock 与 unlock

4.6 DTBO/DTO

为了便于用户对本章节内容的理解，这里先阅读附录章节，明确专业术语：DTB, DTBO, DTC, DTO, DTS, FDT。

它们之间的关系可以描述为：

- DTS 是用于描述 FDT 的文件；

- DTS 经过 DTC 编译后可生成 DTB/DTBO；
- DTB 和 DTBO 通过 DTO 操作可合并成一个新的 DTB；

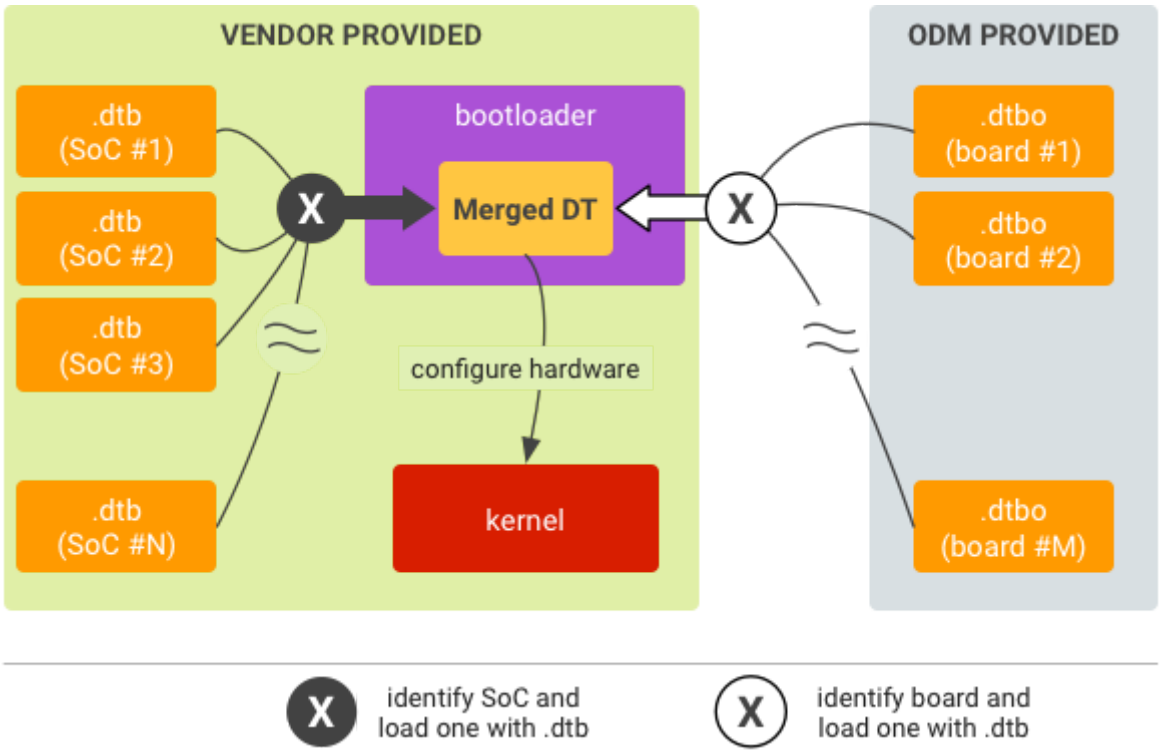
通常情况下，很多用户习惯用“DTBO”来代替“DTO”这一动作含义。下文中会避免混用这两个概念，明确约定：DTO 是动词概念，代表叠加操作；DTBO 是名词概念，指用于叠加的次 DTB。

本章节更多知识可参考：https://source.android.google.cn/devices/architecture/dto。

4.6.1 原理介绍

DTO (Device Tree Overlay) 是 Android P 后引入且必须强制启用的功能，可让次设备树 Blob (DTBO) 叠加到已有的主设备树 Blob 上。DTO 可以维护系统芯片 SoC 设备树，并动态叠加针对特定设备的设备树，从而向树中添加节点并修改现有节点属性。

主设备树 Blob（*.dtb）一般由 Vendor 厂商提供，次设备树 Blob（*.dtbo）可由 ODM/OEM 等厂商提供，最后由 bootloader 合并后再传递给 kernel。如下图：



> 图片来自：https://source.android.google.cn/devices/architecture/dto

需要注意：DTO 操作使用的 DTB 和 DTBO 的编译跟普通的 DTB 编译有区别，语法上有特殊要求：

使用 dtc 编译.dts 时，您必须添加选项-@以在生成的.dtbo 中添加*__symbols__*节点。*__symbols__*节点包含带标签的所有节点的列表，DTO 库可使用这个列表作为参考。如下示例：

1. 编译主.dts 的示例命令：

```
dtc -@ -O dtb -o my_main_dt.dtb my_main_dt.dts
```

2. 编译叠加层 DT `.dts` 的示例命令：

```
dtc -@ -O dtb -o my_overlay_dt.dtbo my_overlay_dt.dts
```

4.6.2 DTO 启用

1. 配置使能：

```
CONFIG_CMD_DTIMG=y
CONFIG_OF_LIBFDT_OVERLAY=y
```

2. board_select_fdt_index() 函数的实现。这是一个 `__weak` 函数，用户可以根据实际情况重新实现它。函数作用是在多份 DTBO 中获取用于执行 DTO 操作的那份 DTBO (返回 index 索引，最小从 0 开始)，默认的 weak 函数返回的 index 为 0。

```
/*
* Default return index 0.
*/
__weak int board_select_fdt_index(ulong dt_table_hdr)
{
    /*
     * User can use "dt_for_each_entry(entry, hdr, idx)" to iterate
     * over all dt entry of DT image and pick up which they want.
     *
     * Example:
     *  struct dt_table_entry *entry;
     *  int index;
     *
     *  dt_for_each_entry(entry, dt_table_hdr, index) {
     *
     *      .... (use entry)
     *  }
     *
     *  return index;
     */
    return 0;
}
```

4.6.3 DTO 结果

1. DTO 执行完成后，在 U-Boot 的开机信息中可以看到结果：

```
// 成功时的打印
ANDROID: fdt overlay OK

// 失败时的打印
ANDROID: fdt overlay failed, ret=-19
```

通常引起失败的原因一般都是因为主/次设备树 blob 的内容存在不兼容，所以用户需要对它们的生成语法和兼容性比较清楚。

2. DTO 执行成功后在给 kernel 的 cmdline 里追加如下信息，表明当前使用哪份 DTBO 进行 DTO 操作：

```
androidboot.dtbo_idx=1     // idx 从 0 开始，这里表示选取 idx=1 的那份 DTBO 进行 DTO 操作
```

3. DTO 执行成功后可以在 U-Boot 命令行使用 `fdt` 命令查看 DTB 内容，确认改动是否生效。

4.7 ENV

4.7.1 框架支持

ENV 是 U-Boot 框架中非常重要的一种数据管理方式，通过 hash table 构建"键值"和"数据"进行映射管理，支持"增/删/改/查"操作。通常，我们把它管理的键值和数据统称为：环境变量。U-Boot 支持把 ENV 数据保存在各种存储介质：NOWHERE / eMMC / FLASH / EEPROM / NAND / SPI_FLASH / UBI...

配置：

```
// 默认配置：ENV保存在内存
CONFIG_ENV_IS_NOWHERE

// ENV保存在各种存储介质
CONFIG_ENV_IS_IN_EEPROM
CONFIG_ENV_IS_IN_FAT
CONFIG_ENV_IS_IN_EXT4
CONFIG_ENV_IS_IN_FLASH
CONFIG_ENV_IS_IN_MMC
CONFIG_ENV_IS_IN_NAND
CONFIG_ENV_IS_IN_NVRAM
CONFIG_ENV_IS_IN_REMOTE
CONFIG_ENV_IS_IN_SPI_FLASH
CONFIG_ENV_IS_IN_FRAGMENT

// 任意已经接入到 BLK 框架层的存储介质（MMC 除外），RK 平台推荐使用！
CONFIG_ENV_IS_IN_BLK
```

框架代码：

```
./env/nowhere.c
./env/blk.c
./env/mmc.c
./env/nand.c
./env/eeprom.c
./env/embedded.c
./env/ext4.c
./env/fat.c
./env/flash.c
......
```

4.7.2 相关接口

```
// 获取环境变量
char *env_get(const char *varname);
ulong env_get_ulong(const char *name, int base, ulong default_val);
ulong env_get_hex(const char *varname, ulong default_val);

// 修改或创建环境变量，value为NULL时等同于删除操作
int env_set(const char *varname, const char *value);
int env_set_ulong(const char *varname, ulong value);
int env_set_hex(const char *varname, ulong value);
```

```
// 把保存在存储介质上的ENV信息全部加载出来
int env_load(void);

// 把当前所有ENV信息保存到存储介质上
int env_save(void);
```

- env_load()：用户不需要调用，U-Boot 框架会在合适的启动流程里调用；
- env_save()：用户在需要的时刻主动调用，会把所有的 ENV 信息保存到 CONFIG_ENV_IS_XXX 指定的存储介质。

4.7.3 高级接口

RK 提供了两个统一处理 ENV 的高级接口，具有创建、追加、替换的功能。主要是为了处理 `bootargs` 环境变量，但同样适用于其他环境变量操作。

```
/**
 * env_update() - update sub value of an environment variable
 *
 * This add/append/replace the sub value of an environment variable.
 *
 * @varname: Variable to adjust
 * @value: Value to add/append/replace
 * @return 0 if OK, 1 on error
 */
int env_update(const char *varname, const char *varvalue);

/**
 * env_update_filter() - update sub value of an environment variable but
 * ignore some key word
 *
 * This add/append/replace/ignore the sub value of an environment variable.
 *
 * @varname: Variable to adjust
 * @value: Value to add/append/replace
 * @ignore: Value to be ignored that in varvalue
 * @return 0 if OK, 1 on error
 */
int env_update_filter(const char *varname, const char *varvalue, const char
*ignore);
```

1. env_update() 使用规则：

- 创建：如果 varname 不存在，则创建 varname 和 varvalue；
- 追加：如果 varname 已存在，varvalue 不存在，则追加 varvalue；
- 替换：如果 varname 已存在，varvalue 已存在，则用当前的 varvalue 替换原来的。比如：原来存在“storagemedia=emmc”，当前传入 varvalue 为“storagemedia=rknand”，则最终更新为"storagemedia=rknand"。

2. env_update_filter() 是 env_update() 的扩展版本：在更新 env 的同时把 varvalue 里的某个关键字剔除；
3. 特别注意：env_update() 和 env_update_filter() 都是以空格和“=”作为分隔符对 ENV 内容进行单元分割，所以操作单元是：单个词、"key=value" 组合词：

- 单个词：sdfwupdate、……

- "key=value"组合词：storagemedia=emmc、init=/init、androidboot.console=ttyFIQ0、……
- 上述两个接口无法处理长字符串单元。比如无法把“console=ttyFIQ0 androidboot.baseband=N/A androidboot.selinux=permissive“作为一个整体单元进行操作。

4.7.4 存储位置

env_save()可以把 ENV 保存到存储介质，RK 平台的 ENV 的存储位置和大小定义如下：

```
config ENV_OFFSET
    hex "Environment offset"
    depends on ENV_IS_IN_EEPROM || ENV_IS_IN_MMC || ENV_IS_IN_NAND || \
            ENV_IS_IN_SPI_FLASH || ENV_IS_IN_BLK || ENV_IS_IN_FRAGMENT
    default 0x0 if ENV_IS_IN_FRAGMENT
    default 0x3f8000
    help
      Offset from the start of the device (or partition).

      This offset may be interpreted differently depending on the chosen
      ENV_IS_IN_* options. For example, for ENV_IS_IN_MMC=y, this offset may
      be negative to indicate an offset backwards from the end of the
      partition. See the relevant help messages for more details.

config ENV_SIZE
    hex "Environment Size"
    default 0x8000
    help
      Size of the environment storage area
```

- 通常，ENV_OFFSET 和 ENV_SIZE 都不建议修改。

4.7.5 通用选项

目前常用的存储介质一般有：eMMC/sdmmc/Nandflash/Norflash 等。但 U-Boot 原生的 Nand、Nor 类 ENV 驱动都走 MTD 框架，而 RK 所有已支持的存储都是走 BLK 框架层，因此这些 ENV 驱动无法使用。

为此，RK 为接入 BLK 框架层的存储提供 `CONFIG_ENV_IS_IN_BLK` 配置选项：

- eMMC/sdmmc 的情况，依然选择 CONFIG_ENV_IS_IN_MMC；
- Nand、Nor 的情况，可以选择 CONFIG_ENV_IS_IN_BLK；

请用户使用前先阅读 Kconfig 的 `CONFIG_ENV_IS_IN_BLK` 定义。

```
// 已经默认被指定好，不需要修改
CONFIG_ENV_OFFSET
CONFIG_ENV_SIZE

// 通常不需要使用到
CONFIG_ENV_OFFSET_REDUND (optional)
```

注意：无论选择哪个 CONFIG_ENV_IS_IN_XXX 配置，请先阅读 Kconfig 中的定义说明，里面都有子配置说明。

4.7.6 fw_printenv 工具

`fw_printenv` 是 U-Boot 提供给 Linux 使用的 env 工具。通过该工具，用户可以在 Linux 上访问和修改 env 内容。使用该工具要求 env 区域必须位于一个 kernel 可见的分区上（建议独立分区），本质上是通过 kernel sys 下的存储节点访问 env 区域。

工具获取方式：

```
./make.sh env
```

执行完命令后获得：

```
./tools/env/fw_printenv      // env读写工具
./tools/env/fw_env.config    // env配置文件
./tools/env/README           // env读写工具说明文档
```

使用方法请参考 README 文档。

4.8 Fastboot

Fastboot 是 Android 提供的一种借助 USB 和 U-Boot 进行交互的方式，一般用于获取设备信息、烧写固件等。

4.8.1 配置选项

```
// 使能配置
CONFIG_FASTBOOT
CONFIG_FASTBOOT_FLASH
CONFIG_USB_FUNCTION_FASTBOOT
CONFIG_FASTBOOT_OEM_BOARD
CONFIG_FASTBOOT_OEM_BOARD_UNLOCK

// 参数配置
CONFIG_FASTBOOT_BUF_ADDR
CONFIG_FASTBOOT_BUF_SIZE
CONFIG_FASTBOOT_FLASH_MMC_DEV
CONFIG_FASTBOOT_USB_DEV
```

4.8.2 触发方式

Fastboot 默认使用 Google adb 的 VID/PID，有如下几种触发方式：

- kernel 的命令行执行：`reboot fastboot`
- U-Boot 的命令行执行：`fastboot usb 0`
- 开机长按组合键：`Ctrl`+`F`

4.8.3 fastboot 命令

U-Boot 下可以通过输入命令进入 fastboot：

```
fastboot usb 0
```

4.8.3.1 命令速览

```
fastboot flash < partition > [ < filename > ]
fastboot erase < partition >
fastboot getvar < variable > | all
fastboot set_active < slot >
fastboot reboot
fastboot reboot-bootloader
fastboot flashing unlock
fastboot flashing lock
fastboot stage [ < filename > ]
fastboot get_staged [ < filename > ]
fastboot oem format
fastboot oem board:unlock
fastboot oem board:lock
fastboot oem board:at-get-unlock-challenge
fastboot oem board:fuse at-perm-attr
fastboot oem board:fuse at-rsa-perm-attr
fastboot oem board:fuse at-bootloader-vboot-key
fastboot oem board:init-ab-metadata
```

4.8.3.2 命令使用

1. fastboot flash < partition > [< filename >]

功能：分区烧写。

例： fastboot flash boot boot.img

2. fastboot erase < partition >

功能：擦除分区。

举例：fastboot erase boot

3. fastboot getvar < variable > | all

功能：获取设备信息

4. 举例：fastboot getvar all （获取设备所有信息）

variable 还可以带的参数：

```
version                                 /* fastboot 版本 */
version-bootloader                      /* U-Boot 版本 */
version-baseband                        /* 基带版本 */
filesize                                /* 最近下载的文件大小 */
downloadsize/max-download-size          /* fastboot 支持单次传输最大字节数 */
serialno                                /* 序列号 */
product                                 /* 产品名称/型号 */
platform                                /* 硬件平台 */
slot-count                              /* slot 数目 */
```

```
has-slot: < partition >                /* 查看slot内是否有该分区名 */
current-slot                           /* 当前启动的slot */
slot-suffixes                          /* 当前设备具有的slot,打印出其name */
slot-successful: < _a | _b >           /* 查看分区是否正确校验启动过 */
slot-unbootable: < _a | _b >           /* 查看分区是否被设置为unbootable */
slot-retry-count: < _a | _b >          /* 查看分区的retry-count次数 */
logical-block-size                     /* 逻辑块数 */
erase-block-size                       /* 擦除块数 */
vboot-state                            /* 验证启动状态 */
unlocked                               /* 设备lock状态 */
flash-unlocked                         /* 设备 flash lock状态 */
partition-type : < partition >         /* 分区类型 */
partition-size : < partition >         /* 分区大小 */
snapshot-update-status                 /* 快照更新状态（虚拟 A/B 系统） */
avb-state                              /* AVB 状态 */
```

1. fastboot set_active < slot >

功能：设置重启的 slot。

举例：`fastboot set_active _a`

2. fastboot reboot

功能：重启设备，正常启动

3. fastboot reboot-bootloader

功能：重启设备，进入 fastboot 模式

4. fastboot flashing unlock

功能：解锁设备，允许烧写固件

5. fastboot flashing lock

功能：锁定设备，禁止烧写

6. fastboot stage [< filename >]

功能：下载数据到设备端内存，内存起始地址为 CONFIG_FASTBOOT_BUF_ADDR

举例：`fastboot stage permanent_attributes.bin`

7. fastboot get_staged [< filename >]

功能：从设备端获取数据

举例：`fastboot get_staged raw_unlock_challenge.bin`

8. fastboot oem board:fuse at-perm-attr

功能：烧写 permanent_attributes.bin 及 hash

举例：

```
fastboot stage permanent_attributes.bin
fastboot oem board:fuse at-perm-attr
```

9. fastboot oem board:lock

功能：锁定设备

10. fastboot oem board:unlock

功能：解锁设备，现支持 authenticated unlock

举例：

```
fastboot oem at-get-vboot-unlock-challenge
fastboot get_staged raw_unlock_challenge.bin
./make_unlock.sh（见 make_unlock.sh 参考）
fastboot stage unlock_credential.bin
fastboot oem board:unlock（无严格校验的话仅需这一句）
```

11. fastboot oem board:fuse at-bootloader-vboot-key

功能：烧写 bootloader key hash

举例：

```
fastboot stage bootloader-pub-key.bin
fastboot oem board:fuse at-bootloader-vboot-key
```

12. fastboot oem format

功能：重新格式化分区，分区信息依赖于 `$partitions`

13. fastboot oem board:at-get-unlock-challenge

功能：authenticated unlock，需要获得 unlock challenge 数据

4.9 FileSystem

4.9.1 框架支持

FAT 和 EXT2/4 是常用的文件系统格式。其中 FAT 采用的是 DOS (MBR) 分区表，常见设备有：SD 卡、U 盘。

目前在 U-Boot 中一般访问这两种文件系统比较多。

FAT 配置：

```
CONFIG_DOS_PARTITION=y
CONFIG_FS_FAT=y
CONFIG_FAT_WRITE=y
CONFIG_FS_FAT_MAX_CLUSTSIZE=65536
CONFIG_CMD_FAT=y
CONFIG_CMD_FS_GENERIC=y
```

FAT 命令：

```
fatinfo fatload fatls fatsize fatwrite
```

EXT2/4 配置：

```
CONFIG_CMD_EXT2=y
CONFIG_CMD_EXT4=y
CONFIG_CMD_FS_GENERIC=y
```

EXT2/4 命令：

```
ext2load ext2ls ext4load ext4ls ext4size
```

4.9.2 相关接口

FAT 函数头文件 `./include/fat.h`：

```
int file_fat_detectfs(void);
int fat_exists(const char *filename);
int fat_size(const char *filename, loff_t *size);
int file_fat_read_at(const char *filename, loff_t pos, void *buffer,
             loff_t maxsize, loff_t *actread);
int file_fat_read(const char *filename, void *buffer, int maxsize);
int fat_set_blk_dev(struct blk_desc *rbdd, disk_partition_t *info);
int fat_register_device(struct blk_desc *dev_desc, int part_no);

int file_fat_write(const char *filename, void *buf, loff_t offset, loff_t len,
           loff_t *actwrite);
int fat_read_file(const char *filename, void *buf, loff_t offset, loff_t len,
          loff_t *actread);
int fat_opendir(const char *filename, struct fs_dir_stream **dirsp);
int fat_readdir(struct fs_dir_stream *dirs, struct fs_dirent **dentp);
void fat_closedir(struct fs_dir_stream *dirs);
void fat_close(void);
```

EXT2/4 函数头文件 `include/ext4fs.h`：

```
int ext4fs_init(void);
void ext4fs_deinit(void);
int ext4fs_filename_unlink(char *filename);
int ext4fs_write(const char *fname, unsigned char *buffer,
         unsigned long sizebytes);
int ext4_write_file(const char *filename, void *buf, loff_t offset, loff_t len,
            loff_t *actwrite);
#endif
#if defined(CONFIG_CMD_EXT4_SPARSE_WRITE)
int ext4_unsparse(struct blk_desc *desc, const u8 *buf, ulong start);
#endif
struct ext_filesystem *get_fs(void);
int ext4fs_open(const char *filename, loff_t *len);
int ext4fs_read(char *buf, loff_t offset, loff_t len, loff_t *actread);
int ext4fs_mount(unsigned part_length);
void ext4fs_close(void);
void ext4fs_reinit_global(void);
int ext4fs_ls(const char *dirname);
int ext4fs_exists(const char *filename);
int ext4fs_size(const char *filename, loff_t *size);
void ext4fs_free_node(struct ext2fs_node *node, struct ext2fs_node *currroot);
```

```
int ext4fs_devread(lbaint_t sector, int byte_offset, int byte_len, char *buf);
void ext4fs_set_blk_dev(struct blk_desc *rbdd, disk_partition_t *info);
long int read_allocated_block(struct ext2_inode *inode, int fileblock);
int ext4fs_probe(struct blk_desc *fs_dev_desc,
         disk_partition_t *fs_partition);
int ext4_read_file(const char *filename, void *buf, loff_t offset, loff_t len,
           loff_t *actread);
int ext4_read_superblock(char *buffer);
int ext4fs_uuid(char *uuid_str);
#endif
```

4.9.3 命令示例

```
// 确认 SD 卡可识别（如果是 U 盘则用 usb 命令进行识别，设备编号一般是：usb 0）
=> mmc dev 1
switch to partitions #0, OK
mmc1 is current device

// 查看信息
=> fatinfo mmc 1
Interface:  MMC
  Device 1: Vendor: Man 000003 Snr e81ec501 Rev: 1.9 Prod: SC16G
  Type: Removable Hard Disk
  Capacity: 15193.5 MB = 14.8 GB (31116288 x 512)
Filesystem: FAT32 "NO NAME    "

// 查看文件
=> fatls mmc 1
            System Volume Information/
       23   hello.txt
       23   linux.txt

2 file(s), 1 dir(s)

// 读取hello.txt文件的大小（结果默认被保存到变量filesize中）
=> fatsize mmc 1 hello.txt
=> echo $filesize
0x17

// 读取hello.txt文件到0x2000000地址
=> fatload mmc 1 0x2000000 hello.txt
reading hello.txt
23 bytes read in 2 ms (10.7 KiB/s)
// 查看读取的hello.txt内容
=> md.l 0x2000000
02000000: 6c6c6568 65682d6f 2d6f6c6c 6c6c6568    hello-hello-hell
02000010: 65682d6f ff6f6c6c ffffffff ffffffff    o-hello.........

// 新建文件：hello-copy.txt。把0x2000000~0x2000017的地址内容写入hello-copy.txt。
=> fatwrite mmc 1 0x2000000 hello-copy.txt 0x17
writing hello-copy.txt
23 bytes written
// 看到新文件：hello-copy.txt
=> fatls mmc 1
            System Volume Information/
```

```
        23    hello.txt
        23    linux.txt
        23    hello-copy.txt

 3 file(s),  1 dir(s)
```

说明：ext2/4 和 FAT 命令的使用方法类似，故不做具体说明。

4.10 HW-ID DTB

RK 平台的 U-Boot 支持检测硬件上的 GPIO 或者 ADC 状态动态加载不同的 Kernel DTB，暂称为 HW-ID DTB (Hardware ID DTB) 功能。

4.10.1 设计原理

通常硬件设计会经常更新版本和一些元器件，比如：屏幕、WiFi 模组等。如果每一个硬件版本都要对应一套软件，维护起来就比较麻烦。所以需要 HW_ID 功能实现一套软件可以适配不同版本的硬件。

针对不同硬件版本，软件上需要提供对应的 DTB 文件，同时还要提供 ADC/GPIO 硬件唯一值用于表征当前硬件版本（比如：固定的 ADC 值、固定的某 GPIO 电平）。

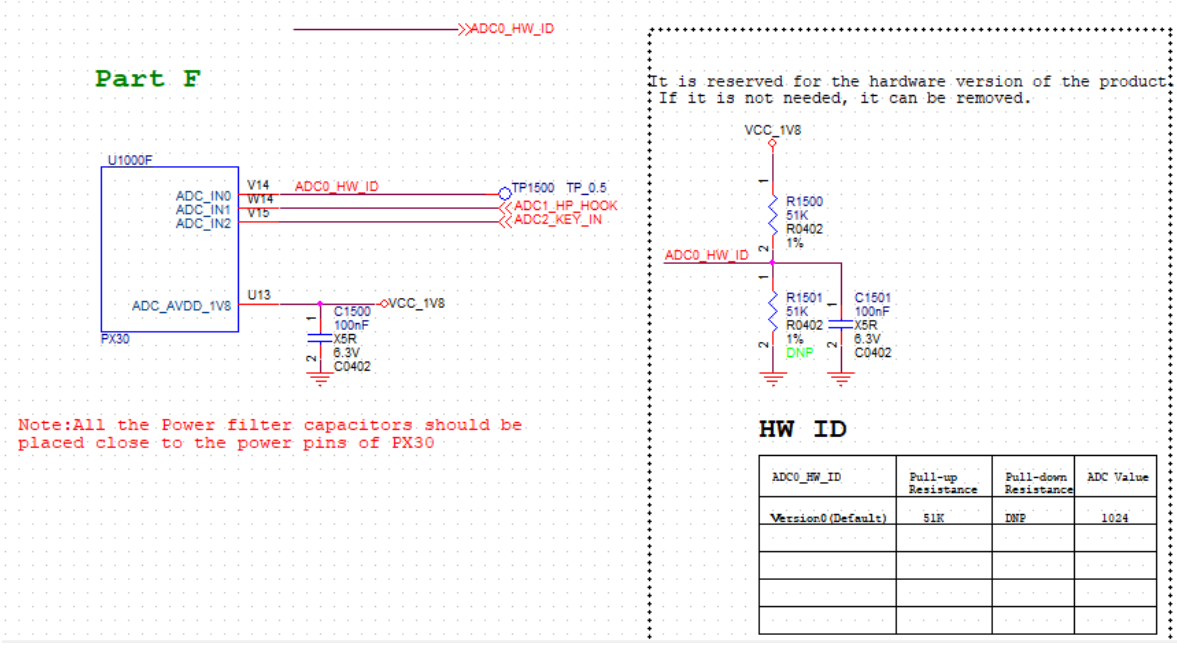
用户把这些和硬件版本对应的 DTB 文件全打包进同一个 resource.img，U-Boot 引导 kernel 时会检测硬件唯一值，从 resource.img 里找出和当前硬件版本匹配的 DTB 传给 kernel。

4.10.2 硬件参考

目前支持 ADC 和 GPIO 两种方式确定硬件版本。

ADC 参考设计

RK3326-EVB/PX30-EVB 主板上有预留分压电阻，不同的电阻分压有不同的 ADC 值，这样可以确定不同硬件版本：



配套使用的 MIPI 屏小板预留有另外一颗下拉电阻：

不同的 MIPI 屏会配置不同的阻值，配合 EVB 主板确定一个唯一的 ADC 参数值。

目前 V1 版本的 ADC 计算方法：ADC 参数最大值为 1024，对应着 ADC_IN0 引脚被直接上拉到供电电压 1.8 V，MIPI 屏上有一颗 10 K 的下拉电阻，接通 EVB 板后 ADC=1024*10K/(10K + 51K) =167.8。

GPIO 参考设计

目前没有 GPIO 的硬件参考设计，用户可自己定制。

4.10.3 DTB 命名

用户需要将ADC/GPIO 的硬件唯一值信息体现在 dtb 文件名里。命名规则如下：

ADC 作为 HW_ID DTB：

- 文件名以".dtb"结尾；
- HW_ID 格式：`#[controller]_ch[channel]=[adcval]`，称为一个完整单元

  `[controller]`: DTS 里面 ADC 控制器的节点名字。

  `[channel]`: ADC 通道。

  `[adcval]`: ADC 的中心值，实际有效范围是：adcval±30。
- 每个完整单元必须使用小写字母，内部不能有空格；
- 多个单元之间通过#进行分隔，最多支持 10 个单元；

范例：

```
rk3326-evb-lp3-v10#saradc_ch2=111#saradc_ch1=810.dtb
rk3326-evb-lp3-v10#_saradc_ch2=569.dtb
```

GPIO 作为 HW_ID DTB：

- 文件名以".dtb"结尾；
- HW_ID 格式：`#gpio[pin]=[level]`，称为一个完整单元

  `[pin]`: GPIO 脚，如 0a2 表示 gpio0a2

  `[level]`: GPIO 引脚电平。
- 每个完整单元必须使用小写字母，内部不能有空格；
- 多个单元之间通过#进行分隔，最多支持 10 个单元；

范例：

```
rk3326-evb-lp3-v10#gpio0a2=0#gpio0c3=1.dtb
```

4.10.4 DTB 打包

kernel 仓库：`scripts/mkmultidtb.py`。通过该脚本可以把多个 DTB 打包进同一个 resource.img。

用户需要打开脚本文件把要打包的 DTB 文件写到 DTBS 字典里面，并填上对应的 ADC/GPIO 的配置信息。

```
...
DTBS = {}
DTBS['PX30-EVB'] = OrderedDict([('rk3326-evb-lp3-v10', '#_saradc_ch0=166'),
                ('px30-evb-ddr3-lvds-v10', '#_saradc_ch0=512')])
...
```

上例中，执行 `scripts/mkmultidtb.py PX30-EVB` 就会生成包含 3 份 DTB 的 resource.img：

- rk-kernel.dtb：RK 默认的 DTB，不体现在上述字典中。所有 DTB 都没匹配成功时默认被使用。打包脚本会使用 DTBS 的第一个 DTB 作为默认的 DTB；
- rk3326-evb-lp3-v10#_saradc_ch0=166.dtb：包含 ADC 信息的 rk3326 DTB 文件；
- px30-evb-ddr3-lvds-v10#_saradc_ch0=512.dtb：包含 ADC 信息的 px30 DTB 文件；

如果是手动执行 `scripts/mkmultidtb.py`，则还要手动把生成的 resource.img 替换进 boot.img。为了方便，用户可以参考下面这样增加调用，内核仓库编译时就会自动包含多 DTB 的 resource.img。

```
diff --git a/scripts/mkimg b/scripts/mkimg
index 1d895602935..5e1355b9da8 100755
--- a/scripts/mkimg
+++ b/scripts/mkimg
@@ -248,6 +248,7 @@ if [ "${srctree}" != "${objtree}" ]; then
 fi
 scripts/resource_tool ${DTB_PATH} ${LOGO} ${LOGO_KERNEL} >/dev/null
 echo "  Image:  resource.img (with ${DTB} ${LOGO} ${LOGO_KERNEL}) is ready"
+./scripts/mkmultidtb.py PX30-EVB

 if [ -f "${BOOT_IMG}" ]; then
        if file -L -p -b ${BOOT_IMG} | grep -q 'Device Tree Blob' ; then
```

4.10.5 功能启用

配置选项：

```
CONFIG_ROCKCHIP_HWID_DTB=y
```

驱动代码：

```
./arch/arm/mach-rockchip/resource.c // 具体实现：rockchip_read_hwid_dtb()
```

DTS 配置：

如果用 GPIO 作为硬件识别，必须在 rkxx-u-boot.dtsi 中保留对应的 pinctrl 和 gpio 节点；ADC 默认已使能。

例如：gpio0 和 gpio1 作为识别：

```
...

&pinctrl {
    bootph-all;  // 追加该属性，让该节点被保留在 U-Boot DTB 中。下同。
};

&gpio0 {
```

```
    bootph-all;
};

&gpio1 {
    bootph-all;
};

...
```

4.10.6 加载结果

```
......
mmc0(part 0) is current device
boot mode: None
DTB: rk3326-evb-lp3-v10#_saradc_ch0=166.dtb // 打印匹配的 DTB，否则默认"rk-
kernel.dtb"
Using kernel DTB
......
```

4.11 SD 和 U 盘

本章节主要介绍 RK 平台上的 SD 和 U 盘的固件启动、升级。

4.11.1 机制原理

启动卡和升级卡制作完成后，都会在固件头部的固定存储偏移位置打上固定的 tag，用于标记当前是启动卡还是升级卡。U-Boot 识别到这个标记后就会走启动或升级流程。其中：

- 启动卡：卡内只有一份完整的固件。U-Boot 直接使用这份完整固件正常启动系统；
- 升级卡：卡内有两份固件。制作升级卡时，PC 工具会烧录两份固件：一份仅含进入 recovery 模式必须的分区镜像（记为 A 固件），一份是完整的 update.img 固件（记为 B 固件）。U-Boot 使用 A 固件引导系统进入 recovery 模式，然后由 recovery 程序使用 B 固件完成升级工作。

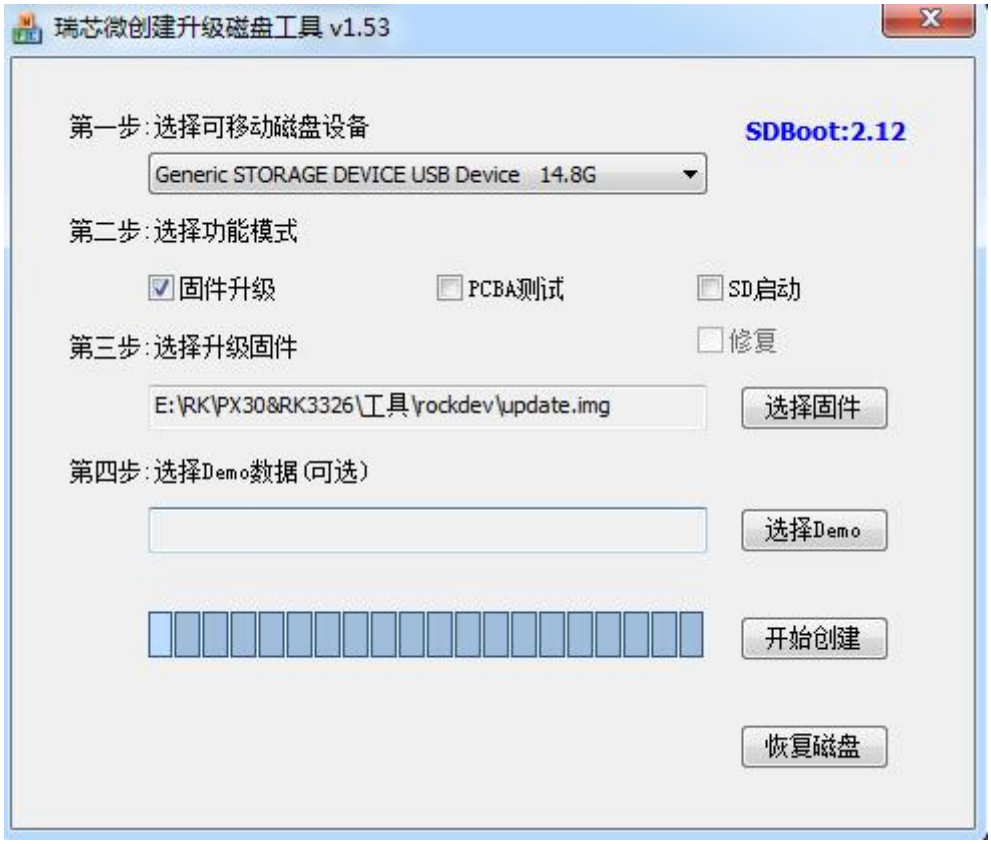
特别注意：

- SD 卡启动/升级是从 BootROM 这一级就开始支持；
- U 盘启动/升级仅从 U-Boot 这一级开始支持，即用户至少要保证 U-Boot 能正常工作！

4.11.2 固件制作

RK 平台上的 SD 和 U 盘启动卡、升级卡的制作流程是完全一致的，仅需两个步骤：

- 使用 SDK 目录下的 `RKTools/linux/Linux_Pack_Firmware/rockdev` 工具生成 update.img。
- 使用 SDDiskTool 工具把 update.img 烧录到 SD 或 U 盘。如图：
    - 选择可移动磁盘
    - 选择`固件升级`或者 `SD 启动`
    - 点击`开始创建`



4.11.3 SD 配置

SD 启动/升级：各平台 SDK 发布的 U-Boot 已经默认使能该功能，用户不需要额外配置。

4.11.4 USB 配置

U 盘启动/升级：各平台 SDK 发布的 U-Boot 默认没有使能。因为 U-Boot 原生的 USB 扫描命令很耗时，所以用户自己按需开启：

- 步骤 1：烧写升级用的整套固件到本地存储（eMMC / Nand / ...等），确认这套固件正常可用。
- 步骤 2：插上 U 盘开机进入 U-Boot 命令行模式。执行 `usb start` 和 `usb info` 命令确认能正常识别 U 盘，否则请先调通 U 盘的识别。
- 步骤 3：将满足步骤 1 的 kernel DTB 拷贝一份命名成 kern.dtb 放到 U-Boot 的 `./dts/` 目录下。这份 kern.dtb 会在编译 U-Boot 时被自动打包进 uboot.img。

  > kern.dtb 用途：当本地存储分区的 kernel DTB 有损坏时，U-Boot 使用 kern.dtb 确保 USB 能被正常初始化。

- 步骤 4：U-Boot 使能 U 盘启动/升级配置

  ```
  CONFIG_ROCKCHIP_USB_BOOT=y
  ```

  重新编译烧写 uboot.img。

  > 如果该过程提示 U-Boot 的固件过大无法打包生成，是因为步骤 3 加入 kern.dtb 引起的，请先裁掉一些不用的 U-Boot 配置。

4.11.5 功能生效

如何确认 SD、U 盘启动或升级功能生效。

用户可以擦除本地存储（eMMC、Nand...）上的 kernel、resource、boot、recovery 等关键分区，确认插上 SD / U 盘后能进入 kernel。

4.11.6 注意事项

- U 盘初始化时会调用 `usb start` 命令，整个过程相对耗时；
- 如果启动/升级卡要支持 GPT 分区表，则 SDDiskTool 工具的版本要求 >= v1.59；
- 如果启动/升级卡要支持 A/B 系统，则 SDDiskTool 工具的版本要求 >= v1.61；
- 如果用户手上的 SDK 比较旧，除了单独增加上述的补丁，建议跟负责 recovery 的工程师确认是否 recovery 有相关补丁。

5. Chapter-5 驱动模块

5.1 AMP

5.1.1 实现思路

U-Boot 框架默认没有 AMP(Asymmetric Multi-Processing) 支持，RK 自己实现了一套 AMP 机制：不同的CPU运行不同的固件。

实现思路：

（1）固件打包

所有的AMP固件（不含Linux）通过its文件指定CPU运行状态、描述固件信息，最后打包成一个FIT格式的amp.img烧写到amp分区。

开机时U-Boot负责加载amp.img固件并进行sha256完整性校验，再由trust指定各CPU的运行状态并派发到相应入口地址。

（2）启动顺序

运行U-Boot的CPU称为主核，主核在完成其它核的状态切换和固件跳转后，最后对自身进行操作。

（3）资源管理

U-Boot不负责AMP方案下各固件之间的资源协调（包括内存和中断等的划分），请开发者自己保证。

（4）Trust配合

AMP功能需要trust配合。如果用户指定的CPU运行状态是默认状态，则SDK的trust已支持；如果非默认状态，则需要trust额外支持（但部分平台SDK已默认支持）。

> 上述CPU默认状态指的是：
>
> 32位芯片默认状态：arch = "arm", thumb = <0>, hyp = 0;
>
> 64位芯片默认状态：arch = "arm64", thumb = <0>, hyp = 1;

（5）Linux+AMP组合

1. 考虑到兼容性，组合方案下的Linux相关固件跟传统SMP固件保持一致，即组合方案的固件形式为：Linux相关固件 + amp.img。
2. 开发者可以在amp的its里通过增加"linux"节点指定CPU运行Linux的状态（不指定就是默认状态）。
3. 如果主核跑Linux或没有指定任何固件，则主核启动完其它AMP固件后会按传统的SMP启动方式，由U-Boot启动Linux。
4. 如果非主核跑Linux，则优先启动Linux、再启动其它AMP固件。注意：想要启动的主核不是CPU0，需要特殊的trust支持。
5. Linux固件的加载地址只由U-Boot配置决定，例如：rk3568_common.h。

5.1.2 框架支持

配置：

```
CONFIG_AMP
CONFIG_ROCKCHIP_AMP
```

框架代码：

```
./drivers/cpu/rockchip_amp.c
```

its 模版：

```
./drivers/cpu/amp.its
```

打包工具：

```
./tools/mkimage   // 完整编译一次U-Boot后会自动生成
```

5.1.3 功能启用

1. 制作amp.img需要一份its文件，请基于 `drivers/cpu/amp.its` 修改：

   如下its：CPU1/2/3跑AMP，CPU0跑Linux，主核是CPU3。启动顺序是：CPU0 => CPU1/2 => CPU3。

```
/dts-v1/;
/ {
    description = "FIT source file for rockchip AMP";
    #address-cells = <1>;

    // 所有AMP固件（不含Linux）请在images节点下指定;
    images {
        amp1 {
            description  = "bare-metal-core1";         // 必选项：描述信息
            data         = /incbin/("./amp1.bin");     // 必选项：amp1固件
            type         = "firmware";  // 必选项：不做改动
            compression  = "none";      // 必选项：不做改动
            arch         = "arm";       // 必选项："arm64"：64位，"arm"：32位
            cpu          = <0x100>;     // 必选项：cpu硬件id(mpidr)
            thumb        = <0>;         // 必选项：0: arm or thumb2; 1: 纯thumb
            hyp          = <0>;         // 必选项：0: el1/svc; 1: el2/hyp
            load         = <0x01800000>;// 必选项：固件加载和运行地址
            udelay       = <1000000>;   // 可选项：启动完当前CPU后做延时后再启动下一个
CPU
            hash {                      // 必选项：不做改动
                algo = "sha256";
            };
        };

        amp2 {
            description  = "bare-metal-core2";
            data         = /incbin/("./amp2.bin");
            type         = "firmware";
            compression  = "none";
            arch         = "arm";
            cpu          = <0x200>;
```

```
            thumb        = <0>;
            hyp          = <0>;
            load         = <0x03800000>;
            udelay       = <1000000>;
            hash {
                algo = "sha256";
            };
        };

        amp3 {
            description  = "bare-metal-core3";
            data         = /incbin/("./amp3.bin");
            type         = "firmware";
            compression  = "none";
            arch         = "arm";
            cpu          = <0x300>;
            thumb        = <0>;
            hyp          = <0>;
            load         = <0x05800000>;
            udelay       = <1000000>;
            hash {
                algo = "sha256";
            };
        };
    };

    configurations {
        default = "conf";
        conf {
            description = "Rockchip AMP images";
            rollback-index = <0x0>;
            // 指定需要被加载的固件及其加载、启动顺序，但是主核不受此顺序限制。
            loadables = "amp1", "amp2", "amp3";

            signature {
                algo = "sha256,rsa2048";
                padding = "pss";
                key-name-hint = "dev";
                sign-images = "loadables";
            };

            // Linux CPU的运行状态指定：
            // （1）只有udelay属性是可选项；
            // （2）启动地址不可配，由U-Boot的平台配置文件决定，例如：rk3568_common.h；
            linux {
                description  = "linux-os";
                arch         = "arm64";
                cpu          = <0x000>;    // CPU0跑linux
                thumb        = <0>;
                hyp          = <0>;
                udelay       = <1000000>;
            };
        };
    };
};
```

说明：

- description：描述信息。
- type：默认使用"firmware"。
- compression：默认使用"none"。
- data：固件路径。该路径是基于amp.its的相对路径。
- arch：CPU 32/64模式。ARMv7只能指定为"arm"；ARMv8可指定为"arm64"或"arm"，分别表示AArch64或AArch32。
- cpu：CPU硬件ID，即mpidr(Multiprocessor Affinity Register)，取低32位即可。例如：

```
cpus {
    #address-cells = <2>;
    #size-cells = <0>;

    cpu0: cpu@0 {
        device_type = "cpu";
        compatible = "arm,cortex-a55";
        reg = <0x0 0x0>;    // mpidr
        ......
    };

    cpu1: cpu@100 {
        device_type = "cpu";
        compatible = "arm,cortex-a55";
        reg = <0x0 0x100>;  // mpidr
        ......
    };
    ......
};
```

- thumb：CPU指令模式。如果是纯thumb则指定为1，否则为0。
- hyp：CPU虚拟机模式。
- load：固件加载和运行地址。
- udelay：启动完成后的延时（可选项），单位us。启动完当前CPU后做相应延时后再启动下一个CPU。
- loadables：需要加载的AMP固件及加载、启动顺序。主CPU一定是最后被启动的，不受此处的顺序约束。
- linux节点：用于Linux+AMP组合方案。具体请参考本章节中“实现思路”的内容。

2. 固件打包：

```
// 0xe00为固件头大小，不建议改变
./tools/mkimage -f ./drivers/cpu/amp.its -E -p 0xe00 amp.img
```

> 需要完整编译一次U-Boot才会自动生成mkimage工具。

3. 分区表增加amp分区

在 parameter.txt 分区表文件中增加 "amp" 分区 ， 然后烧写amp.img。

> U-Boot是直接加载整个amp分区的内容到内存，建议amp分区大小按实际需要配置。

4. Bring up

U-Boot 框架会在合适的时机自动发起所有 AMP 的 bring up，如下是CPU3作为主核跑AMP固件、CPU0/1/2跑Linux固件的启动信息：

```
......
```

```
// 主核加载固件amp3固件
### Loading loadables from FIT Image at 7bdbcf80 ...
   Trying 'amp3' loadables subimage
     Description:  rtthread
     Type:         Firmware
     Compression:  uncompressed
     Data Start:   0x7bdbdd80
     Data Size:    311296 Bytes = 304 KiB
     Architecture: ARM
     Load Address: 0x01800000
     Hash algo:    sha256
     Hash value:
d08db937e4d7bd4125056239154bb30d44a2fcca9e70aa8dea448fabda4838d5
   Verifying Hash Integrity ... sha256+ OK
   Loading loadables from 0x7bdbdd80 to 0x01800000
### Booting FIT Image FIT: No fit blob
FIT: No FIT image
ANDROID: reboot reason: "(none)"
optee api revision: 2.0
TEEC: Warning: Could not find security partition
Not AVB images, AVB skip
ANDROID: Hash OK

// 主核加载Linux固件
Booting IMAGE kernel at 0x03880000 with fdt at 0x0a100000...

Fdt Ramdisk skip relocation
### Booting Android Image at 0x0387f800 ...
Kernel load addr 0x03880000 size 21655 KiB
### Flattened Device Tree blob at 0x0a100000
   Booting using the fdt blob at 0x0a100000
   XIP Kernel Image from 0x03880000 to 0x03880000 ... OK
  'reserved-memory' ramoops@110000: addr=110000 size=f0000
   Using Device Tree in place at 000000000a100000, end 000000000a12322a
vp1 adjust cursor plane from 0 to 1
vp0, plane_mask:0x2a, primary-id:5, curser-id:1
vp1 adjust cursor plane from 1 to 0
vp1, plane_mask:0x15, primary-id:4, curser-id:0
vp2, plane_mask:0x0, primary-id:0, curser-id:-1
Adding bank(fixed): 0x03880000 - 0x80000000 (size: 0x7c780000)

// 至此，所有的固件都被加载完毕，如下开始按方案的优先级规则去启动各个CPU。

// 主核启动CPU0跑Linux （CPU1/2后续通过Linux启动）
AMP: Brought up cpu[0] with state 0x12, entry 0x03880000 ...OK
// 主核启动自己（CPU3）跑AMP固件
AMP: Brought up cpu[300, self] with state 0x10, entry 0x01800000 ...OK

// Linux在CPU0上运行：
[    0.000000] Booting Linux on physical CPU 0x0000000000 [0x412fd050]
[    0.000000] Linux version 4.19.193 (stevenliu@stevenliu) (gcc version 6.3.1
20170404 (Linaro GCC 6.3-2017.05), GNU ld (Linaro_Binutils-2017.05)
2.27.0.20161019) #5 SMP Mon Sep 13 16:22:51 CST 2021
[    0.000000] Machine model: Rockchip RK3568 EVB1 DDR4 V10 Board
......
```

上述打印信息根据用户的its配置会有所不同，以实际为准。

5.2 Button

5.2.1 框架支持

按键驱动使用 button-uclass 框架和标准接口。

配置：

```
CONFIG_BUTTON=y
CONFIG_BUTTON_ADC=y
CONFIG_BUTTON_GPIO=y
CONFIG_BUTTON_RK8XX=y
```

框架代码：

```
drivers/button/button-uclass.c
```

驱动代码：

```
drivers/button/button-adc.c
drivers/button/button-gpio.c
drivers/button/button-rk8xx.c
```

pwrkey 仅以中断方式被识别，其余 gpio 按键以轮询方式被识别。

5.2.2 相关接口

接口：

```
bool button_is_on(u32 code)
enum button_state_t button_get_state(struct udevice *dev);
```

code 定义：

```
/include/dt-bindings/input/linux-event-codes.h
```

返回值：

```
enum button_state_t {
    BUTTON_OFF = 0,
    BUTTON_ON = 1,
    BUTTON_ON_HOLD = 2,
    BUTTON_COUNT,
};
```

KEY_PRESS_LONG_DOWN 默认时长 2000ms，目前只用于 U-Boot 充电的 pwrkey 长按事件。

```
#define KEY_LONG_DOWN_MS    2000
```

5.3 Charge

5.3.1 框架支持

U-Boot 原生代码不支持充电功能，RK自己实现了一套。

充电涉及的模块较多：Display、PMIC、电量计、充电动画、pwrkey、led、CPU低功耗休眠、Timer等。

电量计支持：

```
RK809/RK816/RK817/RK818/cw201/...
```

配置：

```
// 框架
CONFIG_DM_CHARGE_DISPLAY
CONFIG_CHARGE_ANIMATION
CONFIG_DM_FUEL_GAUGE
// 驱动
CONFIG_POWER_FG_CW201X
CONFIG_POWER_FG_RK818
CONFIG_POWER_FG_RK817
CONFIG_POWER_FG_RK816
......
```

充电框架：

```
./drivers/power/charge-display-uclass.c
```

充电动画驱动：

```
// 负责管理整个充电过程，它会获取电量、充电类型、按键事件，发起低功耗休眠等。
./drivers/power/charge_animation.c
```

电量计框架：

```
./drivers/power/fuel_gauge/fuel_gauge_uclass.c
```

电量计驱动：

```
./drivers/power/fuel_gauge/fg_rk818.c
./drivers/power/fuel_gauge/fg_rk817.c   // rk809复用
./drivers/power/fuel_gauge/fg_rk816.c
......
```

逻辑流程：

```
charge-display-uclass.c
    => charge_animation.c
        => fuel_gauge_uclass.c
            => fg_rkxx.c
```

5.3.2 打包图片

充电图片位于`./tools/images/`目录，需要打包进 resource.img 才能被充电框架显示。

内核编译得到的resource.img 默认没打包充电图片，需要在U-Boot 里另外单独打包。

```
$ ls tools/images/
battery_0.bmp  battery_1.bmp  battery_2.bmp  battery_3.bmp  battery_4.bmp
battery_5.bmp  battery_fail.bmp
```

打包命令：

```
./scripts/pack_resource.sh <input resource.img>
```

打包信息：

```
./scripts/pack_resource.sh  /home/cjh/3399/kernel/resource.img

Pack ./tools/images/ & /home/guest/3399/kernel/resource.img to resource.img ...
Unpacking old image(/home/guest/3399/kernel/resource.img):
rk-kernel.dtb logo.bmp logo_kernel.bmp
Pack to resource.img succeeded!
Packed resources:
rk-kernel.dtb battery_1.bmp battery_2.bmp battery_3.bmp battery_4.bmp
battery_5.bmp battery_fail.bmp logo.bmp logo_kernel.bmp battery_0.bmp

resource.img is packed ready
```

成功后会在 U-Boot 根目录下生成包含图片的 resource.img，通过 hd 命令确认内容：

```
hd resource.img | less

00000000  52 53 43 45 00 00 00 00  01 01 01 00 0a 00 00 00  |RSCE............|
00000010  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
......
*
00000400  45 4e 54 52 62 61 74 74  65 72 79 5f 31 2e 62 6d  |ENTRbattery_1.bm|
// 图片1
00000410  70 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |p...............|
00000420  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
00000500  00 00 00 00 4d 00 00 00  9c 18 00 00 00 00 00 00  |....M...........|
00000510  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
00000600  45 4e 54 52 62 61 74 74  65 72 79 5f 32 2e 62 6d  |ENTRbattery_2.bm|
// 图片2
00000610  70 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |p...............|
00000620  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
......
```

5.3.3 DTS 配置

DTS充电节点：

```
charge-animation {
    compatible = "rockchip,uboot-charge";
    status = "okay";

    rockchip,uboot-charge-on = <0>;              // 是否开启U-Boot充电
    rockchip,android-charge-on = <1>;            // 是否开启Android充电

    rockchip,uboot-exit-charge-level = <5>;      // U-Boot充电时，允许开机的最低电量
    rockchip,uboot-exit-charge-voltage = <3650>;// U-Boot充电时，允许开机的最低电压
    rockchip,screen-on-voltage = <3400>;         // U-Boot充电时，允许点亮屏幕的最低电
压

    rockchip,uboot-low-power-voltage = <3350>;   // U-Boot无条件强制进入充电模式的最低
电压

    rockchip,system-suspend = <1>;               // 是否灭屏时进入trust低功耗待机（要
ATF支持）
    rockchip,auto-off-screen-interval = <20>;    // 自动灭屏超时，单位秒，默认15s
    rockchip,auto-wakeup-interval = <10>;        // 休眠自动唤醒时间，单位秒。如果值为0
或没
                                                 // 有这个属性，则禁止休眠自动唤醒，一
般用于
                                                 // 压力测试使用
    rockchip,auto-wakeup-screen-invert = <1>;    // 休眠自动唤醒时是否需要亮/灭屏
};
```

5.3.4 系统休眠

Pwrkey按键：

- 短按 pwrkey 可以亮/灭屏，灭屏时系统会进入低功耗模式；
- 长按 pwrkey 可开机进入系统

低功耗模式有2种，通过 `rockchip,system-suspend = <VAL>` 选择：

- VAL为0：cpu wfi 模式。此时不处理外设，仅仅cpu 进入低功耗模式；
- VAL为1：system suspend 模式，需要ATF/OP-TEE 支持才有效。类同kernel的系统深度待机，整个SoC进入待机。

ATF/OP-TEE 支持U-Boot低功耗待机的最低版本号：请参考平台定义章节。

5.3.5 更换图片

1. 更换 `./tools/images/` 目录下的图片（采用 8bit 或 24bit bmp），使用命令 `ls | sort` 确认图片排列顺序是低电量到高电量，使用 pack_resource.sh 脚本把图片打包进 resource.img；
2. 修改 `./drivers/power/charge_animation.c` 里的图片和电量关系；

```
/*
 * IF you want to use your own charge images, please:
 *
 * 1. Update the following 'image[]' to point to your own images;
 * 2. You must set the failed image as last one and soc = -1 !!!
```

```
 */
static const struct charge_image image[] = {
    { .name = "battery_0.bmp", .soc = 5, .period = 600 },
    { .name = "battery_1.bmp", .soc = 20, .period = 600 },
    { .name = "battery_2.bmp", .soc = 40, .period = 600 },
    { .name = "battery_3.bmp", .soc = 60, .period = 600 },
    { .name = "battery_4.bmp", .soc = 80, .period = 600 },
    { .name = "battery_5.bmp", .soc = 100, .period = 600 },
    { .name = "battery_fail.bmp", .soc = -1, .period = 1000 },
};

// @name：图片的名字；
// @soc：图片对应的电量；
// @period：图片刷新时间（单位：ms）；
// 注意：最后一张图片必须是 fail 图片，且"soc=-1"不可改变 !!
```

5.3.6 充电灯

实际产品中用户对 led 的控制需求各不相同，因此充电框架仅支持 2 个灯。充电时刻 led、充满时刻 led：

- 充电时刻 led：充电时候，电量有变化的时候，才会翻转 led 显示；
- 充满时刻 led：电量 100%充满时，才会点亮 led 灯；

上述2个Led 仅是一个demo，用户请根据自己的需求修改代码。

配置选项：

```
LED_CHARGING_NAME
LED_CHARGING_FULL_NAME
```

这两个配置选项用于指定 led 的 label 属性，请参考Led章节。

5.4 Clock

5.4.1 框架支持

clock 驱动使用 clk-uclass 框架和标准接口。

配置：

```
CONFIG_CLK
```

框架代码：

```
./drivers/clk/clk-uclass.c
```

平台驱动代码：

```
./drivers/clk/rockchip/...
```

5.4.2 相关接口

```
// 申请时钟
int clk_get_by_index(struct udevice *dev, int index, struct clk *clk);
int clk_get_by_name(struct udevice *dev, const char *name, struct clk *clk);

// 使能/关闭时钟
int clk_enable(struct clk *clk);
int clk_disable(struct clk *clk);

// 配置/获取频率
ulong (*get_rate)(struct clk *clk);
ulong (*set_rate)(struct clk *clk, ulong rate);

// 配置/获取相位
int (*get_phase)(struct clk *clk);
int (*set_phase)(struct clk *clk, int degrees);
```

5.4.3 时钟初始化

一共有三类接口涉及时钟初始化，如下先列出cru节点的信息，方便后续介绍。

```
cru: clock-controller@ff2b0000 {
    compatible = "rockchip,px30-cru";
    ......
    assigned-clocks =
        <&pmucru PLL_GPLL>, <&pmucru PCLK_PMU_PRE>,
        <&pmucru SCLK_WIFI_PMU>, <&cru ARMCLK>,
        <&cru ACLK_BUS_PRE>, <&cru ACLK_PERI_PRE>,
        <&cru HCLK_BUS_PRE>, <&cru HCLK_PERI_PRE>,
        <&cru PCLK_BUS_PRE>, <&cru SCLK_GPU>;
    assigned-clock-rates =
        <1200000000>, <100000000>,
        <26000000>, <600000000>,
        <200000000>, <200000000>,
        <150000000>, <150000000>,
        <100000000>, <200000000>;
    ......
}
```

第一类，平台基础时钟默认初始化：`rkclk_init()`

各平台cru驱动probe会调用`rkclk_init()`完成 pll/cpu/bus 频率初始化，频率定义在`cru_rkxxx.h`。
例如 RK3399：

```
#define APLL_HZ         (600 * MHz)
#define GPLL_HZ         (800 * MHz)
#define CPLL_HZ         (384 * MHz)
#define NPLL_HZ         (600 * MHz)
#define PPLL_HZ         (676 * MHz)
#define PMU_PCLK_HZ     ( 48 * MHz)
#define ACLKM_CORE_HZ   (300 * MHz)
#define ATCLK_CORE_HZ   (300 * MHz)
```

```
#define PCLK_DBG_HZ     (100 * MHz)
#define PERIHP_ACLK_HZ  (150 * MHz)
#define PERIHP_HCLK_HZ  ( 75 * MHz)
#define PERIHP_PCLK_HZ  (37500 * KHz)
#define PERILP0_ACLK_HZ (300 * MHz)
#define PERILP0_HCLK_HZ (100 * MHz)
#define PERILP0_PCLK_HZ ( 50 * MHz)
#define PERILP1_HCLK_HZ (100 * MHz)
#define PERILP1_PCLK_HZ ( 50 * MHz)
......
```

第二类，平台基础时钟二次初始化：`clk_set_defaults()`

各平台cru驱动probe可能会调用`clk_set_defaults()`解析且配置cru节点内`assigned-clocks/assigned-clock-parents/assigned-clock-rates`指定的频率（即重新配置频率），但是不包括arm频率。仅当实现`set_armclk_rate()`时才会重新配置arm频率，具体参考下面的CPU提频。

除了cru之外，有需求的外设也可以在自己的probe里主动调用`clk_set_defaults()`，例如 vop、gmac。

第三类，各模块时钟初始化：`clk_set_rate()`

大部分外设模块都是调用`clk_set_rate()`配置自己的频率。

5.4.4 CPU提频

目前各平台U-Boot中的CPU的提频支持情况：请参考平台定义章节。根据实现机制的不同分为如下三类：

第一类：CPU使用APLL

cpu开机提频的实现流程：

- 步骤1：cru 节点的`assigned-clocks`里指定 arm 目标频率；
- 步骤2：cru 驱动probe时调用`clk_set_defaults()`获取（但不会配置）步骤1的arm目标频率；
- 步骤3：实现`set_armclk_rate()`，设置从步骤2获取的 arm 目标频率。部分有需求的平台已默认实现，其它平台可参考已有实现按需增加，例如：`arch\arm\mach-rockchip\px30\px30.c`；

```
int set_armclk_rate(void)
{
    struct px30_clk_priv *priv;
    struct clk clk;
    int ret;

    ret = rockchip_get_clk(&clk.dev);
    if (ret) {
        printf("Failed to get clk dev\n");
        return ret;
    }
    clk.id = ARMCLK;
    priv = dev_get_priv(clk.dev);
    ret = clk_set_rate(&clk, priv->armclk_hz);
    if (ret < 0) {
        printf("Failed to set armclk %lu\n", priv->armclk_hz);
        return ret;
    }
    priv->set_armclk_rate = true;
```

```
    return 0;
}
```

- 步骤4：参考 cpu opp-table 频率电压表，在 arm 的 regulator 节点增加`regulator-init-microvolt = <...>`指定init电压，保证目标频率和电压能匹配。

第二类：CPU使用SCMI CLK

例如RK356X，开机提频需要使用scmi接口设置CPU时钟相关参数。

cpu开机提频的实现流程：

- 步骤1：scmi 节点的`rockchip,clk-init`里指定 arm 目标频率；
- 步骤2：确认UBOOT有打开SCMI，CONFIG_CLK_SCMI宏；
- 步骤3：实现`set_armclk_rate()`，设置从scmi的dts节点中获取的arm目标频率。部分有需求的平台已默认实现，其它平台可参考已有实现按需增加，例如：`arch\arm\mach-rockchip\rk3568\rk3568.c`；

```
#ifdef CONFIG_CLK_SCMI
#include <dm.h>
/*
 * armclk: 1104M:
 *  rockchip,clk-init = <1104000000>,
 *  vdd_cpu : regulator-init-microvolt = <825000>;
 * armclk: 1416M(by default):
 *  rockchip,clk-init = <1416000000>,
 *  vdd_cpu : regulator-init-microvolt = <900000>;
 * armclk: 1608M:
 *  rockchip,clk-init = <1608000000>,
 *  vdd_cpu : regulator-init-microvolt = <975000>;
 */

int set_armclk_rate(void)
{
    struct clk clk;
    u32 *rates = NULL;
    int ret, size, num_rates;

    ret = rockchip_get_scmi_clk(&clk.dev);
    if (ret) {
        printf("Failed to get scmi clk dev\n");
        return ret;
    }

    size = dev_read_size(clk.dev, "rockchip,clk-init");
    if (size < 0)
        return 0;

    num_rates = size / sizeof(u32);
    rates = calloc(num_rates, sizeof(u32));
    if (!rates)
        return -ENOMEM;

    ret = dev_read_u32_array(clk.dev, "rockchip,clk-init",
                 rates, num_rates);
    if (ret) {
        printf("Cannot get rockchip,clk-init reg\n");
```

```
        return -EINVAL;
    }
    clk.id = 0;
    ret = clk_set_rate(&clk, rates[clk.id]);
    if (ret < 0) {
        printf("Failed to set armclk\n");
        return ret;
    }
    return 0;
}
#endif
```

- 步骤4：参考 cpu opp-table 频率电压表，在 arm 的 regulator 节点增加`regulator-init-microvolt = <...>`指定init电压，保证目标频率和电压能匹配。

> SCMI: 参见CH17-附录描述。

第三类：CPU使用SCMI CLK（电压自动提频）

跟第二类的区别是：只需要执行步骤4。cpu 频率会根据电压的提升自动提上去。

5.4.5 时钟树

U-Boot 框架没有提供时钟树管理，各平台增加了`soc_clk_dump()`用于简单打印时钟信息。例如：

```
CLK: (sync kernel. arm: enter 1200000 KHz, init 1200000 KHz, kernel 800000 KHz)
  apll 800000 KHz
  dpll 392000 KHz
  cpll 1000000 KHz
  gpll 1188000 KHz
  npll 24000 KHz
  ppll 100000 KHz
  hsclk_bus 297000 KHz
  msclk_bus 198000 KHz
  lsclk_bus 99000 KHz
  msclk_peri 198000 KHz
  lsclk_peri 99000 KHz
```

第一行打印的含义：

- `sync kernel`：cru 驱动通过`clk_set_defaults()`配置了 kernel cru 节点内指定的各总线频率（arm 频率除外）；否则显示为sync uboot；
- `enter 1200000 KHz`：前级 Loader 进入 U-Boot 时的 arm 频率；
- `init 1200000 KHz`：U-Boot 的 arm 初始化频率，即 APLL_HZ 定义的频率；
- `kernel 800000 KHz`：实现了 set_armclk_rate() 并设置了 kernel cru 节点里`assigned-clocks`指定的 arm 频率；否则显示："kernel 0N/A"；

5.5 Crypto

Crypto 模块主要用于实现硬件级别的加密和哈希等算法。

5.5.1 框架支持

框架代码：

```
./drivers/crypto/crypto_manager.c
./drivers/crypto/cipher/cipher-uclass.c
./drivers/crypto/hash/hash-uclass.c
./drivers/crypto/hmac/hmac-uclass.c
./drivers/crypto/rsa_mod_exp/mod_exp_uclass.c
```

驱动代码：

```
./drivers/crypto/rockchip/
```

5.5.2 相关接口

```
include/u-boot/hmac.h
include/u-boot/hash.h
include/u-boot/cipher.h
include/u-boot/rsa-mod-exp.h
```

接口使用请参考：`./cmd/crypto.c`

5.6 Display

5.6.1 框架支持

RK U-Boot 目前支持的显示接口包括：RGB、LVDS、EDP、MIPI 、HDMI、CVBS 和 DP 等。U-Boot 显示的 logo 图片来自 kernel 根目录下的 logo.bmp 和 logo_kernel.bmp，它们被打包在 resource.img 里。

对图片的要求:

- BI_RGB 8bpp/16bpp/24bpp/32bpp 和 BI_RLE4/BI_RLE8 格式 BMP 图片；
- RK312X/PX30/RK3308/RV1126/RV1106 等基于 VOP LITE 架构的芯片在设计上不支持 mirror 功能，如果显示的 logo 出现 X/Y 方向上的镜像问题，请将 BMP 图片用 photoshop 或 ffmpeg 等工具提前处理为预期的效果。

配置：

```
CONFIG_DM_VIDEO
CONFIG_DISPLAY
CONFIG_DRM_ROCKCHIP
CONFIG_DRM_ROCKCHIP_PANEL
CONFIG_DRM_ROCKCHIP_DW_HDMI
CONFIG_DRM_ROCKCHIP_DW_HDMI_QP
CONFIG_DRM_ROCKCHIP_INNO_HDMI
CONFIG_ROCKCHIP_INNO_HDMI_PHY
CONFIG_DRM_ROCKCHIP_INNO_MIPI_PHY
CONFIG_DRM_ROCKCHIP_INNO_VIDEO_PHY
CONFIG_DRM_ROCKCHIP_INNO_VIDEO_COMBO_PHY
CONFIG_DRM_ROCKCHIP_DW_MIPI_DSI
CONFIG_DRM_ROCKCHIP_DW_MIPI_DSI2
CONFIG_DRM_ROCKCHIP_DW_DP
```

```
CONFIG_DRM_ROCKCHIP_ANALOGIX_DP
CONFIG_DRM_ROCKCHIP_LVDS
CONFIG_DRM_ROCKCHIP_RGB
CONFIG_DRM_ROCKCHIP_RK618
CONFIG_DRM_ROCKCHIP_RK628
CONFIG_DRM_ROCKCHIP_SAMSUNG_MIPI_DCPHY
CONFIG_PHY_ROCKCHIP_SAMSUNG_HDPTX
CONFIG_DRM_ROCKCHIP_TVE
CONFIG_DRM_SII902X
```

框架代码：

```
drivers/video/drm/rockchip_display.c
drivers/video/drm/rockchip_display.h
drivers/video/drm/rockchip_crtc.c
drivers/video/drm/rockchip_crtc.h
drivers/video/drm/rockchip_connector.c
drivers/video/drm/rockchip_connector.h
drivers/video/drm/rockchip_bridge.c
drivers/video/drm/rockchip_bridge.h
drivers/video/drm/rockchip_panel.c
drivers/video/drm/rockchip_panel.h
drivers/video/drm/inno_mipi_phy.c
drivers/video/drm/inno_video_phy.c
drivers/video/drm/inno_video_combo_phy.c
drivers/video/drm/rockchip-inno-hdmi-phy.c
drivers/video/drm/samsung_mipi_dcphy.c
```

驱动文件：

```
vop:
  drivers/video/drm/rockchip_vop.c
  drivers/video/drm/rockchip_vop.h
  drivers/video/drm/rockchip_vop_reg.c
  drivers/video/drm/rockchip_vop_reg.h
  drivers/video/drm/rockchip_vop2.c

rgb:
  drivers/video/drm/rockchip_rgb.c
  drivers/video/drm/rockchip_rgb.h

lvds:
  drivers/video/drm/rockchip_lvds.c
  drivers/video/drm/rockchip_lvds.h

mipi:
  drivers/video/drm/drm_mipi_dsi.c
  drivers/video/drm/dw_mipi_dsi.c
  drivers/video/drm/dw_mipi_dsi2.c

edp:
  drivers/video/drm/analogix_dp.c
  drivers/video/drm/analogix_dp.h
  drivers/video/drm/analogix_dp_reg.c
  drivers/video/drm/analogix_dp_reg.h
```

```
hdmi:
  drivers/video/drm/dw_hdmi.c
  drivers/video/drm/dw_hdmi.h
  drivers/video/drm/rockchip_dw_hdmi.c
  drivers/video/drm/rockchip_dw_hdmi.h
  drivers/video/drm/dw_hdmi_qp.c
  drivers/video/drm/dw_hdmi_qp.h
  drivers/video/drm/rockchip_dw_hdmi_qp.c
  drivers/video/drm/rockchip_dw_hdmi_qp.h

cvbs:
  drivers/video/drm/rockchip_tve.c
  drivers/video/drm/rockchip_tve.h

dp:
  drivers/video/drm/dw-dp.c

bridge:
  drivers/video/drm/rk618.c
  drivers/video/drm/rk618.h
  drivers/video/drm/rk618_lvds.c
  drivers/video/drm/rk628/
  drivers/video/drm/sii902x.c
```

5.6.2 相关接口

```
// 显示 U-Boot logo 和 kernel logo:
void rockchip_show_logo(void);

// 显示 bmp 图片，目前主要用于充电图片显示:
void rockchip_show_bmp(const char *bmp);

// 将 U-Boot 中的一些变量通过 dtb 传给内核。
// 包括 kernel logo 的大小、地址、格式、bcsh/csc配置、crtc 输出扫描时序以及过扫描的配置等。
void rockchip_display_fixup(void *blob);
```

5.6.3 DTS 配置

```
reserved-memory {
    #address-cells = <2>;
    #size-cells = <2>;
    ranges;

    drm_logo: drm-logo@00000000 {
        compatible = "rockchip,drm-logo";
        // 预留buffer用于kernel logo的存放，具体地址和大小在U-Boot中会修改
        reg = <0x0 0x0 0x0 0x0>;
    };
};

&route-edp {
    status = "okay";                        // 使能U-Boot logo显示功能
    logo,uboot = "logo.bmp";                // 指定U-Boot logo显示的图片
```

```
    logo,kernel = "logo_kernel.bmp";        // 指定kernel logo显示的图片
    logo,mode = "center";                   // center：居中显示，fullscreen：全屏显示
    logo,rotate = <90>;                     // 旋转角度：90/180/270
    charge_logo,mode = "center";            // center：居中显示，fullscreen：全屏显示
    connect = <&vopb_out_edp>;              // 确定显示通路，vopb->edp->panel
};

&edp {
    status = "okay"; // 使能edp
};

&vopb {
    status = "okay"; // 使能vopb
};

&panel {
    "simple-panel";
    ...
    status = "okay";

    disp_timings: display-timings {
        native-mode = <&timing0>;
        timing0: timing0 {
            ...
        };
    };
};
```

5.6.4 defconfig

目前除了一些对启动速度有要求或内存较小的平台外，U-Boot 的 defconfig 已经默认支持显示，只要在 dts 中将显示相关的信息配置好即可。

RK3308/RV1103/RV1106 等平台考虑到启动速度等一些原因默认不支持显示，需要在 defconfig 中加入如下修改：

```
--- a/configs/evb-rk3308_defconfig
+++ b/configs/evb-rk3308_defconfig
@@ -4,7 +4,6 @@ CONFIG_SYS_MALLOC_F_LEN=0x2000
CONFIG_ROCKCHIP_RK3308=y
CONFIG_ROCKCHIP_SPL_RESERVE_IRAM=0x0
CONFIG_RKIMG_BOOTLOADER=y
-# CONFIG_DM_KERNEL_DTB is not set
CONFIG_TARGET_EVB_RK3308=y
CONFIG_DEFAULT_DEVICE_TREE="rk3308-evb"
CONFIG_DEBUG_UART=y
@@ -55,6 +54,11 @@ CONFIG_USB_GADGET_DOWNLOAD=y
CONFIG_G_DNL_MANUFACTURER="Rockchip"
CONFIG_G_DNL_VENDOR_NUM=0x2207
CONFIG_G_DNL_PRODUCT_NUM=0x330d
+CONFIG_DM_VIDEO=y
+CONFIG_DISPLAY=y
+CONFIG_DRM_ROCKCHIP=y
+CONFIG_DRM_ROCKCHIP_RGB=y
+CONFIG_LCD=y
```

```
CONFIG_USE_TINY_PRINTF=y
CONFIG_SPL_TINY_MEMSET=y
CONFIG_ERRNO_STR=y
```

或者使能相应的 .config 配置：

```
// rk3308
make rk3308_defconfig rk3308-display.config
// rv1103/rv1106
make rv1106_defconfig rv1106-display.config
```

关于 upstream defconfig 配置的说明：

upstream 维护了一套 Rockchip U-Boot 显示驱动，目前主要支持 RK3288 和 RK3399 两个平台：

```
./drivers/video/rockchip/
```

如果要使用这套驱动，可以打开 CONFIG_VIDEO_ROCKCHIP，同时关闭 CONFIG_DRM_ROCKCHIP。跟我们目前 SDK 使用的显示驱动对比，后者的优势有：

- 支持的平台和显示接口更全面；
- HDMI、DP 等显示接口可以根据用户的设定输出指定分辨率，过扫描效果，显示效果调节效果等；
- U-Boot logo 可以平滑过渡到 kernel logo 直到系统起来。

5.6.5 LOGO 分区

用户如果有动态更新开机 LOGO 的需求（一般在应用层发起更新），可以通过独立的 LOGO 分区实现。

操作步骤：

- 分区表中增加独立的 LOGO 分区
- 用户根据需要以某种方式动态更新 LOGO 分区中的图片。更新时，用户直接把原始图片更新到 LOGO 分区中即可，不需要任何打包。当 LOGO 分区的图片无效时，则仍旧使用 resource 文件中默认的图片。

LOGO 分区支持情况：

如果代码只包含如下提交，则 LOGO 分区仅支持1张图片，只能替换默认的 logo.bmp：

```
  1d30bcc rockchip: resource: support parse "logo" partition picture
```

LOGO 分区支持2张图片：图片1用于替换 logo.bmp，图片2用于替换 logo_kernel.bmp。两张图片紧挨着，图片之间保持512字节对齐，顺序不可更换。

假设图片分别为 logo.bmp 和 logo_kernel.bmp。logo.img 打包命令：

```
cat logo.bmp > logo.img && truncate -s %512 logo.img && cat logo_kernel.bmp >>
logo.img
```

把生成的 logo.img 烧写到 logo 分区即可，开机后看到 "LOGO: " 打印：

```
U-Boot v5-g042c01531e-210512-dirty #cjh (May 14 2021 - 11:25:03 +0800)
```

```
Model: Rockchip RK3568 Evaluation Board
PreSerial: 2, raw, 0xfe660000
DRAM:  2 GiB
Sysmem: init
Relocation Offset: 7d34f000, fdt: 7b9f8758
Using default environment

dwmmc@fe2b0000: 1, dwmmc@fe2c0000: 2, sdhci@fe310000: 0
Bootdev(atags): mmc 0
MMC0: HS200, 200Mhz
PartType: EFI
boot mode: normal
FIT: No fdt blob
Android 11.0, Build 2021.4, v2
Found DTB in boot part
// 如下打印说明logo.img分区的图片被正确识别到。
LOGO: logo.bmp
LOGO: logo_kernel.bmp
DTB: rk-kernel.dtb
HASH(c): OK
......
```

5.6.6 常见问题分析

Q1：如果希望默认就是 X/Y 方向上镜像显示，是否有什么办法？

A1：默认 Y 方向上镜像显示是不支持的。默认 X 方向镜像显示对于 VOP LITE 架构的平台是不支持的，而 RK3568、RK3588 等 VOP2 架构平台，可以通过如下配置保证从 U-Boot 开始就是 X 方向镜像显示的：

```
&vp1 {
    xmirror-enable;
};
```

Q2：可以支持 4K 分辨率的 BMP logo 图片吗？

A2：是可以支持的，但是各平台的 defconfig 是无法支持 4K logo 图片正常显示的，需要添加如下修改（以 rk3576 平台为例）：

```
diff --git a/drivers/video/drm/rockchip_display.c
b/drivers/video/drm/rockchip_display.c
index b1773ba6942..d1606db1ba5 100644
--- a/drivers/video/drm/rockchip_display.c
+++ b/drivers/video/drm/rockchip_display.c
@@ -52,7 +52,7 @@
 #define RK_BLK_SIZE 512
 #define BMP_PROCESSED_FLAG 8399
 #define BYTES_PER_PIXEL sizeof(uint32_t)
-#define MAX_IMAGE_BYTES (8 * 1024 * 1024)
+#define MAX_IMAGE_BYTES (32 * 1024 * 1024)

 DECLARE_GLOBAL_DATA_PTR;
 static LIST_HEAD(rockchip_display_list);
diff --git a/include/configs/rk3576_common.h b/include/configs/rk3576_common.h
```

```
index 16abba314c5..b4a8ec3b898 100644
--- a/include/configs/rk3576_common.h
+++ b/include/configs/rk3576_common.h
@@ -22,7 +22,7 @@
 #endif
 #define CONFIG_SPL_LOAD_FIT_ADDRESS    0x42000000

-#define CONFIG_SYS_MALLOC_LEN          (32 << 20)
+#define CONFIG_SYS_MALLOC_LEN          (32 << 21)
 #define CONFIG_SYS_CBSIZE              1024

 #ifdef CONFIG_SUPPORT_USBPLUG
```

并且需要将配置项 CONFIG_DRM_MEM_RESERVED_SIZE_MBYTES 修改为 64 MB。

上述修改的原因为：

- 出于内存占用及默认 parameter 分区表配置等方面考虑，不建议使用太大的 BMP logo 图片，所以将 MAX_IMAGE_BYTES 限制到 8 MB，超过此大小的建议使用 BI_RLE4/BI_RLE8 格式 BMP 图片。
- 常见的 4K BI_RGB 24bpp BMP 图片大小约为 24 MB，各平台默认的 malloc heap 大小通常为 32 MB，会出现 BMP decode 相关函数无法申请到内存问题。
- 为了保证 U-Boot 到 Kernel logo 显示的平滑切换，通常两张 logo 图片会是相同的分辨率和格式，这样预留给 logo 显示功能的内存区域大小（由 CONFIG_DRM_MEM_RESERVED_SIZE_MBYTES 决定，默认为 32 MB）就不够了。

5.7 Efuse/Otp

5.7.1 框架支持

efuse/otp 驱动使用 misc-uclass.c 框架和标准接口。通常情况，efuse/otp 一般会有 secure 和 non-secure 之分。U-Boot 提供 non-secure 的访问，U-Boot SPL 提供 secure otp 某些区域的访问。

non-secure 配置：

```
CONFIG_MISC
// 2选1，各平台的defconfig已默认使能对应配置。
CONFIG_ROCKCHIP_EFUSE
CONFIG_ROCKCHIP_OTP
```

secure 配置：

```
CONFIG_SPL_MISC=y
CONFIG_SPL_ROCKCHIP_SECURE_OTP=y
// 禁用maskrom升级方式的支持，仅部分平台支持。
CONFIG_SPL_OTP_DISABLE_SD=y
CONFIG_SPL_OTP_DISABLE_USB=y
CONFIG_SPL_OTP_DISABLE_UART=y
CONFIG_SPL_OTP_DISABLE_SPI2APB=y
// 撤销已写入OTP的rsa公钥hash，仅部分平台支持。
CONFIG_SPL_REVOKE_PUB_KEY=y
```

如何判断上述secure功能是否支持，以rk3572为例：

例如rk3572_common.h中包含如下定义：

```
// 有如下定义则说明支持禁用maskrom升级方式
#define OTP_DISABLE_UPGRADE_ADDR    0x194
#define OTP_DISABLE_USB_VAL     0x3
#define OTP_DISABLE_SD_VAL      0xc
#define OTP_DISABLE_UART_VAL        0x30
#define OTP_DISABLE_SPI2APB_VAL     0xc0
// 有如下定义则说明支持撤销已写入OTP的rsa公钥hash，默认仅支持撤销一次，需要支持多次撤销请联系
RK工程师处理
#define OTP_NEXT_RSA_HASH_ADDR      0x1e0
#define OTP_NEXT_RSA_HASH_SIZE      32
#define OTP_RSA_HASH_REVOKE_VAL     0x3
#define OTP_RSA_HASH_REVOKE_ADDR    0x23
#define OTP_RSA_HASH_REVOKE_SIZE    1
// 有如下定义则说明支持读UDS(android DICE功能中使用)
#define OTP_DICE_UDS_ADDR       0x240
#define OTP_DICE_UDS_SIZE       32
```

框架代码：

```
./drivers/misc/misc-uclass.c
```

驱动代码：

```
// non-secure:
./drivers/misc/rockchip-efuse.c
./drivers/misc/rockchip-otp.c
// secure，以RK3588为例：
./drivers/misc/rk3588-secure-otp.S
```

5.7.2 相关接口

```
int misc_read(struct udevice *dev, int offset, void *buf, int size)
int misc_write(struct udevice *dev, int offset, void *buf, int size)
int misc_ioctl(struct udevice *dev, unsigned long request, void *buf)
// 自带回读校验
int misc_otp_write_verify(struct udevice *dev, int offset, const uint8_t
*write_buf, int size)
```

5.7.3 DTS 配置

以 rk3572 为例：

non-secure: 默认使用内核dtb中的otp节点，如需在spl中使用，则参考如下进行定义：

```
otp: otp@2a480000 {
    bootph-all;
    compatible = "rockchip,rk3308-otp";
    reg = <0x0 0x2a480000 0x0 0x400>;
};
```

secure: 会在u-boot.dtsi中定义好，不建议修改。

```
secure-otp@2a400000 {
    bootph-all;
    compatible = "rockchip,rk3572-secure-otp";
    mask_addr = <0x2a420000>;
    reg = <0x0 0x2a400000 0x0 0x4000>;
};
```

5.7.4 调用示例

non-secure 示例:

```
char data[10] = {0};
struct udevice *dev;

/* retrieve the device */
ret = uclass_get_device_by_driver(UCLASS_MISC,
                  DM_DRIVER_GET(rockchip_otp), &dev);
if (ret) {
    printf("no OTP device found\n");
    return 0;
}

misc_read(dev, 0x10, &data, 10);
```

secure 示例:

```
char data[10] = {0};
struct udevice *dev;
int i;

dev = misc_otp_get_device(OTP_S);
if (!dev)
    return -ENODEV;

for (i = 0; i < 10; i++)
    data[i] = i;

misc_otp_write(dev, 0x10, &data, 10);
memset(data, 0, 10);
misc_otp_read(dev, 0x10, &data, 10);
// 推荐，自动进行回读校验
misc_otp_write_verify(dev, 0x10, &data, 10);
```

5.7.5 其他

U-Boot下需通过tee访问secure OTP，具体支持情况，请参考文档：《Rockchip OTP 开发指南》。

5.8 Ethernet

5.8.1 框架支持

框架代码：

```
./net/*
./drivers/net/*
./drivers/net/phy/*
```

驱动代码：

```
./drivers/net/designware.c
./drivers/net/gmac_rockchip.c
./drivers/net/dwc_eth_qos.c
./drivers/net/dwc_eth_qos_rockchip.c
```

menuconfig 配置：

- 驱动配置
  Rockchip 以太网驱动有两套驱动，如果对驱动的选择有疑问，请参考我们对应的 sdk config 配置。

```
// designware:
CONFIG_DM_ETH=y
CONFIG_ETH_DESIGNWARE=y
CONFIG_GMAC_ROCKCHIP=y
```

```
// dwc_eth_qos:
CONFIG_DM_ETH=y
CONFIG_DM_ETH_PHY=y
CONFIG_DWC_ETH_QOS=y
CONFIG_DWC_ETH_QOS_ROCKCHIP=y
```

需要启用 VENDOR Storage, U-Boot 可以将 MAC 地址保存下来，每次开机使用相同的 MAC 地址，同时会传递给内核，内核也保持和 U-Boot 使用一样的 MAC 地址。

```
-# CONFIG_ROCKCHIP_VENDOR_PARTITION is not set
```

另外，GMAC 会用到时钟 clock-names，把它从 CONFIG_OF_REMOVE_PROPS 中移除；如果 GMAC DTS 也会用到 assigned-clocks，就都需要从 CONFIG_OF_REMOVE_PROPS 中移除:

```
-CONFIG_OF_REMOVE_PROPS="clock-names interrupt-parent assigned-clocks assigned-
clock-rates assigned-clock-parents"
+CONFIG_OF_REMOVE_PROPS="interrupt-parent"
```

- cmd 配置
  需要的功能手动配置上。

```
Command line interface  ---> Network commands  --->

[*] bootp, tftpboot
[ ] tftp put
[ ] tftp download and bootm
[ ] tftp download and flash
```

```
[ ] tftpsrv
[ ] rarpboot
-*- dhcp
-*- pxe
[ ] nfs
-*- mii
-*- ping
[ ] cdp
[ ] sntp
[ ] dns
[ ] linklocal
[ ] ethsw
```

5.8.2 相关接口

- 数据结构初始化接口

```
void net_init(void);
int eth_register(struct eth_device *dev);
int phy_init(void);
```

- 设备注册接口

```
int eth_register(struct eth_device *dev);
int phy_register(struct phy_driver *drv);
```

- 网络数据读写 和 phy 读写
  U-Boot的数据收发需要主动调用，没有采用中断或轮询方式，具体实现可参照 NetLoop().

```
int eth_send(void *packet, int length);
int eth_rx(void);

int phy_read(struct phy_device *phydev, int devad, int regnum);
int phy_write(struct phy_device *phydev, int devad, int regnum, u16 val);
```

5.8.3 DTS 配置

DTS 节点与 kernel 一样，需要关注的是以下板级相关的属性配置：

- phy 接口配置(phy-mode)
- phy 复位脚与复位时间(snps,reset-gpio) (snps,reset-delays-us)
- 针对主控的时钟输出方向(clock_in_out)
- 时钟源选择与频率设定(assigned-clock-parents) (assigned-clock-rates)
- RGMII Delayline，RGMII 接口需要(tx_delay) (rx_delay)

```
&gmac {
    phy-mode = "rgmii";
    clock_in_out = "input";

    snps,reset-gpio = <&gpio3 RK_PA0 GPIO_ACTIVE_LOW>;
    snps,reset-active-low;
```

```
    /* Reset time is 20ms, 100ms for rtl8211f */
    snps,reset-delays-us = <0 20000 100000>;

    assigned-clocks = <&cru CLK_GMAC_SRC>, <&cru CLK_GMAC_TX_RX>, <&cru
CLK_GMAC_ETHERNET_OUT>;
    assigned-clock-parents = <&cru CLK_GMAC_SRC_M1>, <&cru RGMII_MODE_CLK>;
    assigned-clock-rates = <125000000>, <0>, <25000000>;

    pinctrl-names = "default";
    pinctrl-0 = <&rgmiim1_pins &clk_out_ethernetm1_pins>;

    tx_delay = <0x2a>;
    rx_delay = <0x1a>;

    phy-handle = <&phy>;
    status = "okay";
};
```

5.8.4 使用示例

常用的网络命令：

- DHCP

```
Usage:
dhcp [loadAddress] [[hostIPaddr:]bootfilename]
```

使用这条命令，就不需要设置 serverip，ipaddr，以及 gateway 了。
当 dhcp 成功从 dhcp 服务器上面拿到 ip 地址后，其就会从 hostIPaddr 地址，以 tftp 的方式获取文件。

100M 环境：

```
=> dhcp 0x20000000 192.168.0.100:kernel.img
ethernet@ffc40000 Waiting for PHY auto negotiation to complete. done
BOOTP broadcast 1
DHCP client bound to address 192.168.0.106 (2 ms)
Using ethernet@ffc40000 device
TFTP from server 192.168.0.100; our IP address is 192.168.0.106
Filename 'kernel.img'.
Load address: 0x20000000
Loading: ##################################################################
         ##################################################################
         ##################################################################
         ##################################################################
         ##################################################################
         ##################################################################
         ##################################################################
         ##################################################################
         ##################################################################
         ##################################################################
         ##################################################################
         ##################################################################
         ##################################################################
         ##################################################################
         ##################################################################
```

```
         #################################################################
         #################################################################
         #################################################################
         #################################################################
         ###################################################################
         1.5 MiB/s
done
Bytes transferred = 19054084 (122be04 hex)
```

- PING

```
=> ping 192.168.0.1
ethernet@ffc40000 Waiting for PHY auto negotiation to complete. done
Using ethernet@ffc40000 device
host 192.168.0.1 is alive
```

- TFTP

1000M 环境：

```
Usage:
tftp [loadAddress] [[hostIPaddr:]bootfilename]
```

也可以自己设置 IP 地址：

```
=> setenv ipaddr 192.168.1.101
=> setenv serverip 192.168.1.100

=> tftp kernel.img 0x20000000
ethernet@ffc40000 Waiting for PHY auto negotiation to complete. done
Using ethernet@ffc40000 device
TFTP from server 192.168.1.100; our IP address is 192.168.1.101
Filename 'kernel.img'.
Load address: 0x20000000
Loading: ###############################################################
         ###############################################################
         ###############################################################
         
```

```
         12.2 MiB/s
done
Bytes transferred = 20275220 (1356014 hex)
```

5.8.5 网络故障排查

1. 网络环境，常见的有下面几个方向

- 电脑端防火墙是否没关；
- 如果是跨网段的，确认网关是否设置；
- TFTP 服务器配置是否正确；
- 某些路由器的 TFTP 功能是否被关闭。

2. 代码问题，一般来说，主要确认以下3个地方：

- pinctrl 配置是否正确。检查相关 pin 的 iomux 和 驱动强度是否正确，也可以 dump 相关寄存器与内核比较是否一致，大部分情况下，我们是先调通了内核的网络再调 U-Boot 的。
- clock 配置是否正确。时钟配置的检查相对麻烦一些，主要检查分频比，时钟源，以及时钟方向，大部分寄存器在 CRU 里面，也存在有的芯片部分寄存器在 GRF，同样可以 dump 相关寄存器 与内核的比较是否一致。
- PHY 复位脚。主要检测复位脚配置是否正确，以及复位波形是否符合 PHY 的要求。

5.9 Gpio

5.9.1 框架支持

GPIO 驱动使用 gpio-uclass 框架和标准接口。

配置：

```
CONFIG_DM_GPIO
CONFIG_ROCKCHIP_GPIO
```

框架代码：

```
./drivers/gpio/gpio-uclass.c
```

驱动代码：

```
./drivers/gpio/rk_gpio.c
```

5.9.2 DM接口

DM 标准接口。用户必须通过 `struct gpio_desc` 才能访问到gpio，推荐使用的类型。

```
// 申请/释放GPIO
int gpio_request_by_name(struct udevice *dev, const char *list_name,
                         int index, struct gpio_desc *desc, int flags);
int gpio_request_by_name_nodev(ofnode node, const char *list_name, int index,
```

```
                                struct gpio_desc *desc, int flags);
int gpio_request_list_by_name(struct udevice *dev, const char *list_name,
                              struct gpio_desc *desc_list, int max_count, int
flags);
int gpio_request_list_by_name_nodev(ofnode node, const char *list_name,
                                    struct gpio_desc *desc_list, int max_count,
                                    int flags);
int dm_gpio_free(struct udevice *dev, struct gpio_desc *desc)

// 配置GPIO方向。@flags: GPIOD_IS_OUT（输出）和 GPIOD_IS_IN（输入）
int dm_gpio_set_dir_flags(struct gpio_desc *desc, ulong flags);

// 设置/获取GPIO电平
int dm_gpio_get_value(const struct gpio_desc *desc)
int dm_gpio_set_value(const struct gpio_desc *desc, int value)
```

注意事项：

`dm_gpio_get_value()` 的返回值表示active状态，而非电平的高或低。例如：

- gpios = <&gpio2 RK_PD0 GPIO_ACTIVE_LOW>，低电平时返回值为 1，高电平为 0。
- gpios = <&gpio2 RK_PD0 GPIO_ACTIVE_HIGH>，低电平时返回值为 0，高电平为 1。

`dm_gpio_set_value()` 的参数value也是同理，1：active，0：inactive。

5.9.3 Legacy接口

兼容接口类型。该接口类型主要是兼容旧U-Boot的API，函数内部的实现本质还是走DM框架，但对外屏蔽了 `struct gpio_desc`。功能可用，从DM代码标准化的角度考虑并不推荐。

```
int gpio_request(unsigned gpio, const char *label)
int gpio_free(unsigned gpio)
int gpio_direction_input(unsigned gpio)
int gpio_direction_output(unsigned gpio, int value)
int gpio_get_value(unsigned gpio)
int gpio_set_value(unsigned gpio, int value)
```

`@gpio` 是根据每组GPIO有32个pin，每个bank有8个pin的规律计算得到的。例如：

```
gpio0_a7 = (0 * 32) + (0 * 8) + 7 = 7
gpio1_b6 = (1 * 32) + (1 * 8) + 6 = 46
gpio3_c2 = (3 * 32) + (2 * 8) + 2 = 114
```

`@value`：函数跟上面`dm_gpio_`类型的接口一致。

5.10 Interrupt

5.10.1 框架支持

U-Boot 原生代码没有中断框架，RK 自己实现了一套用于支持 GICv2/v3，默认使能。

目前用到中断的场景：

- Pwrkey：U-Boot 充电时 CPU 会进入低功耗休眠，需要通过Pwrkey 中断唤醒 CPU；
- Timer：U-Boot 充电和测试用例中会用到 Timer 中断；
- Debug：使能 CONFIG_IRQ_TIMER_DUMP 调试功能；

配置：

```
CONFIG_IRQ
CONFIG_GICV2
CONFIG_GICV3
```

框架代码：

```
./drivers/irq/irq-gpio-switch.c
./drivers/irq/irq-gpio.c
./drivers/irq/irq-generic.c
./drivers/irq/irq-gic.c
./drivers/irq/virq.c
./include/irq-generic.h
```

5.10.2 相关接口

```c
// CPU本地中断开关
void enable_interrupts(void);
int disable_interrupts(void);

// GPIO转换成中断号
int gpio_to_irq(struct gpio_desc *gpio);
int phandle_gpio_to_irq(u32 gpio_phandle, u32 pin);
int hard_gpio_to_irq(unsigned gpio);

// 注册/释放中断回调
void irq_install_handler(int irq, interrupt_handler_t *handler, void *data);
void irq_free_handler(int irq);

// 使能/关闭中断
int irq_handler_enable(int irq);
int irq_handler_disable(int irq);

// 中断触发类型
int irq_set_irq_type(int irq, unsigned int type);
```

申请 IRQ

- 拥有独立硬件中断号的外设不需要额外转换，例如：pwm, timer等。
- GPIO 的 pin 脚没有独立的硬件中断号，需要额外转换申请。

一共有 3 种方式申请 GPIO 的 pin 脚中断号：

（1）传入 `struct gpio_desc` 结构体

```c
// 此方法可以动态解析dts配置，比较灵活、常用。
int gpio_to_irq(struct gpio_desc *gpio);
```

范例：

```
battery {
    compatible = "battery,rk817";
    ......
    dc_det_gpio = <&gpio2 7 GPIO_ACTIVE_LOW>;
    ......
};
```

```
struct gpio_desc dc_det;
int ret, irq;

ret = gpio_request_by_name_nodev(dev_ofnode(dev), "dc_det_gpio", 0,
                                 &dc_det, GPIOD_IS_IN);
// 为了示例简单，省去返回值判断
if (!ret) {
    irq = gpio_to_irq(&dc_det);
    irq_install_handler(irq, ...);
    irq_set_irq_type(irq, IRQ_TYPE_EDGE_FALLING);
    irq_handler_enable(irq);
}
```

（2）传入 gpio 的 phandle 和 pin

```
// 此方法可以动态解析dts配置，比较灵活、常用。
int phandle_gpio_to_irq(u32 gpio_phandle, u32 pin);
```

范例（rk817 的中断引脚为 GPIO0_A7）：

```
rk817: pmic@20 {
    compatible = "rockchip,rk817";
    reg = <0x20>;
    ......
    interrupt-parent = <&gpio0>;              // "&gpio0": 指向gpio0节点的phandle;
    interrupts = <7 IRQ_TYPE_LEVEL_LOW>;      // "7": pin脚;
    ......
};
```

```
u32 interrupt[2], phandle;
int irq, ret;

phandle = dev_read_u32_default(dev->parent, "interrupt-parent", -1);
if (phandle < 0) {
    printf("failed get 'interrupt-parent', ret=%d\n", phandle);
    return phandle;
}

ret = dev_read_u32_array(dev->parent, "interrupts", interrupt, 2);
if (ret) {
    printf("failed get 'interrupt', ret=%d\n", ret);
    return ret;
}

// 为了示例简单，省去返回值判断
irq = phandle_gpio_to_irq(phandle, interrupt[0]);
irq_install_handler(irq, pwrkey_irq_handler, dev);
```

```
irq_set_irq_type(irq, IRQ_TYPE_EDGE_FALLING);
irq_handler_enable(irq);
```

（3）强制指定 gpio

```
// 此方法直接强制指定 gpio，传入的 gpio 必须通过特殊的宏来声明才行，不够灵活，不建议使用。
int hard_gpio_to_irq(unsigned gpio);
```

范例（GPIO0_A0 申请中断）：

```
int gpio0_a0, irq;

// 为了示例简单，省去返回值判断
gpio0_a0 = RK_IRQ_GPIO(RK_GPIO0, RK_PA0);
irq = hard_gpio_to_irq(gpio0_a0);
irq_install_handler(irq, ...);
irq_handler_enable(irq);
```

5.11 I2C

5.11.1 框架支持

i2c 驱动使用 i2c-uclass 框架和标准接口。

配置：

```
CONFIG_DM_I2C
CONFIG_SYS_I2C_ROCKCHIP
```

框架代码：

```
./drivers/i2c/i2c-uclass.c
```

驱动代码：

```
./drivers/i2c/rk_i2c.c
./drivers/i2c/i2c-gpio.c    // gpio模拟i2c通讯，目前用不到
```

5.11.2 相关接口

```
// i2c 读写
int dm_i2c_read(struct udevice *dev, uint offset, uint8_t *buffer, int len)
int dm_i2c_write(struct udevice *dev, uint offset, const uint8_t *buffer, int
len)

// 对上面接口的封装
int dm_i2c_reg_read(struct udevice *dev, uint offset)
int dm_i2c_reg_write(struct udevice *dev, uint offset, unsigned int val);
```

5.12 Led

5.12.1 框架支持

Led 驱动使用 led-uclass.c 框架和标准接口。

配置：

```
CONFIG_LED_GPIO
```

框架代码：

```
drivers/led/led-uclass // 默认编译
```

驱动代码：

```
drivers/led/led_gpio.c // 支持 compatible = "gpio-leds"
```

5.12.2 相关接口

```
// 获取led device
int led_get_by_label(const char *label, struct udevice **devp);
// 设置/获取led状态
int led_set_state(struct udevice *dev, enum led_state_t state);
enum led_state_t led_get_state(struct udevice *dev);
// 忽略，目前未做底层驱动实现
int led_set_period(struct udevice *dev, int period_ms);
```

5.12.3 DTS 节点

U-Boot 的 led_gpio.c 功能相对简单，只解析 led 节点下的 3 个属性：

- gpios：led 控制引脚和有效状态；
- label：led 名字；
- default-state：默认状态，驱动 probe 时会被设置；

```
leds {
    compatible = "gpio-leds";
    status = "okay";

    blue-led {
        gpios = <&gpio2 RK_PA1 GPIO_ACTIVE_LOW>;
        label = "battery_full";
        default-state = "off";
    };

    green-led {
        gpios = <&gpio2 RK_PA0 GPIO_ACTIVE_LOW>;
        label = "greenled";
```

```
        default-state = "off";
    };

    ......
};
```

5.13 Mtd

MTD (Memory Technology Device) 即内存技术设备，支持并口 nand、spi nand、spi nor。

5.13.1 框架支持

```
CONFIG_MTD=y
CONFIG_CMD_MTD=y
```

5.13.2 相关接口

常用接口如下：

```
/* 获取 MTD 设备 */
struct mtd_info *get_mtd_device_nm(const char *name);
void put_mtd_device(struct mtd_info *mtd);

/* 基本读写擦除接口 */
int mtd_read(struct mtd_info *mtd, loff_t from, size_t len, size_t *retlen,
u_char *buf);
int mtd_write(struct mtd_info *mtd, loff_t to, size_t len, size_t *retlen, const
u_char *buf);
int mtd_erase(struct mtd_info *mtd, struct erase_info *instr);

/* 坏块管理接口 */
int mtd_block_isbad(struct mtd_info *mtd, loff_t ofs);
int mtd_block_markbad(struct mtd_info *mtd, loff_t ofs);

/* OOB 操作接口 */
int mtd_read_oob(struct mtd_info *mtd, loff_t from, struct mtd_oob_ops *ops);
int mtd_write_oob(struct mtd_info *mtd, loff_t to, struct mtd_oob_ops *ops);

/* 坏块跳过读写接口（NAND 专用） */
int nand_read_skip_bad(struct mtd_info *mtd, loff_t offset, size_t *length,
                       size_t *actual, loff_t lim, u_char *buffer);
int nand_write_skip_bad(struct mtd_info *mtd, loff_t offset, size_t *length,
                        size_t *actual, loff_t lim, u_char *buffer, int flags);
```

5.13.3 使用示例

spi nor 加载固件示例

以 flash offset 0x400000 byte、0x800 bytes 数据到内存地址 0x4000000 为例：

```c
#include <mtd.h>

#define MTD_SPINOR_NAME     "nor0"

static int mtd_demo(void)
{
    char *mtd_name = MTD_SPINOR_NAME;
    struct mtd_info *mtd;
    size_t retlen, off, size;
    u_char *des_buf;
    int ret;

    mtd = get_mtd_device_nm(mtd_name);
    if (IS_ERR_OR_NULL(mtd)) {
        printf("MTD device %s not found, ret %ld\n",
               mtd_name, PTR_ERR(mtd));
        return CMD_RET_FAILURE;
    }

    des_buf = (u_char *)0x4000000;

    off = 0x400000;
    size = 0x800;

    ret = mtd_read(mtd, off, size, &retlen, des_buf);
    if (ret || size != retlen) {
        pr_err("mtd read fail, ret=%d retlen=%ld size=%ld\n", ret, retlen,
size);
    }

    put_mtd_device(mtd);

    return ret;
}
```

Nand 示例

建议：

- 参考 drivers/mtd/nand/raw/nand_util.c，使用有坏块识别的读/写/擦除接口，例如：
  - `nand_read_skip_bad()` - 跳过坏块读取
  - `nand_write_skip_bad()` - 跳过坏块写入（支持 WITH_DROP_FFS、WITH_WR_VERIFY 等标志）
  - `nand_erase_opts()` - 带选项的擦除
- 对于数据量较少的一次完整写行为（通常每次上电写数据量少于 2KB），可以考虑使用 MTD_BLK 相关接口，频繁调用该接口会影响 flash 的寿命
- nand_read_skip_bad/nand_write_skip_bad 参数说明：
  - `mtd`: MTD 设备指针
  - `offset`: flash 偏移地址
  - `length`: 读/写数据长度
  - `actual`: 实际需要的长度（考虑坏块后），可为 NULL
  - `lim`: 最大允许长度限制
  - `buffer`: 数据缓冲区
  - `flags`: 写操作标志（仅 nand_write_skip_bad）

5.14 Mtd_blk

RK 自定义了一套 MTD Block 层，支持并口 nand、spi nand、spi nor，注册对应的 MTD block 设备以支持相应的 block 接口。

特点：

- 单位为 sector，即 512B
- 无论单次写请求的数据量多大都会擦除数据对应的 flash block，所以对于零碎且频繁的写行为如果调用该接口将会影响 flash 的寿命

5.14.1 框架支持

U-Boot 配置：

```
// MTD 驱动
CONFIG_MTD=y
CONFIG_CMD_MTDPARTS=y

// MTD block 设备驱动
CONFIG_CMD_MTD_BLK=y
CONFIG_MTD_BLK=y

// 其他 nand 设备驱动 config
......
```

SPL 配置：

```
CONFIG_MTD=y
CONFIG_CMD_MTDPARTS=y
CONFIG_SPL_MTD=y

// 其他 nand 设备驱动 config
......
```

框架代码：

```
drivers/mtd/mtd-uclass.c        // MTD uclass 框架
drivers/mtd/mtdcore.c           // MTD 核心层，提供
get_mtd_device_nm/mtd_read/mtd_write 等接口
drivers/mtd/mtd_uboot.c         // U-Boot MTD 设备探测和分区解析
drivers/mtd/mtd_blk.c           // MTD block 层，提供 BLK 接口转换
drivers/mtd/mtdpart.c           // MTD 分区管理
```

驱动为各个控制器驱动，把读写等接口挂接到 MTD 层。

5.14.2 相关接口

MTD block 设备通过标准 BLK 接口访问，使用前需要通过设备名或 uclass 查找：

```
/* 通过设备类型名称获取 block 设备 */
struct blk_desc *blk_get_by_str(const char *ifname, int devnum);

/* 基本读写接口 */
unsigned long blk_dread(struct blk_desc *block_dev, lbaint_t start,
                        lbaint_t blkcnt, void *buffer);
unsigned long blk_dwrite(struct blk_desc *block_dev, lbaint_t start,
                        lbaint_t blkcnt, const void *buffer);

/* MTD 设备访问接口（MTD block 层内部使用） */
int mtd_blk_map_table_init(struct blk_desc *desc, loff_t offset, size_t length);
```

MTD block 层特点：

- 将 MTD 设备的坏块映射转换为线性 LBA 地址
- 支持 NAND、SPI NAND 等设备的坏块管理
- 写操作会自动进行块擦除

SPL配置：

```
CONFIG_MTD=y
CONFIG_CMD_MTDPARTS=y
CONFIG_SPL_MTD=y

// 其他nand设备驱动config
......
```

框架代码：

```
drivers/mtd/mtd-uclass.c
drivers/mtd/mtdcore.c
drivers/mtd/mtd_uboot.c
drivers/mtd/mtd_blk.c
```

驱动为各个控制器驱动，把读写等接口挂接到 MTD 层。

5.14.3 相关接口

```
unsigned long blk_dread(struct blk_desc *block_dev, lbaint_t start,
                        lbaint_t blkcnt, void *buffer)
unsigned long blk_dwrite(struct blk_desc *block_dev, lbaint_t start,
                        lbaint_t blkcnt, const void *buffer)
```

5.15 Optee Client

U-Boot在ARM TrustZone里属于Non-Secure World，需要借助OPTEE Client才能访问安全资源。

5.15.1 框架支持

基于U-Boot 框架默认OP-TEE驱动进行适配。

配置：

```
// OP-TEE API接口使能
CONFIG_OPTEE_LIB

// OP-TEE驱动使能
CONFIG_TEE
CONFIG_OPTEE

// 当 eMMC 的 RPMB 不能用时必须开启此配置，即启用security分区！
CONFIG_OPTEE_ALWAYS_USE_SECURITY_PARTITION
```

框架和驱动：

```
// OP-TEE API源码路径
lib/optee/

// OP-TEE驱动源码路径
drivers/tee
```

5.15.2 固件说明

使用的 uboot.img 必须启用 TA 功能，否则无法跟OPTEE Client交互。

```
//32位平台
rkbin/bin/rk35/rk3506_tee_v1.26.bin //未开启TA功能
rkbin/bin/rk35/rk3506_tee_ta_v1.00.bin //开启TA功能

//64位平台，默认都有开启TA功能
rkbin/bin/rk35/rk3588_bl32_v1.18.bin
```

5.15.3 接口说明

OP-TEE API 请参考《TEE_Client_API_Specification-V1.0_c.pdf》。
下载地址为：https://globalplatform.org/specs-library/tee-client-api-specification/

基于 OP-TEE 内置TA的功能，RK封装了使用内置TA功能的接口。
接口源码见`lib/optee/optee_otp_api.c`等文件，使用时请包含头文件`include/tee/optee.h`。

下面对部分API进行说明。

5.15.3.1 适用性

如下接口在各平台上的适用性：请参考平台定义章节。

```
optee_base_write_security_data()
optee_base_read_security_data()
optee_read_vbootkey_hash()
optee_write_vbootkey_hash()
optee_read_vbootkey_enable_flag()
optee_read_attribute_hash()
```

```
optee_write_attribute_hash()
optee_write_oem_otp_key()
optee_oem_otp_key_is_written()
optee_set_oem_hr_otp_read_lock()
optee_oem_otp_key_cipher()
optee_read_oem_ns_otp()
optee_write_oem_ns_otp()
optee_write_ta_encryption_key()
optee_ta_encryption_key_is_written()
optee_write_oem_encrypt_data()
optee_oem_encrypt_data_is_written()
optee_check_security_level_flag()
optee_write_oem_huk()
```

5.15.3.2 返回值

若无特殊说明，以下API的返回值，见上述文档`《TEE_Client_API_Specification-V1.0_c.pdf》`的`Return Codes`章节。

5.15.3.3 optee_base_write_security_data

```
uint32_t optee_base_write_security_data(char *filename,
                        uint32_t filename_size,
                        uint8_t *data,
                        uint32_t data_size)
```

功能

U-Boot写入数据到安全存储。

若是emmc/ufs设备则数据存储到RPMB分区，其他设备比如NAND则存储到security分区。

若定义了CONFIG_OPTEE_ALWAYS_USE_SECURITY_PARTITION则固定存储到security分区。

参数

- [in] filename - 文件名
- [in] filename_size - 文件名长度
- [in] data - 将要写入的数据
- [in] data_size - 写入数据长度

5.15.3.4 optee_base_read_security_data

```
uint32_t optee_base_read_security_data(char *filename,
                           uint32_t filename_size,
                           uint8_t *data,
                           uint32_t data_size)
```

功能

U-Boot从安全存储读取数据。

若是emmc/ufs设备则从RPMB分区读取，其他设备比如NAND则从security分区读取。

若定义了CONFIG_OPTEE_ALWAYS_USE_SECURITY_PARTITION则固定从security分区读取。

参数

- [in] filename - 文件名
- [in] filename_size - 文件名长度
- [in] data - 将要读取的数据
- [in] data_size - 读取数据长度

5.15.3.5 optee_read_vbootkey_hash

```
uint32_t optee_read_vbootkey_hash(uint32_t *buf, uint32_t length);
```

功能

读取OTP或eFuse中的secure boot public key的hash值。
Secure boot相关说明，见`Rockchip_Developer_Guide_Secure_Boot_Application_Note_EN`文档。

参数

- [out] buf - 将要读取的hash buffer
- [in] length - 哈希长度，具体支持的哈希算法长度以secure boot文档为准，长度以word（32bits）为单位。

5.15.3.6 optee_write_vbootkey_hash

```
uint32_t optee_write_vbootkey_hash(uint32_t *buf, uint32_t length);
```

功能

写OTP或eFuse中的secure boot public key的hash值，**同时使能secure boot flag，开启secure boot**。
Secure boot相关说明，见`Rockchip_Developer_Guide_Secure_Boot_Application_Note_EN`文档。

参数

- [in] buf - 将要写入的hash buffer
- [in] length - 哈希长度，具体支持的哈希算法长度以secure boot文档为准，长度以word（32bits）为单位。

5.15.3.7 optee_read_vbootkey_enable_flag

```
uint32_t optee_read_vbootkey_enable_flag(uint8_t *flag);
```

功能

读取secure boot是否开启的标志。
Secure boot相关说明，见`Rockchip_Developer_Guide_Secure_Boot_Application_Note_EN`文档。

参数

- [in] flag - 1 Byte，1表示开启secure boot，0表示关闭。

5.15.3.8 optee_read_attribute_hash

```
uint32_t optee_read_attribute_hash(uint32_t *buf, uint32_t length)
```

功能

U-Boot读取atx_permanent_attributes.bin的哈希。该哈希用于验证atx_permanent_attributes.bin合法性。

参数

- [out] buf - 哈希数据
- [in] length - 长度

5.15.3.9 optee_write_attribute_hash

```
uint32_t optee_write_attribute_hash(uint32_t *buf, uint32_t length)
```

功能

U-Boot写入atx_permanent_attributes.bin的哈希。

参数

- [in] buf - 哈希数据
- [in] length - 长度

5.15.3.10 optee_write_oem_otp_key

```
uint32_t optee_write_oem_otp_key(enum RK_OEM_OTP_KEYID key_id,
                                 uint8_t *byte_buf,
                                 uint32_t byte_len);
```

功能

把明文密钥写到指定的OEM OTP区域。
OEM OTP的相关特性说明，见`Rockchip_Developer_Guide_OTP_CN`文档。

参数

- [in] key_id - 将要写的key_id，默认支持`RK_OEM_OTP_KEY0 - 3`共4个密钥，对于rv1126/rv1109，额外支持key_id为`RK_OEM_OTP_KEY_FW`的密钥
  `RK_OEM_OTP_KEY_FW`：Boot ROM解密loader时用的密钥，`optee_oem_otp_key_cipher`接口支持用这个密钥去做业务数据加解密或者解密kernel image
- [in] byte_buf - 明文密钥
- [in] byte_len - 明文密钥长度，对于`RK_OEM_OTP_KEY_FW`，byte_len仅支持16，对于其他密钥，byte_len支持16、24、32

5.15.3.11 optee_oem_otp_key_is_written

```
uint32_t optee_oem_otp_key_is_written(enum RK_OEM_OTP_KEYID key_id, uint8_t
*value);
```

功能

判断密钥是否已经写入指定的OEM OTP区域。
OEM OTP的相关特性说明，见`Rockchip_Developer_Guide_OTP_CN`文档。

参数

- [in] key_id - 将要写的key区域索引，默认支持`RK_OEM_OTP_KEY0 - 3`共4个密钥，对于rv1126/rv1109，额外支持key_id为`RK_OEM_OTP_KEY_FW`的密钥。
- [out] value - 判断是否已经写入秘钥，1表示已写入，0表示未写入。

返回值

当返回值为#define TEEC_SUCCESS 0x00000000时，value值才有意义。

RK3588平台还会判断key_id是否被lock，若对应key_id被lock则会返回 #define TEEC_ERROR_ACCESS_DENIED 0xFFFF0001错误。

5.15.3.12 optee_set_oem_hr_otp_read_lock

```
uint32_t optee_set_oem_hr_otp_read_lock(enum RK_OEM_OTP_KEYID key_id);
```

功能

设置指定OEM OTP区域的read lock标志，设置成功后，该区域禁止写数据，并且该区域已有的数据CPU软件不可读，可通过`optee_oem_otp_key_cipher`接口使用密钥。
OEM OTP的相关特性说明，见`Rockchip_Developer_Guide_OTP_CN`文档。

注意：当设置的key_id为`RK_OEM_OTP_KEY0`或者`RK_OEM_OTP_KEY1`或者`RK_OEM_OTP_KEY2`时，设置成功后，会影响其他OTP区域的属性，例如部分OTP区域变为不可写，详见`Rockchip_Developer_Guide_OTP_CN`文档。

参数

- [in] key_id - 将要设置的key_id，支持`RK_OEM_OTP_KEY0 - 3`

5.15.3.13 optee_oem_otp_key_cipher

```
uint32_t optee_oem_otp_key_cipher(enum RK_OEM_OTP_KEYID key_id,
                                  rk_cipher_config *config,
                                  uint32_t src_phys_addr,
                                  uint32_t dst_phys_addr,
                                  uint32_t len);
```

功能

选择OEM OTP区域的密钥，执行cipher单次计算。

参数

- [in] key_id - 将要使用的key_id，默认支持`RK_OEM_OTP_KEY0 - 3`，对于rv1126/rv1109，额外支持`RK_OEM_OTP_KEY_FW`
- [in] config - 算法、模式、密钥、iv等

算法支持AES, SM4

模式支持ECB/CBC/CTS/CTR/CFB/OFB

密钥长度支持16、24、32 Bytes，若是rv1109/rv1126平台，密钥长度仅支持16、32，当key_id为`RK_OEM_OTP_KEY_FW`时密钥长度仅支持16

- [in] src_phys_addr - 待计算数据的buffer地址，支持与dst_phys_addr相同，即支持原地加解密
- [out] dst_phys_addr - 计算结果的buffer地址，支持与src_phys_addr相同
- [in] len - 输入和输出数据buffer的Byte长度，要求与所用算法的block对齐

5.15.3.14 optee_read_oem_ns_otp

```
uint32_t optee_read_oem_ns_otp(uint32_t byte_off, uint8_t *byte_buf, uint32_t
byte_len)
```

功能

U-Boot读取安全OTP中Non-Protected OEM Zone数据。

参数

- [in] byte_off - 待读取的OTP区域的位置偏移
- [out] byte_buf - 返回的OTP数据，byte_buf内存空间必须不小于byte_len
- [in] byte_len - 待读取的长度

5.15.3.15 optee_write_oem_ns_otp

```
uint32_t optee_write_oem_ns_otp(uint32_t byte_off, uint8_t *byte_buf, uint32_t
byte_len)
```

功能

U-Boot写入数据到安全OTP中Non-Protected OEM Zone。

参数

- [in] byte_off - 待写入的OTP区域的位置偏移
- [in] byte_buf - 待写入到OTP的数据
- [in] byte_len - 长度

5.15.3.16 optee_write_ta_encryption_key

```
uint32_t optee_write_ta_encryption_key(uint32_t *buf, uint32_t length)
```

功能

U-Boot写入TA加密密钥。

参数

- [in] buf - 密钥
- [in] length - 长度

5.15.3.17 optee_ta_encryption_key_is_written

```
uint32_t optee_ta_encryption_key_is_written(uint8_t *value)
```

功能

U-Boot判断TA加密密钥是否已写入。

参数

- [out] value - 0未写入，1已写入

5.15.3.18 optee_write_oem_encrypt_data

```
uint32_t optee_write_oem_encrypt_data(uint32_t *buf, uint32_t length)
```

功能

U-Boot写入防抄板私有数据。

参数

- [in] buf - 数据
- [in] length - 长度

5.15.3.19 optee_oem_encrypt_data_is_written

```
uint32_t optee_oem_encrypt_data_is_written(uint8_t *value)
```

功能

U-Boot判断防抄板私有数据是否已写入。

参数

- [out] value - 0未写入，1已写入

5.15.3.20 optee_check_security_level_flag

```
uint32_t optee_check_security_level_flag(uint8_t flag)
```

功能

U-Boot设置TEE的强弱安全等级。

参数

- [in] flag - 安全等级

5.15.3.21 optee_write_oem_huk

```
uint32_t optee_write_oem_huk(uint32_t *buf, uint32_t length)
```

功能

U-Boot写入OEM HUK。

参数

- [in] buf - 数据
- [in] length - 长度

5.15.4 共享内存

U-Boot 与 Optee 通信时的数据需放在共享内存中。用户可通过 `TEEC_AllocateSharedMemory()` 申请共享内存，但建议不超过 1M。若超过则建议分割数据进行多次传递，使用完需调用 `TEEC_ReleaseSharedMemory()` 释放共享内存。

5.15.5 测试命令

作用：测试安全存储功能。U-Boot 命令行：

```
=> mmc testsecurestorage
```

该测试用例将循环测试安全存储读写功能，当硬件使用 emmc 时将测试 rpmb 与 security 分区两种安全存储方式；当硬件使用 nand 时只测试 security 分区安全存储。

5.15.6 常见错误打印

- 没有找到 emmc 或者 nand 设备。此时请检查 U-Boot 是否缺少配置，或者硬件是否损坏。

  ```
  "TEEC: Could not find device"
  ```

- 没有找到 security 分区。当没有RPMB可用时，需要在 parameter.txt 中定义 security 分区。

  ```
  "TEEC: Could not find security partition"
  ```

- 第一次使用 security 分区进行安全存储或 security 分区数据被非法篡改时会出现该打印。

  ```
  "TEEC: verify [%d] fail, cleanning ...."
  ```

- 安全存储的空间不足。请检查存储的数据是否过大，或者之前存储过大量的数据但没有删除。

  ```
  "TEEC: Not enough space available in secure storage !"
  ```

5.16 PCIe

5.16.1 开发须知

确认在 u-boot 启动的哪个阶段应用 PCIe，并依此做相应 dts 配置：

1. 作为启动设备的 NVME，需要尽早初始化，所有后续固件在这个 NVME 中，所以 PCIe 只能使用 u-boot 的 dts 配置
2. 不作为启动设备，如支持网卡等设备，允许较迟初始化，由于 u-boot 框架支持使用 kernel dtb，所以使用 boot.img 中的 kernel dtb 中配置
3. u-boot 阶段 PCIe RC 只注册 mem 32bits range，不适用 mem 64bits-pref 空间

5.16.2 框架支持

框架代码：

```
./drivers/pci/*
./drivers/phy/*
```

驱动代码：

```
drivers/pci/pcie_dw_rockchip.c
drivers/phy/rockchip/phy-rockchip-snps-pcie3.c
drivers/phy/rockchip/phy-rockchip-naneng-combphy.c
```

menuconfig 配置：

- 驱动配置

Rockchip PCIe驱动目前支持的平台请查看pcie_dw_rockchip.c文件中的compatible属性，如果对驱动的选择有疑问，请参考我们对应的 sdk config 配置。

```
CONFIG_DM_REGULATOR_GPIO=y
CONFIG_DM_REGULATOR_FIXED=y
CONFIG_PCI=y
CONFIG_DM_PCI=y
CONFIG_DM_PCI_COMPAT=y
CONFIG_PCI_PNP=y
CONFIG_PCIE_DW_ROCKCHIP=y
CONFIG_PHY_ROCKCHIP_SNPS_PCIE3=y
CONFIG_PHY_ROCKCHIP_NANENG_COMBOPHY=y
CONFIG_PHY=y
CONFIG_CMD_PCI=y
//添加NVMe支持
CONFIG_NVME=y
CONFIG_CMD_NVME=y
//添加PCIe转USB支持
CONFIG_USB_XHCI_PCI=y
//添加Embedded DTB 支持，增加Embedded DTB 支持后镜像大小会变大
CONFIG_EMBED_KERNEL_DTB_ALWAYS=y
CONFIG_SPL_FIT_IMAGE_KB=2560
```

5.16.3 DTS 配置

加载方案选择建议：

- flash + PCIe NVMe 双存储方案：加载 kernel dtb 之前使用 PCIe
- AMP 方案中使用 PCIe 支持：加载 Embedded dtb 之后使用 PCIe

- 通用做法：加载 kernel dtb 之后使用 PCIe

加载 kernel dtb 之前使用 PCIe

建议参考 kernel DTB 节点配置来设置 uboot dtsi 相关节点，并添加 bootph-all 属性：

- phy 供电，如果默认已开启可不加
- vcc 3v3 供电
- phy 节点
- 控制器节点

以 RK3588 PCIe3x4 为例：

```
diff --git a/arch/arm/dts/rk3588-u-boot.dtsi b/arch/arm/dts/rk3588-u-boot.dtsi
index 3fe8054aac..a8e2defbad 100644
--- a/arch/arm/dts/rk3588-u-boot.dtsi
+++ b/arch/arm/dts/rk3588-u-boot.dtsi
@@ -22,6 +22,28 @@
                compatible = "rockchip,rk3588-secure-otp";
                reg = <0x0 0xfe3a0000 0x0 0x4000>;
        };
+
+       vcc12v_dcin: vcc12v-dcin {
+               bootph-all;
+               compatible = "regulator-fixed";
+               regulator-name = "vcc12v_dcin";
+               regulator-always-on;
+               regulator-boot-on;
+               regulator-min-microvolt = <12000000>;
+               regulator-max-microvolt = <12000000>;
+       };
+
+       vcc3v3_pcie30: vcc3v3-pcie30 {
+               bootph-all;
+               compatible = "regulator-fixed";
+               regulator-name = "vcc3v3_pcie30";
+               regulator-min-microvolt = <3300000>;
+               regulator-max-microvolt = <3300000>;
+               enable-active-high;
+               gpio = <&gpio3 RK_PC3 GPIO_ACTIVE_HIGH>;
+               startup-delay-us = <5000>;
+               vin-supply = <&vcc12v_dcin>;
+       };
 };

 &firmware {
@@ -117,6 +139,19 @@
        status = "okay";
 };

+&pcie30phy {
+       bootph-all;
+       rockchip,pcie30-phymode = <PHY_MODE_PCIE_AGGREGATION>;
+       status = "okay";
+};
+
+&pcie3x4 {
+       bootph-all;
```

```
+       reset-gpios = <&gpio4 RK_PB6 GPIO_ACTIVE_HIGH>;
+       vpcie3v3-supply = <&vcc3v3_pcie30>;
+       status = "okay";
+};
+
 &uart2 {
        bootph-all;
        status = "okay";
```

以 RK3566 为例：

```
From b58a47956bbd03de0fcef572fa06cdeea974e2a9 Mon Sep 17 00:00:00 2001
From: Jon Lin <jon.lin@rock-chips.com>
Date: Thu, 9 Mar 2023 15:29:37 +0800
Subject: [PATCH] TEST: uboot: rk3566_evb2_v11: nvme

Change-Id: I87b3786a433691f3c385460fa8636291bce8ed9a
Signed-off-by: Jon Lin <jon.lin@rock-chips.com>
---
 arch/arm/dts/rk3568-u-boot.dtsi | 43 +++++++++++++++++++++++++++++++++++++++
 configs/rk3568_defconfig        | 13 ++++++++++
 2 files changed, 56 insertions(+)

diff --git a/arch/arm/dts/rk3568-u-boot.dtsi b/arch/arm/dts/rk3568-u-boot.dtsi
index a0678e35db..1ab8ea4436 100644
--- a/arch/arm/dts/rk3568-u-boot.dtsi
+++ b/arch/arm/dts/rk3568-u-boot.dtsi
@@ -26,6 +26,27 @@
        cru_rst_addr = <0xfdd20470>;
        bootph-all;
    };
+
+   dc_12v: dc-12v {
+       compatible = "regulator-fixed";
+       regulator-name = "dc_12v";
+       regulator-always-on;
+       regulator-boot-on;
+       regulator-min-microvolt = <12000000>;
+       regulator-max-microvolt = <12000000>;
+   };
+
+   vcc3v3_pcie: gpio-regulator {
+       bootph-all;
+       compatible = "regulator-fixed";
+       regulator-name = "vcc3v3_pcie";
+       regulator-min-microvolt = <3300000>;
+       regulator-max-microvolt = <3300000>;
+       enable-active-high;
+       gpio = <&gpio0 RK_PC2 GPIO_ACTIVE_HIGH>;
+       startup-delay-us = <5000>;
+       vin-supply = <&dc_12v>;
+   };
 };

 &psci {
@@ -386,6 +407,28 @@
    status = "okay";
```

```
 };

+&pipegrf {
+   bootph-all;
+   status = "okay";
+};
+
+&pipe_phy_grf2 {
+   bootph-all;
+   status = "okay";
+};
+
+&combphy2_psq {
+   bootph-all;
+   status = "okay";
+};
+
+&pcie2x1 {
+   bootph-all;
+   reset-gpios = <&gpio1 RK_PB2 GPIO_ACTIVE_HIGH>;
+   vpcie3v3-supply = <&vcc3v3_pcie>;
+   status = "okay";
+};
+
 &pinctrl {
    bootph-all;
    status = "okay";
```

加载 Embedded dtb 之后使用 PCIe

u-boot 工程支持内嵌 dtb 方案，该方案能避免来自 kernel dtb 变化所带来的影响，包括没有内核支持的产品方案，通常 Embedded dtb 来源为内核标准 dtb 文件，主要步骤如下：

- 编译 kernel 固件，生成目标 dtb 文件
- U-Boot 开启 Embedded dtb 配置

```
CONFIG_EMBED_KERNEL_DTB=y
CONFIG_EMBED_KERNEL_DTB_ALWAYS=y
CONFIG_EMBED_KERNEL_DTB_PATH="./dts/rk目标芯片-目标设备.dtb"  # 例如
CONFIG_EMBED_KERNEL_DTB_PATH="dts/rk3588-evb1.dtb"
```

- 编译生成 u-boot 镜像

建议：

- 部分 AMP 方案，既无 PCIe early init 需求，又无内核，且为了先于 AMP 加载前完成 PCIe 枚举，可参考以下补丁在启动流程中枚举 PCIe：

```
diff --git a/arch/arm/mach-rockchip/board.c b/arch/arm/mach-rockchip/board.c
index 979598ff7b..87d131e118 100644
--- a/arch/arm/mach-rockchip/board.c
+++ b/arch/arm/mach-rockchip/board.c
@@ -537,6 +537,11 @@ int board_init(void)
        io_domain_init();
 #endif
        set_armclk_rate();
+
```

```
+#ifdef CONFIG_PCI
+       pci_init();
+#endif
+
 #ifdef CONFIG_DM_DVFS
        dvfs_init(true);
 #endif
```

加载 kernel dtb 之后使用 PCIe

可考虑将相关调用置于 ”RK u-boot using kernel DTB 阶段“ 后即可直接复用 kernel DTB，相关文档请参考内核PCIe配置说明。

5.16.4 使用示例

常用的命令：

5.16.4.1 PCIe CMD

```
## Chapter-5 PCI 枚举，其中 CFG 映射的内存地址为 0x00000000f0000000
=> pci enum
pcie@fe150000: PCIe Linking... LTSSM is 0x1
pcie@fe150000: PCIe Linking... LTSSM is 0x6
pcie@fe150000: PCIe Linking... LTSSM is 0x4
pcie@fe150000: PCIe Linking... LTSSM is 0x210023
pcie@fe150000: PCIe Link up, LTSSM is 0x230011
pcie@fe150000: PCIE-0: Link up (Gen3-x2, Bus0)
pcie@fe150000: invalid flags type!
pcie@fe150000: Config space: [0x00000000f0000000 - 0x00000000f0100000, size
0x100000]

## Chapter-5 扫描所有设备
=> pci
BusDevFun  VendorId   DeviceId   Device Class       Sub-Class
_____________________________________________________________
00.00.00   0x1d87     0x3588     Bridge device           0x04
01.00.00   0x144d     0xa809     Mass storage controller 0x08

## Chapter-5 显示 bus 01 设备详细信息
=> pci 01 long
Scanning PCI devices on bus 1

Found PCI device 01.00.00:
  vendor ID =                   0x144d
  device ID =                   0xa809
  command register ID =         0x0006
  status register =             0x0010
  revision ID =                 0x00
  class code =                  0x01 (Mass storage controller)
  sub class code =              0x08
  programming interface =       0x02
  cache line =                  0x08
  latency time =                0x00
  header type =                 0x00
```

```
  BIST =                          0x00
  base address 0 =                0xf0300004
  base address 1 =                0x00000000
  base address 2 =                0x00000000
  base address 3 =                0x00000000
  base address 4 =                0x00000000
  base address 5 =                0x00000000
  cardBus CIS pointer =           0x00000000
  sub system vendor ID =          0x144d
  sub system ID =                 0xa801
  expansion ROM base address =    0x00000000
  interrupt line =                0xff
  interrupt pin =                 0x01
  min Grant =                     0x00
  max Latency =                   0x00

## Chapter-5 显示 bdf 01.00.00 设备 bar 映射地址，示例代码显示 bar0 映射内存地址
0xf0300000，size 0x4000，
=> pci bar 01.00.00
ID   Base                Size                Width  Type
-------------------------------------------------------------
 0   0x00000000f0300000  0x0000000000004000  64     MEM

## Chapter-5 读取 bdf 01.00.00 设备 CFG 空间信息
=> pci d.w 01.00.00 0
00000000: 144d a809 0006 0010 0200 0108 0008 0000
00000010: 0004 f030 0000 0000 0000 0000 0000 0000
00000020: 0000 0000 0000 0000 0000 0000 144d a801
00000030: 0000 0000 0040 0000 0000 0000 01ff 0000

## Chapter-5 AER功能，用于读取指定设备的AER寄存器信息，输出格式与Linux lspci工具一致。
=> pci aer 01.00.0
    AER Capability found at offset 0x40
    UESta: DLP-- SDES-- TLP-- FCP-- CmpltTO-- CmpltAbrt-- UnxCmplt-- RxOF--
MalfTLP-- ECRC-- UnsupReq-- ACSViol--
    UEMsk: DLP-- SDES-- TLP-- FCP-- CmpltTO-- CmpltAbrt-- UnxCmplt-- RxOF--
MalfTLP-- ECRC-- UnsupReq-- ACSViol--
    UESvrt: DLP+ SDES- TLP+ FCP- CmpltTO- CmpltAbrt- UnxCmplt- RxOF- MalfTLP-
ECRC- UnsupReq- ACSViol-
    CESta: RxErr-- BadTLP-- BadDLLP-- Rollover-- Timeout-- NonFatalErr--
    CEMsk: RxErr-- BadTLP-- BadDLLP-- Rollover-- Timeout-- NonFatalErr--
    AERCap: First Error Pointer: 00, GenCap- CGenEn- ChkCap- ChkEn-

## Chapter-5 retrain link功能，用于链路重新建立
=> pci x 01.00.0
    Retrain triggered for device 0001:0000
    Link Status for device 0001:0000: 0x1022
    Speed: Gen3
    Width: x2
    Link Up: Yes

## Chapter-5 FLR(Function Level Reset)功能，用于复位外设功能
=> pci flr 01.00.0
    FLR completed and state restored for device 01:00.0
```

5.16.4.2 NVMe

```
## Chapter-5 发起nvme 扫描
=> nvme scan

## Chapter-5 罗列nvme设备详细信息
=> nvme details
    Blk device 0: Optional Admin Command Support:
            Namespace Management/Attachment: no
            Firmware Commit/Image download: yes
            Format NVM: yes
            Security Send/Receive: no
    Blk device 0: Optional NVM Command Support:
            Reservation: yes
            Save/Select field in the Set/Get features: yes
            Write Zeroes: yes
            Dataset Management: yes
            Write Uncorrectable: yes
    Blk device 0: Format NVM Attributes:
            Support Cryptographic Erase: No
            Support erase a particular namespace: Yes
            Support format a particular namespace: Yes
    Blk device 0: LBA Format Support:
    Blk device 0: End-to-End DataProtect Capabilities:
            As last eight bytes: No
            As first eight bytes: No
            Support Type3: No
            Support Type2: No
            Support Type1: No
    Blk device 0: Metadata capabilities:
            As part of a separate buffer: No
            As part of an extended data LBA: No

## Chapter-5 看到一个256GB的NVMe, 如果看不到容量，需要拔出设备确保完全掉电，重来。
=> nvme info
    Device 0: Vendor: 0x144d Rev: EXD7201Q Prod: S444NA0M384608
                Type: Hard Disk
                Capacity: 244198.3 MB = 238.4 GB (500118192 x 512)

## Chapter-5 选择ID为0的nvme设备
=> nvme device 0

Device 0: Vendor: 0x144d Rev: EXD7201Q Prod: S444NA0M384608
            Type: Hard Disk
            Capacity: 244198.3 MB = 238.4 GB (500118192 x 512)
... is now current device

## Chapter-5 将0x40000000内存设置位0x55aa55aa
=> md.l 0x40000000 1
    40000000: d08ec033                                  3...
=> mw.l 0x40000000 0x55aa55aa
=> md.l 0x40000000 1
    40000000: 55aa55aa                                  .U.U

## Chapter-5 从0x40000000内存开始取1个block数据，写入NVME的LBA 0地址
=> nvme write 0x40000000 0x0 0x1

    nvme write: device 0 block # 0, count 1 ... 1 blocks written: OK
```

```
## Chapter-5 检查下0x44000000内存，确认原始数据
=> md.l 0x44000000 1
    44000000: ffffffff                                ....

## Chapter-5 从NVMe的LBA 0地址，读取1个block数据，写入内存0x44000000
=> nvme read 0x44000000 0x0 0x1

    nvme read: device 0 block # 0, count 1 ... 1 blocks read: OK

## Chapter-5 确认0x44000000内存数据是从NVMe读回来的
=> md.l 0x44000000 1
    44000000: 55aa55aa

## Chapter-5 从0地址开始，擦除一个block
=> nvme erase 0x0 0x1
    nvme erase: device 0 block # 0, count 1 ... 1 blocks erased: OK

## Chapter-5 从NVMe的LBA 0地址，读取1个block数据，写入内存0x44000000
=> nvme read 0x44000000 0x0 0x1
    nvme read: device 0 block # 0, count 1 ... 1 blocks read: OK

## Chapter-5 确认0x44000000内存数据是从NVMe读回来的
=> md.l 0x44000000 0x200        #NVMe第一个block的数据已经清除了
    44000000: 00000000 00000000 00000000 00000000    ................
    44000010: 00000000 00000000 00000000 00000000    ................
    44000020: 00000000 00000000 00000000 00000000    ................
    ...

## Chapter-5 使用write zero功能清空0x100地址开始的一个block数据
=> nvme write_zeroes 0x100 0x1
    nvme write_zeroes: device 0 block # 0x100, count 1 ... 1 blocks
write_zeroed: OK
```

5.16.4.3 RK3588 RC dma

```
=> pci
BusDevFun  VendorId   DeviceId   Device Class       Sub-Class
_____________________________________________________________
00.00.00   0x1d87     0x3588     Bridge device           0x04
01.00.00   0x1d87     0x356a     ???                     0x00
=> pci 1 long
Scanning PCI devices on bus 1

Found PCI device 01.00.00:
  vendor ID =                   0x1d87
  device ID =                   0x356a
  command register ID =         0x0006
  status register =             0x0010
  revision ID =                 0x01
  class code =                  0x12 (???)
  sub class code =              0x00
  programming interface =       0x00
  cache line =                  0x08
  latency time =                0x00
  header type =                 0x00
```

```
  BIST =                          0x00
  base address 0 =                0xf0400000    # BAR0 映射的 CPU 地址，由于 RK PCIe
采用 CPU-BUS 一一映射，所以 bus addr 相同值
  base address 1 =                0x00000000
  base address 2 =                0x0400000c
  base address 3 =                0x00000000
  base address 4 =                0xf0800000
  base address 5 =                0x00000000
  cardBus CIS pointer =           0x00000000
  sub system vendor ID =          0x0000
  sub system ID =                 0x0000
  expansion ROM base address =    0x00000000
  interrupt line =                0xff
  interrupt pin =                 0x01
  min Grant =                     0x00
  max Latency =                   0x00
=>

## Chapter-5 BAR CPU 访问，PIO 访问
md.l 0xf0400000 0x40

## Chapter-5 DMA read
mw.l 0x3c000000 0xffffffff
dcache flush 0x3c000000 0x100      # flush
mw.l 0xf538002c 0x1
mw.l 0xf5380300 0x4000008
mw.l 0xf5380304 0x0
mw.l 0xf5380308 0x100
mw.l 0xf538030c 0xf0400000
mw.l 0xf5380310 0x0
mw.l 0xf5380314 0x3c000000
mw.l 0xf5380318 0x0
mw.l 0xf5380030 0x0
dcache invalidate 0x3c000000 0x100  # invalidate
md.l 0x3c000000

## Chapter-5 DMA write
mw.l 0x3c000000 0xffffffff
dcache flush 0x3c000000 0x100 # flush
mw.l 0xf538000c 0x1
mw.l 0xf5380200 0x4000008
mw.l 0xf5380204 0x0
mw.l 0xf5380208 0x100
mw.l 0xf538020c 0x3c000000
mw.l 0xf5380210 0x0
mw.l 0xf5380214 0xf0400000
mw.l 0xf5380218 0x0
mw.l 0xf5380010 0x0
md.l 0x3c000000 0x40
```

说明：

- dcache flush/clean 宏开关 CONFIG_CMD_CACHE，要求添加以下支持补丁：

```
commit b46a81a12dd4a1514a6522e33a1d16194f242d62
Author: Joseph Chen <chenjh@rock-chips.com>
Date:   Wed Sep 28 01:36:45 2022 +0000

    cmd: cache: Add flush/invalidate dcache range support

    Signed-off-by: Joseph Chen <chenjh@rock-chips.com>
    Change-Id: Id0e0cd9019072e8c557ebd2987b439057cb4ae3b
```

5.16.4.4 RK3568 RC dma

```
=> pci
BusDevFun  VendorId   DeviceId   Device Class       Sub-Class
_____________________________________________________________
00.00.00   0x1d87     0x356a     Bridge device           0x04
01.00.00   0x1d87     0x356a     ???                     0x00
=> pci 1 long
Scanning PCI devices on bus 1

Found PCI device 01.00.00:
  vendor ID =                   0x1d87
  device ID =                   0x356a
  command register ID =         0x0006
  status register =             0x0010
  revision ID =                 0x01
  class code =                  0x12 (???)
  sub class code =              0x00
  programming interface =       0x00
  cache line =                  0x08
  latency time =                0x00
  header type =                 0x00
  BIST =                        0x00
  base address 0 =              0xf0400000     # BAR0 映射的 CPU 地址，由于 RK PCIe
采用 CPU-BUS 一一映射，所以 bus addr 相同值
  base address 1 =              0x00000000
  base address 2 =              0x0400000c
  base address 3 =              0x00000000
  base address 4 =              0xf0800000
  base address 5 =              0x00000000
  cardBus CIS pointer =         0x00000000
  sub system vendor ID =        0x0000
  sub system ID =               0x0000
  expansion ROM base address =  0x00000000
  interrupt line =              0xff
  interrupt pin =               0x01
  min Grant =                   0x00
  max Latency =                 0x00
=>

## Chapter-5 BAR CPU 访问，PIO 访问
md.l 0xf0400000 0x40

## Chapter-5 DMA read
mw.l 0x3c000000 0xffffffff
dcache flush 0x3c000000 0x100      # flush
```

```
mw.l 0xf638002c 0x1
mw.l 0xf6380300 0x4000008
mw.l 0xf6380304 0x0
mw.l 0xf6380308 0x100
mw.l 0xf638030c 0xf0400000
mw.l 0xf6380310 0x0
mw.l 0xf6380314 0x3c000000
mw.l 0xf6380318 0x0
mw.l 0xf6380030 0x0
dcache invalidate 0x3c000000 0x100  # invalidate
md.l 0x3c000000

## Chapter-5 DMA write
mw.l 0x3c000000 0xffffffff
dcache flush 0x3c000000 0x100 # flush
mw.l 0xf638000c 0x1
mw.l 0xf6380200 0x4000008
mw.l 0xf6380204 0x0
mw.l 0xf6380208 0x100
mw.l 0xf638020c 0x3c000000
mw.l 0xf6380210 0x0
mw.l 0xf6380214 0xf0400000
mw.l 0xf6380218 0x0
mw.l 0xf6380010 0x0
md.l 0x3c000000 0x40
```

说明：

- dcache flush/clean 宏开关 CONFIG_CMD_CACHE，要求添加以下支持补丁：

```
commit b46a81a12dd4a1514a6522e33a1d16194f242d62
Author: Joseph Chen <chenjh@rock-chips.com>
Date:   Wed Sep 28 01:36:45 2022 +0000

    cmd: cache: Add flush/invalidate dcache range support

    Signed-off-by: Joseph Chen <chenjh@rock-chips.com>
    Change-Id: Id0e0cd9019072e8c557ebd2987b439057cb4ae3b
```

5.16.5 常见问题分析

RK3568 Linux5.10 U-Boot 关闭 PCIE ASPM 节能

默认控制器已经关闭 ASPM 支持，确认 PCIe3x2 U-Boot shell 下的命令方式参考：

```
pci display.l 20.00.00 0x80 1
```

其中 BITS[1:0] PCIE_CAP_ACTIVE_STATE_LINK_PM_CONTROL：

```
Values:
0x0 (DISABLED): Disabled
0x1 (L0S_ENTRY_EN): L0s Entry Enabled
0x2 (L1_ENTRY_En): L1 Entry Enabled
0x3 (L0S_L1_ENTRY_EN): L0s and L1 Entry Enabled
```

RC DMA 访问 FPGA 限制

FPGA 及大部分外设的 BAR 访问有不同表现：

- 部分设备支持不同长度的 memory read/write TLP 包
- 部分设备仅支持 4Bytes memory read/write TLP 包

所以仅支持 4Bytes memory read/write TLP 导致：

- CPU 访问 BAR 空间正常
- RC DMA 发起超过 4B address 传输长度的请求会出现不同的 error，比如 ca abort

```
## Chapter-5 RK3568 DMA read ca abort 信息
=> md.l 0xf63800b8 1
f63800b8: 00000100                                ....
```

其他说明：

- 抓此类问题，PCIe 协议分析仪应使用 TLP trigger，不应该使用 memory trigger，否则 error 后没有 cpl 无法触发
- 怀疑是外设收到超过 4B length 的 tlp 请求后 PCIe 对总线的访问受限，导致 ca abort，但具体需求由外设原厂协助分析

5.17 Pinctrl

5.17.1 框架支持

pinctrl 驱动使用 pinctrl-uclass 框架和标准接口。

配置：

```
CONFIG_PINCTRL_GENERIC
CONFIG_PINCTRL_ROCKCHIP
```

框架代码：

```
./drivers/pinctrl/pinctrl-uclass.c
```

驱动代码：

```
./drivers/pinctrl/rockchip/pinctrl-rockchip-core.c
```

5.17.2 相关接口

```
int pinctrl_select_state(struct udevice *dev, const char *statename)     // 设置状态
int pinctrl_get_gpio_mux(struct udevice *dev, int banknum, int index)    // 获取状态
```

pinctrl 框架会在各个 driver probe 时由框架自动为其设置"default"状态，用户一般不需要调用 pinctrl 接口。

5.18 Pmic/Regulator

5.18.1 框架支持

PMIC/Regulator 驱动使用 pmic-uclass、regulator-uclass 框架和标准接口。

PMIC支持：

```
rk805/rk808/rk809/rk816/rk817/rk818/...
```

Regulator支持：

```
rk805/rk808/rk809/rk816/rk817/rk818/syr82x/tcs452x/fan53555/pwm/gpio/fixed/...
```

配置：

```
CONFIG_DM_PMIC
CONFIG_PMIC_CHILDREN
CONFIG_PMIC_RK8XX         // 适用于目前所有RK8XX系列芯片
CONFIG_DM_REGULATOR
CONFIG_REGULATOR_PWM
CONFIG_REGULATOR_RK8XX    // 适用于目前所有RK8XX系列芯片
CONFIG_DM_REGULATOR_FAN53555
```

框架代码：

```
./drivers/power/pmic/pmic-uclass.c
./drivers/power/regulator/regulator-uclass.c
```

驱动文件：

```
./drivers/power/pmic/rk8xx.c
./drivers/power/regulator/rk8xx.c
./drivers/power/regulator/fixed.c
./drivers/power/regulator/gpio-regulator.c
./drivers/power/regulator/pwm_regulator.c
./drivers/power/regulator/fan53555.c
......
```

5.18.2 相关接口

```
// 获取regulator。@platname：“regulator-name”指定的名字，如：vdd_arm、vdd_logic；
int regulator_get_by_platname(const char *platname, struct udevice **devp);

// 使能/关闭
int regulator_get_enable(struct udevice *dev);
int regulator_set_enable(struct udevice *dev, bool enable);
int regulator_set_suspend_enable(struct udevice *dev, bool enable);
int regulator_get_suspend_enable(struct udevice *dev);
```

```
// 配置/获取电压
int regulator_get_value(struct udevice *dev);
int regulator_set_value(struct udevice *dev, int uV);
int regulator_set_suspend_value(struct udevice *dev, int uV);
int regulator_get_suspend_value(struct udevice *dev);
```

5.18.3 init 电压

目前有两种方式为某路regulator设置初始化电压输出，前提是必须配置`regulator-boot-on`：

- 配置`regulator-min-microvolt`和`regulator-max-microvolt`为相同值；
- 配置`regulator-init-microvolt = <...>`

```
vdd_arm: DCDC_REG1 {
    regulator-name = "vdd_arm";
    regulator-boot-on;                          // 必须配置
    regulator-min-microvolt = <712500>;
    regulator-max-microvolt = <1450000>;
    regulator-init-microvolt = <1100000>;    // 设置初始化电压为1.1v
    ......
};
```

5.18.4 跳过初始化

如果想跳过某路regulator的初始化，可增加`regulator-loader-ignore`。

```
vdd_arm: DCDC_REG1 {
    regulator-name = "vdd_arm";
    regulator-loader-ignore;// 仅对U-Boot阶段的regulator初始化有效，kernel无效
    ......
};
```

5.19 Reset

5.19.1 框架支持

reset 驱动使用 reset-uclass.c 框架和标准接口。RK 平台上reset 的实质是进行 CRU 软复位。

配置：

```
CONFIG_DM_RESET
CONFIG_RESET_ROCKCHIP
```

框架代码：

```
./drivers/reset/reset-uclass.c
```

驱动代码：

```
./drivers/reset/reset-rockchip.c
```

5.19.2 相关接口

```
// 获取reset句柄
int reset_get_by_index(struct udevice *dev, int index, struct reset_ctl
*reset_ctl);
int reset_get_by_name(struct udevice *dev, const char *name,
                      struct reset_ctl *reset_ctl);
// 释放reset
int reset_free(struct reset_ctl *reset_ctl);
// 请求reset
int reset_request(struct reset_ctl *reset_ctl);
// 触发reset、释放reset
int reset_assert(struct reset_ctl *reset_ctl);
int reset_deassert(struct reset_ctl *reset_ctl);
```

范例：

```
struct reset_ctl reset_ctl;

ret = reset_get_by_name(dev, "mac-phy", &reset_ctl);
if (ret) {
    debug("reset_get_by_name() failed: %d\n", ret);
    return ret;
}

ret = reset_request(&reset_ctl);
if (ret)
    return ret;

ret = reset_assert(&reset_ctl);
if (ret)
    return ret;

......

ret = reset_deassert(&reset_ctl);
if (ret)
    return ret;

......

ret = reset_free(&reset_ctl);
if (ret)
    return ret;
```

5.19.3 DTS 配置

U-Boot 默认启用reset功能，用户只需在外设节点里指定要操作的 reset 对象即可：

```
// 格式：
reset-names = <name-string-list>
resets = <cru-phandle-list>
```

例如 gmac2phy：

```
gmac2phy: ethernet@ff550000 {
    compatible = "rockchip,rk3328-gmac";
    ......

    // 指定reset属性
    reset-names = "stmmaceth", "mac-phy";
    resets = <&cru SRST_GMAC2PHY_A>, <&cru SRST_MACPHY>;
};
```

5.20 Rng

5.20.1 框架支持

RNG用于实现硬件随机数功能。

框架代码：

```
./drivers/rng/rng-uclass.c
```

驱动代码：

```
./drivers/rng/rockchip_rng.c
```

配置：

```
CONFIG_DM_RNG=y
CONFIG_RNG_ROCKCHIP=y
```

5.20.2 相关接口

```
// @buffer: 保存随机数输出
// @size: 随机数长度，单位：byte
int dm_rng_read(struct udevice *dev, void *buffer, size_t size)
```

5.20.3 DTS 配置

因为RNG是Crypto硬件模块的功能之一，所以RNG节点跟Crypto节点一样有V1/2之分。RNG节点的compatible字段有两种：

```
compatible = "rockchip,cryptov1-rng";
compatible = "rockchip,cryptov2-rng";
```

完整的节点配置请参考rv1126.dtsi、rk3568.dtsi、rk3399.dtsi等文件。

5.21 Spi

5.21.1 框架支持

框架代码：

```
./drivers/spi/spi-uclass.c
```

驱动代码：

```
./drivers/spi/rk_spi.c
```

menuconfig 配置：

```
CONFIG_ROCKCHIP_SPI=y
CONFIG_CMD_SPI=y
```

5.21.2 相关接口

./include/spi.h

```
// 初始化对应 SPI 总线
struct spi_slave *spi_setup_slave(unsigned int bus, unsigned int cs, unsigned
int max_hz, unsigned int mode);

// 获取\释放总线
int spi_claim_bus(struct spi_slave *slave);
void spi_release_bus(struct spi_slave *slave);

// 常用读写接口
int spi_xfer(struct spi_slave *slave, unsigned int bitlen, const void *dout,
                void *din, unsigned long flags);
int spi_write_then_read(struct spi_slave *slave, const u8 *opcode,
                        size_t n_opcode, const u8 *txbuf, u8 *rxbuf,
                        size_t n_buf);
```

5.21.3 DTS 配置

```
&spi0 {
    bootph-all;
    status = "okay";
};
```

5.21.4 调用示例

可以参考 drivers/power/power_spi.c。

也可以参考如下 demo：

```
static u32 spi_bus_test(int bus, int cs)
{
    struct spi_slave *spi_slave;
    u32 tx_data, rx_data;
    int ret;

#ifdef CONFIG_DM_SPI
    struct udevice *dev;
    char name[30], *str;

    snprintf(name, sizeof(name), "generic_%d:%d", bus, cs);
    str = strdup(name);
    if (!str)
        return -ENOMEM;
    ret = spi_get_bus_and_cs(bus, cs, 50000000, SPI_MODE_0, "spi_generic_drv",
str, &dev, &spi_slave);
    if (ret)
        return ret;
#else
    spi_slave = spi_setup_slave(bus, cs, 50000000, SPI_MODE_0);
    if (!spi_slave) {
        /*
         * Invalid bus 1 (err=-19) means that spi1 is disabled in dts
         * Invalid chip select 1:0 (err=-19) means that there is no dev under
spi1 bus in dts
         * check it in uboot dtb or kernel dtb(if is enabled)
         *
         * btw, spi_get_bus_and_cs support no sub dev operation but
spi_setup_slave can't
         */
        return -ENODEV;
    }
#endif

    if (spi_claim_bus(spi_slave))
        return -ENODEV;

    tx_data = 0x12345678;
    ret = spi_xfer(spi_slave, 32, &tx_data, &rx_data, SPI_XFER_BEGIN |
SPI_XFER_END);

    spi_release_bus(spi_slave);

    pr_err("%s success\n", __func__);

    return ret;
}
```

说明：

- spi 外设支持通过 dts 设备子节点的 spi-max-frequency 属性指定速率，也支持在 spi_get_bus_and_cs/spi_setup_slave 时传输设定速率

5.21.5 测试命令

使用 cmd_spi 相关命令：

```
sspi 3:0.0 @1000000 24 AAA          # bus3:cs0:mode0 接口速率为1M 传输长度为24bits
传输数据为 "AAA"
```

5.21.6 常见问题分析

Q1：没有信号？

A1：请确认对应的 iomux 和 clk 是否配置合理。

Q2：cmd spi 异常？

A2：cmd 阶段默认使用kernel dtb，请确认 kernel dtsi aliases 是否有相应 spi 指定。

Q3：SPI 为什么调用 spi_setup_slave 失败？

A3：SPI bus 节点要挂设备才能正常 spi_setup_slave，但 u-boot 有为 bus 提供 driver 为 spi_generic_drv 的标准 dev，设备名为 generic_1:0，调用 spi_setup_slave 声明为该设备即可。

A4：如何确认 U-Boot 阶段 spi 频率？

A4：打开 debug 开关：

```
diff --git a/drivers/spi/rk_spi.c b/drivers/spi/rk_spi.c
index 962cc596591..3ca47f0cfd3 100644
--- a/drivers/spi/rk_spi.c
+++ b/drivers/spi/rk_spi.c
@@ -26,7 +26,8 @@
 #include "rk_spi.h"

 /* Change to 1 to output registers at the start of each transaction */
-#define DEBUG_RK_SPI    0
+#define DEBUG
+#define DEBUG_RK_SPI    1

 /*
  * ctrlr1 is 16-bits, so we should support lengths of 0xffff + 1. However,
```

以 cmd/spi.c 测试为例，关键 debug log 说明：

```
=> sspi 0:0.0 @1000000 24 AAA
rockchip_spi_of_to_plat: base=2c0e0000, max-frequency=50000000,
deactivate_delay=0, rsd=0
rockchip_spi_probe: probe
rockchip_spi_probe: rate = 198000000
spi speed 1000000, div 198
rockchip_spi_xfer: dout=00000000fddeab50, din=00000000fddeab30, len=0, flags=3
ctrl0:          0x00002401
ctrl1:          0x00000000
ssienr:                 0x00000000
ser:            0x00000000
baudr:          0x000000c6
```

```
txftlr:                 0x00000000
rxftlr:                 0x00000000
txflr:          0x00000000
rxflr:          0x00000000
sr:             0x0000004c
imr:            0x00000000
isr:            0x00000000
dmacr:          0x00000000
dmatdlr:        0x00000000
dmardlr:        0x00000000
activate cs0
deactivate cs0
```

5.22 Storage

存储驱动使用标准的存储框架，访问接口对接到 BLK 层用于支持文件系统。目前支持的存储设备：eMMC、Nand flash、SPI Nand flash、SPI Nor flash，其中 flash 相关的框架如下：

简称	主要支持的颗粒类型	主控驱动	**flash** 框架	注册设备类型	主要支持文件系统
rknand 方案	MLC TLC Nand	drivers/rkand	drivers/rkand	block 设备	FAT、EXT、SquashFS
rkflash 方案	SLC Nand、SPI Nand	drivers/rkflash	drivers/rkflash	block 设备	FAT、EXT、SquashFS
rkflash 方案（SPI Nor 支持）	SPI Nor	drivers/rkflash	drivers/rkflash	block 或 mtd 设备	SquashFS、JFFS2
SLC Nand 开源方案	SLC Nand	drivers/mtd/nand/raw	drivers/mtd/nand/raw	mtd 设备	UBI
SPI Nand 开源方案	SPI Nand	drivers/spi/rockchip_sfc.c	drivers/mtd/nand/spi	mtd 设备	UBI
SPI Nor 开源方案	SPI Nor	drivers/spi/rockchip_sfc.c	drivers/mtd/spi	mtd 或 mtd block 设备	SquashFS、JFFS2

> 说明：
>
> 1. rkflash 与 开源方案中关于 Nand flash 的支持主要区别在于：rkflash 集成 rk ftl（Flash Transfer Layer）在存储驱动中，而开源方案 ftl 部分则依赖于文件系统自身的 flash 的管理，例如 UBI 文件系统支持坏块管理、磨损均衡等适合 Nand flash 的文件系统特性。

5.22.1 框架支持

MMC & SD

MMC 为多媒体卡，比如 eMMC；SD 是一种基于半导体快闪记忆器的新一代记忆设备。在 Rockchip 平台，它们共用一个 dw_mmc 控制器（除了 rk3399、rk3399pro）。

配置：

```
CONFIG_MMC_DW=y
CONFIG_MMC_DW_ROCKCHIP=y
CONFIG_CMD_MMC=y
```

驱动文件：

```
./drivers/mmc/
```

SLC Nand & SPI Nand & SPI Nor 开源方案

由于开源社区的不断完善及 UBI 文件系统的可行性，RK 也完善 flash 结合较多开源代码的方案，且开源方案默认选用 pre loader 为 SPL 的启动方案，所以大部分配置都是结合 SPL 相关配置来完成。

配置：

```
// MTD 驱动支持
CONFIG_MTD=y
CONFIG_CMD_MTD_BLK=y
CONFIG_SPL_MTD=y
CONFIG_MTD_BLK=y

// spi nand 驱动支持
CONFIG_MTD_SPI_NAND=y
CONFIG_ROCKCHIP_SFC=y
CONFIG_SPL_SPI_FLASH_SUPPORT=y
CONFIG_SPL_SPI_SUPPORT=y

// nand 驱动支持
CONFIG_NAND=y
CONFIG_CMD_NAND=y
CONFIG_NAND_ROCKCHIP=y /* NandC v6 可根据 TRM NANDC->NANDC_NANDC_VER 寄存器确认，
0x00000801 */
//CONFIG_NAND_ROCKCHIP_V9=y  /* NandC v9 可根据 TRM NANDC->NANDC_NANDC_VER 寄存器确
认，0x56393030，例如：RK3326/PX30 为此版本 */
CONFIG_SPL_NAND_SUPPORT=y
CONFIG_SYS_NAND_U_BOOT_LOCATIONS=y
// nand page size需要按真实大小定义，如果使用容量大于等于512MB的NAND，一般需要配置为4096
#define CONFIG_SYS_NAND_PAGE_SIZE   2048

// spi nor 驱动支持
CONFIG_CMD_SF=y
CONFIG_CMD_SPI=y
CONFIG_SPI_FLASH=y
CONFIG_SF_DEFAULT_MODE=0x1
CONFIG_SF_DEFAULT_SPEED=50000000
CONFIG_SPI_FLASH_GIGADEVICE=y
CONFIG_SPI_FLASH_MACRONIX=y
CONFIG_SPI_FLASH_WINBOND=y
CONFIG_SPI_FLASH_MTD=y
CONFIG_ROCKCHIP_SFC=y
```

```
CONFIG_SPL_SPI_SUPPORT=y
CONFIG_SPL_MTD=y
CONFIG_SPL_SPI_FLASH_SUPPORT=y
```

去除 rkflash/rknand宏配置：

驱动文件：

```
./drivers/mtd/nand/raw          //SLC Nand 主控驱动及协议层
./drivers/mtd/nand/spi          //SPI Nand 协议层
./drivers/spi/rockchip_sfc.c    //SPI Flash 主控驱动
./drivers/mtd/spi               //SPI Nor 协议层
./drivers/mtd/mtd_blk.c         //MTD block 层
```

5.22.2 相关接口

存储驱动的访问接口都对挂到BLK层，所以无论何种存储都通过如下接口访问：

```
// 获取存储句柄
struct blk_desc *plat_bootdev(void)

// 访问接口
unsigned long blk_dread(struct blk_desc *block_dev, lbaint_t start,
                        lbaint_t blkcnt, void *buffer)
unsigned long blk_dwrite(struct blk_desc *block_dev, lbaint_t start,
                         lbaint_t blkcnt, const void *buffer)
unsigned long blk_derase(struct blk_desc *block_dev, lbaint_t start,
                         lbaint_t blkcnt)
```

5.22.3 类型区分

U-Boot 当前启动存储类型有两种方式进行区分：

- 通过环境变量 `devtype` 和 `devnum` 对应的字符串。
- 通过当前 `struct blk_desc` 结构体（句柄）内的 `if_type` 和 `devnum` 成员变量。

devtype	if_type	devnum	存储类型	说明
mmc	IF_TYPE_MMC	0	eMMC	-
mmc	IF_TYPE_MMC	1	SD card	-
mtd	IF_TYPE_MTD	0	Nand	mtd 开源方案
mtd	IF_TYPE_MTD	1	SPI Nand	mtd 开源方案
mtd	IF_TYPE_MTD	2	SPI Nor	mtd 开源方案
rknand	IF_TYPE_RKNAND	0	Nand	rkflash 方案
spinand	IF_TYPE_SPINAND	0	SPI Nand	rkflash 方案
spinor	IF_TYPE_SPINOR	1	SPI Nor	rkflash 方案
nvme	IF_TYPE_NVME	0	SSD	-

devtype	if_type	devnum	存储类型	说明
scsi	IF_TYPE_SCSI	0	SATA	-

5.22.4 DTS 配置

eMMC配置：

```
// rkxxxx.dtsi配置
emmc: dwmmc@ff390000 {
    compatible = "rockchip,px30-dw-mshc", "rockchip,rk3288-dw-mshc";
    reg = <0x0 0xff390000 0x0 0x4000>;      // 控制器寄存器base address及长度
    max-frequency = <150000000>;            // eMMC普通模式时钟为50MHz,当配置为eMMC
                                            // HS200模式，该max-frequency生效
    clocks = <&cru HCLK_EMMC>, <&cru SCLK_EMMC>,
         <&cru SCLK_EMMC_DRV>, <&cru SCLK_EMMC_SAMPLE>;   // 控制器对应时钟编号
    clock-names = "biu", "ciu", "ciu-drv", "ciu-sample";  // 控制器时钟名
    fifo-depth = <0x100>;                                 // fifo深度，默认配置
    interrupts = <GIC_SPI 53 IRQ_TYPE_LEVEL_HIGH>;        // 中断配置
    status = "disabled";
};

// rkxxxx-u-boot.dtsi
&emmc {
    bootph-all;
    status = "okay";
}

// rkxxxx.dts
&emmc {
    bus-width = <8>;                    // 设备总线位宽
    cap-mmc-highspeed;                  // 标识此卡槽支持highspeed mmc
    mmc-hs200-1_8v;                     // 支持HS200
    supports-emmc;                      // 标识此插槽为eMMC功能，必须添加，否则无法初始
化外设
    disable-wp;                         // 对于无物理WP管脚，需要配置
    non-removable;                      // 此项表示该插槽为不可移动设备。此项为必须添加项
    num-slots = <1>;                    // 标识为第几插槽
    status = "okay";
};
```

Nandc 配置：

```
&nandc0 {
    bootph-all;
    status = "okay";
    #address-cells = <1>;
    #size-cells = <0>;

    nand@0 {
        bootph-all;
        reg = <0>;
        nand-ecc-mode = "hw_syndrome";
        nand-ecc-strength = <16>;
        nand-ecc-step-size = <1024>;
```

```
    };
}
```

SFC 配置：

```
&sfc {
    bootph-all;
    status = "okay";
    spi_nand: flash@0 {
        bootph-all;
        compatible = "spi-nand";
        reg = <0>;
        spi-tx-bus-width = <1>;
        spi-rx-bus-width = <4>;
        spi-max-frequency = <96000000>;
    };
    spi_nor: flash@1 {
        bootph-all;
        compatible = "jedec,spi-nor";
        reg = <0>;
        spi-tx-bus-width = <1>;
        spi-rx-bus-width = <4>;
        spi-max-frequency = <96000000>;
    };
};
```

> 注意：
>
> 1. 考虑到软件的兼容性，u-boot 下仅支持 spi-tx-bus-width = <1> 的一线 SPI flash 传输;

5.22.5 双存储扩展

详细参考 Rockchip_Developer_Guide_Dual_Storage_CN.pdf 文档。

5.22.6 常见问题分析

Q1：如何调整并确认开源方案 FSPI/SFC 控制器输出的时钟频率？

A1：设定 rkxxxx-u-boot.dtsi 中 sfc 节点下的设备子节点的 spi-max-frequency 属性值，并关闭无效的子设备，然后开启驱动内的 debug 信息：

```
diff --git a/drivers/spi/rockchip_sfc.c b/drivers/spi/rockchip_sfc.c
index 939b48e377c..62a425a29f4 100644
--- a/drivers/spi/rockchip_sfc.c
+++ b/drivers/spi/rockchip_sfc.c
@@ -790,7 +790,7 @@ static int rockchip_sfc_set_speed(struct udevice *bus, uint
speed)
        sfc->cur_speed = speed;
        sfc->cur_real_speed = clk_get_rate(&sfc->clk);

-       dev_dbg(sfc->dev, "set_freq=%dHz real_freq=%dHz\n",
+       dev_err(sfc->dev, "set_freq=%dHz real_freq=%dHz\n",
                sfc->cur_speed, sfc->cur_real_speed);
 #else
        dev_dbg(sfc->dev, "sfc failed, CLK not support\n");
```

5.23 Thermal

5.23.1 框架支持

Thermal 模块用于获取tsadc采集到的芯片温度，默认获取的是CPU温度。

框架代码：

```
./drivers/thermal/thermal-uclass.c
```

驱动代码：

```
./drivers/thermal/rockchip_thermal.c
```

配置：

```
CONFIG_DM_THERMAL=y
CONFIG_ROCKCHIP_THERMAL=y
```

5.23.2 相关接口

```
// @temp: 保存获取到的温度
int thermal_get_temp(struct udevice *dev, int *temp)
```

5.23.3 DTS配置

内核的dts一般默认都有完整配置和使能。

5.24 Uart

serial 使用 serial-uclass.c 框架和标准接口，目前主要是UART debug在使用。

配置：

```
// 使能配置
CONFIG_DEBUG_UART
CONFIG_SYS_NS16550

// 参数配置
CONFIG_DEBUG_UART_BASE
CONFIG_DEBUG_UART_CLOCK
CONFIG_BAUDRATE
```

框架代码：

```
./drivers/serial/serial-uclass.c
```

驱动代码：

```
./drivers/serial/ns16550.c
```

5.24.1 单独更换

单独更换 U-Boot 阶段的UART debug 流程如下（uart2 为例）：

- `CONFIG_ROCKCHIP_PRELOADER_SERIAL` 禁用；
- `board_debug_uart_init()` 里配置 uart iomux（注意：某些平台有m0、m1...模式要配置 ）；
- `board_debug_uart_init()` 里配置 uart clock ， 保证时钟源是 24MHz；
- defconfig 更新 `CONFIG_BAUDRATE`
- defconfig 更新 `CONFIG_DEBUG_UART_BASE`

5.24.2 全局替换

Pre-loader serial 是实现前级固件共享UART debug 配置的机制，这些固件包括：ddr、miniloader、bl31、op-tee、U-Boot。原理：由最早阶段的 ddr bin 配置好UART debug 并且通过 ATAGS 传参机制逐级传递下去，各级固件获取 UART debug 配置进行使用（不包括 kernel）。

用户可以通过修改 ddr bin里的串口配置实现UART debug的全局替换，步骤：

DDR bin 配置

rkbin 仓库里提供了工具给用户配置不同的参数，包括串口更换：

```
tools/ddrbin_tool
tools/ddrbin_param.txt
tools/ddrbin_tool_user_guide.txt
```

U-Boot 配置

1 使能配置：CONFIG_ROCKCHIP_PRELOADER_SERIAL // 已经默认使能

2 rkxx-u-boot.dtsi 中把使用到的 uart 节点加上属性“bootph-all;”；

3 aliases 建立 serial 别名，因为U-Boot是通过aliaes找到目标节点并初始化它的。

例如：./arch/arm/dts/rk1808-u-boot.dtsi 里为了方便，为所有 uart 都建立别名；

```
aliases {
    mmc0 = &emmc;
    mmc1 = &sdmmc;

// 必须创建别名
    serial0 = &uart0;
    serial1 = &uart1;
    serial2 = &uart2;
    serial3 = &uart3;
    serial4 = &uart4;
    serial5 = &uart5;
    serial6 = &uart6;
    serial7 = &uart7;
};

// 必须增加bootph-all属性
&uart0 {
    bootph-all;
};
&uart1 {
    bootph-all;
};
&uart2 {
    bootph-all;
    clock-frequency = <24000000>;
    status = "okay";
};
&uart3 {
    bootph-all;
};
&uart4 {
    bootph-all;
};
```

5.24.3 关闭串口

```
CONFIG_DISABLE_CONSOLE=y
```

5.24.4 相关接口

```
// UART debug接口
void putc(const char c);
void puts(const char *s);
int printf(const char *fmt, ...);
void flushc(void);

// 跟外设通信功能的普通UART接口
int serial_dev_getc(struct udevice *dev);
int serial_dev_tstc(struct udevice *dev);
void serial_dev_putc(struct udevice *dev, char ch);
void serial_dev_puts(struct udevice *dev, const char *str);
void serial_dev_setbrg(struct udevice *dev, int baudrate);
void serial_dev_clear(struct udevice *dev);
```

5.25 USB

U-Boot USB 主要包括 Device、Host、PHY 和 USB 外设驱动几个部分。本章节将详细介绍各部分的框架配置、板级配置以及相关命令的使用方法。

5.25.1 框架支持

Device

Device基于Gadget框架（不使用DM_USB），一般配置为rockusb或fastboot模式，用于固件升级、固件校验等。rockusb协议基于UMS协议，其状态机是以HOOK的方式嵌入到UMS框架中，具体可参考rockusb驱动的实现。

目前，USB Device控制器有DWC2和DWC3两种，通常一款芯片只会集成其中一种OTG控制器，所以CONFIG中只需使能一种控制器相关配置即可，具体是哪款控制器需要参阅芯片TRM或Rockchip USB开发指南。

配置：

```
CONFIG_USB=y

// gadget配置
CONFIG_USB_GADGET=y
CONFIG_USB_GADGET_MANUFACTURER="Rockchip"
CONFIG_USB_GADGET_VENDOR_NUM=0x2207
CONFIG_USB_GADGET_PRODUCT_NUM=0x330a // 根据芯片ID配置
CONFIG_USB_GADGET_VBUS_DRAW=2
CONFIG_USB_GADGET_DUALSPEED=y

// rockusb配置
#define CONFIG_USB_FUNCTION_MASS_STORAGE // 位于include/configs/rkxxx_common.h中
CONFIG_USB_GADGET_DOWNLOAD=y
CONFIG_CMD_ROCKUSB=y

// DWC3控制器配置
CONFIG_USB_DWC3=y
CONFIG_USB_DWC3_GADGET=y

// DWC2控制器配置
```

```
CONFIG_USB_GADGET_DWC2_OTG=y
```

框架代码：

```
// gadget框架
drivers/usb/gadget/g_dnl.c
drivers/usb/gadget/config.c
drivers/usb/gadget/epautoconf.c
drivers/usb/gadget/usbstring.c
drivers/usb/gadget/f_mass_storage.c
```

驱动代码：

```
// rockusb
cmd/rockusb.c
drivers/usb/gadget/f_rockusb.c

// controller
drivers/usb/gadget/dwc2_udc_otg* // dwc2 OTG控制器
drivers/usb/dwc3 // dwc3 OTG控制器
```

Host

Host控制器包括OHCI，EHCI和xHCI，其中xHCI可支持USB3设备，但并不是所有的芯片都有集成xHCI控制器，具体集成情况需要参阅芯片TRM或Rockchip USB开发指南。Host控制器驱动全部使用UPSTREAM代码并开启DM_USB配置，因此Host开发主要是控制器对应DTS的适配工作。

配置：

```
CONFIG_USB=y
CONFIG_DM_USB=y

// xHCI
CONFIG_USB_HOST=y
CONFIG_USB_XHCI_HCD=y
CONFIG_USB_XHCI_DWC3=y
CONFIG_USB_DWC3_GENERIC=y

// EHCI
CONFIG_USB_EHCI_HCD=y
CONFIG_USB_EHCI_GENERIC=y

// OHCI
#define CONFIG_USB_OHCI_NEW // 位于include/configs/rkxxx_common.h中
#define CONFIG_SYS_USB_OHCI_MAX_ROOT_PORTS      1
CONFIG_USB_OHCI_HCD=y
CONFIG_USB_OHCI_GENERIC=y
```

框架代码：

```
// 框架代码
cmd/usb.c
drivers/usb/host/usb-uclass.c

// EHCI
```

```
drivers/usb/host/ehci-generic.c
drivers/usb/host/ehci-hcd.c

// OHCI
drivers/usb/host/ohci-generic.c
drivers/usb/host/ohci-hcd.c

// xHCI
drivers/usb/host/xhci.c
drivers/usb/host/xhci-dwc3.c
drivers/usb/host/xhci-mem.c
drivers/usb/host/xhci-ring.c
```

PHY

U-Boot USB PHY主要包括USB2和USB3 PHY驱动，使用DM_USB配置，兼容Linux内核DTB。具体PHY IP的集成情况需要参阅芯片TRM或Rockchip USB PHY开发指南。

配置：

```
CONFIG_PHY=y
CONFIG_PHY_ROCKCHIP_INNO_USB2=y
CONFIG_PHY_ROCKCHIP_USBDP=y
CONFIG_PHY_ROCKCHIP_NANENG_COMBOPHY=y
CONFIG_PHY_ROCKCHIP_TYPEC=y
```

框架代码：

```
drivers/phy/phy-uclass.c
```

驱动代码：

```
drivers/phy/rockchip/phy-rockchip-inno-usb2.c
drivers/phy/rockchip/phy-rockchip-usbdp.c
drivers/phy/rockchip/phy-rockchip-naneng-combphy.c
drivers/phy/rockchip/phy-rockchip-typec.c
```

其它外设

U-Boot USB 外设支持主要为USB HUB、USB键盘和UMS设备。

配置：

```
// USB键盘
CONFIG_USB_KEYBOARD=y
CONFIG_USB_KEYBOARD_FN_KEYS=y // 支持F1-F12, INS, HOME等快捷键。

// USB存储设备
CONFIG_USB_STORAGE=y
```

框架代码：

```
// 框架代码
common/usb.c
drivers/usb/host/usb-uclass.c

// USB键盘
drivers/input/usb_kbd.c
drivers/input/keyboard-uclass.c

// USB存储设备
common/usb_storage.c
```

5.25.2 板级配置

Device

因为USB Device不使用DM_USB方式，所以需要在Board文件中配置对应控制器的Properties，比如USB控制器的地址，TX FIFO的大小等。

```
// DWC3控制器配置
// board/rockchip/evb_rk3588/evb_rk3588.c
#ifdef CONFIG_USB_DWC3
#define CRU_BASE            0xfd7c0000
#define CRU_SOFTRST_CON42   0x0aa8
#define U3PHY_BASE          0xfed80000

static struct dwc3_device dwc3_device_data = {
    .maximum_speed = USB_SPEED_SUPER,
    .base = 0xfc000000,
    .dr_mode = USB_DR_MODE_PERIPHERAL,
    .hsphy_mode = USBPHY_INTERFACE_MODE_UTMIW,
    .index = 0,
    .dis_u2_susphy_quirk = 1,
    .dis_u1u2_quirk = 1,
};

.....
#endif

// DWC2控制器配置已在rockchip通用board.c文件实现，一般不需要修改。
// arch/arm/mach-rockchip/board.c
```

USB键盘

如果使用USB键盘做为U-Boot 标准输入设备，需要在stdin环境变量中添加usbkbd，参考代码如下。

```
// 环境变量配置位于各板级头文件中
// include/configs/evb_rk3568.h

#define ROCKCHIP_DEVICE_SETTINGS \
        "stdin=serial,usbkbd\0" \
```

5.25.3 DTS 配置

如前文所述，USB Device不使用DM_USB方式，所以不需要相关DT节点的配置；USB Host和USB PHY驱动兼容Linux内核DTB，因此，可以直接使用Linux内核DTB，如果要配置使用U-Boot DTB可参考Linux内核相关节点的实现。

5.25.4 相关命令

rockusb

```
rockusb - Use the rockusb Protocol

Usage:
rockusb <USB_controller> <devtype> <dev[:part]>  e.g. rockusb 0 mmc 0
```

有如下三种方法进入U-Boot loader升级模式：

- 通过reset + revovery按键方式进入；
- 进入U-Boot命令行，执行上述命令开启rockusb，进入升级模式；
- 进入系统后，命令行执行 “reboot loader” 软重启进入升级模式。

usb

```
usb - USB sub-system

Usage:
usb start - start (scan) USB controller
usb reset - reset (rescan) USB controller
usb stop [f] - stop USB [f]=force stop
usb tree - show USB device tree
usb info [dev] - show available USB devices
usb test [dev] [port] [mode] - set USB 2.0 test mode
    (specify port 0 to indicate the device's upstream port)
    Available modes: J, K, S[E0_NAK], P[acket], F[orce_Enable]
usb storage - show details of USB storage devices
usb dev [dev] - show or set current USB storage device
usb part [dev] - print partition table of one or all USB storage    devices
usb read addr blk# cnt - read `cnt' blocks starting at block `blk#'
    to memory address `addr'
usb write addr blk# cnt - write `cnt' blocks starting at block `blk#'
    from memory address `addr'
```

U-Boot USB不支持设备热拔插，因此需要执行USB命令枚举和断开设备。

- 通过 “usb start” 或 “usb reset” 命令，解析控制器节点并扫描所有port上接入的设备；
- 通过 “usb stop” 命令断开所有设备，析构控制器设备；
- “usb info” 和 “usb tree” 命令可以查看控制器信息和当前连接的设备信息；
- “usb storage”及其下面的命令用于UMS功能，具体使用方法参考命令说明。

fastboot

fastboot配置及使用请参考CH04-系统模块 Fastboot章节。

5.26 Vendor Storage

Vendor Storage 用于存放 SN、MAC 等不需要加密的小数据。数据存放在 NVM（eMMC、NAND 等）的保留分区中，有多个备份，更新数据时数据不丢失，可靠性高。

详细的资料参考文档《appnote rk vendor storage》。

5.26.1 原理概述

一共把 vendor 的存储块分成 4 个分区，vendor0、vendor1、vendor2、vendor3。每个 vendorX（X=0、1、2、3）的 hdr 里都有一个单调递增的 version 字段用于表明 vendorX 被更新的时刻点。每次读操作只读取最新的 vendorX（即 version 最大），写操作的时候会更新 version 并且把整个原有信息和新增信息搬移到 vendorX+1 分区里。例如当前从 vendor2 读取到信息，经过修改后再回写，此时写入的是 vendor3。这样做只是为了起到一个简单的安全防护作用。

5.26.2 框架支持

U-Boot 框架没有支持 Vendor Storage 功能，Rockchip 自己实现了一套 Vendor Storage 驱动。

配置：

```
CONFIG_ROCKCHIP_VENDOR_PARTITION
```

驱动文件：

```
./arch/arm/mach-rockchip/vendor.c
./arch/arm/include/asm/arch-rockchip/vendor.h
```

5.26.3 相关接口

```
int vendor_storage_read(u16 id, void *pbuf, u16 size)
int vendor_storage_write(u16 id, void *pbuf, u16 size)
```

关于 id 的定义和使用，请参考《appnote rk vendor storage》。

5.26.4 功能自测

U-Boot 串口命令行下使用"rktest vendor"命令可以进行 Vendor Storage 功能自测。

5.27 Watchdog

5.27.1 框架支持

watchdog 驱动使用 wdt-uclass.c 框架和标准接口。

配置：

```
CONFIG_WDT
CONFIG_WDT_ROCKCHIP
```

框架代码：

```
./drivers/watchdog/wdt-uclass.c
```

驱动代码：

```
./drivers/watchdog/rockchip_wdt.c
```

5.27.2 相关接口

```
// 设置喂狗超时时间且启动wdt（@flags默认填0）
int wdt_start(struct udevice *dev, u64 timeout_ms, ulong flags);
// 关闭wdt
int wdt_stop(struct udevice *dev);
int wdt_stop_all(void);
// 喂狗
int wdt_reset(struct udevice *dev);
// 忽略，目前未做底层驱动实现
int wdt_expire_now(struct udevice *dev, ulong flags)
```

目前 U-Boot 的默认流程里不启用、也不使用 wdt 功能，用户可根据自己的产品需求进行启用。

6. Chapter-6 进阶原理

6.1 kernel-DTB

6.1.1 设计背景

U-Boot 的原生架构要求一块板子必须对应一份 U-Boot dts，并且 U-Boot dts 生成的 dtb 会被打包到 U-Boot 自己的镜像中。这样就会出现各 SoC 平台上，N 块板子需要 N 份 U-Boot 镜像。

我们不难发现，其实一个SoC平台不同的板子之间主要是外设的差异，SoC核心部分是一致的。RK平台为了实现一个SoC平台仅需要一份U-Boot镜像，因此增加了 kernel DTB 机制。本质就是在较早的阶段切到kernel DTB，用它的配置信息初始化外设。

所以 RK 平台通过支持 kernel DTB 可以达到兼容板子差异，如：display、pmic/regulator、pinctrl、clk 等。

kernel DTB 的启用需要依赖 OF_LIVE（Live DT）。

```
config USING_KERNEL_DTB
        bool "Using dtb from Kernel/resource for U-Boot"
        depends on RKIMG_BOOTLOADER && OF_LIVE
        default y
        help
          This enable support to read dtb from resource and use it for U-Boot,
          the uart and emmc will still using U-Boot dtb, but other devices like
          regulator/pmic, display, usb will use dts node from kernel.
```

6.1.2 Live DT

背景和原理：

引入kernel DTB后U-Boot阶段存在两份DTB，其中存储、串口、Crypto等模块与U-Boot DTB相关，其余模块与kernel DTB相关。那么在U-Boot阶段可能在不同时刻需要交叉访问到这两类的模块，而模块又可能需要访问自己的DTB节点信息。

那么，这两类模块隶属于不同的DTB，而`gd->fdt_blob`又只能指向其中一份且不方便随意切换，这要怎么办？因为kernel 的 dts 最终要传递给 kernel 使用，所以也不能把 U-Boot dts 中的某些节点直接 overlay 到 kernel dts 上合成一份。

Live DT 可以解决这个问题。Live DT 的原理是：初始化阶段 U-Boot 直接扫描整个 DTB，把所有 DTB 节点转换成 `struct device_node` 节点链表，并且和具体的 device-driver 绑定。以后 device-driver 要访问 DTB 节点时直接访问自己的 `device_node` 即可，不需要再访问原有 DTB。

所以，相当于U-Boot和kernel 的DTB都绑定了各自的device-driver群组，且不再需要直接访问DTB文件。

这就解决了访问两份DTB引起的冲突。

更多参考:

```
./doc/driver-model/livetree.txt
```

fdt 和 Live DT 转换：

ofnode 类型（`include/dm/ofnode.h`）是两种 dt 都支持的一种封装格式，使用 Live DT 时用 `device_node` 来访问 dt 节点，使用 fdt 时用 offset 访问 dt 节点。当需要同时支持两种类型的驱动时请使用 ofnode 类型。

ofnode 结构：

```
/*
 * @np: Pointer to device node, used for live tree
 * @of_offset: Pointer into flat device tree, used for flat tree. Note that this
 *  is not really a pointer to a node: it is an offset value. See above.
 */
typedef union ofnode_union {
    const struct device_node *np;    /* will be used for future live tree */
    long of_offset;
} ofnode;
```

- "dev_"、"ofnode_"开头的函数为支持两种 dt 访问方式；
- "of_"开头的函数是只支持 Live DT 的接口；
- "fdtdec_"、"fdt_"开头的函数是只支持 fdt 的接口；

6.1.3 机制实现

kernel dtb 切换是在`./arch/arm/mach-rockchip/board.c` 的 `init_kernel_dtb()`里实现的。此时 U-Boot 的 dts 已经扫描完成，mmc/nand/nor 等存储驱动可正常工作。

此时从固件中读取 kernel dtb，然后进行 Live DT 建表并 bind 所有 device-driver，最后更新 `gd->fdt_blob` 指针指向 kernel dtb 即可。

6.1.4 U-Boot

- U-Boot 编译完成后会在./dts/目录下生成两个 dtb：
  - dt.dtb： 由 defconfig 里 `CONFIG_DEFAULT_DEVICE_TREE` 指定的 dts 编译得到的；
  - dt-spl.dtb：把 dt.dtb 中带`bootph-all`属性的节点全部抽取出来，再去掉 defconfig 里 `CONFIG_OF_REMOVE_PROPS` 指定的 property 得到。一般仅包含串口、DDR、存储等驱动必须依赖的节点：DMC、UART、MMC、NAND、GRF、CRU等。
- 不启用 `CONFIG_DM_KERNEL_DTB` 时系统使用 dt.dtb；启用 `CONFIG_DM_KERNEL_DTB` 时系统使用 dt-spl.dtb。
- dt.dtb或者dt-spl.dtb在U-Boot编译结束后都被命名为u-boot.dtb，然后追加到u-boot.bin的末尾。用户可以通过fdtdump命令检查u-boot.dtb的内容。

6.2 内核传参

本章节介绍U-Boot如何向kernel传递参数。

6.2.1 cmdline

U-Boot把kernel DTB里的`/chosen/bootargs`读取出来，修改/追加新内容后重新写回`/chosen/bootargs`节点，达到传递cmdline的目的。

6.2.2 内存容量

U-Boot修改kernel DTB里的`/memory`节点，把可用的内存容量信息填写进去。开机信息有相关打印：

```
......
## Chapter-6 Booting Android Image at 0x0027f800 ...
Kernel load addr 0x00280000 size 23387 KiB
RAM disk load addr 0x0a200000 size 782 KiB
## Chapter-6 Flattened Device Tree blob at 08300000
   Booting using the fdt blob at 0x8300000
   XIP Kernel Image ... OK
  'reserved-memory' ramoops@110000: addr=110000 size=f0000
   Using Device Tree in place at 0000000008300000, end 0000000008314648

// kernel可用的内存空间
Adding bank: 0x00200000 - 0x08400000 (size: 0x08200000)
Adding bank: 0x0a200000 - 0x80000000 (size: 0x75e00000)
Total: 473.217 ms

Starting kernel ...
```

6.2.3 其它方式

其它的传参方式本质也都是修改kernel DTB。如下：

节点/属性	操作	作用
/serial-number	创建	序列号
/memory	修改	kernel 可见内存
/display-subsystem/route/route-edp/	追加	显示相关参数(edp 为例)
/chosen/linux,initrd-start	创建	ramdisk 起始地址
/chosen/linux,initrd-end	创建	ramdisk 结束地址
/bootargs	修改	kernel 可见 cmdline
GMAC 节点内的 mac-address 或 local-mac-address	修改	mac 地址
arch/arm/mach-rockchip/board.c: board_fdt_fixup()	修改	板级的 fdt fixup

6.3 AB系统

6.3.1 AB 数据格式

A/B的数据结构位于 misc 分区偏移 2KB 位置。

```
/* Magic for the A/B struct when serialized. */
#define AVB_AB_MAGIC "\0AB0"
#define AVB_AB_MAGIC_LEN 4

/* Versioning for the on-disk A/B metadata - keep in sync with avbtool. */
#define AVB_AB_MAJOR_VERSION 1
#define AVB_AB_MINOR_VERSION 0

/* Size of AvbABData struct. */
#define AVB_AB_DATA_SIZE 32

/* Maximum values for slot data */
#define AVB_AB_MAX_PRIORITY 15
#define AVB_AB_MAX_TRIES_REMAINING 7

typedef struct AvbABSlotData {
  /* Slot priority. Valid values range from 0 to AVB_AB_MAX_PRIORITY,
   * both inclusive with 1 being the lowest and AVB_AB_MAX_PRIORITY
   * being the highest. The special value 0 is used to indicate the
   * slot is unbootable.
   */
  uint8_t priority;

  /* Number of times left attempting to boot this slot ranging from 0
   * to AVB_AB_MAX_TRIES_REMAINING.
   */
  uint8_t tries_remaining;

  /* Non-zero if this slot has booted successfully, 0 otherwise. */
  uint8_t successful_boot;

  /* Reserved for future use. */
  uint8_t reserved[1];
} AVB_ATTR_PACKED AvbABSlotData;

/* Struct used for recording A/B metadata.
 *
 * When serialized, data is stored in network byte-order.
 */
typedef struct AvbABData {
  /* Magic number used for identification - see AVB_AB_MAGIC. */
  uint8_t magic[AVB_AB_MAGIC_LEN];

  /* Version of on-disk struct - see AVB_AB_{MAJOR,MINOR}_VERSION. */
  uint8_t version_major;
  uint8_t version_minor;

  /* Padding to ensure |slots| field start eight bytes in. */
  uint8_t reserved1[2];

  /* Per-slot metadata. */
  AvbABSlotData slots[2];

  uint8_t last_boot;
  /* Reserved for future use. */
  uint8_t reserved2[11];
```

```
  /* CRC32 of all 28 bytes preceding this field. */
  uint32_t crc32;
} AVB_ATTR_PACKED AvbABData;
```

对于小容量存储，没有 misc 分区但有 vendor 分区，可以考虑存储到 vendor。

在此基础上增加 lastboot，标记最后一个可启动固件。主要应用于低电情况或工厂生产测试时 retry 次数用完，而还没有进入系统调用 boot_ctrl 服务。参考如下：

```
typedef struct AvbABSlotData {
  /* Slot priority. Valid values range from 0 to AVB_AB_MAX_PRIORITY,
   * both inclusive with 1 being the lowest and AVB_AB_MAX_PRIORITY
   * being the highest. The special value 0 is used to indicate the
   * slot is unbootable.
   */
  uint8_t priority;

  /* Number of times left attempting to boot this slot ranging from 0
   * to AVB_AB_MAX_TRIES_REMAINING.
   */
  uint8_t tries_remaining;

  /* Non-zero if this slot has booted successfully, 0 otherwise. */
  uint8_t successful_boot;

  /* Reserved for future use. */
  uint8_t reserved[1];
} AVB_ATTR_PACKED AvbABSlotData;

typedef struct AvbABData {
  /* Magic number used for identification - see AVB_AB_MAGIC. */
  uint8_t magic[AVB_AB_MAGIC_LEN];

  /* Version of on-disk struct - see AVB_AB_{MAJOR,MINOR}_VERSION. */
  uint8_t version_major;
  uint8_t version_minor;

  /* Padding to ensure |slots| field start eight bytes in. */
  uint8_t reserved1[2];

  /* Per-slot metadata. */
  AvbABSlotData slots[2];

  /* mark last boot slot */
  uint8_t last_boot;
  /* Reserved for future use. */
  uint8_t reserved2[11];

  /* CRC32 of all 28 bytes preceding this field. */
  uint32_t crc32;
} AVB_ATTR_PACKED AvbABData;
```

最后表格来说明各个参数的含义：

AvbABSlotData：

参数	含义
priority	标志 slot 优先级，0 为不可启动，15 为最高优先级
tries_remaining	尝试启动次数，设置为 7 次
successful_boot	系统启动成功后会配置该参数，1：该 slot 成功启动，0：该 slot 未成功启动

AvbABData：

参数	含义
magic	结构体头部信息：\0AB0
version_major	主版本信息
version_minor	次版本信息
slots	slot 引导信息，参见 AvbABSlotData
last_boot	上一次成功启动的 slot，0：slot A 上次成功启动，1：slot B 上次成功启动
crc32	数据校验

6.3.2 AB 启动模式

目前 system bootctrl 设计两套控制模式，bootloader 支持这两种模式启动。

6.3.2.1 successful-boot

正常进入系统后，boot_ctrl 依据 androidboot.slot_suffix，设置当前 slot 的变量：

```
successful_boot = 1;
priority = 15;
tries_remaining = 0;
last_boot = 0 or 1;    // refer to androidboot.slot_suffix
```

升级系统中，boot_ctrl 设置：

```
升级的slot设置：
successful_boot = 0;
priority = 14;
tries_remaining = 7;
last_boot = 0 or 1;    // refer to androidboot.slot_suffix

当前slot设置：
successful_boot = 1;
priority = 15;
tries_remaining = 0;
last_boot = 0 or 1;    // refer to androidboot.slot_suffix
```

升级系统完成，boot_ctrl 设置：

```
升级的slot设置：
successful_boot = 0;
priority = 15;
tries_remaining = 7;
last_boot = 0 or 1;    // refer to androidboot.slot_suffix

当前slot设置：
successful_boot = 1;
priority = 14;
tries_remaining = 0;
last_boot = 0 or 1;    // refer to androidboot.slot_suffix
```

6.3.2.2 reset-retry

正常进入系统后，boot_ctrl 依据 androidboot.slot_suffix，设置当前 slot 的变量：

```
successful_boot = 0;
priority = 15;
tries_remaining = 7;
last_boot = 0 or 1;    // refer to androidboot.slot_suffix
```

升级系统中，boot_ctrl 设置：

```
升级的slot设置：
successful_boot = 0;
priority = 14;
tries_remaining = 7;
last_boot = 0 or 1;    // refer to androidboot.slot_suffix

当前slot设置：
successful_boot = 0;
priority = 15;
tries_remaining = 7;
last_boot = 0 or 1;    // refer to androidboot.slot_suffix
```

升级系统完成，boot_ctrl 设置：

```
升级的slot设置：
successful_boot = 0;
priority = 15;
tries_remaining = 7;
last_boot = 0 or 1;    // refer to androidboot.slot_suffix

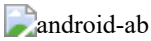
当前slot设置：
successful_boot = 0;
priority = 14;
tries_remaining = 7;
last_boot = 0 or 1;    // refer to androidboot.slot_suffix
```

6.3.2.3 模式对比

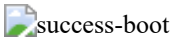
- successful_boot 模式

    - 优点：只要正常启动系统，不会回退到旧版本固件，除非 system bootctrl 配置
    - 缺点：设备长时间工作后，如果存储某些颗粒异常，会导致系统一直重启
- reset retry 模式
    - 优点：始终保持 retry 机制，可以应对存储异常问题
    - 缺点：会回退到旧版本固件

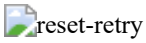
6.3.3 启动流程



AB successful_boot 模式数据流程：



AB reset retry 模式数据流程：



6.3.4 升级和异常

- 系统 升级：参考《Rockchip Linux 升级方案开发指南》
- recovery 升级：AB system 不考虑支持 recovery 升级

6.3.5 验证方法

6.3.5.1 successful-boot

1. 只烧写 slot A，系统从 slot A 启动。设置从 slot B 启动，系统从 slot A 启动。测试完成，清空 misc 分区
2. 烧写 slot A 与 slot B，启动系统，当前系统为 slot A。设置系统从 slot B 启动，reboot 系统，当前系统为 slot B。测试完成，清空 misc 分区
3. 烧写 slot A 与 slot B，迅速 reset 系统 14 次后，retry counter 用完，还能从 last_boot 指定的系统启动，即能正常从 slot A 启动。测试完成，清空 misc 分区
4. 烧写 slot A 与 slot B，启动系统，当前系统为 slot A。设置系统从 slot B 启动，reboot 系统，当前系统为 slot B。设置系统从 slot A 启动，reboot 系统，当前系统为 slot A。测试完成，清空 misc 分区

6.3.5.2 reset-retry

1. 只烧写 slot A，系统从 slot A 启动。设置从 slot B 启动，系统从 slot A 启动。测试完成，清空 misc 分区
2. 烧写 slot A 与 slot B，启动系统，当前系统为 slot A。设置系统从 slot B 启动，reboot 系统，当前系统为 slot B。测试完成，清空 misc 分区
3. 烧写 slot A 与 slot B，迅速 reset 系统 14 次后，retry counter 用完，还能从 last_boot 指定的系统启动，即能正常从 slot A 启动。测试完成，清空 misc 分区
4. 烧写 slot A 与 slot B，其中 slot B 的 boot.img 损坏，启动系统，当前系统为 slot A。设置系统从 slot B 启动，reboot 系统，系统会重启 7 次后，从 slot A 正常启动系统。测试完成，清空 misc 分区
5. 烧写 slot A 与 slot B，启动系统，当前系统为 slot A。设置系统从 slot B 启动，reboot 系统，当前系统为 slot B。设置系统从 slot A 启动，reboot 系统，当前系统为 slot A。测试完成，清空 misc 分区

6.3.6 引用参考

《Rockchip-Secure-Boot2.0.md》

《Rockchip-Secure-Boot-Application-Note.md》

《Android Verified Boot 2.0》

6.4 AVB安全启动

6.4.1 引用参考

《Rockchip-Secure-Boot-Application-Note.md》

《Android Verified Boot 2.0》

《Rockchip_Developer_Guide_Linux4.4_SecureBoot_CN.pdf》

6.4.2 术语

AVB : Android Verified Boot

OTP & efuse : One Time Programmable

Product RootKey (PRK)：AVB 的 root key 由签名 loader，uboot 的 root key 校验

ProductIntermediate Key (PIK)：中间 key，中介作用

ProductSigning Key (PSK)：用于签固件的 key

ProductUnlock Key (PUK)：用于解锁设备

各种key分离，职责明确，可以降低key被泄露的风险。

6.4.3 简介

本文介绍 Rockchip 安全验证引导流程。所谓的安全验证引导流程分为安全性校验与完整性校验。安全性校验是加密公钥的校验，流程为从安全存储（OTP & efuse）中读取公钥 hash，与计算的公钥 hash 对比，是否一致，然后公钥用于解密固件 hash。完整性校验为校验固件的完整性，流程为从存储里加载固件，计算固件的 hash 与解密出来的 hash 对比是否一致。

6.4.4 加密示例

设备的安全验证启动流程与通信中的数据加密校验流程类似，通过该例子可以加速对 avb 校验流程的理解。假如现在 Alice 向 Bob 传送数字信息，为了保证信息传送的保密性、真实性、完整性和不可否认性，需要对传送的信息进行数字加密和签名，其传送过程为：

1.Alice 准备好要传送的数字信息（明文）；

2.Alice 对数字信息进行哈希运算，得到一个信息摘要；

3.Alice 用自己的私钥对信息摘要进行加密得到 Alice 的数字签名，并将其附在数字信息上；

4.Alice 随机产生一个加密密钥，并用此密码对要发送的信息进行加密，形成密文；

5.Alice 用 Bob 的公钥对刚才随机产生的加密密钥进行加密，将加密后的 DES 密钥连同密文一起传送给 Bob；

6.Bob 收到 Alice 传送来的密文和加密过的 DES 密钥，先用自己的私钥对加密的 DES 密钥进行解密，得到 Alice 随机产生的加密密钥；

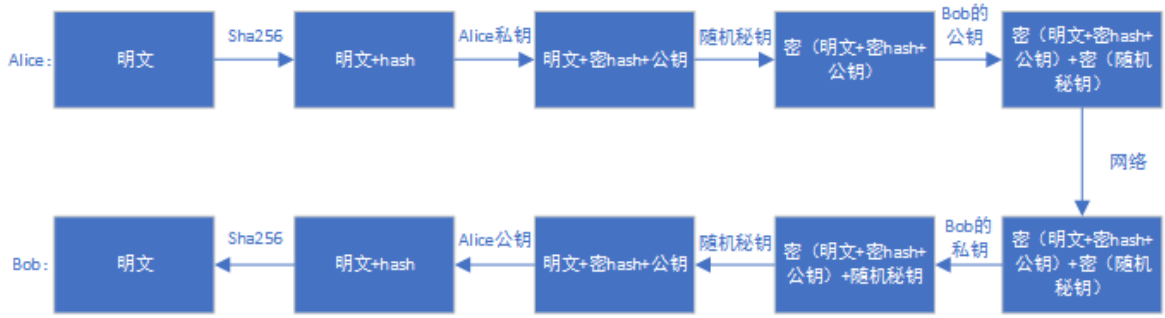
7.Bob 然后用随机密钥对收到的密文进行解密，得到明文的数字信息，然后将随机密钥抛弃；

8.Bob 用 Alice 的公钥对 Alice 的数字签名进行解密，得到信息摘要；

9.Bob 用相同的哈希算法对收到的明文再进行一次哈希运算，得到一个新的信息摘要；

10.Bob 将收到的信息摘要和新产生的信息摘要进行比较，如果一致，说明收到的信息没有被修改过。

上面提及的 DES 算法可以更换其他算法，如 AES 加密算法，公私钥算法可以采用 RSA 算法，流程如下：



6.4.5 AVB

AVB 为 Android Verified Boot，谷歌设计的一套固件校验流程，主要用于校验 boot system 等固件。Rockchip Secure Boot 参考通信中的校验方式及 AVB，实现一套完整的 Secure Boot 校验方案。

6.4.5.1 AVB 特性

- 安全校验
- 完整性校验
- 防回滚保护
- persistent partition 支持
- chained partitions 支持，可以与 boot，system 签名私钥一致，也可以由 oem 自己保存私钥，但必须由 PRK 签名

6.4.5.2 生成key，签名和证书

```
#!/bin/sh
touch temp.bin
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 -outform PEM -out
testkey_prk.pem
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 -outform PEM -out
testkey_psk.pem
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 -outform PEM -out
testkey_pik.pem
python avbtool make_atx_certificate --output=pik_certificate.bin --
subject=temp.bin --subject_key=testkey_pik.pem --
subject_is_intermediate_authority --subject_key_version 42 --
authority_key=testkey_prk.pem
python avbtool make_atx_certificate --output=psk_certificate.bin --
subject=product_id.bin --subject_key=testkey_psk.pem --subject_key_version 42 --
authority_key=testkey_pik.pem
python avbtool make_atx_metadata --output=metadata.bin --
intermediate_key_certificate=pik_certificate.bin --
product_key_certificate=psk_certificate.bin
```

permanent_attributes.bin 生成：

```
python avbtool make_atx_permanent_attributes --output=permanent_attributes.bin -
-product_id=product_id.bin --root_authority_key=testkey_prk.pem
```

其中 product_id.bin 需要自己定义，占 16 字节，可作为产品 ID 定义。

boot.img 签名示例：

```
avbtool add_hash_footer --image boot.img --partition_size 33554432 --
partition_name boot --key testkey_psk.pem --algorithm SHA256_RSA4096
```

注意：partition size 要至少比原固件大 64K，大小还要 4K 对齐，且不大于 parameter.txt 定义的分区大小。

system.img 签名示例：

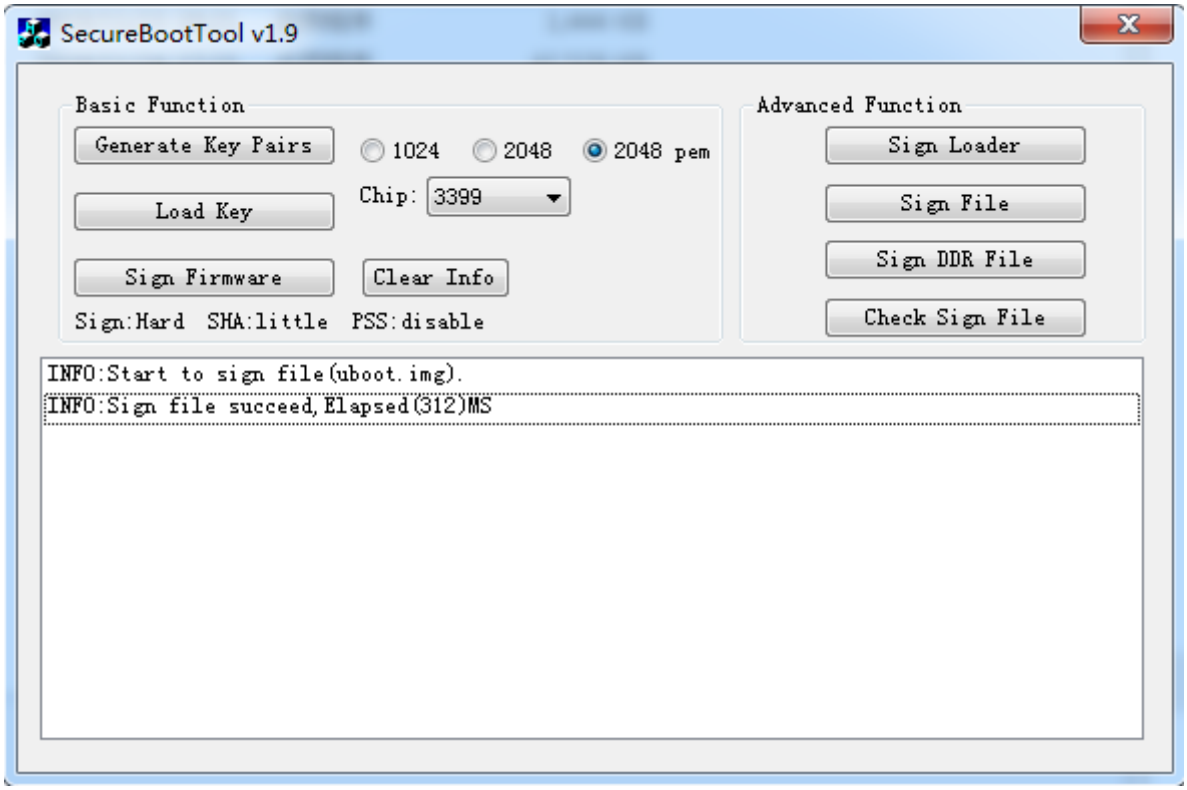
```
avbtool add_hashtree_footer --partition_size 536870912 --partition_name system -
-image system.img --algorithm SHA256_RSA4096 --key testkey_psk.pem
```

生成 vbmeta 包含 metadata.bin，命令示例如下：

```
python avbtool make_vbmeta_image --public_key_metadata metadata.bin --
include_descriptors_from_image boot.img --include_descriptors_from_image
system.img --generate_dm_verity_cmdline_from_hashtree system.img --algorithm
SHA256_RSA4096 --key testkey_psk.pem --output vbmeta.img
```

最终把生成的 vbmeta.img 烧写到对应的分区，如 vbmeta 分区。

通过 SecureBootTool 生成 PrivateKey.pem 和 PublicKey.pem。



对 permanent_attributes.bin 进行签名：

```
openssl dgst -sha256 -out permanent_attributes_cer.bin -sign PrivateKey.pem
permanent_attributes.bin
```

permanent_attributes.bin 是整个系统的安全认证数据，它需要烧写它的 hash 到 efuse 或 OTP，或它的数据由前级的安全认证（pre-load）。

- 如果平台使用的是efuse，由于规划的 efuse 一般不足以存放证书hash，所以 permanent_attributes.bin 的验证由前级的公钥加 permanent_attributes.bin 的证书进行认证
- 如果平台使用的是OTP，安全数据空间足够，会直接烧写。

各个平台efuse与OTP支持情况：**请参考文档：《Rockchip OTP 开发指南》**。

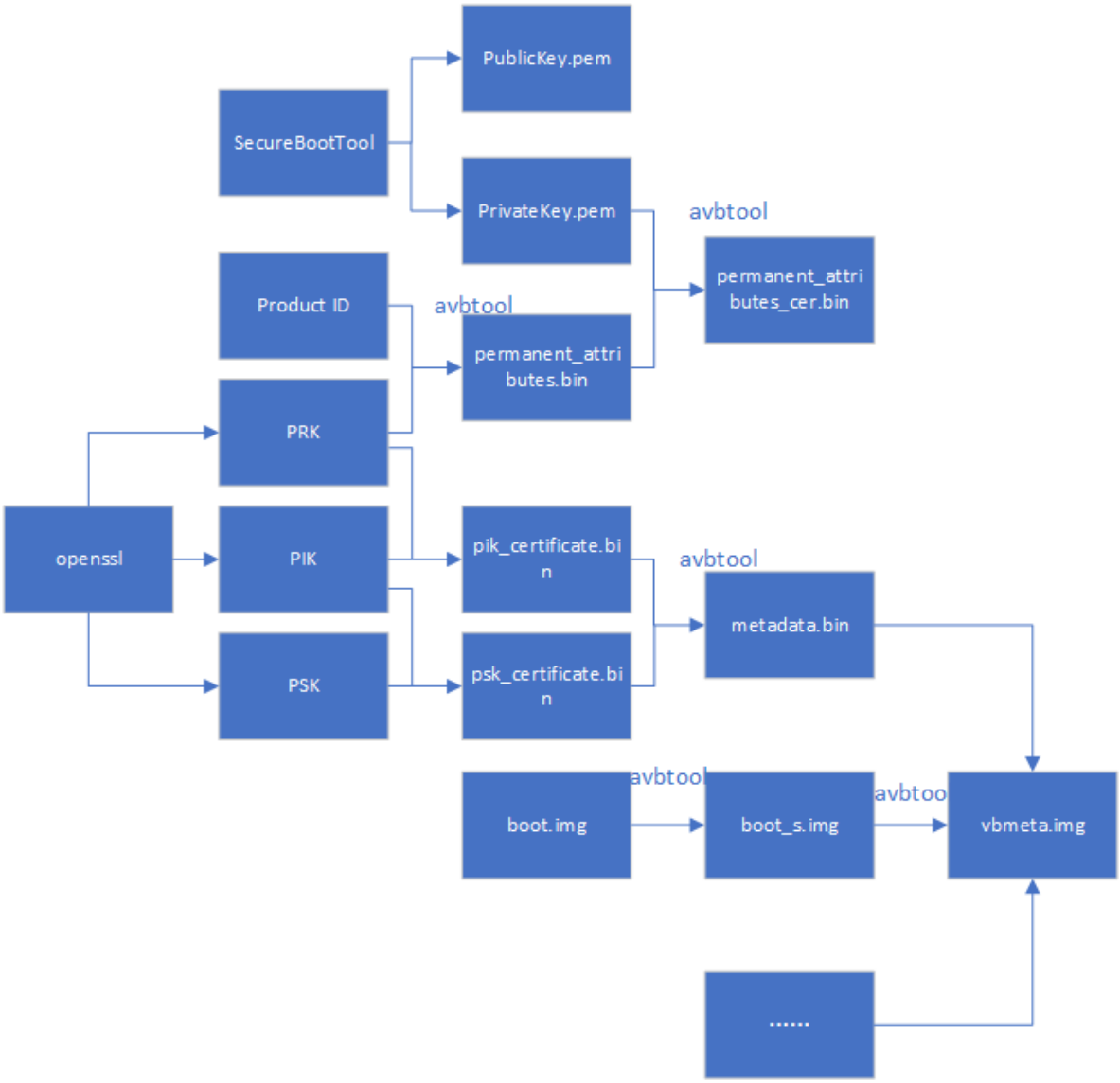
efuse 平台 pub_key 烧写：

```
fastboot stage permanent_attributes.bin
fastboot oem board:fuse at-perm-attr
fastboot stage permanent_attributes_cer.bin
fastboot oem board:fuse at-rsa-perm-attr
```

OTP 平台 pub_key 烧写：

```
fastboot stage permanent_attributes.bin
fastboot oem board:fuse at-perm-attr
```

整个签名流程：



6.4.5.3 AVB lock

```
fastboot oem board:lock
```

如何进入 fastboot 见 fastboot 命令支持章节。

6.4.5.4 AVB unlock

目前 Rockchip 采用严格安全校验，需要在对应的defconfig内添加

```
CONFIG_LIBAVB_ATH_UNLOCK_SUPPORT=y
```

否则输入 fastboot oem board:unlock 就可以解锁设备，启动校验 vbmeta.img，boot.img 失败也会成功启动设备。

首先，需要生成 PUK：

```
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 -outform PEM -out
testkey_puk.pem
```

unlock_credential.bin 为需要下载到设备解锁的证书，其生成过程如下：

```
python avbtool make_atx_certificate --output=puk_certificate.bin --
subject=product_id.bin --subject_key=testkey_puk.pem --
usage=com.google.android.things.vboot.unlock --subject_key_version 42 --
authority_key=testkey_pik.pem
```

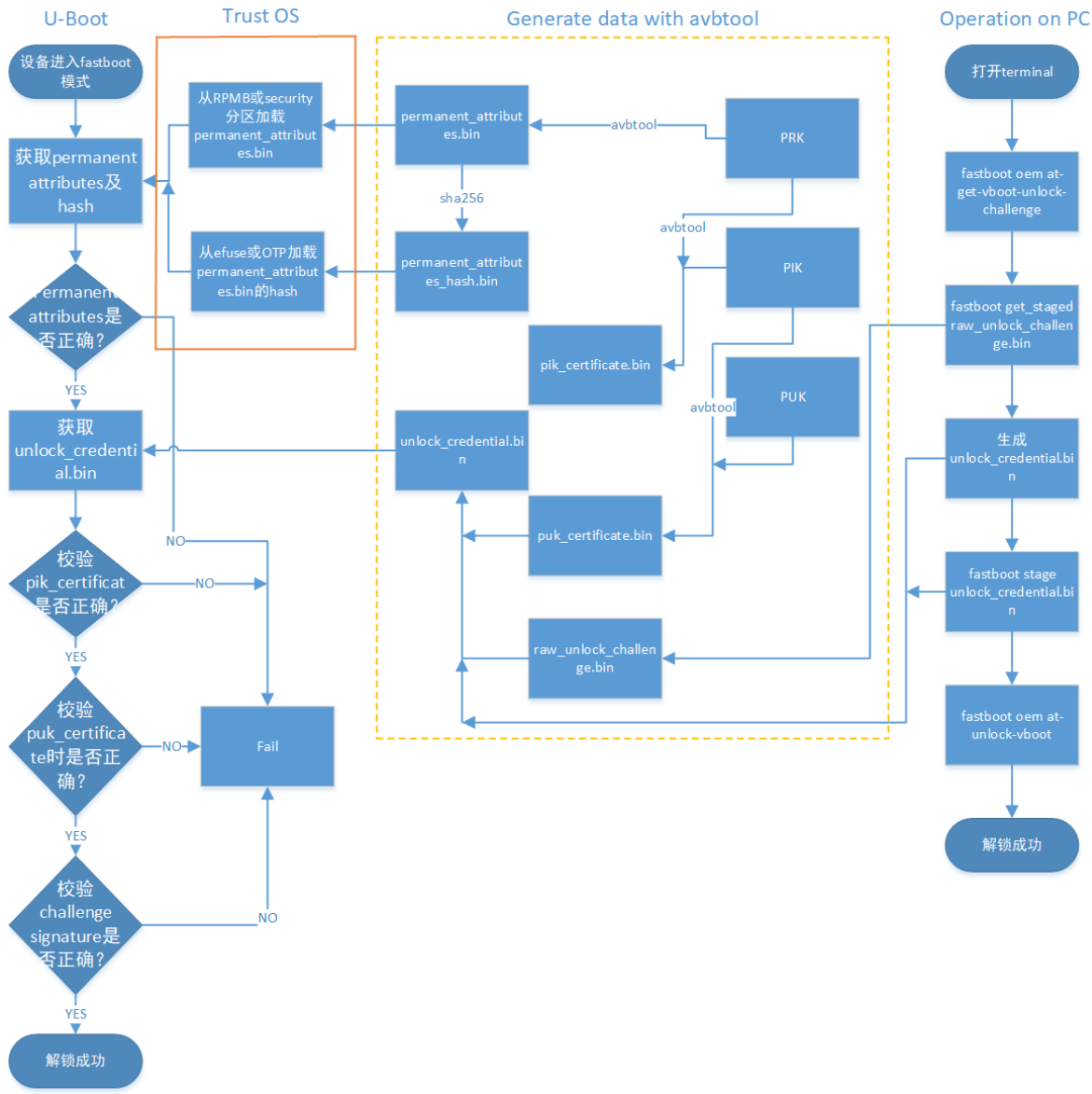
然后生成 unlock_credential.bin，执行下列命令：

```
python avbtool make_atx_unlock_credential --output=unlock_credential.bin --
intermediate_key_certificate=pik_certificate.bin --
unlock_key_certificate=puk_certificate.bin --challenge=unlock_challenge.bin --
unlock_key=testkey_puk.pem
```

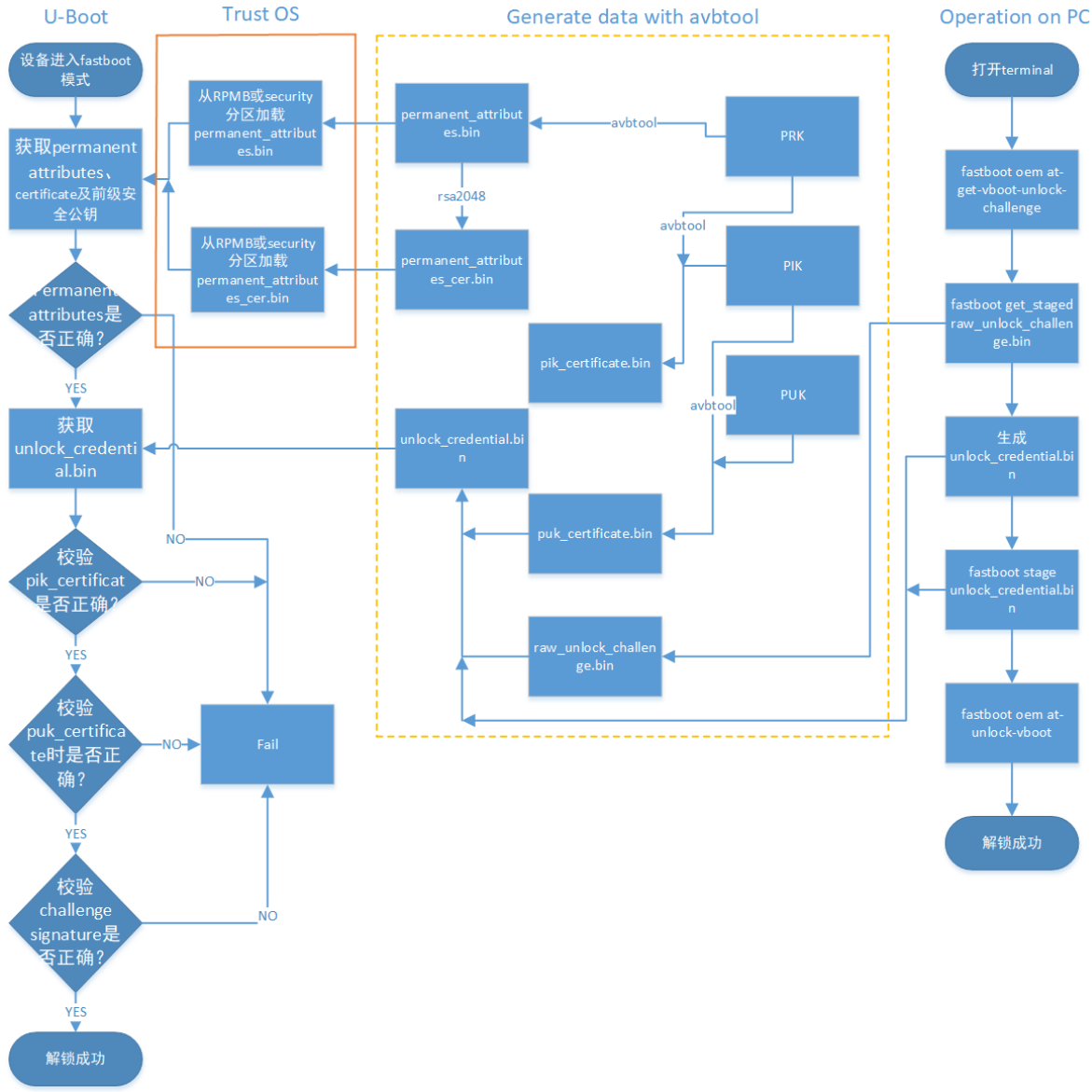
最终可以把证书通过 fastboot 命令下载到设备，并解锁设备，fastboot 命令如下：

```
fastboot stage unlock_credential.bin
fastboot oem board:unlock
```

OTP 设备解锁流程：



efuse 设备解锁流程：



操作流程如下：

1. 设备进入 fastboot 模式，电脑端输入

```
fastboot oem board:at-get-vboot-unlock-challenge
fastboot get_staged raw_unlock_challenge.bin
```

获得带版本、Product Id 与 16 字节的随机数的数据，取出随机数作为 unlock_challenge.bin。

2. 使用 avbtool 生成 unlock_credential.bin，参考make_unlock.sh。
3. 电脑端输入

```
fastboot stage unlock_credential.bin
fastboot oem board:unlock
```

注意：此时设备状态一直处于第一次进入 fastboot 模式状态，在此期间不能断电、关机、重启。因为步骤 1.做完后，设备存储着生成的随机数，如果断电、关机、重启，会导致随机数丢失，后续校验 challenge signature 会因为随机数不匹配失败。

如果开启：

```
CONFIG_MISC=y
CONFIG_ROCKCHIP_EFUSE=y
CONFIG_ROCKCHIP_OTP=y
```

就会使用 CPUID 作为 challenge number，而 CPUID 是与机器匹配的，数据不会因为关机而丢失，生成的unlock_credential.bin 可以重复使用。省去重复生成unlock_challenge.bin，制作unlock_credential.bin的步骤。再次解锁步骤变为：

```
fastboot oem board:at-get-vboot-unlock-challenge
fastboot stage unlock_credential.bin
fastboot oem board:unlock
```

4. 设备进入解锁状态，开始解锁。

make_unlock.sh 参考

```
#!/bin/sh
python avb-challenge-verify.py raw_unlock_challenge.bin product_id.bin
python avbtool make_unlock_credential --output=unlock_credential.bin --
intermediate_key_certificate=pik_certificate.bin --
unlock_key_certificate=puk_certificate.bin --challenge=unlock_challenge.bin --
unlock_key=testkey_puk.pem
```

avb-challenge-verify.py 源码

```
#!/user/bin/env python
"This is a test module for getting unlock_challenge.bin"
import sys
import os
from hashlib import sha256

def challenge_verify():
    if (len(sys.argv) != 3) :
        print "Usage: rkpublickey.py [challenge_file] [product_id_file]"
        return
    if ((sys.argv[1] == "-h") or (sys.argv[1] == "--h")):
        print "Usage: rkpublickey.py [challenge_file] [product_id_file]"
        return
    try:
        challenge_file = open(sys.argv[1], 'rb')
        product_id_file = open(sys.argv[2], 'rb')
        challenge_random_file = open('unlock_challenge.bin', 'wb')
        challenge_data = challenge_file.read(52)
        product_id_data = product_id_file.read(16)
        product_id_hash = sha256(product_id_data).digest()
        print("The challege version is %d" %ord(challenge_data[0]))
        if (product_id_hash != challenge_data[4:36]) :
            print("Product id verify error!")
            return
        challenge_random_file.write(challenge_data[36:52])
        print("Success!")

    finally:
        if challenge_file:
            challenge_file.close()
```

```
        if product_id_file:
            product_id_file.close()
        if challenge_random_file:
            challenge_random_file.close()

if __name__ == '__main__':
    challenge_verify()
```

6.4.5.5 Cmdline 新增内容

```
Kernel command line: androidboot.verifiedbootstate=green
androidboot.slot_suffix=_a dm="1 vroot none ro 1,0 1031864 verity 1
PARTUUID=b2110000-0000-455a-8000-44780000706f PARTUUID=b2110000-0000-455a-8000-
44780000706f 4096 4096 128983 128983 sha1
90d1d406caac04b7e3fbf48b9a4dcd6992cc628e
4172683f0d6b6085c09f6ce165cf152fe3523c89 10 restart_on_corruption
ignore_zero_blocks use_fec_from_device PARTUUID=b2110000-0000-455a-8000-
44780000706f fec_roots 2 fec_blocks 130000 fec_start 130000" root=/dev/dm-0
androidboot.vbmeta.device=PARTUUID=f24f0000-0000-4e1b-8000-791700006a98
androidboot.vbmeta.avb_version=1.1 androidboot.vbmeta.device_state=unlocked
androidboot.vbmeta.hash_alg=sha512 androidboot.vbmeta.size=6528
androidboot.vbmeta.digest=41991c02c82ea1191545c645e2ac9cc7ca08b3da0a2e3115aff479
d2df61feaccdd35b6360cfa936f6f4381e4557ef18e381f4b236000e6ecc9ada401eda4cae
androidboot.vbmeta.invalidate_on_error=yes androidboot.veritymode=enforcing
```

这里说明几个参数：

1. 为什么传递 vbmeta 的 PARTUUID？因为确保后续使用 vbmeta hash-tree 的合法性，需要 kernel 再校验一遍 vbmeta，digest 为 androidboot.vbmeta.digest。
2. skip_initramfs：boot ramdisk 是否打包到 boot.img 的问题，在 A/B system 中，ramdisk 是没有打包到 boot.img，cmdline 需要传递这个参数。
3. root=/dev/dm-0 开启 dm-verity，指定 system。
4. androidboot.vbmeta.device_state：android verify 状态
5. androidboot.verifiedbootstate：校验结果。

green：If in LOCKED state and an the key used for verification was not set by the end user。

yellow：If in LOCKED state and an the key used for verification was set by the end user。

orange：If in the UNLOCKED state。

这里特别说明一下 dm="1 vroot none ro……"参数生成：

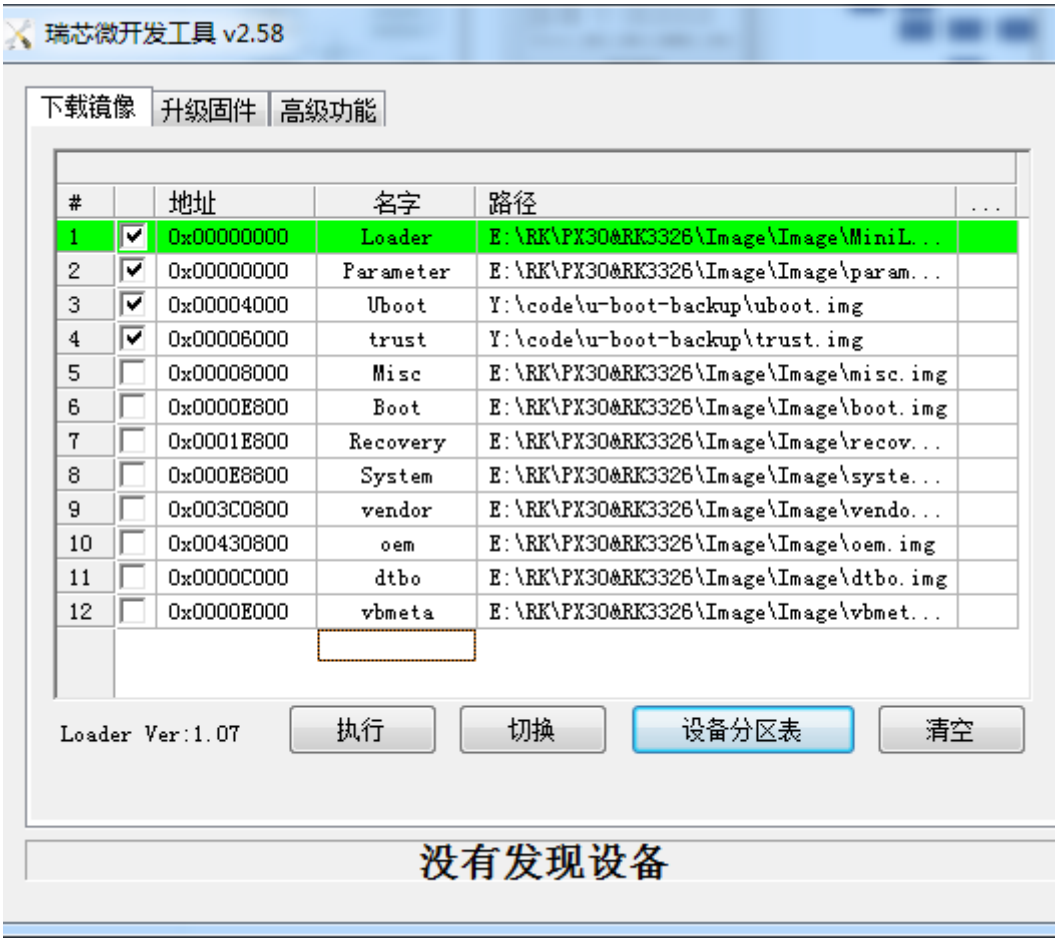
```
avbtool make_vbmeta_image --include_descriptors_from_image boot.img --
include_descriptors_from_image system.img --
generate_dm_verity_cmdline_from_hashtree system.img --
include_descriptors_from_image vendor.img --algorithm SHA512_RSA4096 --key
testkey_psk.pem --public_key_metadata metadata.bin --output vbmeta.img
```

avbtool 生成 vbmeta 时，对 system 固件加--generate_dm_verity_cmdline_from_hashtree 即可。dm="1 vroot none ro……"这些信息会保存到 vbmeta。这部分安卓专用，如果分区只校验到 boot.img，无需增加该参数。

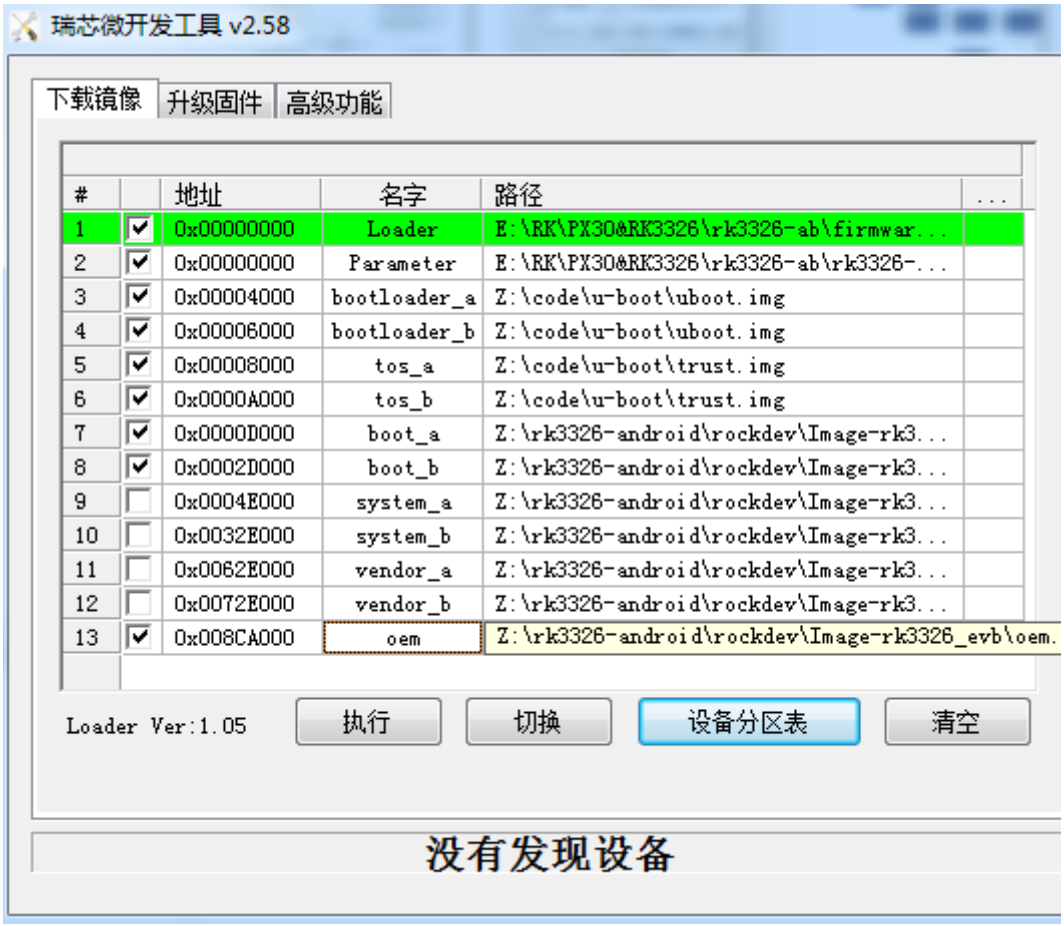
Android SDK 开启AVB相关配置 会把这些信息加到 vbmeta 内。

6.4.6 固件烧写

如下是windows固件烧写工具：

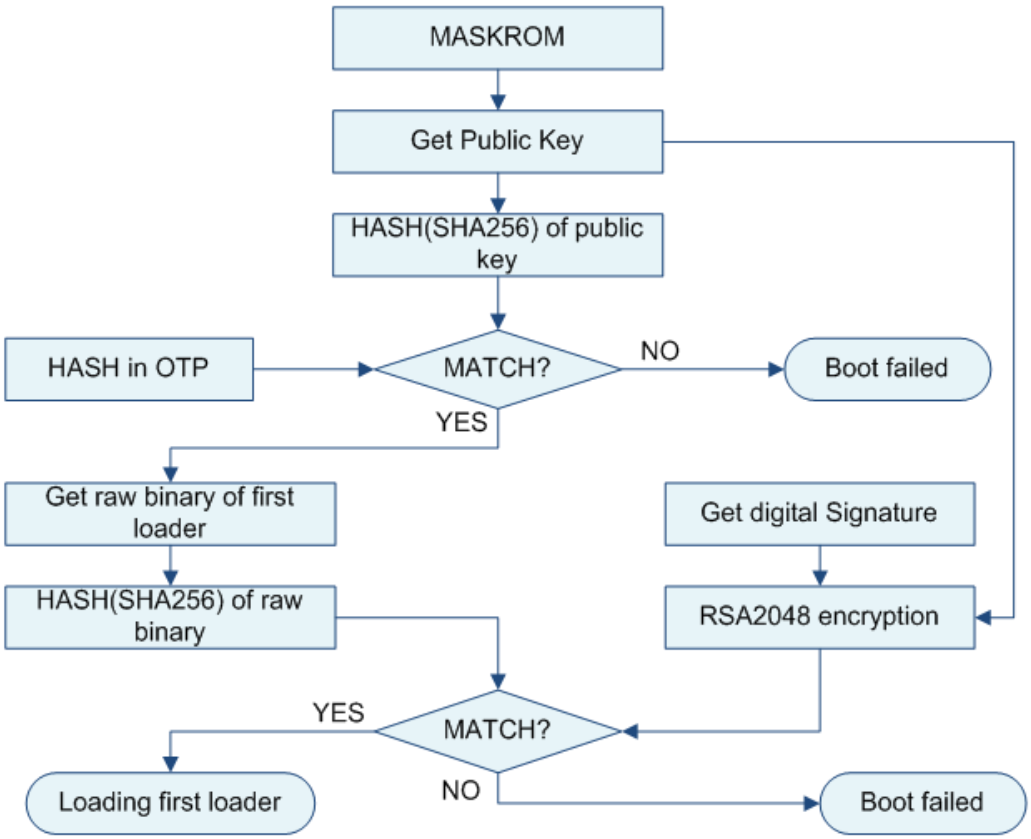


A/B System 烧写



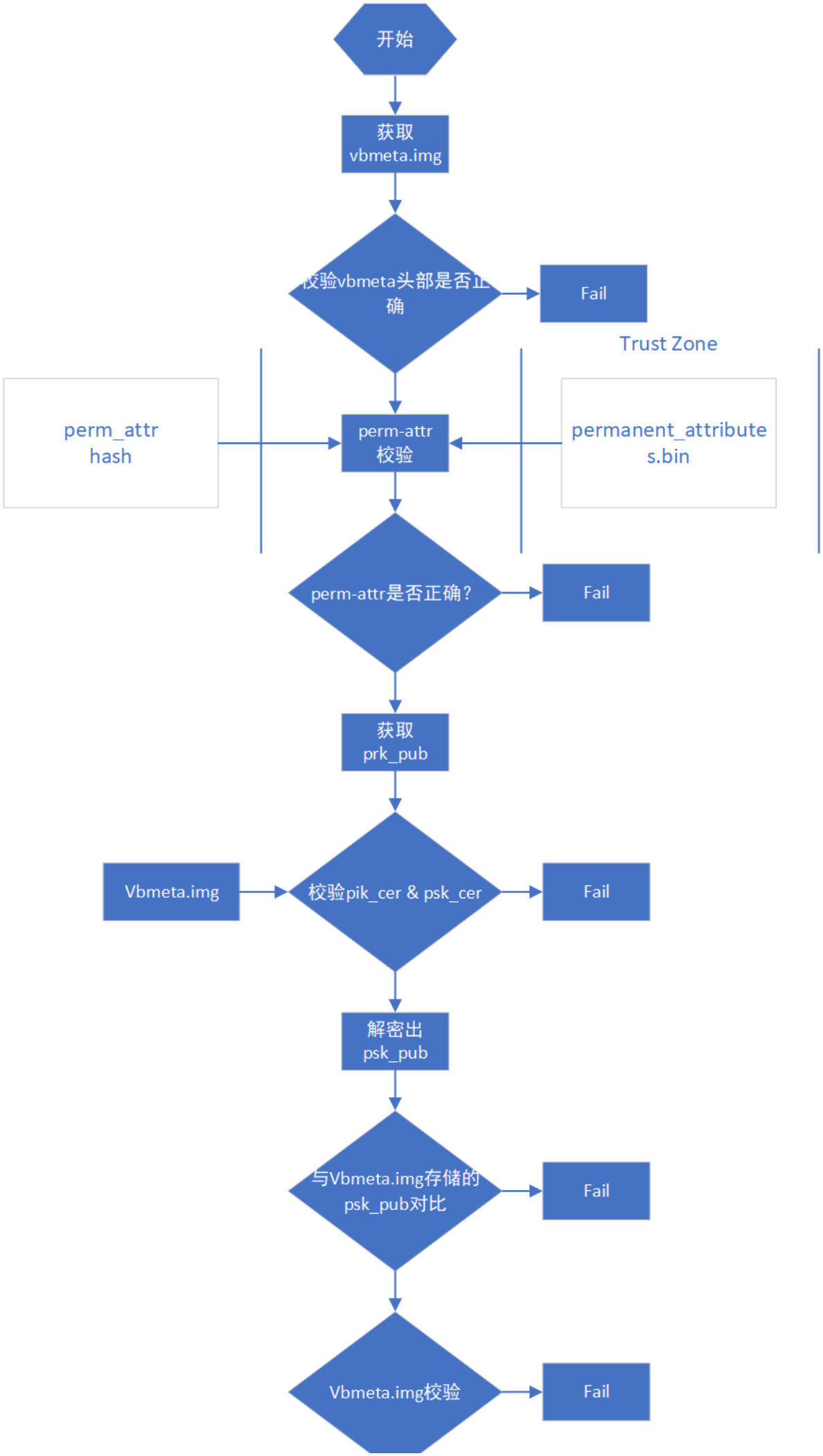
6.4.7 Pre-loader verified

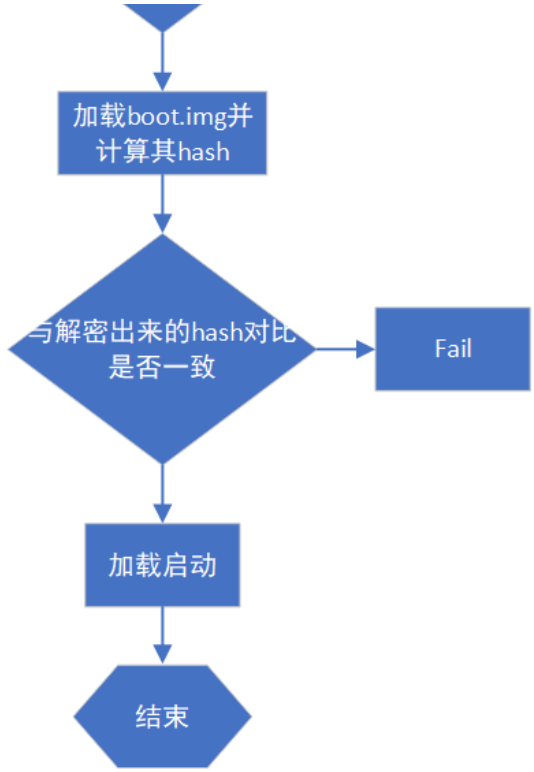
参见《Rockchip-Secure-Boot-Application-Note.md》



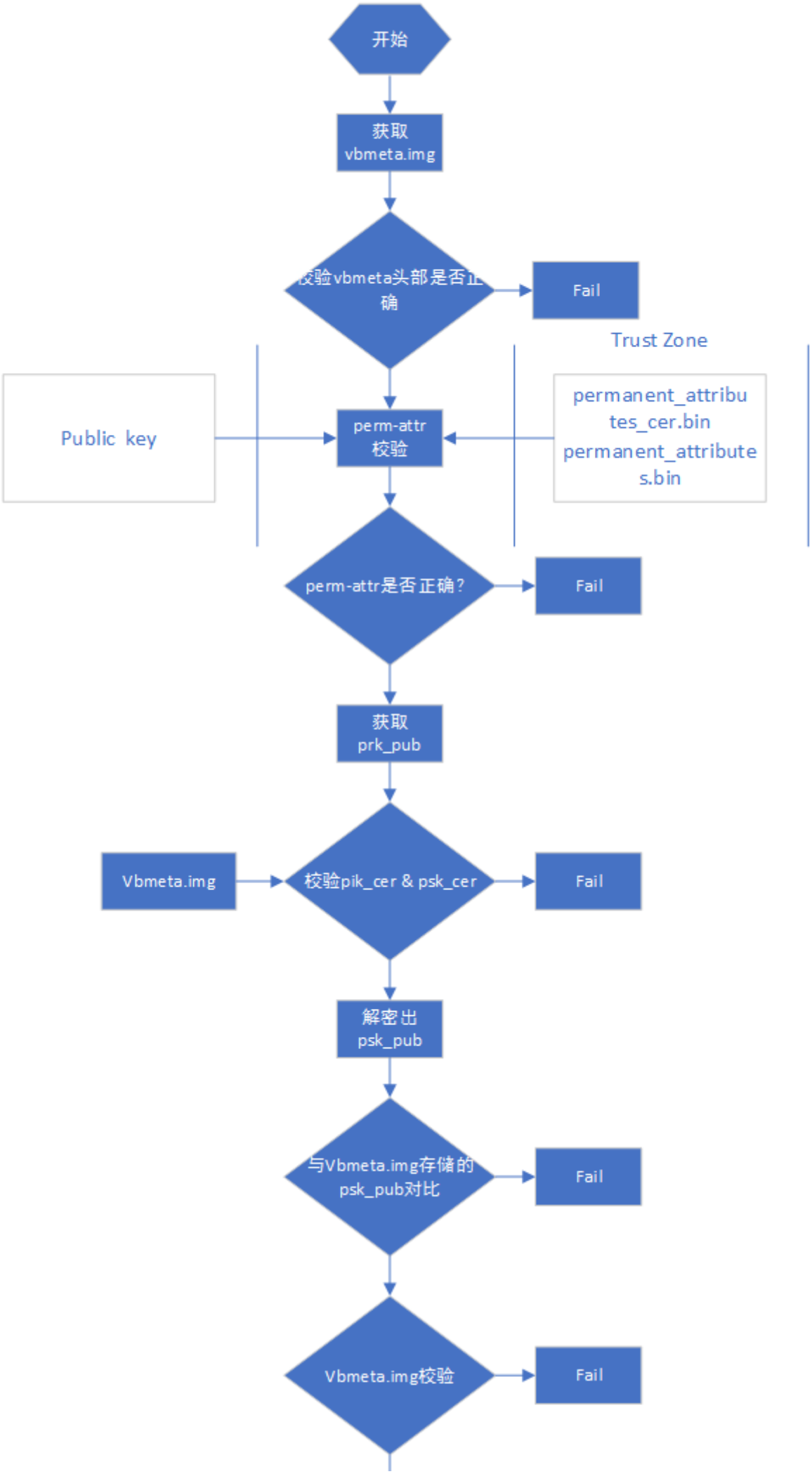
6.4.8 U-boot verified

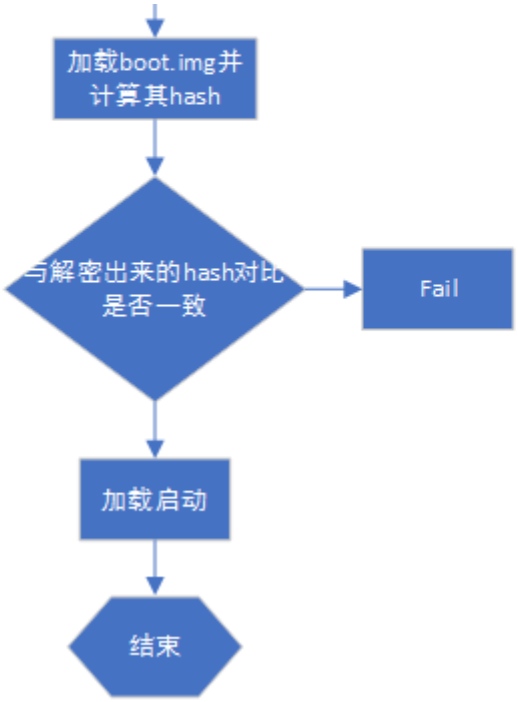
OTP 设备校验流程：


开始
获取 vbmeta.img
校验vbmeta头部是否正确
Fail
Trust Zone
perm_attr hash
perm-attr 校验
permanent_attributes.bin
perm-attr是否正确?
Fail
获取 prk_pub
Vbmeta.img
校验pik_cer & psk_cer
Fail
解密出 psk_pub
与Vbmeta.img存储的psk_pub对比
Fail
Vbmeta.img校验
Fail

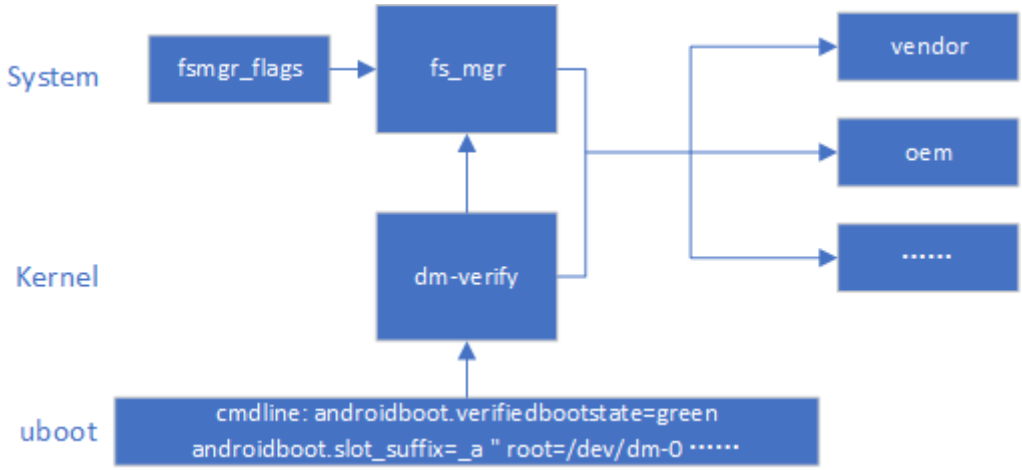


efuse设备校验流程：


开始
获取 vbmeta.img
校验vbmeta头部是否正确
Fail
Trust Zone
Public key
perm-attr 校验
permanent_attributes_cer.bin
permanent_attributes.bin
perm-attr是否正确?
Fail
获取 prk_pub
Vbmeta.img
校验pik_cer & psk_cer
Fail
解密出 psk_pub
与Vbmeta.img存储的 psk_pub对比
Fail
Vbmeta.img校验
Fail



6.4.9 系统校验启动



系统启动到 kernel，kernel 首先解析 U-Boot 传递的 cmdline 参数，确认系统启动是否使用 dm-verity。然后加载启用 system 的 fs_mgr 服务。fs_mgr 依据 fsmgr_flags 的参数来校验加载固件，固件 hash & hash tree 存放于 vbmeta.img。主要有如下参数：

avb：使用 avb 的方式加载校验分区

slotselect：该分区分 A/B，加载时会使用到 cmdline 的"androidboot.slot_suffix=_a"这个参数。

6.4.10 Linux AVB

如下介绍基于linux 环境的 AVB 操作及验证流程。

6.4.10.1 操作流程

1. 生成整套固件
2. 使用 SecureBootConsole 生成 PrivateKey.pem 与 PublicKey.pem，工具为 rk_sign_tool，命令如下：

```
rk_sign_tool cc --chip 3399
rk_sign_tool kk --out .
```

3. load key

```
rk_sign_tool lk --key privateKey.pem --pubkey publicKey.pem
```

4. 签名 loader

```
rk_sign_tool sl --loader loader.bin
```

5. 签名 uboot.img & trust.img

```
rk_sign_tool si --img uboot.img
rk_sign_tool si --img trust.img
```

6. avb 签名固件准备：准备空的 temp.bin，16 字节的 product_id.bin，待签名的 boot.img，运行下列代码

```
#!/bin/bash
touch temp.bin
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 -outform PEM -out
testkey_prk.pem
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 -outform PEM -out
testkey_psk.pem
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 -outform PEM -out
testkey_pik.pem
python avbtool make_atx_certificate --output=pik_certificate.bin --
subject=temp.bin --subject_key=testkey_pik.pem --
subject_is_intermediate_authority --subject_key_version 42 --
authority_key=testkey_prk.pem
python avbtool make_atx_certificate --output=psk_certificate.bin --
subject=product_id.bin --subject_key=testkey_psk.pem --subject_key_version 42 --
authority_key=testkey_pik.pem
python avbtool make_atx_metadata --output=metadata.bin --
intermediate_key_certificate=pik_certificate.bin --
product_key_certificate=psk_certificate.bin
python avbtool make_atx_permanent_attributes --output=permanent_attributes.bin -
-product_id=product_id.bin --root_authority_key=testkey_prk.pem
python avbtool add_hash_footer --image boot.img --partition_size 33554432 --
partition_name boot --key testkey_psk.pem --algorithm SHA256_RSA4096
python avbtool make_vbmeta_image --public_key_metadata metadata.bin --
include_descriptors_from_image boot.img --algorithm SHA256_RSA4096 --key
testkey_psk.pem  --output vbmeta.img
openssl dgst -sha256 -out permanent_attributes_cer.bin -sign PrivateKey.pem
permanent_attributes.bin
```

生成 vbmeta.img，permanent_attributes_cer.bin，permanent_attributes.bin。

该步骤就签名了 boot.img……

7. 固件烧写

```
rkdeveloptool db loader.bin
rkdeveloptool ul loader.bin
rkdeveloptool gpt parameter.txt
rkdeveloptool wlx uboot uboot.img
rkdeveloptool wlx trust trust.img
rkdeveloptool wlx boot boot.img
rkdeveloptool wlx system system.img
```

rkdeveloptool 可以参考https://github.com/rockchip-linux/rkdeveloptool

8. 烧写 permanent_attributes_cer.bin，permanent_attributes.bin

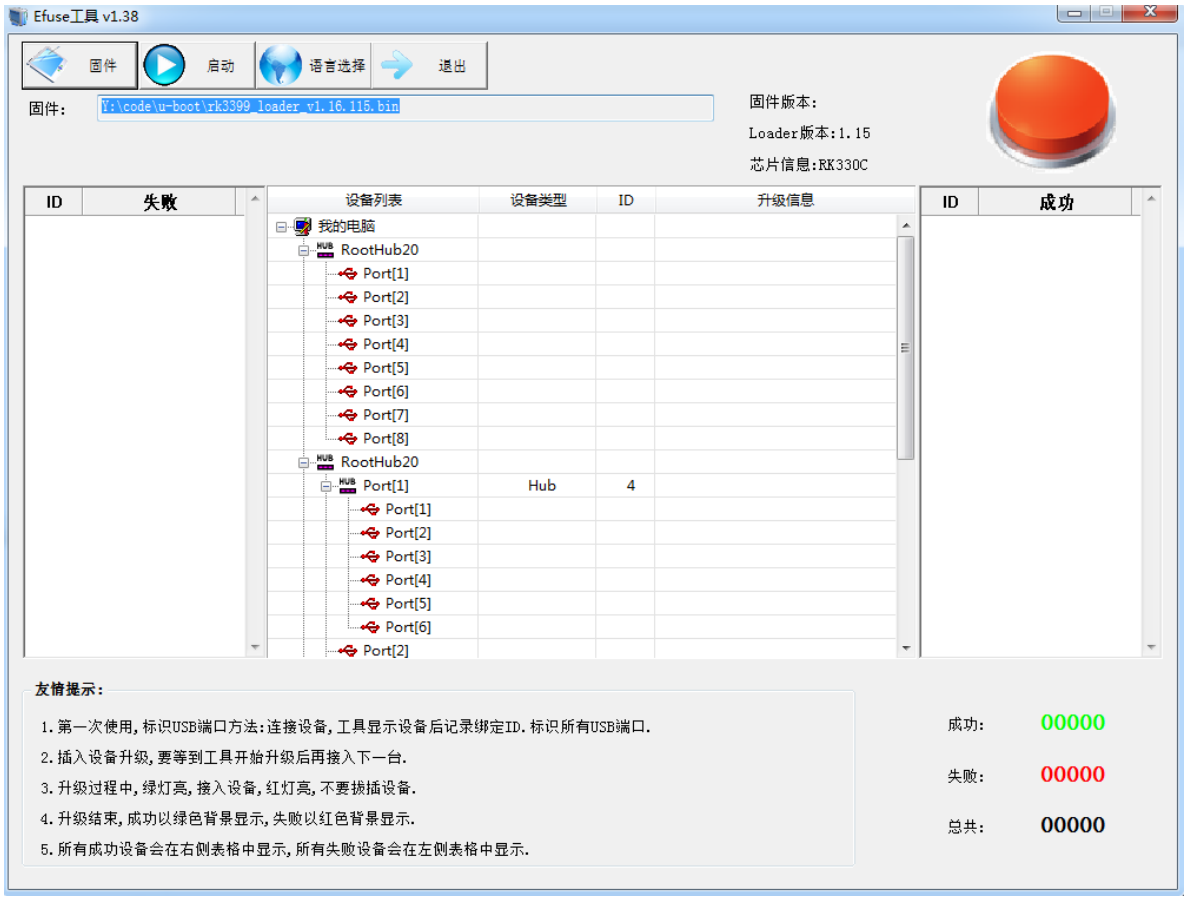
有OTP平台：

```
fastboot stage permanent_attributes.bin
fastboot oem board:fuse at-perm-attr
```

有 efuse 平台：

```
fastboot stage permanent_attributes.bin
fastboot oem board:fuse at-perm-attr
fastboot stage permanent_attributes_cer.bin
fastboot oem board:fuse at-rsa-perm-attr
```

9. efuse 烧写（efuse 工具目前只有 windows 版本），选择特定的 loader，选择对应的设备，点击启动烧写。



10. OTP 平台 loader public key 烧写

参考《Rockchip-Secure-Boot-Application-Note.md》

6.4.10.2 验证流程

验证流程待补充，建议结合量产环境补充完整测试步骤。

6.5 SD启动和升级

6.5.1 简介

Rockchip 现将 SD 卡划分为常规 SD 卡，SD 升级卡，SD 启动卡，SD 修复卡。可以通过瑞芯微创建升级磁盘工具将 update.img 下载到 SD 卡内，制作不同的卡类型。

卡类型	功能
常规 SD 卡	普通的存储设备
SD 升级卡	设备从 SD 卡内启动到 recovery，由 recovery 负责把 sd 内固件更新到设备存储
SD 启动卡	设备直接从 SD 卡启动
SD 修复卡	从 pre-loader 开始拷贝 SD 卡内的固件到设备存储

6.5.2 分类

6.5.2.1 常规卡

普通 SD 卡与 PC 使用完全一样，可以在 U-Boot 和 Kernel 系统中作为普通的存储空间使用，无需工具对 SD 卡做任何操作。

6.5.2.2 升级卡

SD 升级卡是通过 RK 的工具制作，实现 SD 卡对本地存储(如 eMMC，nand flash)固件的升级。SD 卡升级是可以脱离 PC 机或网络的一种固件升级方法。具体是将 SD 卡启动代码写到 SD 卡的保留区，然后将固件拷贝到 SD 卡可见分区上，主控从 SD 卡启动时，SD 卡启动代码和升级代码将固件烧写到本地主存储中。同时 SD 升级卡支持 PCBA 测试和 Demo 文件的拷贝。SD 升级卡的这些功能可以使固件升级做到脱离 PC 机进行，提高生产效率。
已经制作好的升级用 SD 卡，如果只需要更新固件和 demo 文件时，可以按下面步骤来完成：

1. 拷贝固件到 SD 卡根目录，并重命名为 sdupdate.img
2. 拷贝 demo 文件到 SD 卡根目录下的 Demo 目录中

SD 引导升级卡格式(非 GPT)

偏移	数据段
扇区 0	MBR
扇区 64-4M	IDBLOCK(启动标志置 0)
4M-8M	Parameter
12M-16M	uboot
16M-20M	trust
……	misc
……	resource
……	kernel
……	recovery
剩下空间	Fat32 存放 update.img

SD 引导升级卡格式(GPT)

偏移	数据段
扇区 0	MBR
扇区 1-34	GPT 分区表
扇区 64-4M	IDBLOCK(启动标志置 0)
4M-8M	Parameter
……	uboot
……	trust
……	misc
……	resource
……	kernel
……	recovery
剩下空间	Fat32 存放 update.img

6.5.2.3 启动卡

SD 启动卡是通过 RK 的工具制作，实现直接从 SD 卡启动，极大的方便用户更新启动新固件而不用重新烧写固件到设备存储内。具体实现是将固件烧写到 SD 卡中，把 SD 卡当作主存储使用。主控从 SD 卡启动时，固件以及临时文件都存放在 SD 卡上，有没有本地主存储都可以正常工作。目前主要用于设备系统从 SD 卡启动，或用于 PCBA 测试。**注意**：PCBA 测试只是 recovery 下面的一个功能项，可用于升级卡与启动卡。

SD 引导启动卡格式(非 GPT)

偏移	数据段
扇区 0	MBR
扇区 64-4M	IDBLOCK(启动标志置 1)
4M-8M	Parameter
8M-12M	uboot
12M-16M	trust
……	misc
……	resource
……	boot
……	kernel
……	recovery
……	system
……	user

SD 引导启动卡格式(GPT)

偏移	数据段
扇区 0	MBR
扇区 1-34	GPT 分区表
扇区 64-4M	IDBLOCK(启动标志置 1)
……	uboot
……	Boot
……	trust
……	resource
……	kernel
……	recovery
……	system
……	vendor
……	oem
……	user
最后 33 扇区	备份 GPT

6.5.2.4 修复卡

SD 修复卡类似于 SD 卡升级功能，但固件升级工作由 miniloader完成。首先工具会将启动代码写到 SD 卡的保留区，然后将固件拷贝到 SD 卡可见分区上，主控从 SD 卡启动时，SD 卡升级代码将固件升级到本地主存储中。主要用于设备固件损坏，SD 卡可以修复设备。
SD 修复卡格式(非 GPT)

偏移	数据段
扇区 0	MBR
扇区 64-4M	IDBLOCK(启动标志置 2)
4M-8M	Parameter
8M-12M	uboot
12M-16M	trust
……	misc
……	resource
……	boot
……	kernel
……	recovery
……	system
……	user

SD 修复卡格式(GPT)

偏移	数据段
扇区 0	MBR
扇区 1-34	GPT 分区表
扇区 64-4M	IDBLOCK(启动标志置 2)
……	uboot
……	Boot
……	trust
……	resource
……	kernel
……	recovery
……	system
……	vendor
……	oem
……	user
最后 33 扇区	备份 GPT

6.5.3 固件标志

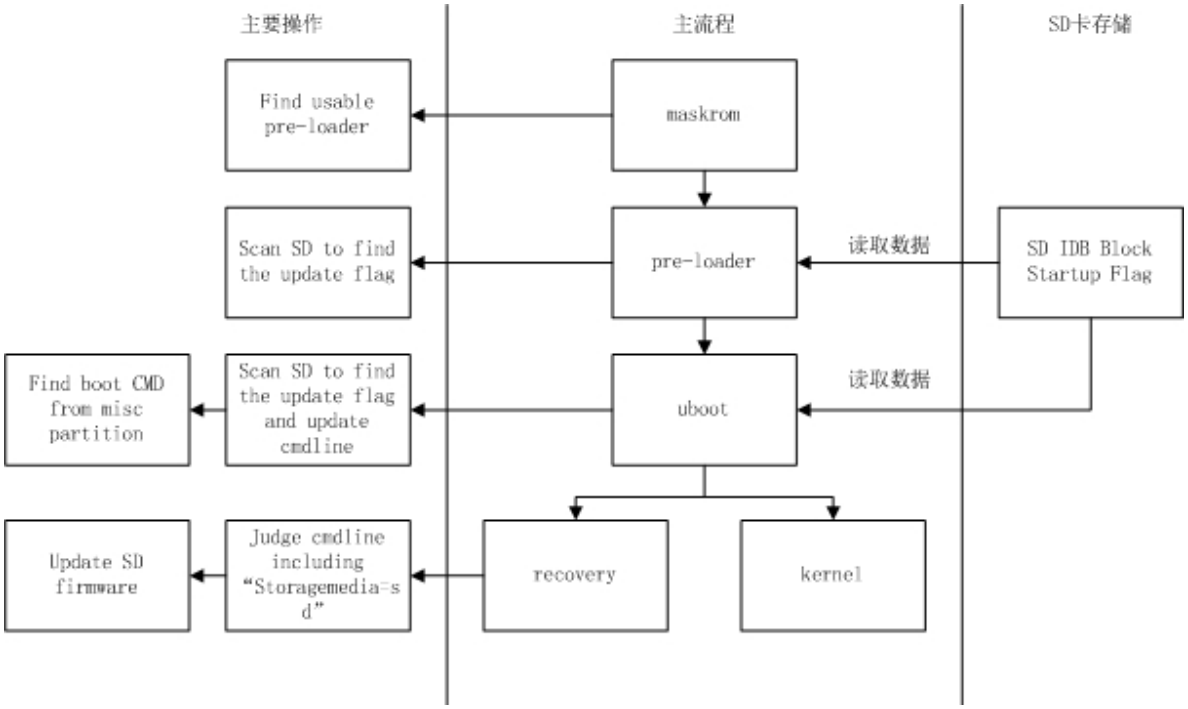
SD 卡作为各种不同功能的卡，会在 sd 卡内做一些标志。

在 SD 卡的第 64 扇区处，起始标志若为（magic number）为 0xFCDC8C3B，则为一些特殊卡，会从 SD 卡内读取固件，启动设备。如果不是，则作为普通 SD 卡看待。在第（64 扇区 + 616bytes）地方，存放各种卡的标志。目前有三种类型：

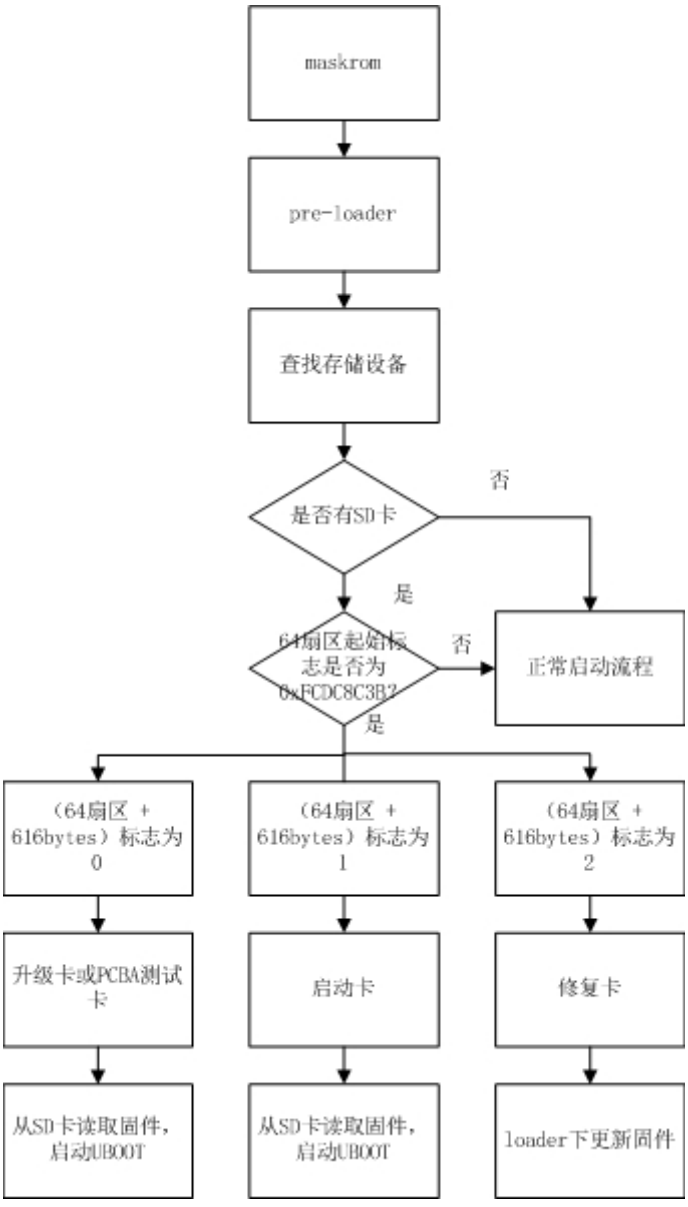
标志	卡类型
0	升级卡或 PCBA 测试卡
1	启动卡
2	修复卡

6.5.4 启动流程

SD 卡的 boot 流程可分为 pre-loader 启动流程与 uboot 启动流程，这两个流程都需要加载检测 SD 卡及 SD 卡内 IDB Block 内 Startup Flag 标志，并且会依据这些标志执行不同的功能。流程如下：

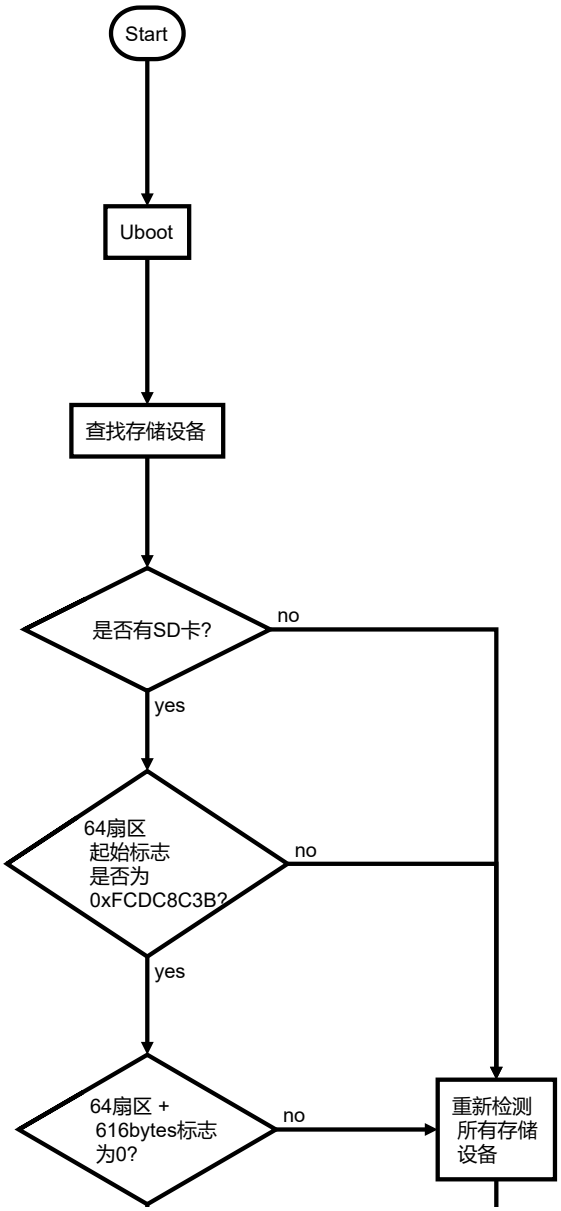


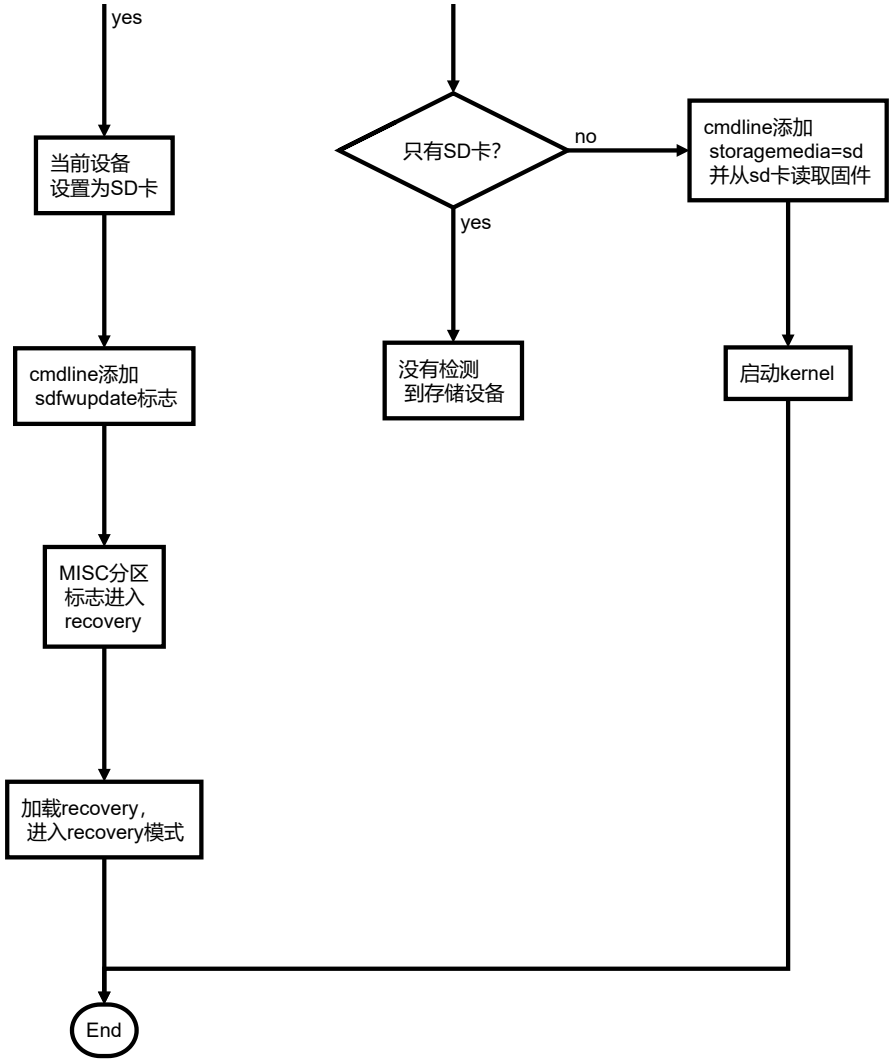
6.5.4.1 pre-loader 启动



maskrom 首先先找到一份可用的 miniloader 固件（可以从 TRM 确定 Maskrom 支持的启动存储介质和优先顺序，maskrom 会依次扫描可用存储里的固件），然后跳转到 miniloader。miniloader 重新查找存储设备，如果检测到 SD 卡，检测 SD 卡是否包含 IDB 格式固件。如果是，再判断卡标志。如果 SD 卡可用且标志位为 '0' 或 ‘1’，则从 SD 卡内读取 U-Boot 固件，加载启动 U-Boot。如果标志为‘2’，则进入修复卡流程，在 loader 下更新固件。正常启动流程为扫描其他存储，加载启动下级 loader。

6.5.4.2 U-Boot 启动





升级卡：U-Boot 重新查找存储设备，如果检测到 SD 卡，检测 SD 卡是否包含 IDB 格式固件。如果是，再判断卡标志是否为 0，传递给 kernel 的 cmdline 添加'sdfwupdate'。最后读取 SD 卡的 misc 分区，读取卡启动模式，若为 recovery 模式，加载启动 recovery。
启动卡：U-Boot 重新查找存储设备，如果检测到 SD 卡，检测 SD 卡是否包含 IDB 格式固件。如果是，再判断卡标志是否为 1。最后读取 SD 卡的 misc 分区，读取卡启动模式，如果为 recovery，加载启动 recovery。如果是 normal 模式，则加载启动 kernel。

6.5.4.3 Recovery 和 PCBA

具体可参考《Rockchip Recovery 用户操作指南 V1.03.pdf》。

6.5.5 注意事项

- 制作非GPT 格式固件时，U-Boot 需要配置 CONFIG_RKPARM_PARTITION。
- 制作 SD 升级卡时，update.img 必须包含 MiniloaderAll.bin，parameter.txt，uboot.img，trust.img，misc.img，resource.img，recovery.img 这些固件，否则烧写 update.img 会出现写入 MBR 失败的提示。

7. Chapter-7 配置裁剪

TODO

8. Chapter-8 调试手段

本章节主要介绍一些 U-Boot 阶段常用的调试手段，包括使用命令、配置选项、开机打印等。

8.1 DEBUG

功能：让全局 `debug()` 打印生效。

可在各平台的 rkxxx_common.h 中增加宏定义进行使能：

```
#define DEBUG
```

8.2 Initcall

功能：打印启动流程。

U-Boot 的启动实质是一系列 initcall 调用，把 `initcall_run_list()` 函数内的 `debug()` 改成 `printf()`。例：

```
U-Boot v5-01725-g03b8d3b-dirty (Jul 06 2018 - 10:08:27 +0800)

initcall: 0000000000214388
initcall: 0000000000214724
Model: Rockchip RK3399 Evaluation Board
initcall: 0000000000214300
DRAM:  initcall: 0000000000203f68
initcall: 0000000000214410  // 结合反汇编找出地址对应的函数
initcall: 00000000002140dc
....
3.8 GiB
initcall: 00000000002143b8
....
Relocation Offset is: f5c03000
initcall: 00000000f5e176bc
initcall: 00000000002146a4 (relocated to 00000000f5e176a4)
initcall: 0000000000214668 (relocated to 00000000f5e17668)

...
```

8.3 io 命令

功能：读写内存

```
// 读操作
md - memory display
Usage: md [.b, .w, .l, .q] address [# of objects]

// 写操作
mw - memory write (fill)
Usage: mw [.b, .w, .l, .q] address value [count]
```

读操作。范例：显示 0x76000000 地址开始的连续 0x10 个数据。

```
=> md.l 0x76000000 0x10
76000000: fffffffe ffffffff ffffffff ffffffff    ................
76000010: fffffdf ffffffff feffffff ffffffff    ................
76000020: ffffffff ffffffff ffffffff ffffffff    ................
76000030: ffffffff ffffffff ffffffff ffffffff    ................
```

写操作。范例：对 0x76000000 地址赋值为 0x1234；

```
=> mw.l 0x76000000 0xffff1234 // 高16位有mask
=> md.l 0x76000000 0x10       // 回读
76000000: ffff1234 ffffffff ffffffff ffffffff    ................
76000010: fffffdf ffffffff feffffff ffffffff    ................
76000020: ffffffff ffffffff ffffffff ffffffff    ................
76000030: ffffffff ffffffff ffffffff ffffffff    ................
```

8.4 iomem 命令

功能：读内存。比 md 命令更灵活，通过自动解析 DTS 节点获取基地址信息。

```
=> iomem
iomem - Show iomem data by device compatible

Usage:
// @<compatible>：节点的compatible部分关键词匹配
iomem <compatible> <start offset>  <end offset>
eg: iomem -grf 0x0 0x200
```

范例：RK3228 读取 GRF 里 0x00 ~ 0x20 的数据：

```
// 这里为了和 "rockchip,rk3288-pmugrf" 区分开，所以用 "-grf" 作为关键词
=> iomem -grf 0x0 0x20
rockchip,rk3228-grf:
11000000:  00000000 00000000 00004000 00002000
11000010:  00000000 00005028 0000a5a5 0000aaaa
11000020:  00009955
```

8.5 i2c 命令

功能：I2C 设备读写。

```
=> i2c
i2c - I2C sub-system

Usage:
i2c dev [dev] - show or set current I2C bus
i2c md chip address[.0, .1, .2] [# of objects] - read from I2C device
i2c mw chip address[.0, .1, .2] value [count] - write to I2C device (fill)
......
```

读操作。范例：

```
=> i2c dev 0                      // 切到i2c0（指定一次即可）
Setting bus to 0

=> i2c md 0x1b 0x2e 0x20          // i2c设备地址为1b(7位地址)，读取0x2e开始的连续0x20个寄存器值
002e: 11 0f 00 00 11 0f 00 00 01 00 00 00 09 00 00 0c    ................
003e: 00 0a 0a 0c 0c 0c 00 07 07 0a 00 0c 0c 00 00 00    ................
```

写操作。范例：

```
=> i2c dev 0                      // 切到i2c0（指定一次即可）
Setting bus to 0

=> i2c mw 0x1b 0x2e 0x10          // i2c设备地址为1b(7位地址)，对0x2e寄存器赋值为0x10
=> i2c md 0x1b 0x2e 0x20          // 回读
002e: 10 0f 00 00 11 0f 00 00 01 00 00 00 09 00 00 0c    ................
003e: 00 0a 0a 0c 0c 0c 00 07 07 0a 00 0c 0c 00 00 00    ................
```

8.6 gpio 命令

功能：pin 脚输入输出读写

```
=> gpio
gpio - query and control gpio pins

Usage:
gpio <input|set|clear|toggle> <pin>
    - input/set/clear/toggle the specified pin
gpio status [-a] [<bank> | <pin>]  - show [all/claimed] GPIOs
```

查看 pin 脚状态：如 RV1126

```
=> gpio status -a
Bank A:
A0: input: 0 [ ]
A1: output: 1 [ ]
A2: input: 1 [ ]
...
A29: unused: 1 [ ]
A30: unknown
A31: unused: 0 [ ]
```

```
...
D6: input: 0 [ ]
D7: output: 1 [x] vcc18-lcd-n.gpio
...
D31: input: 0 [ ]

Bank E:
E0: input: 0 [ ]
E1: input: 0 [ ]
```

pin 输入：

```
=> gpio input A7
```

pin 输出 INACTIVE：

```
=> gpio clear A7
```

pin 输出 ACTIVE：

```
=> gpio set A7
```

pin 状态切换：如 A7: input: 0 改为 A7: output: 1

```
=> gpio toggle A7
```

8.7 fdt 命令

功能：打印 DTB 内容。

```
=> fdt
fdt - flattened device tree utility commands

Usage:
fdt addr [-c]  <addr> [<length>]    - Set the [control] fdt location to <addr>
fdt print  <path> [<prop>]          - Recursive print starting at <path>
fdt list   <path> [<prop>]          - Print one level starting at <path>
......
NOTE: Dereference aliases by omitting the leading '/', e.g. fdt print ethernet0.
```

其中如下两条组合命令一起使用，可以把 device-tree 完整 dump 出来：

```
=> fdt addr $fdt_addr_r   // 指定fdt地址
=> fdt print              // 把fdt内容全部打印出来
```

8.8 mmc 命令

功能：MMC 设备读写、切换。

MMC 设备查看：

```
=> mmc info
Device: dwmmc@ff0f0000                  // 设备节点
Manufacturer ID: 15
OEM: 100
Name: 8GME4
Timing Interface: High Speed            // 速度模式
Tran Speed: 52000000                    // 当前速度
Rd Block Len: 512
MMC version 5.1
High Capacity: Yes
Capacity: 7.3 GiB                       // 存储容量
Bus Width: 8-bit                        // 总线宽度
Erase Group Size: 512 KiB
HC WP Group Size: 8 MiB
User Capacity: 7.3 GiB WRREL
Boot Capacity: 4 MiB ENH
RPMB Capacity: 512 KiB ENH
```

MMC 设备切换：

```
=> mmc dev 0                            // 切换到 eMMC
=> mmc dev 1                            // 切换到 sd 卡
```

MMC 设备读写：

```
mmc read addr blk# cnt
mmc write addr blk# cnt
mmc erase blk# cnt
例：
=> mmc read 0x70000000 0 1    // 读取 MMC 设备第一个 block，大小为 1 sector 的数据到内存 0x70000000
=> mmc write 0x70000000 0 1   // 把内存 0x70000000 起 1 sector 的数据写到存储第一个 block 起位置
=> mmc erase 0 1              // 擦除存储第一个 block 起 1 sector 数据
```

如果 MMC 设备读写异常，可以通过以下简单步骤快速定位：

把 drivers/mmc/dw_mmc.c 内的 `debug()` 改为 `printf()` 后重新编译烧写。查看 MMC 设备的打印信息：

- 如果最后的打印为 Sending CMD0，请检查硬件供电、管脚连接；检查软件 IOMUX 是否被其他 IP 切走；
- 如果最后打印为 Sending CMD8，安全软件部分请配置 MMC 设备允许访问安全存储；
- 如果初始化命令都已通过，最后打印为 Sending CMD18，请检查 MMC 硬件供电、靠近 MMC 供电端的电容是否足够（可以更换大电容）、软件可以降低时钟频率、切换 MMC 设备的速度模式。

8.9 时间戳

功能：给 U-Boot 打印信息增加时间戳（相对时间）。

在 defconfig 中添加：

```
CONFIG_BOOTSTAGE_PRINTF_TIMESTAMP=y
```

范例：

```
[    0.259266] U-Boot v5-01739-g856f373-dirty (Jul 10 2018 - 20:26:05 +0800)
[    0.260596] Model: Rockchip RK3399 Evaluation Board
[    0.261332] DRAM:  3.8 GiB
Relocation Offset is: f5bfd000
Using default environment

[    0.354038] dwmmc@fe320000: 1, sdhci@fe330000: 0
[    0.521125] Card did not respond to voltage select!
[    0.521188] mmc_init: -95, time 9
[    0.671451] switch to partitions #0, OK
[    0.671500] mmc0(part 0) is current device
[    0.675507] boot mode: None
[    0.683738] DTB: rk-kernel.dtb
[    0.706940] Using kernel dtb
......
```

时间戳打印的是系统 timer 的当前计数值。由于 timer 在 U-Boot 启动前已经开始运行，因此该值并非从 0 起始，只表示相对时间，不能作为绝对时间使用。

8.10 dm tree

功能：查看所有 device-driver 之间的绑定、probe 状态。

```
=> dm tree

 Class      Probed        Driver               Name
-----------------------------------------------------------
 root       [ + ]   root_driver               root_driver
 syscon     [   ]   rk322x_syscon             |-- syscon@11000000
 serial     [ + ]   ns16550_serial            |-- serial@11030000 *
 clk        [ + ]   clk_rk322x                |-- clock-controller@110e0000
 sysreset   [   ]   rockchip_sysreset         |   |-- sysreset
 reset      [   ]   rockchip_reset            |   `-- reset
 mmc        [ + ]   rockchip_rk3288_dw_mshc   |-- dwmmc@30020000 *
 blk        [ + ]   mmc_blk                   |   `-- dwmmc@30020000.blk *
 ram        [   ]   rockchip_rk322x_dmc       |-- dmc@11200000
 serial     [ + ]   ns16550_serial            |-- serial@11020000
 i2c        [ + ]   i2c_rockchip              |-- i2c@11050000
 ......
```

打印含义：

- 列出所有已经完成 bind 的 device-driver
- 列出所有 uclass-device-driver 之间的隶属关系
- [+] 表示当前 driver 已经完成 probe
- * 表示当前 device-driver 来自于 U-Boot 的 DTB，否则来自 kernel DTB

8.11 dm uclass

功能：查看某类 uclass 下的所有设备。

```
=> dm uclass

uclass 0: root
- * root_driver @ 7be54c88, seq 0, (req -1)

uclass 11: adc
- * saradc@ff100000 @ 7be56220, seq 0, (req -1)
......
uclass 40: backlight
- * backlight @ 7be81178, seq 0, (req -1)

uclass 77: key
-   rockchip-key @ 7be811f0
......
```

8.12 stacktrace.sh

利用调用栈回溯机制分析 abort、dump_stack() 的现场。请参考“RK 架构”章节中的 `Stacktrace` 小节。

8.13 系统卡死

功能：打印当前 CPU 的现场和调用栈，适用于系统卡死时使用。串口会每隔 5s dump 出和 abort 时类似的信息。

```
CONFIG_IRQ_TIMER_DUMP
```

获取到调用栈信息后再使用 stacktrace 脚本转换。请参考“RK 架构”章节中的 `Stacktrace` 小节。

在 defconfig 中添加：

```
CONFIG_IRQ_TIMER_DUMP=y
```

8.14 修改 DDR 容量

开机时 DDR 初始化代码会把 DDR 容量传递给 U-Boot，U-Boot 会去除一些安全内存后再传递给内核。用户可以在 U-Boot 阶段修改传递给内核的 DDR 容量。

传递范例：

```
......
// 传递给内核的可用内存块（已去除安全内存块）。
Adding bank: 0x00200000 - 0x08400000 (size: 0x08200000)
Adding bank: 0x0a200000 - 0x40000000 (size: 0x35e00000)
Total: 895.411 ms

Starting kernel ...
[    0.000000] Booting Linux on physical CPU 0x0
```

代码位置：

```
./arch/arm/mach-rockchip/param.c
```

修改位置：

```
struct memblock *param_parse_ddr_mem(int *out_count)
{
    ......

    // 这里就是ddr传递给U-Boot的容量信息。
    // 因为可能出现不连续的地址，所以会分块传递，分别指明各个内存块的起始地址和大小。
    // PS: 一般情况下都是连续内存，不会需要分块。
    for (i = 0, n = 0; i < count; i++, n++) {
        // 比如2GB容量（连续地址），则：count=1, base = 0, size = 0x80000000。
        // 用户调试时可以在这里按需修改。
        base = t->u.ddr_mem.bank[i];
        size = t->u.ddr_mem.bank[i + count];

        /* 0~4GB */
        if (base < SZ_4GB) {
            mem[n].base = base;
            mem[n].size = ddr_mem_get_usable_size(base, size);
            if (base + size > SZ_4GB) {
                n++;
                mem[n].base_u64 = SZ_4GB;
                mem[n].size_u64 = base + size - SZ_4GB;
            }
        } else {
            /* 4GB+ */
            mem[n].base_u64 = base;
            mem[n].size_u64 = size;
        }

        assert(n < count + MEM_RESV_COUNT);
    }
    ......
}
```

8.15 跳转信息

功能：确认固件版本和流程。某些情况下，开机信息也可以帮助用户定位一些死机问题。

1. trust 跑完后就卡死

trust 跑完后就卡死的可能性：固件打包或者烧写有问题，导致 trust 跳转到错误的 U-Boot 启动地址。此时用户可以通过 trust 开机打印的 U-Boot 启动地址来确认。

64 位平台 U-Boot 启动地址一般是偏移 0x200000（DRAM 起始地址是 0x0）：

```
NOTICE:  BL31: v1.3(debug):d98d16e
NOTICE:  BL31: Built : 15:03:07, May 10 2018
NOTICE:  BL31: Rockchip release version: v1.1
INFO:    GICv3 with legacy support detected. ARM GICV3 driver initialized in EL3
INFO:    Using opteed sec cpu_context!
INFO:    boot cpu mask: 0
INFO:    plat_rockchip_pmu_init(1151): pd status 3e
INFO:    BL31: Initializing runtime services
INFO:    BL31: Initializing BL32
INFO:    BL31: Preparing for EL3 exit to normal world
INFO:    Entry point address = 0x200000  // U-Boot地址
INFO:    SPSR = 0x3c9
```

32 位平台 U-Boot 启动地址一般是偏移 0x0（DRAM 起始地址是 0x60000000）：

```
INF [0x0] TEE-CORE:init_primary_helper:378: Release version: 1.9
INF [0x0] TEE-CORE:init_primary_helper:379: Next entry point address: 0x60000000
// U-Boot地址
INF [0x0] TEE-CORE:init_teecore:83: teecore inits done
```

2. U-Boot 版本回溯：

通过 U-Boot 开机信息可回溯编译版本。如下对应提交点是 commit: b34f08b。

```
U-Boot v5-01730-gb34f08b (Jul 06 2018 - 17:47:52 +0800)
```

开机信息中出现"dirty"，说明编译时有本地改动没有提交进仓库，编译点不干净。

```
U-Boot v5-01730-gb34f08b-dirty (Jul 06 2018 - 17:35:04 +0800)
```

8.16 启动信息

用户通过 U-Boot 开机信息可获知当前 U-Boot 的流程和各外设的状态，方便快速定位异常。

目前 U-Boot 支持三种固件类型引导：Android 格式 > FIT 格式 > DISTRO 格式。RK 发布的 SDK 主要是前两种固件格式，DISTRO 一般是开源用户会使用。

说明：如果用户的代码不够新，有些打印可能看不到，这不影响用户对于 U-Boot 开机信息的整体了解。

8.16.1 Android 固件

```
// U-Boot 第一行打印，包含了 commit 版本、编译时间等信息
// 注意：这里只是U-Boot"相对早"的第一行正规打印，而不是U-Boot能做出的最早打印
// 打开debug信息，可以看到更早的调试打印
U-Boot v5-03033-g81b79f7-dirty (Jul 04 2019 - 15:04:00 +0800)

// U-Boot dts的"model"字段内容，通过这个信息可以知道我们使用了U-Boot哪份dts
Model: Rockchip RK3399 Evaluation Board
// 启用了preloader-serial功能，即沿用前级loader的串口配置，当前使用UART2作为打印口
PreSerial: 2
```

```
// 板子的总内存容量是2GB
DRAM: 2 GiB
// 当前版本支持了sysmem内存卡管理机制
Sysmem: init
// U-Boot会把自己的代码进行自搬移，从当前ddr靠前的位置搬到靠后的位置（详见U-Boot开发文档启动流程）
// 自搬移后的代码起始地址是0x7dbe2000，这个信息在反汇编调试时可能有用到
Relocation Offset is: 7dbe2000
// ENV默认保存在ddr里。如果是选择保存在eMMC、Nand等存储介质里，则不会有这个打印
Using default environment

// 当前的存储介质是mmc0，即eMMC(如果是sd卡，则为mmc1)
dwmmc@fe320000: 1, sdhci@fe330000: 0
// 存储介质类型是通过atags，由前级miniloader传参告知U-Boot
Bootdev(atags): mmc 0
// 当前的eMMC的工作在HS400模式，时钟频率是150M
MMC0: HS400, 150Mhz
// 当前采用GPT分区表（如果是RK parameter分区表，则打印：RKPARM）
PartType: EFI
// 当前是recovery模式
// kernel里执行的"reboot xxx"命令，最终也是由这个打印来体现
boot mode: recovery
// Kernel DTB来自于recovery.img，其正常被加载
Load FDT from recovery part
DTB: rk-kernel.dtb
HASH: OK(c)

// ==> 注意：自此之后，U-Boot已经切到kernel dtb，后续所有外设驱动都使用kernel dtb的信息！！

// DTBO执行成功
ANDROID: fdt overlay OK
// I2C的速度，这个是U-Boot开机速度的影响因素之一，尤其对于DCDC和LDO非常多的PMIC，如果I2C速度慢，
// 那么对开机速度有一定影响。如果用户关心开机速度，可以关注这个信息
I2c speed: 400000Hz
// 当前PMIC是RK818
// on值对应ON_SOURCE寄存器，表明了当前这次PMIC上电的原因
// off值对应OFF_SOURCE寄存器，表明前一次关机或掉电的原因
// on和off信息，对于系统出现异常重启或关机等情况时，这是个有价值的信息
PMIC: RK818 (on=0x20 off=0x40)
// 各路可调压regulator的当前电压值，一般是DCDC且对应的是RK平台的arm、logic、center等电压
// 在出现系统启动异常、开机不稳定等问题时，这是个有价值的信息vdd_center 900000 uV
vdd_cpu_l 900000 uV
vdd_log 900000 uV
// Kernel dts的"model"字段内容，通过这个信息可以知道我们使用了Kernel的哪份dts
Model: Rockchip RK3399 Excavator Board edp avb (Android)
enter Recovery mode!
// 显示驱动的相关信息
Rockchip UBOOT DRM driver version: v1.0.1
Using display timing dts
Detailed mode clock 200000 kHz, flags[a]
  H: 1536 1548 1564 1612
  V: 2048 2056 2060 2068
bus_format: 100e
// clk-tree信息，具体含义请参考U-Boot开发文档的CLK章节
CLK: (uboot. arml: enter 816000 KHz, init 816000 KHz, kernel 0N/A)
CLK: (uboot. armb: enter 24000 KHz, init 24000 KHz, kernel 0N/A)
```

```
 aplll 816000 KHz
 apllb 24000 KHz
 dpll 800000 KHz
 cpll 200000 KHz
 gpll 800000 KHz
 npll 600000 KHz
 vpll 24000 KHz
 aclk_perihp 133333 KHz
 hclk_perihp 66666 KHz
 pclk_perihp 33333 KHz
 aclk_perilp0 266666 KHz
 hclk_perilp0 88888 KHz
 pclk_perilp0 44444 KHz
 hclk_perilp1 100000 KHz
 pclk_perilp1 50000 KHz
// GMAC驱动使能
Net:  eth0: ethernet@fe300000
// 开机长按ctrl+c，可在如下打印之后进入U-Boot命令行模式
Hit key to stop autoboot('CTRL+C'): 0
// 再一次知道当前是recovery模式
ANDROID: reboot reason: "recovery"
// vboot=0表示没有启用secureboot；当前是AVB固件，所以会走AVB的常规校验流程
Vboot=0, AVB images, AVB verify
// 设备是否unlock
read_is_device_unlocked() ops returned that device is UNLOCKED
// 原生的U-Boot默认是把整个boot.img/recovery.img加载起来，然后再把ramdisk、fdt、kernel再进行
// 一次搬移（称为relocation），搬到用户预定的地址上，这样是比较耗时的，尤其当ramdisk非常大的时候。
// RK平台做了修改，一次性直接从存储上把ramdisk、fdt、kernel搬到预定的内存地址。
// 有如下打印则说明启用了这种一次性搬移的操作，更省时间
Fdt Ramdisk skip relocation

// 加载Android格式的固件，把kernel加载到0x00280000，fdt加载到0x8300000
// 假如是LZ4压缩内核，这里可能打印：
// Booting LZ4 kernel at 0x00680000(Uncompress to 0x00280000) with fdt at 0x8300000...
Booting IMAGE kernel at 0x00280000 with fdt at 0x8300000...

// 忽略，可不关心
### Booting Android Image at 0x0027f800 ...
// kernel和ramdisk的加载地址以及大小
Kernel load addr 0x00280000 size 19081 KiB
RAM disk load addr 0x0a200000 size 9627 KiB
// fdt的加载地址
### Flattened Device Tree blob at 08300000
   Booting using the fdt blob at 0x8300000
// 忽略，可不关心
   XIP Kernel Image ... OK
// 这里仅仅是打印kernel dts指定的reserved-memory，可作为出内核启动出问题时的一个分析信息
   'reserved-memory' secure-memory@20000000: addr=20000000 size=10000000
// fdt的起始和结束地址
   Using Device Tree in place at 0000000008300000, end 000000000831c6f7
// 传递给内核，告知内核可使用的内存空间范围（ATF、optee等空间已经被除去）
Adding bank: 0x00200000 - 0x08400000 (size: 0x08200000)
Adding bank: 0x0a200000 - 0x80000000 (size: 0x75e00000)
// U-Boot阶段开机耗时
Total: 367.128 ms
```

```
// 由U-Boot打印，这个打印之后，U-Boot会完成一些ARM架构相关（比如：清cache、关中断、
// cpu状态切换等）和U-Boot的dm设备注销等清零工作，出问题的概率极低。
// 完成上述工作后就跳到kernel，因此也可以理解为，出现这个打印就是到了内核阶段。
Starting kernel ...

// kernel阶段的打印信息
[    0.000000] Booting Linux on physical CPU 0x0
[    0.000000] Initializing cgroup subsys cpuset
[    0.000000] Initializing cgroup subsys cpu
[    0.000000] Initializing cgroup subsys cpuacct
[    0.000000] Initializing cgroup subsys schedtune
[    0.000000] Linux version 4.4.167 (hgc@ubuntu) (gcc version 6.3.1 20170404
(Linaro
GCC 6.3-2017.05) ) #83 SMP PREEMPT Thu Mar 21 09:31:08 CST 2019
[    0.000000] Boot CPU: AArch64 Processor [410fd034]
[    0.000000] earlycon: Early serial console at MMIO32 0xff1a0000 (options '')
[    0.000000] bootconsole [uart0] enabled
[    0.000000] Reserved memory: failed to reserve memory for node 'stb-
devinfo@00000000': base 0x0000000000000000, size 0 MiB
[    0.000000] cma: Reserved 16 MiB at 0x000000007f000000
......
```

9. Chapter-9 测试用例

TODO

10. Chapter-10 SPL

10.1 固件引导

SPL 的作用是代替 miniloader 完成 `trust.img` 和 `uboot.img` 的加载与引导工作。SPL 目前支持引导以下两种固件：

- FIT 固件：默认使能；
- RKFW 固件：默认关闭，需要用户单独配置和使能；

10.1.1 FIT 固件

FIT（flattened image tree）格式是 SPL 支持的一种比较新颖的固件格式，支持多个 image 打包和校验。FIT 使用 DTS 的语法对打包的 image 进行描述，描述文件为 u-boot.its，最终生成的 FIT 固件为 u-boot.itb。

FIT 的优点是复用 DTS 的语法和编译规则，描述方式更灵活，固件解析也可以直接使用 `libfdt` 库。

u-boot.its 文件：

- `/images`：静态定义了所有可获取的资源配置（最后可用、可不用），类似 dtsi 的角色；
- `/configurations`：每个 config 节点描述了一套可启动的配置，类似一个板级 dts。
- 使用 `default =` 指定当前选用的默认配置；

范例：

```
/dts-v1/;

/ {
    description = "Configuration to load ATF before U-Boot";
    #address-cells = <1>;

    images {
        uboot@1 {
            description = "U-Boot (64-bit)";
            data = /incbin/("u-boot-nodtb.bin");
            type = "standalone";
            os = "U-Boot";
            arch = "arm64";
            compression = "none";
            load = <0x00200000>;
        };

        atf@1 {
            description = "ARM Trusted Firmware";
            data = /incbin/("bl31_0x00010000.bin");
            type = "firmware";
            arch = "arm64";
            os = "arm-trusted-firmware";
            compression = "none";
            load = <0x00010000>;
```

```
            entry = <0x00010000>;
        };

        atf@2 {
            description = "ARM Trusted Firmware";
            data = /incbin/("bl31_0xff091000.bin");
            type = "firmware";
            arch = "arm64";
            os = "arm-trusted-firmware";
            compression = "none";
            load = <0xff091000>;
        };

        optee@1 {
            description = "OP-TEE";
            data = /incbin/("bl32.bin");
            type = "firmware";
            arch = "arm64";
            os = "op-tee";
            compression = "none";
            load = <0x08400000>;
        };

        fdt@1 {
            description = "rk3328-evb.dtb";
            data = /incbin/("arch/arm/dts/rk3328-evb.dtb");
            type = "flat_dt";
            compression = "none";
        };
    };

    configurations {
        default = "config@1";
        config@1 {
            description = "rk3328-evb.dtb";
            firmware = "atf@1";
            loadables = "uboot@1", "atf@2", "optee@1" ;
            fdt = "fdt@1";
        };
    };
};
```

u-boot.itb 文件：

```
                          mkimage + dtc
[u-boot.its] + [images]         ==>          [u-boot.itb]
```

上述展示了 itb 文件的生成过程。FIT 固件可以理解为一种特殊的 DTB 文件，只是其内容是 image。用户可以通过 `fdtdump` 命令查看 itb 文件：

```
cjh@ubuntu:~/uboot-nextdev/u-boot$ fdtdump u-boot.itb | less

/dts-v1/;
// magic:               0xd00dfeed
// totalsize:           0x497 (1175)
// off_dt_struct:       0x38
```

```
// off_dt_strings:      0x414
// off_mem_rsvmap:      0x28
// version:             17
// last_comp_version:   16
// boot_cpuid_phys:     0x0
// size_dt_strings:     0x83
// size_dt_struct:      0x3dc

/ {
    timestamp = <0x5d099c85>;
    description = "Configuration to load ATF before U-Boot";
    #address-cells = <0x00000001>;
    images {
        uboot@1 {
            data-size = <0x0009f8a8>;
            data-offset = <0x00000000>;
            description = "U-Boot (64-bit)";
            type = "standalone";
            os = "U-Boot";
            arch = "arm64";
            compression = "none";
            load = <0x00600000>;
        };
        atf@1 {
            data-size = <0x0000c048>;   // 编译过程自动增加了该字段，描述atf@1固件大小
            data-offset = <0x0009f8a8>; // 编译过程自动增加了该字段，描述atf@1固件偏移
            description = "ARM Trusted Firmware";
            type = "firmware";
            arch = "arm64";
            os = "arm-trusted-firmware";
            compression = "none";
            load = <0x00010000>;
            entry = <0x00010000>;
        };
        atf@2 {
            data-size = <0x00002000>;
            data-offset = <0x000ab8f0>;
            description = "ARM Trusted Firmware";
            type = "firmware";
            arch = "arm64";
            os = "arm-trusted-firmware";
            compression = "none";
            load = <0xfff82000>;
        };
        fdt@1 {
            data-size = <0x00005793>;
            data-offset = <0x000ad8f0>;
            description = "rk3308-evb.dtb";
            type = "flat_dt";
            ......
        };
        ......
    };
};
```

更多 FIT 信息请参考：

```
./doc/uImage.FIT/
```

10.1.2 RKFW 固件

为了更直接地替换 miniloader，同时避免修改后级固件的分区和打包格式，RK 平台增加了对 RKFW 格式的引导支持（即独立分区的固件：`trust.img` 和 `uboot.img`）。

配置：

```
CONFIG_SPL_LOAD_RKFW            // 使能开关
CONFIG_RKFW_TRUST_SECTOR        // trust.img 分区地址，需要和分区表的定义保持一致
CONFIG_RKFW_U_BOOT_SECTOR       // uboot.img 分区地址，需要和分区表的定义保持一致
```

代码：

```
./include/spl_rkfw.h
./common/spl/spl_rkfw.c
```

10.1.3 存储优先级

U-Boot dts 中通过 `u-boot,spl-boot-order` 指定存储设备的启动优先级。

```
/ {
    aliases {
        mmc0 = &emmc;
        mmc1 = &sdmmc;
    };

    chosen {
        u-boot,spl-boot-order = &sdmmc, &nandc, &emmc;
        stdout-path = &uart2;
    };
    ......
};
```

10.2 编译打包

10.2.1 代码编译

U-Boot 根据**不同的编译路径** 对同一份 U-Boot 代码编译获得 SPL 固件，当编译 SPL 时会自动生成 `CONFIG_SPL_BUILD` 宏。U-Boot 会在编译完 u-boot.bin 之后继续编译 SPL，并创建独立的输出目录 `./spl/`。

```
  // 编译 u-boot
  ......
  DTC     arch/arm/dts/rk3399-puma-ddr1866.dtb
  DTC     arch/arm/dts/rv1108-evb.dtb
make[2]: `arch/arm/dts/rk3328-evb.dtb' is up to date.
```

```
  SHIPPED dts/dt.dtb
  FDTGREP dts/dt-spl.dtb
  CAT     u-boot-dtb.bin
  MKIMAGE u-boot.img
  COPY    u-boot.dtb
  MKIMAGE u-boot-dtb.img
  COPY    u-boot.bin

  // 编译 spl，有独立的 spl/ 目录
  LD      spl/arch/arm/cpu/built-in.o
  CC      spl/board/rockchip/evb_rk3328/evb-rk3328.o
  LD      spl/dts/built-in.o
  CC      spl/common/init/board_init.o
  COPY    tpl/u-boot-tpl.dtb
  CC      spl/cmd/nvedit.o
  CC      spl/env/common.o
  CC      spl/env/env.o
  .....
  LD      spl/drivers/block/built-in.o

  ......
```

编译结束后得到：

```
./spl/u-boot-spl.bin
```

10.3 系统模块

10.3.1 分区表

SPL 使用 GPT 分区表。

配置：

```
CONFIG_SPL_LIBDISK_SUPPORT=y
CONFIG_SPL_EFI_PARTITION=y
CONFIG_PARTITION_TYPE_GUID=y
```

驱动：

```
./disk/part.c
./disk/part_efi.c
```

接口：

```
int part_get_info(struct blk_desc *dev_desc, int part, disk_partition_t *info);
int part_get_info_by_name(struct blk_desc *dev_desc,
                          const char *name, disk_partition_t *info);
```

10.3.2 A/B system

SPL 支持 A/B 系统启动。

配置：

```
CONFIG_SPL_AB=y
```

驱动：

```
./common/spl/spl_ab.c
```

接口：

```
int spl_get_current_slot(struct blk_desc *dev_desc, char *partition, char
*slot);
int spl_get_partitions_sector(struct blk_desc *dev_desc, char *partition,u32
*sectors);
```

10.3.3 启动优先级

- SPL 使用 `u-boot,spl-boot-order` 定义的启动顺序，位于 rkxxxx-u-boot.dtsi：

  ```
  chosen {
  stdout-path = &uart2;
  u-boot,spl-boot-order = &sdmmc, &sfc, &nandc, &emmc;
  };
  ```

- Maskrom 的启动优先级：

  ```
   spi nor > spi nand > emmc > sd
  ```

- Pre-loader(SPL) 的启动优先级：

  ```
  sd > spi nor > spi nand > emmc
  ```

  把 sd 卡的优先级提到最高可以方便系统从 sd 卡启动。

10.3.4 ATAGS

SPL 与 U-Boot 通过 ATAGS 机制实现传参。传递的信息有：启动的存储设备、打印串口等。

配置：

```
CONFIG_ROCKCHIP_PRELOADER_ATAGS=y
```

驱动：

```
./arch/arm/include/asm/arch-rockchip/atags.h
./arch/arm/mach-rockchip/atags.c
```

接口：

```
int atags_set_tag(u32 magic, void *tagdata);
struct tag *atags_get_tag(u32 magic);
```

10.3.5 kernel boot

通常 kernel 是由 U-Boot 加载和引导，SPL 也可以支持加载 kernel。目前支持加载 android head version 2 的 boot.img，支持 RK 格式固件。

启动顺序：

```
Maskrom -> ddr -> SPL -> Trust -> Kernel
```

10.3.6 pinctrl

配置：

```
CONFIG_SPL_PINCTRL_GENERIC=y
CONFIG_SPL_PINCTRL=y
```

驱动：

```
./drivers/pinctrl/pinctrl-uclass.c
./drivers/pinctrl/pinctrl-generic.c
./drivers/pinctrl/pinctrl-rockchip.c
```

DTS 配置：

以 sdmmc 为例：

```
&pinctrl {
    bootph-all;
};

&pcfg_pull_none_4ma {
    bootph-all;
};

&pcfg_pull_up_4ma {
    bootph-all;
};

&sdmmc {
    bootph-all;
};

&sdmmc_pin {
    bootph-all;
};

&sdmmc_clk {
```

```
    bootph-all;
};

&sdmmc_cmd {
    bootph-all;
};

&sdmmc_bus4 {
    bootph-all;
};

&sdmmc_pwren {
    bootph-all;
};
```

注意事项：

SPL 启用 pinctrl 时要修改 defconfig 里的 `CONFIG_OF_SPL_REMOVE_PROPS` 定义，删除其中的 `pinctrl-0 pinctrl-names` 字段。

10.4 驱动模块

10.4.1 MMC

配置：

```
CONFIG_SPL_MMC_SUPPORT=y  // 默认已使能
```

驱动：

```
./common/spl/spl_mmc.c
```

接口：

```
int spl_mmc_load_image(struct spl_image_info *spl_image,
                       struct spl_boot_device *bootdev);
```

10.4.2 MTD block

SPL 统一 nand、spi nand、spi nor 接口到 block 层。

配置：

```
// MTD 驱动支持
CONFIG_MTD=y
CONFIG_CMD_MTD_BLK=y
CONFIG_SPL_MTD_SUPPORT=y
CONFIG_MTD_BLK=y
CONFIG_MTD_DEVICE=y

// spi nand 驱动支持
```

```
CONFIG_MTD_SPI_NAND=y
CONFIG_ROCKCHIP_SFC=y
CONFIG_SPL_SPI_FLASH_SUPPORT=y
CONFIG_SPL_SPI_SUPPORT=y

// nand 驱动支持
CONFIG_NAND=y
CONFIG_CMD_NAND=y
CONFIG_NAND_ROCKCHIP=y /* NandC v6 可根据 TRM NANDC->NANDC_NANDC_VER 寄存器确认，
0x00000801 */
//CONFIG_NAND_ROCKCHIP_V9=y  /* NandC v9 可根据 TRM NANDC->NANDC_NANDC_VER 寄存器确
认，0x56393030，例如：RK3326/PX30 为此版本 */
CONFIG_SPL_NAND_SUPPORT=y
CONFIG_SYS_NAND_U_BOOT_LOCATIONS=y
CONFIG_SYS_NAND_U_BOOT_OFFS=0x8000
CONFIG_SYS_NAND_U_BOOT_OFFS_REDUND=0x10000
// nand page size 需要按真实大小定义，如果使用容量大于等于 512MB 的 NAND，一般需要配置为
4096
#define CONFIG_SYS_NAND_PAGE_SIZE   2048

// spi nor 驱动支持
CONFIG_CMD_SF=y
CONFIG_CMD_SPI=y
CONFIG_SPI_FLASH=y
CONFIG_SF_DEFAULT_MODE=0x1
CONFIG_SF_DEFAULT_SPEED=50000000
CONFIG_SPI_FLASH_GIGADEVICE=y
CONFIG_SPI_FLASH_MACRONIX=y
CONFIG_SPI_FLASH_WINBOND=y
CONFIG_SPI_FLASH_MTD=y
CONFIG_ROCKCHIP_SFC=y
CONFIG_SPL_SPI_SUPPORT=y
CONFIG_SPL_MTD_SUPPORT=y
CONFIG_SPL_SPI_FLASH_SUPPORT=y
```

驱动：

```
./common/spl/spl_mtd_blk.c
```

接口：

```
int spl_mtd_load_image(struct spl_image_info *spl_image,
                       struct spl_boot_device *bootdev);
```

10.4.3 OTP

用于存储不可更改数据，secure boot 中用到。

配置：

```
CONFIG_SPL_MISC=y
CONFIG_SPL_ROCKCHIP_SECURE_OTP=y
```

驱动：

```
./drivers/misc/misc-uclass.c
./drivers/misc/rockchip-secure-otp.S
```

接口：

```
int misc_read(struct udevice *dev, int offset, void *buf, int size);
int misc_write(struct udevice *dev, int offset, void *buf, int size);
```

11. Chapter-11 TPL

TODO

12. Chapter-12 FIT

12.1 前言

本章节将介绍 FIT 格式和基于 FIT 格式的安全/非安全启动方案细节。本章节为了便于介绍，全文主要以 boot.img 为说明和操作对象，但是同样适用于 recovery.img。

12.2 简介

12.2.1 基础介绍

FIT（flattened image tree）是 U-Boot 支持的一种新固件类型的引导方案，支持任意多个 image 打包和校验。FIT 使用 its (image source file) 文件描述 image 信息，最后通过 mkimage 工具生成 itb (flattened image tree blob) 镜像。its 文件使用 DTS 的语法规则，非常灵活，可以直接使用 libfdt 库和相关工具。

FIT 是 U-Boot 默认支持且主推的固件格式，SPL 和 U-Boot 阶段都支持对 FIT 格式的固件引导。更多信息请参考：

```
./doc/usage/fit/
```

因为官方的 FIT 功能无法满足实际产品需求，所以 RK 平台对 FIT 进行了适配和优化。因此 FIT 方案中必须使用 RK U-Boot 编译生成的 mkimage 工具，不能使用 PC 自带的 mkimage。

12.2.2 范例介绍

如下以u-boot.its和u-boot.itb作为范例进行介绍。

- `/images`：静态定义了所有的资源，相当于一个 dtsi文件；
- `/configurations`：每个 config 节点都描述了一套可启动的配置，相当于一个板级dts文件。
- `default =`：指明默认启用的config；

```
/dts-v1/;

/ {
    description = "Simple image with OP-TEE support";
    #address-cells = <1>;

    images {
        uboot {
            description = "U-Boot";
            data = /incbin/("./u-boot-nodtb.bin");
            type = "standalone";
            os = "U-Boot";
            arch = "arm";
            compression = "none";
            load = <0x00400000>;
```

```
            hash {
                algo = "sha256";
            };
        };
        optee {
            description = "OP-TEE";
            data = /incbin/("./tee.bin");
            type = "firmware";
            arch = "arm";
            os = "op-tee";
            compression = "none";
            load = <0x8400000>;
            entry = <0x8400000>;
            hash {
                algo = "sha256";
            };
        };
        fdt {
            description = "U-Boot dtb";
            data = /incbin/("./u-boot.dtb");
            type = "flat_dt";
            compression = "none";
            hash {
                algo = "sha256";
            };
        };
    };

    // configurations 节点下可以定义任意多个不同的conf节点，但实际产品方案上我们只需要一个
conf即可。
    configurations {
        default = "conf";
        conf {
            description = "Rockchip armv7 with OP-TEE";
            rollback-index = <0x0>;
            firmware = "optee";
            loadables = "uboot";
            fdt = "fdt";
            signature {
                algo = "sha256,rsa2048";
                padding = "pss";
                key-name-hint = "dev";
                sign-images = "fdt", "firmware", "loadables";
            };
        };
    };
};
```

使用 mkimage 工具和 its 文件可以生成 itb 文件：

```
                          mkimage + dtc
[u-boot.its] + [images]    ========>       [u-boot.itb]
```

fdtdump 命令可以查看 itb 文件内容：

```
cjh@ubuntu:~/uboot-nextdev/u-boot$ fdtdump fit/u-boot.itb | less
```

```
/dts-v1/;
// magic:       0xd00dfeed
// totalsize:       0x600 (1536)
// off_dt_struct:   0x48
// off_dt_strings:  0x48c
// off_mem_rsvmap:  0x28
// version:     17
// last_comp_version:   16
// boot_cpuid_phys: 0x0
// size_dt_strings: 0xc3
// size_dt_struct:  0x444

/memreserve/ 7f34d3411000 600;
/ {
    version = <0x00000001>;                 // 新增固件版本号
    totalsize = <0x000bb600>;               // 新增字段描述整个itb文件的大小
    timestamp = <0x5ecb3553>;               // 新增当前固件生成时刻的时间戳
    description = "Simple image with OP-TEE support";
    #address-cells = <0x00000001>;
    images {
        uboot {
            data-size = <0x0007ed54>;       // 新增字段描述固件大小
            data-position = <0x00000a00>; // 新增字段描述固件偏移
            description = "U-Boot";
            type = "standalone";
            os = "U-Boot";
            arch = "arm";
            compression = "none";
            load = <0x00400000>;
            hash {
                // 新增固件的sha256校验和
                value = <0xeda8cd52 0x8f058118 0x00000003 0x35360000 0x6f707465
0x0000009f 0x00000091 0x00000000>;
                algo = "sha256";
            };
        };
        optee {
            data-size = <0x0003a058>;
            data-position = <0x0007f800>;
            description = "OP-TEE";
            type = "firmware";
            arch = "arm";
            os = "op-tee";
            compression = "none";
            load = <0x08400000>;
            entry = <0x08400000>;
            hash {
                value = <0xa569b7fc 0x2450ed39 0x00000003 0x35360000 0x66647400
0x00001686 0x000b9a00 0x552d426f>;
                algo = "sha256";
            };
        };
        fdt {
            data-size = <0x00001686>;
            data-position = <0x000b9a00>;
            description = "U-Boot dtb";
            type = "flat_dt";
```

```
            compression = "none";
            hash {
                value = <0x0f718794 0x78ece7b2 0x00000003 0x35360000 0x00000001
0x6e730000 0x636f6e66 0x00000000>;
                algo = "sha256";
            };
        };
    };
    configurations {
        default = "conf";
        conf {
            description = "Rockchip armv7 with OP-TEE";
            rollback-index = <0x00000001>; // 固件防回滚版本号，没有手动指定时默认为0
            firmware = "optee";
            loadables = "uboot";
            fdt = "fdt";
            signature {
                algo = "sha256,rsa2048";
                padding = "pss";
                key-name-hint = "dev";
                sign-images = "fdt", "firmware", "loadables";
            };
        };
    };
};
```

12.2.3 itb结构

itb 本质是 fdt_blob + images 的文件集合，有如下两种打包方式，RK 平台方案采用结构 2 方式。

```
             fdt blob
|-----------------------------------|
|    |------|   |------|   |------|    |
|    | img0 |   | img1 |   | img2 |    | 结构1: image在fdt_blob内，即:itb =
fdt_blob(含img)
|    |------|   |------|   |------|    |
|-----------------------------------|

|--------------|------|------|------|
|              |      |      |      |
|   fdt blob   | img0 | img1 | img2 | 结构2: image在fdt_blob外，即itb = fdt_blob +
img
|              |      |      |      |
|--------------|------|------|------|
```

12.3 平台配置

12.3.1 芯片支持

目前已经作为正式 Feature 发布在 SDK 的平台：请参考首页各芯片 **feature** 支持状态。

12.3.2 代码配置

代码：

```
// 框架代码
./boot/image.c
./boot/image-fit.c
./common/spl/spl_fit.c

// 平台代码：
./arch/arm/mach-rockchip/fit.c
./cmd/bootfit.c

// 工具代码
./tools/mkimage.c
./tools/fit_image.c
```

配置：

```
// U-Boot 阶段支持 FIT
CONFIG_ROCKCHIP_FIT_IMAGE=y

// U-Boot阶段：安全启动、防回滚、硬件crypto
CONFIG_FIT_SIGNATURE=y
CONFIG_FIT_ROLLBACK_PROTECT=y
CONFIG_DM_CRYPTO=y
CONFIG_FIT_HW_CRYPTO=y

// SPL阶段：安全启动、防回滚、硬件crypto
CONFIG_SPL_FIT_SIGNATURE=y
CONFIG_SPL_FIT_ROLLBACK_PROTECT=y
CONFIG_SPL_DM_CRYPTO=y
CONFIG_SPL_FIT_HW_CRYPTO=y

// uboot.img镜像包含几份uboot.itb，单份uboot.itb多大
CONFIG_SPL_FIT_IMAGE_KB=3072
CONFIG_SPL_FIT_IMAGE_MULTIPLE=2

// uboot工程编译后默认输出fit格式的uboot.img; 否则为传统的RK格式uboot.img和trust.img。
CONFIG_ROCKCHIP_FIT_IMAGE_PACK=y
```

由于不同平台的 crypto 可能不同，RSA 功能的配置参数也不同。具体请参考当前平台的通用 defconfig。

```
CONFIG_RSA_N_SIZE
CONFIG_RSA_E_SIZE
CONFIG_RSA_C_SIZE
```

> 通用 defconfig：[芯片]_defconfig，例如：rv1126_defconfig、rk3568_defconfig。

如果 FIT 方案是作为 SDK 正式发布的 feature，那么大部分基础配置已使能，用户需要自己配置的选项有：

```
// U-Boot 安全启动和防回滚机制
CONFIG_FIT_SIGNATURE=y
CONFIG_FIT_ROLLBACK_PROTECT=y

// SPL 安全启动和防回滚机制
CONFIG_SPL_FIT_SIGNATURE=y
CONFIG_SPL_FIT_ROLLBACK_PROTECT=y
```

- CONFIG_FIT_SIGNATURE 没有使能：uboot 可以同时支持引导两种格式的固件：android、fit（发布的 SDK 会根据平台需求选择开启哪几种支持）。
- CONFIG_FIT_SIGNATURE 使能：uboot 只支持引导 fit 固件。

12.3.3 镜像文件

FIT 方案上最终输出两个 FIT 格式的固件用于烧写，分别是 uboot.img 和 boot.img，还有一个 SPL 文件用于打包成 loader。

- uboot.img 文件

  uboot.itb = trust + u-boot.bin + mcu.bin(option)

  uboot.img = uboot.itb * N份（N一般是2）

  > trust 和 mcu 文件来自rkbin工程，编译脚本会自动从rkbin工程索引并获取它们。

- boot.img 文件

  boot.itb = kernel + fdt + resource + ramdisk(optional)

  boot.img = boot.itb * M份（M一般是1）

- MCU 配置

  目前某些平台可能带有 MCU 固件，不同产品可以根据相应的 TRUST ini 配置来决定是否启用。例如：

  ```
  // 文件：RKTRUST/RV1126TOS_TB.ini，用于快速开机产品，启用了 MCU。
  [TOS]
  TOSTA=bin/rv11/rv1126_tee_ta_tb_v1.04.bin
  ADDR=0x00040000

  // MCU 配置格式：固件路径，启动地址，状态（okay 或 disabled）。
  // 如果为 disabled，则 mcu 不会被打包进 uboot.img 中。
  [MCU]
  MCU=bin/rv11/rv1126_mcu_v1.02.bin,0x108000,okay
  ```

- 固件压缩

  目前某些平台可以支持 uboot.img 内部子固件的压缩，支持如下：

平台	压缩格式	固件
RV1126	gzip、none	u-boot.bin, trust, mcu(optional)

  用户可以在 rkbin 工程中对应的 TRUST ini 增加属性来启用。例如：

```
// 文件：RKTRUST/RV1126TOS_SPI_NOR_TINY.ini，用于小容量SPI Nor产品。
[TOS]
TOS=bin/rv11/rv1126_tee_v1.02.bin
ADDR=0x08400000
[MCU]
MCU=bin/rv11/rv1126_mcu_v1.00.bin,0x208000,disabled

// 压缩格式：gzip 或 none，不存在如下配置字段则默认非压缩。
[COMPRESSION]
COMPRESSION=gzip
```

- SPL 文件

  SPL文件指的是编译完成后生成的 `spl/u-boot-spl.bin`，负责引导 FIT 格式的 uboot.img。用户需要用它替换 RK 平台上不开源的 miniloader，最终打包成 loader。

- `./fit`目录

  U-Boot 编译完成后会在目录下生成 `./fit` 文件夹，包含了一些中间文件，后续章节会介绍。

boot.img 和 uboot.img 分别在 sdk 工程和 uboot 工程下被编译生成。但是支持安全启动的 boot.img 必须放在 U-Boot 工程下重新打包签名，后续章节会介绍。

12.3.4 its 文件

- uboot 的 its 文件为./fit/u-boot.its，由 defconfig 中 `CONFIG_SPL_FIT_GENERATOR` 指定的脚本动态创建，固件编译成功后可见。
- boot 的 its 文件位于 SDK 工程下：

```
device/rockchip/[platform]/xxx.its  // [platform] 是平台目录
```

12.3.5 相关工具

```
// 核心打包工具，编译完成后会自动生成，U-Boot 和 rkbin 仓库下都有（U-Boot 仓库下是实时编译生成）。
./tools/mkimage
// 固件打包脚本
./make.sh
// 固件重签名脚本
scripts/fit-resign.sh
// 固件解包脚本
scripts/fit-unpack.sh
// 固件替换脚本
./scripts/fit-repack.sh
```

脚本工具的使用在后续章节会介绍，此处先重点介绍make.sh的参数：

可选项(用户根据实际情况决定是否传递)：

- `--spl-new`：传递此参数，表示使用当前编译的spl文件打包loader；否则使用rkbin工程里的spl文件。
- `--version-uboot [n]`：指定uboot.img的固件版本号，n必须是十进制正整数。
- `--version-boot [n]`：指定boot.img的固件版本号，n必须是十进制正整数；
- `--version-recovery [n]`：指定recovery.img的固件版本号，n必须是十进制正整数；

必选项(启用安全启动的情况)：

- `--rollback-index-uboot [n]`：指定uboot.img 固件防回滚版本号，n必须是十进制正整数；
- `--rollback-index-boot [n]`：指定boot.img 固件防回滚版本号，n必须是十进制正整数；
- `--rollback-index-recovery [n]`：指定recovery.img 固件防回滚版本号，n必须是十进制正整数；
- `--no-check`：打包安全固件时被使用，用于跳过安全固件打包脚本的自校验。

> 说明：
>
> 1. 固件防回滚版本号：只有在启用了安全启动的前提下才允许被激活使用，该版本号保存在OTP或者其它安全存储中。主要作用：为了防止固件版本被回退后进行漏洞攻击。
> 2. 固件版本号：可选，不指定的情况下默认为0。主要作用：只是作为固件版本标识，方便用户对固件进行版本管理。

12.4 非安全启动

12.4.1 uboot.img

编译命令：

```
./make.sh rv1126 --spl-new --version-uboot 10  // 可不指定 --spl-new 和 --version-uboot
```

编译结果：

```
  CC      spl/common/spl/spl.o
  CC      spl/lib/display_options.o
  LD      spl/common/spl/built-in.o
  LD      spl/lib/built-in.o
  LD      spl/u-boot-spl
  OBJCOPY spl/u-boot-spl-nodtb.bin
  CAT     spl/u-boot-spl-dtb.bin
  COPY    spl/u-boot-spl.bin
  CFGCHK  u-boot.cfg

out:rv1126_spl_loader_v1.00.100.bin
fix opt:rv1126_spl_loader_v1.00.100.bin
merge success(rv1126_spl_loader_v1.00.100.bin)
/home4/cjh/uboot-nextdev

// 生成 rv1126_spl_loader_v1.00.100.bin（用spl替代了RK平台传统的miniloader
// loader ini 文件来源
pack loader(SPL) okay! Input: /home4/cjh/rkbin/RKBOOT/RV1126MINIALL.ini
// 来自 --spl-new 参数的提示；用户可以选择不加这个参数。
pack loader with new: spl/u-boot-spl.bin

// 生成 uboot.img（包含trust和uboot），版本号为10
Image(no-signed, version=10):  uboot.img (FIT with uboot, trust...) is ready
// trust ini文件来源
pack uboot.img okay! Input: /home4/cjh/rkbin/RKTRUST/RV1126TOS.ini

Platform RV1126 is build OK, with exist .config
```

打包备份：通过defconfig配置指定uboot.img的多备份：

```
CONFIG_SPL_FIT_IMAGE_KB=3072    // 单份 itb 大小
CONFIG_SPL_FIT_IMAGE_MULTIPLE=2 // 打包的份数
```

SPL 根据这个配置去探测和引导 U-Boot 和 trust，主要是应对 OTA 升级过程中异常掉电引起的固件损坏，而无法启动的问题。

12.4.2 boot.img

FIT 方案如果作为 SDK 正式发布的 feature，SDK 编译完成后会生成 FIT 格式的 boot.img。

如果要生成安全启动用的 boot.img，必须把 SDK 生成的 boot.img 放到 U-Boot 工程下重新打包并签名，因为安全固件打包的签名工具、配置、参数等都来源于 U-Boot 工程。

12.5 安全启动

FIT 方案支持安全启动，相关的 feature：

- sha256 + rsa2048 + pkcs-v2.1(pss) padding
- 固件防回滚
- 固件重签名(远程签名)
- Crypto硬件加速

12.5.1 原理

12.5.1.1 校验流程

- Maskrom 校验 loader（包含 SPL, ddr, usbplug）
- SPL 校验 uboot.img（包含 trust、U-Boot...）
- U-Boot 校验 boot.img（包含 kernel，fdt，ramdisk...）

目前默认只支持 sha256+rsa2048+pkcs-v2.1(pss) padding 的安全校验模式。

12.5.1.2 key存放

RSA key 被 mkimage 打包在 u-boot.dtb 和 u-boot-spl.dtb 中，然后它们再被打包进 u-boot.bin 和 u-boot-spl.bin。

u-boot.dtb 里 RSA key 的格式如下（同理 u-boot-spl.dtb）：

```
cjh@ubuntu:~/uboot-nextdev$ fdtdump u-boot.dtb | less
/dts-v1/;
....

/ {
    #address-cells = <0x00000001>;
    #size-cells = <0x00000001>;
    compatible = "rockchip,rv1126-evb", "rockchip,rv1126";
```

```
    model = "Rockchip RV1126 Evaluation Board";

    // signature节点由mkimage工具自动插入生成，节点里保存了RSA-SHA算法类型、RSA核心因子参数等信息。
    signature {
        key-dev {
            required = "conf";
            algo = "sha256,rsa2048";
            rsa,np = <0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x00000000 0x00000000 0x00000000 0x1327f633 0x00000003 0x00000003 0x00000003
0xc7aead6a 0xb4c79f40 0xa82bdf76 0xfb2f8387 0xa1e06dce 0xd451a706 0xc7f865e3
0x3e2d7ca8 0x6a71762e 0x125f1828 0x36ab1a41 0xb7e9e852 0x7bd0011a 0x7279e0b8
0xf37e189c 0x8cf00963 0x00000100 0x00000000 0x00000000 0x00000000 0x00000000
0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000377 0x00000004
0x00000004 0x00000004 0x00000002 0x00000003 0x69616c40 0x00000003 0x6d634066
0x00000010 0x66633630 0x73797363>;
            rsa,c = <0x00000000>;
            rsa,r-squared = <0x00000000>;
            rsa,modulus = <0xc25ae693 0xc359f2a4 0xa866c89d 0xb7b1994f
0xf9f9f690 0x518d54a7 0xda0b83e8 0x06606e12 0x6ad1cbf9 0x92438edd 0x81e039c0
0x5d7322cc 0x124cdc80 0xa0c3288a 0x9265c3ae 0x6ac47a4b 0x00000003 0x00000000
0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x00000000 0x00000008 0x00000003 0x00000003 0x00000003 0x00000002 0x73657300
0x2f736572 0x00000000 0x2f64776d 0x00000003 0x6d634066 0x00000001 0x30303000
0x726f636b 0x67726600 0x00000008 0x00000003 0x00000004 0x00000001 0x30303000
0x726f636b 0x706d7567 0x00000003 0x00001000 0x00000003 0x00000002 0x6e616765
0x30000000 0x726f636b 0x706d7500 0x00000008>;
            rsa,exponent-BN = <0x00000000 0x00000000 0x00000000 0x00000000
0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000003 0x00010001
0xe95771c5 0x00000800 0x64657600 0x616c6961 0x0000002c 0x30303030 0x00000034
0x30303000 0x2f64776d 0x00000002 0x65303030 0x0000001b 0x3132362d 0x00000003
0x00020000 0x00000003 0x00000002 0x65303230 0x0000001b 0x3132362d 0x6e000000
0xfe020000 0x00000042 0x0000006d 0x722d6d61 0x65303030 0x0000001b 0x3132362d
0x00000003 0x00001000 0x00000002 0x6e74726f 0x30000000 0x726f636b 0x706d7563
0x0000003e 0x00000004 0x00000004 0x00000004 0x00000000 0x00000050 0x636c6f63
0x40666634 0x00000014 0x2c727631 0x00000008>;
            rsa,exponent = <0x00000000 0x00000368>;
            rsa,n0-inverse = <0xe95771c5>;
            rsa,num-bits = <0x00000800>;
            key-name-hint = "dev";
        };
    };
```

SPL 支持烧写 key hash 的功能，u-boot-spl.dtb 的 key-dev 会多出 `burn-key-hash = <0x00000001>;`。

12.5.1.3 key使用

从 Maskrom 到 kernel 为止的安全启动，统一使用一把 RSA 公钥完成安全校验：

- Maskrom 校验 loader。

  RSA 公钥需要使用 PC 工具 `rk_sign_tool` 写入 loader 的文件头中。安全启动时，Maskrom 首先从 loader 固件头中获取 RSA 公钥并校验合法性；然后再使用该公钥校验 loader 的固件签名。

> rk_sign_tool 可从 rkbin 仓库中获取，U-Boot 会自动完成对 loader 的签名。

- SPL 校验 U-Boot 和 trust。

  SPL 把 RSA 公钥保存在 u-boot-spl.dtb 中，u-boot-spl.dtb 会被打包进 u-boot-spl.bin 文件（最后打包进 loader）；安全启动时 SPL 从自己的 dtb 文件中拿出 RSA 公钥对 uboot.img 进行安全校验。

- U-Boot 校验 boot。

  U-Boot 把 RSA 公钥保存在 u-boot.dtb 中，u-boot.dtb 会被打包进 u-boot.bin 文件（最后打包为 uboot.img）；安全启动时 U-Boot 从自己的 dtb 文件中拿 RSA 公钥对 boot.img 进行校验。

所以当前这级的 RSA Key 已经作为自身固件的一部分，由前一级 loader 完成了安全校验，从而保证了 Key 的安全。

12.5.1.4 签名存放

RSA 的签名结果被保存在 itb 文件中；被签名内容由 `hashed-nodes` 指定：包括了整个 `conf` 节点的属性、被打包固件的节点等。

如下是 u-boot.itb 的签名信息，同理 boot.itb：

```
cjh@ubuntu:~/uboot-nextdev$ fdtdump uboot.img | less
/dts-v1/;
......

    configurations {
        default = "conf";
        conf {
            description = "Rockchip armv7 with OP-TEE";
            // 当前的固件版本号
            rollback-index = <0x0000001c>;
            firmware = "optee";
            loadables = "uboot";
            fdt = "fdt";

            // 被签名内容和签名结果，由mkimage自动插入
            signature {
                hashed-strings = <0x00000000 0x000000da>;
                // 指定被签名内容
                hashed-nodes = "/", "/configurations", "/configurations/conf",
"/images/fdt", "/images/fdt/hash", "/images/optee", "/images/optee/hash",
"/images/uboot", "/images/uboot/hash";
                // 进行签名的时间、签名者、版本
                timestamp = <0x5e9427b4>;
                signer-version = "v5-g8bb63db-200413-dirty #cjh";
                signer-name = "mkimage";
                // 签名结果！！(采用sha256+rsa2048)
                value = <0x78397d5d 0xb9219a0b 0xa7cb91a7 0xe1f32867 0x62719d9b
0x8901200c 0xfcbac03a 0x1295ccc8 0x1cff9608 0xdf5f69d2 0x21391225 0x7af10ca7
0x5527864f 0xb13f527e 0xddf9ee62 0xea50199d 0x00000003 0x35362c72 0x00000004
0x00000017 0x77617265 0x00000002 0x00000009 0x23616464 0x6d616765 0x73006172
0x6f6e006c 0x72790064 0x61636b2d 0x7265006c 0x006b6579 0x69676e2d 0x706f7369
0x7a650074 0x75650073 0x69676e65 0x73686564 0x642d7374 0x00000000 0x00000000
0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000 0x00000000
0x00000000 0x00000000 0x00000000>;
```

```
                algo = "sha256,rsa2048";
                key-name-hint = "dev";
                sign-images = "fdt", "firmware", "loadables";
            };
        };
    };
```

12.5.1.5 防回滚

- 安全启动支持对 boot.img 和 uboot.img 分别指定当前固件版本号，如果当前固件版本号小于机器上的最小版本号，则不允许启动。
- 最小版本号的更新：完成安全校验且确认系统可以正常启动后，被更新到 OTP 或安全存储中。

12.5.2 前期准备

12.5.2.1 Key

U-Boot 工程下执行如下三条命令可以生成签名用的 RSA 密钥对。通常情况下只需要生成一次，此后都用这对密钥签名和验证固件，请妥善保管。

```
// 1. 放 key 的目录：keys
mkdir -p keys

// 2. 使用 RK 的 "rk_sign_tool" 工具生成 RSA2048 的私钥 privateKey.pem 和 publicKey.pem（请参考 rk_sign_tool 的使用手册），分别更名存放为：keys/dev.key 和 keys/dev.pubkey。

// 3. 使用 -x509 和私钥生成一个自签名证书：keys/dev.crt（效果本质等同于公钥）
openssl req -batch -new -x509 -key keys/dev.key -out keys/dev.crt
```

> 如果报错用户目录下没有.rnd文件：
>
> Can't load /home4/cjh//.rnd into RNG
> 140522933268928:error:2406F079:random number generator:RAND_load_file:Cannot open file:../crypto/rand/randfile.c:88:Filename=/home4/cjh//.rnd
>
> 请先手动创建：touch ~/.rnd

`ls keys/` 查看结果：

```
dev.crt  dev.key  dev.pubkey
```

> 注意：上述的 "keys"、"dev.key"、"dev.crt"、"dev.pubkey" 名字都不可变。因为这些名字已经在 its 文件中静态定义，如果改变则会打包失败。

12.5.2.2 配置

U-Boot 的 defconfig 打开如下配置：

```
// 必选。
CONFIG_FIT_SIGNATURE=y
CONFIG_SPL_FIT_SIGNATURE=y

// 可选。
CONFIG_FIT_ROLLBACK_PROTECT=y        // boot.img 防回滚
CONFIG_SPL_FIT_ROLLBACK_PROTECT=y    // uboot.img 防回滚
```

> 建议通过 make menuconfig 的方式选中配置后，再通过 make savedefconfig 更新原本的 defconfig 文件。这样可以避免因为强加 defconfig 配置而导致依赖关系不对，进而导致编译失败的情况。

12.5.2.3 固件

把 SDK 工程下生成的 boot.img 复制一份到 U-Boot 根目录下。

12.5.3 编译打包

（1）基础命令（不防回滚）：

```
./make.sh rv1126 --spl-new --boot_img boot.img --recovery_img recovery.img
```

编译结果：

```
......

// 编译完成后，生成已签名的uboot.img和boot.img。
start to sign rv1126_spl_loader_v1.00.100.bin
......
sign loader ok.
......
Image(signed, version=0):  uboot.img (FIT with uboot, trust...) is ready
Image(signed, version=0):  recovery.img (FIT with kernel, fdt, resource...) is
ready
Image(signed, version=0):  boot.img (FIT with kernel, fdt, resource...) is ready
Image(signed):  rv1126_spl_loader_v1.05.106.bin (with spl, ddr, usbplug) is
ready
pack uboot.img okay! Input: /home4/cjh/rkbin/RKTRUST/RV1126TOS.ini

Platform RV1126 is build OK, with new .config(make rv1126-secure_defconfig)
```

（2）扩展命令1：

如果开启防回滚，必须对上述（1）追加 rollback 参数。例如：

```
// 指定 uboot.img 和 boot.img 的最小版本号分别为 10、12。
./make.sh rv1126 --spl-new --boot_img boot.img --recovery_img recovery.img --
rollback-index-uboot 10 --rollback-index-boot 12 --rollback-index-recovery 12
```

编译结果：

```
......

// 编译完成后，生成已签名的uboot.img和boot.img，且包含防回滚版本号。
start to sign rv1126_spl_loader_v1.00.100.bin
......
sign loader ok.
......
Image(signed, version=0, rollback-index=10):  uboot.img (FIT with uboot, trust)
is ready
Image(signed, version=0, rollback-index=12):  recovery.img (FIT with kernel,
fdt, resource...) is ready
Image(signed, version=0, rollback-index=12):  boot.img (FIT with kernel, fdt,
resource...) is ready
Image(signed):  rv1126_spl_loader_v1.00.100.bin (with spl, ddr, usbplug) is
ready
```

（3）扩展命令2：

如果要把公钥 hash 烧写到 OTP/eFUSE，必须对上述（1）或（2）追加参数 `--burn-key-hash`。例如：

```
// 指定 uboot.img 和 boot.img 的最小版本号分别为 10、12。
// 要求 SPL 阶段把公钥 hash 烧写到 OTP/eFUSE 中。
./make.sh rv1126 --spl-new --boot_img boot.img --recovery_img recovery.img --
rollback-index-uboot 10 --rollback-index-boot 12 --rollback-index-recovery 12 --
burn-key-hash
```

编译结果：

```
......

// 使能 burn-key-hash
### spl/u-boot-spl.dtb: burn-key-hash=1

// 编译完成后，生成已签名的uboot.img和boot.img，且包含防回滚版本号。
start to sign rv1126_spl_loader_v1.00.100.bin
......
sign loader ok.
......
Image(signed, version=0, rollback-index=10):  uboot.img (FIT with uboot, trust)
is ready
Image(signed, version=0, rollback-index=12):  recovery.img (FIT with kernel,
fdt, resource...) is ready
Image(signed, version=0, rollback-index=12):  boot.img (FIT with kernel, fdt,
resource...) is ready
Image(signed):  rv1126_spl_loader_v1.00.100.bin (with spl, ddr, usbplug) is
ready
```

> 上电开机时能看到 SPL 打印：RSA: Write key hash successfully。

（4）注意事项：

- `--boot_img`：可选。指定待签名的 boot.img。
- `--recovery_img`：可选。指定待签名的 recovery.img。
- `--rollback-index-uboot`、`--rollback-index-boot`、`--rollback-index-recovery`：可选。指定防回滚版本号。

- `--spl-new`：如果编译命令不带此参数，则默认使用 rkbin 中的 spl 文件打包生成 loader；否则使用当前编译的 spl 文件打包 loader。

  因为 u-boot-spl.dtb 中需要被打包进 RSA 公钥（来自于用户），所以 RK 发布的 SDK 不会在 rkbin 仓库提交支持安全启动的 spl 文件。因此，用户编译时要指定该参数。但是用户也可以把自己的 spl 版本提交到 rkbin 工程，此后编译固件时就可以不再指定此参数，每次都使用这个稳定版的 spl 文件。

- 编译后会生成三个固件：loader、uboot.img、boot.img，只要 RSA key 没有更换，就允许单独更新其中的任意固件。

12.5.4 校验原则

（1）Maskrom 校验 SPL

OTP 没有烧写 key：Maskrom 走非安全启动流程。

OTP 有烧写 key：Maskrom 校验 Loader 里的 key，必须跟 OTP 里的一致才会开始进行安全校验，不一致就不让启动。

（2）SPL 校验 U-Boot

CONFIG_SPL_FIT_SIGNATURE=y：SPL 一定会对 uboot.img 进行安全校验，校验成功才启动；uboot.img 没有签名或校验失败，不启动。

CONFIG_SPL_FIT_SIGNATURE=n：SPL 本身没包含安全启动相关的代码，一定不会校验 uboot.img（无论是否签名）。

（3）U-Boot 校验 boot/recovery

CONFIG_FIT_SIGNATURE=y：U-Boot 一定会对 boot.img/recovery.img 进行安全校验，校验成功才启动；boot.img/recovery.img 没有签名或校验失败，不启动。

CONFIG_FIT_SIGNATURE=n：U-Boot 本身没包含安全启动相关的代码，一定不会校验 boot.img/recovery.img（无论是否签名）。

> 注意：当前这一级是否会去校验后一级，跟当前这级固件是否被签名没有任何关系。只取决于自身是否包含安全启动的相关代码，即上述配置是否为 y。

12.5.5 启动信息

如下是安全启动的信息：

```
BW=32 Col=10 Bk=8 CS0 Row=15 CS=1 Die BW=16 Size=1024MB
out
U-Boot SPL board init
U-Boot SPL v5-gacb99c5-200408-dirty #cjh (Apr 09 2020 - 20:51:21)
unrecognized JEDEC id bytes: 00, 00, 00

Trying to boot from MMC1
// SPL完成签名校验
sha256,rsa2048:dev+
// 防回滚检测：当前uboot.img固件版本号是10，本机的最小版本号是9
rollback index: 10 >= 9, OK
// SPL完成各子镜像的hash校验
### Checking optee ... sha256+ OK
```

```
### Checking uboot ... sha256+ OK
### Checking fdt ... sha256+ OK

Jumping to U-Boot via OP-TEE
I/TC:
E/TC:0 0 plat_rockchip_pmu_init:2003 0
E/TC:0 0 plat_rockchip_pmu_init:2006 cpu off
E/TC:0 0 plat_rockchip_pmusram_prepare:1945 pmu sram prepare 0x14b10000
0x8400880 0x1c
E/TC:0 0 plat_rockchip_pmu_init:2020 pmu sram prepare
E/TC:0 0 plat_rockchip_pmu_init:2056 remap
I/TC: OP-TEE version: 3.6.0-233-g35ecf936 #1 Tue Mar 31 08:46:13 UTC 2020 arm
I/TC: Next entry point address: 0x00400000
I/TC: Initialized

U-Boot v5-gacb99c5-200408-dirty #cjh (Apr 09 2020 - 20:51:21 +0800)

Model: Rockchip RV1126 Evaluation Board
PreSerial: 2
DRAM:  1023.5 MiB
Sysmem: init
Relocation Offset: 00000000, fdt: 3df404e0
Using default environment

dwmmc@ffc50000: 0
Bootdev(atags): mmc 0
MMC0: HS200, 200Mhz
PartType: EFI
boot mode: normal
conf: sha256,rsa2048:dev+
resource: sha256+
DTB: rk-kernel.dtb
FIT: signed, conf required
HASH(c): OK

I2c0 speed: 400000Hz
PMIC:  RK8090 (on=0x10, off=0x00)
vdd_logic 800000 uV
vdd_arm 800000 uV
vdd_npu init 800000 uV
vdd_vepu init 800000 uV
......

Hit key to stop autoboot('CTRL+C'):  0
### Booting FIT Image at 0x3d8122c0 with size 0x0052b200
Fdt Ramdisk skip relocation
### Loading kernel from FIT Image at 3d8122c0 ...
   Using 'conf' configuration
   // uboot完成签名校验
   Verifying Hash Integrity ... sha256,rsa2048:dev+ OK
   // 防回滚检测：当前boot.img固件版本号是22，本机的最小版本号是21
   Verifying Rollback-index ... 22 >= 21, OK
   Trying 'kernel' kernel subimage
     Description:  Kernel for arm
     Type:         Kernel Image
     Compression:  uncompressed
     Data Start:   0x3d8234c0
     Data Size:    5349248 Bytes = 5.1 MiB
```

```
     Architecture: ARM
     OS:           Linux
     Load Address: 0x02008000
     Entry Point:  0x02008000
     Hash algo:    sha256
     Hash value:
64b4a0333f7862967be052a67ee3858884fcefebf4565db5c3828a941a15f34a
   Verifying Hash Integrity ... sha256+ OK  // 完成kernel的hash校验
### Loading ramdisk from FIT Image at 3d8122c0 ...
   Using 'conf' configuration
   Trying 'ramdisk' ramdisk subimage
     Description:  Ramdisk for arm
     Type:         RAMDisk Image
     Compression:  uncompressed
     Data Start:   0x3dd3d4c0
     Data Size:    0 Bytes = 0 Bytes
     Architecture: ARM
     OS:           Linux
     Load Address: 0x0a200000
     Entry Point:  unavailable
     Hash algo:    sha256
     Hash value:
e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855
   Verifying Hash Integrity ... sha256+ OK  // 完成ramdisk的hash校验
   Loading ramdisk from 0x3dd3d4c0 to 0x0a200000
### Loading fdt from FIT Image at 3d8122c0 ...
   Using 'conf' configuration
   Trying 'fdt' fdt subimage
     Description:  Device tree blob for arm
     Type:         Flat Device Tree
     Compression:  uncompressed
     Data Start:   0x3d812ec0
     Data Size:    66974 Bytes = 65.4 KiB
     Architecture: ARM
     Load Address: 0x08300000
     Hash algo:    sha256
     Hash value:
8fb1f170766270ed4f37cce4b082a51614cb346c223f96ddfe3526fafc5729d7
   Verifying Hash Integrity ... sha256+ OK // 完成fdt的hash校验
   Loading fdt from 0x3d812ec0 to 0x08300000
   Booting using the fdt blob at 0x8300000
   Loading Kernel Image from 0x3d8234c0 to 0x02008000 ... OK
   Using Device Tree in place at 08300000, end 0831359d
Adding bank: 0x00000000 - 0x08400000 (size: 0x08400000)
Adding bank: 0x0848a000 - 0x40000000 (size: 0x37b76000)
Total: 236.327 ms

Starting kernel ...

[    0.000000] Booting Linux on physical CPU 0xf00
[    0.000000] Linux version 4.19.111 (cjh@ubuntu) (gcc version 6.3.1 20170404
(Linaro GCC 6.3-2017.05)) #28 SMP PREEMPT Wed Mar 25 16:03:27 CST 2020
[    0.000000] CPU: ARMv7 Processor [410fc075] revision 5 (ARMv7), cr=10c5387d
```

12.6 独立签名

基于 FIT 格式的特点，独立签名是对未签名固件进行解包再重新打包并签名，本章节将先介绍操作流程，再介绍相关工具。独立签名功能于 2025 年 1 月份才支持，请确保 uboot 和 rkbin 仓库包含相关补丁：

uboot 仓库：

```
91b5d8722da scripts: Add fit-sign.sh
554ddf99bfb scripts: fit-core.sh: Add fit_signcfg export support
cb458c93a05 scripts: fit-core.sh: Add --no-sign support
bc02f36c3b2 scripts: fit-unpack: Support none digest node
```

rkbin 仓库：

```
44216040fad tools: build mkimage/fit_check_sign with -static
88d46662a9b tools: Add fit sign tools support
```

12.6.1 签名流程

编译固件的人员：

1. 参考前面的安全启动章节，打开相关配置项。
2. 编译：

   ```
   ./make.sh rk3588 --spl-new --no-sign   # 必须追加 --spl-new --no-sign
   ```

   编译后会生成：

   ```
   uboot.img
   rk3588_spl_loader_v1.18.113.bin
   fit/fit_signcfg/
   ```

3. 把生成的三个文件、其他待签名FIT文件（例如：boot.img、recovery.img）发给负责签名的人员或服务器。

   注意：生成的 loader、uboot 跟 fit_signcfg 有严格的匹配关系，不能跟其他时刻编译的同名固件混在一起参与签名，否则签名脚本会报错。

签名的人员或服务器：

1. 创建工作目录（名字可任意）：

   ```
   $ mkdir workspace
   $ cd workspace
   $ mkdir keys src output
   ```

2. 下载 rkbin 仓库，把 rkbin 里的 tools 目录拷贝或软链接到 workspace 目录下：

   ```
   $ cp ~/rkbin/tools/ ./ -R
   ```

3. 参考前面的安全启动章节把 RSA 密钥对放到 keys 目录下，把待签名的固件和 fit_signcfg 目录放到 src 目录下。

其中 loader、uboot 和 fit_signcfg 是必须的，还可以放其他待签名的 FIT 固件（如：boot.img、recovery.img）等。例如：

```
cjh@sys2:/home4/cjh/rkbin/workspace$ tree
├── keys
│   ├── dev.crt
│   ├── dev.key
│   └── dev.pubkey
├── output
├── src
│   ├── fit_signcfg
│   │   ├── MINIALL.ini
│   │   ├── sign.readonly_config
│   │   └── test_images.tar.gz
│   ├── boot.img
│   ├── recovery.img
│   ├── rk3588_spl_loader_v1.18.113.bin
│   └── uboot.img
└── tools
    ├── bmp2gray16
    ├── boot_merger
    ├── burner_image_kits
    │   ├── make.sh
    │   ├── make_slc_nand.sh
    ....
```

4. 执行签名：

```
$ ./tools/fit-sign.sh --key-dir keys/ --src-dir src/ --out-dir output/
```

签名过程不会对src目录的源文件做任何修改。签名成功后会在output目录下生成所有签名固件：

```
$ tree output/
output/
├── boot.img
├── recovery.img
├── rk3588_spl_loader_v1.18.113.bin
└── uboot.img
```

12.6.2 签名脚本

独立签名的相关工具都已提交在 rkbin 仓库，供开发者调用的脚本是：`./tools/fit-sign.sh`。

```
$ ./tools/fit-sign.sh -h

Usage:
    ./tools/fit-sign.sh [args]

Args:
    --key-dir                   <dir>                           | Mandatory
    --src-dir                   <dir>                           | Mandatory
    --out-dir                   <dir>                           | Mandatory
    --burn-key-hash                                             | Optional
    --rollback-index            <image1 n1> <image2 n2> ...     | Optional
    --version                   <image1 n1> <image2 n2> ...     | Optional
```

- --key-dir：必选。指向 RSA 密钥目录。
- --src-dir：必选。指向要待签名固件目录，该目录必须包含 loader、uboot、fit_signcfg 目录。
- --out-dir：必选。指向签名后的输出目录。
- --burn-key-hash：可选。不需要跟额外参数，用于使能 SPL 烧写 RSA 公钥 hash 到 otp。
- --rollback-index：可选。给对应固件指定防回滚版本号，十进制表示。
- --version：可选。给对应固件指定常规的文件版本号，十进制表示。

例如：

```
./tools/fit-sign.sh --key-dir keys/ --src-dir src/ --out-dir output/ --version
uboot.img 1 boot.img 3 --rollback-index uboot.img 3 boot.img 5
```

12.6.3 签名配置

为了严格保证签名时指定的 feature 在 spl、uboot 代码中有被使能，编译固件时会额外生成配置记录相关的文件。签名脚本需要使用到这些文件。

```
$ tree fit_signcfg/
fit_signcfg/
├── MINIALL.ini             # 打包 loader 用的 ini 文件。
├── sign.readonly_config    # 当前编译配置、uboot、loader 版本相关的记录文件。
└── test_images.tar.gz      # 暂时无用，忽略。
```

12.7 远程签名

独立签名需要把完整固件上传给负责签名的人员或服务器，这对传输的数据量、速度等有一定影响，而远程签名机制是一种更轻便的方案。两种方案各有利弊，请用户按需选择。

12.7.1 实现思路

- 因为只能拿到服务器的公钥，所以用户先用临时私钥+服务器公钥在本地 PC 上对固件进行一次打包签名，会生成带有临时签名的安全固件和被签名数据；

  > 公钥的作用是为了把公钥打包进dtb文件，在安全启动流程时使用；私钥的作用是做临时签名。

- 用户把被签名数据发送给服务器即可（不需整个固件，更节省时间），服务器使用私钥对被签名数据进行签名，然后把签名返回给用户；
- 用户使用这份签名替换安全固件中的临时签名即可获得最后用于烧写的安全固件。

12.7.2 被签名数据

上述章节提到的被签名数据包含：fdt blob配置 + 子镜像hash集合。

- fdt blob 节点配置

  `hashed-nodes`指定了一系列节点，这些节点的内容都会纳入被签名数据。

```
cjh@ubuntu:~/uboot-nextdev$ fdtdump uboot.img | less
/dts-v1/;
......

configurations {
        default = "conf";
        conf {
            description = "Rockchip armv7 with OP-TEE";
            rollback-index = <0x0000001c>;
            firmware = "optee";
            loadables = "uboot";
            fdt = "fdt";

            signature {
                hashed-strings = <0x00000000 0x000000da>;
                // 这些节点的内容都会纳入被签名数据
                hashed-nodes = "/", "/configurations/conf", "/images/fdt",
"/images/fdt/hash", "/images/optee", "/images/optee/hash", "/images/uboot",
"/images/uboot/hash";
                ......
```

- 子镜像hash的集合。

  mkimage会为各个子镜像自动生成hash值，并追加进hash节点。`sign-images`指定的所有子镜像hash值都会纳入被签名数据（本质是通过`hashed-nodes`进行指定了hash节点）。例如：

```
cjh@ubuntu:~/uboot-nextdev/u-boot$ fdtdump fit/u-boot.itb | less

/dts-v1/;
......

/ {
    totalsize = <0x000bb600>;
    timestamp = <0x5ecb3553>;
    description = "Simple image with OP-TEE support";
    #address-cells = <0x00000001>;
    images {
        uboot {
            data-size = <0x0007ed54>;
            data-position = <0x00000a00>;
            description = "U-Boot";
            type = "standalone";
            os = "U-Boot";
            arch = "arm";
```

```
            compression = "none";
            load = <0x00400000>;
            hash {
                // uboot镜像的hash，由mkimage工具自动计算生成
                value = <0xeda8cd52 0x8f058118 0x00000003 0x35360000
0x6f707465 0x0000009f 0x00000091 0x00000000>;
                algo = "sha256";
            };
        };
        ......
```

12.7.3 具体步骤

用于签名固件的 RSA 密钥对是：dev.key、dev.pubkey 和 dev.crt。dev.key 作为私钥由远程服务器持有，用户只有 dev.pubkey 和 dev.crt。

步骤1：

在本地 U-Boot 工程环境下：用户把 dev.crt 放到 keys 目录下，然后用 RK 的 "rk_sign_tool" 工具随机生成一把临时私钥，命名为 dev.key 放到 keys 目录下。参考上面的章节（但是编译参数要追加 `--no-check`）生成签名固件 uboot.img 和 boot.img（实际最后不会被使用，用户需要的是中间文件）。

注意：**编译命令要指定参数 `--no-check`**，否则会因为 dev.key 和 dev.crt 不匹配导致打包脚本自校验失败。比如：

```
./make.sh rv1126 --spl-new --boot_img boot.img --rollback-index-uboot 10 --
rollback-index-boot 12 --no-check
```

除了生成签名固件 uboot.img 和 boot.img，用户还可以在 `fit/` 目录下得到中间文件：

```
// 被签名内容(data2sign意为：data to sign)
fit/uboot.data2sign
fit/boot.data2sign

// 已签名itb文件（使用临时私钥），我们的img文件由它们进行多备份后获得。
fit/uboot.itb
fit/boot.itb
```

步骤2：

用户把 uboot.data2sign 发送给远程服务器。假设远程服务器持有的私钥为 dev.key，使用如下命令签名并输出签名结果：uboot.sig

```
openssl dgst -sha256 -sign dev.key -sigopt rsa_padding_mode:pss -out uboot.sig
uboot.data2sign
```

服务器把签名结果文件 uboot.sig 返回给用户，用户使用 uboot.sig 替换 uboot.itb 中的临时签名：

```
./scripts/fit-resign.sh -f fit/uboot.itb -s uboot.sig // 会生成新的 uboot.img，用于
烧写
```

同理 boot.itb 文件。由此用户获得了最终有效的签名固件 uboot.img 和 boot.img。

注意事项：

- fit-resign.sh 时 -f 指定的 itb 文件，不是 img 文件。脚本会对 itb 重签名后生成 img 文件。
- 执行 fit-resign.sh 时用的 itb 文件必须是步骤 1 编译生成的，即 itb 文件和 data2sign 文件是一对一对应的，因为 data2sign 信息中包含了生成 itb 文件的时间戳，即 `/timestamp = <...>`。所以即使当前没有任何代码改动，重新编译获得一个新的 uboot.itb，把 uboot.sig 替换进新的 uboot.itb 中也会引起安全启动失败！
- 由于没有私钥，loader 需要单独发送到服务器端进行签名。

12.8 固件解包

用户可以借助脚本对固件解包，例如 boot.img：

```
cjh@ubuntu:~/uboot-nextdev$ ./scripts/fit-unpack.sh -f boot.img -o out
Unpack to directory out:
  fdt                 : 82813 bytes... sha256+
  kernel              : 5844640 bytes... sha256+
  ramdisk             : 0 bytes... sha256+
  resource            : 120832 bytes... sha256+
```

> 如果 img 包含多备份，脚本只解包第一份 itb；sha256+ 表示固件没有损坏，否则显示 sha256-。

12.9 固件替换

用户可以借助脚本批量替换子固件。例如：想用自己的 bl31.elf 替换进别人的 uboot_legacy.img 里：

1. 用自己的bl31.elf编译出自己的uboot.img
2. 用fit-unpack.sh解包uboot_legacy.img 到 out/目录下

   ```
   cjh@ubuntu:~/uboot-nextdev$ ./scripts/fit-unpack.sh -f uboot_legacy.img -o
   out/

   uboot_legacy.img: Device Tree Blob version 17, size=2560, boot CPU=0, string
   block size=197, DT structure block size=2204
   Unpack to directory out:
     uboot               : 576352 bytes... sha256+
     atf-1               : 69089 bytes... sha256+
     atf-2               : 36864 bytes... sha256+
     atf-3               : 24576 bytes... sha256+
     optee               : 228134 bytes... sha256+
     fdt                 : 8867 bytes... sha256+
   ```

3. 把 out/ 目录下的 atf-xxx 文件都删掉
4. 用 fit-repack.sh 把 out/ 里的所有子镜像替换进自己的 uboot.img。至此，新的 uboot.img 就包含了自己的 bl31 和 uboot_legacy.img 里的其他子镜像，实现了预期的替换效果。

```
cjh@ubuntu:~/uboot-nextdev$ ./scripts/fit-repack.sh -f uboot.img -d out/

uboot.img: Device Tree Blob version 17, size=2560, boot CPU=0, string block
size=197, DT structure block size=2204
Unpack to directory out/repack/:
  uboot               : 576352 bytes... sha256+
  atf-1               : 69089 bytes... sha256+
  atf-2               : 36864 bytes... sha256+
  atf-3               : 24576 bytes... sha256+
  optee               : 228134 bytes... sha256+
  fdt                 : 8867 bytes... sha256+
  ......
Image(repack):  uboot.img is ready
```

原理说明：

子镜像替换采用的策略不是"把我的子镜像替换进他的 uboot.img 里"，而是"把他的子镜像替换进我的 uboot.img"。

原因是：uboot.img 中的 atf-xxx 来自于 bl31.elf，新旧不同的 bl31.elf 里包含的 atf-xxx 个数有可能不同，若不同则无法平替。虽然 atf-xxx 以外的 u-boot、bl32、mcu 等子镜像个数都是固定的，但为了支持 bl31.elf 的替换，所以采用了这种逆向替换的策略。

上述介绍了 bl31.elf 的替换方式，其他子镜像的替换也采用相同策略。

12.10 安全校验Step-by-Step

1. 进入 u-boot 目录，打开对应平台的 configs/rxxxxx_defconfig，选择如下配置：

```
// 必选。
CONFIG_FIT_SIGNATURE=y
CONFIG_SPL_FIT_SIGNATURE=y

// 可选。
CONFIG_FIT_ROLLBACK_PROTECT=y        // boot.img 防回滚
CONFIG_SPL_FIT_ROLLBACK_PROTECT=y    // uboot.img 防回滚
```

2. 执行如下操作生成 keys：

```
mkdir -p keys
../rkbin/tools/rk_sign_tool kk --bits 2048 --out .
cp privateKey.pem keys/dev.key && cp publicKey.pem keys/dev.pubkey
openssl req -batch -new -x509 -key keys/dev.key -out keys/dev.crt
```

注意：该步骤执行一次即可，然后妥善保存这些 keys。

3. 编译签名，以 rv1126 为例（如果编译签名其他芯片固件，如 rk3566，将下列命令内的 rv1126 改为 rk3566 即可）：

```
// Linux：拷贝 boot.img，recovery.img 到 u-boot 文件下，执行下列脚本签名 loader,uboot,boot,recovery，设置 uboot,boot,recovery 的防版本回滚号，注意防版本回滚号依据需要配置
./make.sh rv1126 --spl-new --boot_img boot.img --recovery_img recovery.img --rollback-index-uboot 1 --rollback-index-boot 2

// Android：签名 loader,uboot，设置 uboot 的防版本回滚号，注意防版本回滚号依据需要配置
./make.sh rv1126 --spl-new --rollback-index-uboot 1
```

如果编译出现：

```
Can't load XXXXXX//.rnd into RNG
```

执行：

```
touch ~/.rnd
```

4. 公钥hash烧写：

```
// Linux：拷贝 boot.img，recovery.img 到 u-boot 文件下，执行下列脚本签名 loader,uboot,boot,recovery，设置 uboot,boot,recovery 的防版本回滚号，注意防版本回滚号依据需要配置，使能烧写 key hash
./make.sh rv1126 --spl-new --boot_img boot.img --recovery_img recovery.img --rollback-index-uboot 1 --rollback-index-boot 2 --burn-key-hash

// Android：签名 loader,uboot，设置 uboot 的防版本回滚号，注意防版本回滚号依据需要配置，使能烧写 key hash
./make.sh rv1126 --spl-new --rollback-index-uboot 1 --burn-key-hash
```

注意：该步骤在整个产品开发验证完后再配置 `--burn-key-hash`，否则安全开启，产品开发过程中每次只能更新签名过的固件。

5. Android其他固件签名：

参考《Rockchip_Developer_Guide_Secure_Boot_for_UBoot_Next_Dev_CN.md》

13. Chapter-13 快速开机

TODO

14. Chapter-14 平台定义

14.1 ATF/OPTEE

U-Boot 充电待机要求的 ATF/OPTEE 最低版本：

芯片	最低版本号
RK3576	rk3576_bl31_v1.04.elf
RK3588	rk3588_bl31_v1.24.elf
RK3538	N/A
RK3572	N/A

14.2 Clock

CPU 提频功能支持列表：

芯片	支持情况	提频处理项
RK3576	N/A	N/A
RK3588	N/A	N/A
RK3538	N/A	N/A
RK3572	N/A	N/A

14.3 Defconfig

1. 各平台的 defconfig 支持情况（以 SDK 发布版本为准）：

> `[芯片]_defconfig` 或 `[芯片].config` 通常是全功能版本，其余为特定 feature 版本。具体文件请以实际仓库情况为准。

芯片	defconfig	支持 Kernel DTB	说明
RK3576	rk3576_defconfig rk3576-usbplug.config rk3576-car.config rk3576-ab-car.config rk3576-eink.config	Y	通用版本 开源usbplug 车载版本 支持ab系统车载版本 电子书版本
RK3588	rk3588_defconfig rk3588-ramboot.config rk3588-sata.config rk3588-aarch32.config rk3588-ipc.config	Y	通用版本 无存储器件(内存启动) 双存储支持sata启动 支持aarch32模式 ipc sdk上使用
RK3538	rk3538_defconfig rk3538-usbplug.config	Y	通用版本 开源usbplug
RK3572	rk3572_defconfig rk3572-usbplug.config rk3572-car.config rk3572-ab-car.config rk3572-eink.config	Y	通用版本 开源usbplug 车载版本 支持ab系统车载版本 电子书版本

14.4 DFU

TODO

14.5 OP-TEE

Optee client 接口在各平台上的适用性：

API	RK3588	RK3576	RK3538	RK3572
optee_read_vbootkey_hash	√	√	√	√
optee_write_vbootkey_hash	√	√	√	√
optee_read_vbootkey_enable_flag	√	√	√	√
optee_write_oem_otp_key	√	√	√	√
optee_oem_otp_key_is_written	√	√	√	√
optee_set_oem_hr_otp_read_lock	√	√	√	√
optee_oem_otp_key_cipher	√	√	√	√

15. Chapter-15 注意事项

15.1 配置修改

- 修改 defconfig 或 fragment 后，应重新确认启动、烧写和升级流程。
- 涉及 SPL/TPL、内存布局、分区大小的修改时，需要同步检查打包结果。

15.2 文档使用

- 文档中的配置项、路径和命令应以当前仓库为准。
- 若平台存在差异化实现，请优先参考对应 defconfig 和代码注释。

15.3 调试与发布

- 发布版本建议关闭不必要的调试打印。
- 保留必要的恢复和烧写路径，避免裁剪过度导致无法救砖。

16. Chapter-16 工具

本章节相关的开发工具路径（以U-Boot根目录为参考点）：

```
./scripts/mkbootimg
./scripts/unpack_bootimg
./scripts/repack-bootimg
./scripts/unpack_resource.sh
./scripts/stacktrace.sh
../rkbin/tools/resource_tool
../rkbin/tools/boot_merger
```

16.1 boot_merger

功能：根据 ini 配置文件打包 miniloader + ddr + usb plug，生成 loader 固件。

ini 文件：

以 RK3288MINIALL.ini 文件为例：

```
[CHIP_NAME]
NAME=RK320A                 ----芯片名称："RK"加上与 maskrom 约定的 4 位字符芯片型号
[VERSION]
MAJOR=2                     ----主版本号
MINOR=36                    ----次版本号
[CODE471_OPTION]            ----code471，目前设置为ddr bin
NUM=1
Path1=tools/rk_tools/bin/rk32/rk3288_ddr_400MHz_v1.06.bin
[CODE472_OPTION]            ----code472，目前设置为usbplug bin
NUM=1
Path1=tools/rk_tools/bin/rk32/rk3288_usbplug_v2.36.bin
[LOADER_OPTION]
NUM=2
LOADER1=FlashData           ----flash data，目前设置为ddr bin
LOADER2=FlashBoot           ----flash boot，目前设置为miniloader bin
FlashData=tools/rk_tools/bin/rk32/rk3288_ddr_400MHz_v1.06.bin
FlashBoot=tools/rk_tools/bin/rk32/rk3288_miniloader_v2.36.bin
[OUTPUT]                    ----输出文件名
PATH=rk3288_loader_v1.06.236.bin
```

打包命令：

```
// @[ini file]: 必选。ini文件

./tools/boot_merger [ini file]
```

范例：

```
./tools/boot_merger ./RKBOOT/RK3399MINIALL.ini
out:rk3399_loader_v1.17.115.bin
fix opt:rk3399_loader_v1.17.115.bin
merge success(rk3399_loader_v1.17.115.bin)
```

解包命令：

```
// @[input image]: 必选。用于解包的固件，一般是loader文件

./tools/boot_merger --unpack [input image]
```

范例：

```
./tools/boot_merger --unpack rk3399_loader_v1.17.115.bin
unpack entry(rk3399_ddr_800MHz_v1.17)
unpack entry(rk3399_usbplug_v1.15)
unpack entry(FlashData)
unpack entry(FlashBoot)
unpack success
```

16.2 resource_tool

功能：用于打包任意资源文件，生成 `resource.img`。

打包命令：

```
../rkbin/tools/resource_tool [--pack] [--image=<resource.img>] <file list>
```

范例：

```
../rkbin/tools/resource_tool ./arch/arm/boot/dts/rk3126-evb.dtb logo.bmp
logo_kernel.bmp
Pack to resource.img succeeded!
```

解包命令：

```
../rkbin/tools/resource_tool --unpack --image=<resource.img> [output dir]
```

范例：

```
../rkbin/tools/resource_tool --unpack --image=resource.img ./out/

Dump header:
partition version:0.0
header size:1
index tbl:
        offset:1        entry size:1    entry num:3
Dump Index table:
entry(0):
        path:rk-kernel.dtb
        offset:4        size:33728
```

```
entry(1):
        path:logo.bmp
        offset:70       size:170326
entry(2):
        path:logo_kernel.bmp
        offset:403      size:19160
Unpack resource.img to ./out succeeded!
```

16.3 mkimage

功能：生成 SPL 模式下的Loader固件。

例如：通过下面的命令生成 Rockchip 的 bootrom 所需 IDBLOCK 格式，这个命令会同时修改 u-boot-tpl.bin 的头 4 个 byte 为 Bootrom 所需校验的 ID：

```
./tools/mkimage -n rk3328 -T rksd -d tpl/u-boot-tpl.bin idbloader.img
```

详细参考：

```
./doc/mkimage.1
```

16.4 stacktrace.sh

功能：解析调用栈信息，请参考 CH02-RK架构_CN.md 中的 `Stacktrace` 章节。

16.5 mkbootimg

功能：打包固件生成 boot 和 recovery.img，源文件来自 Android 工程。

范例：

```
./scripts/mkbootimg --kernel zImage --second resource.img --ramdisk ramdisk.img
--out boot.img
```

16.6 unpack_bootimg

功能：用于 boot 和 recovery.img 解包，源文件来自 Android 工程。

范例：

```
./scripts/unpack_bootimg --boot_img boot.img --out out/
```

16.7 repack-bootimg

功能：替换boot和recovery.img中的固件。

范例：

```
// 例如：只替换kernel
./scripts/repack-bootimg --boot_img boot.img --kernel zImage -o boot_repack.img

// 例如：只替换resource
./scripts/repack-bootimg --boot_img boot.img --second resource.img -o
boot_repack.img
```

16.8 pack_resource.sh

功能：打包`./tools/images/`目录下的充电图片进resource.img。

范例：

```
./scripts/pack_resource.sh resource.img

Pack ./tools/images/ & resource.img to resource.img ...
Unpacking old image(resource.img):
rk-kernel.dtb 1
Pack to resource.img succeeded!
Packed resources:
rk-kernel.dtb battery_1.bmp battery_2.bmp battery_3.bmp battery_4.bmp
battery_5.bmp battery_fail.bmp battery_0.bmp

上例末尾数字表示本次打包的资源项数量。

resource.img is packed ready
```

17. Chapter-17 附录

17.1 术语

- U-Boot：Universal Boot Loader
- AOSP：Android Open-Source Project
- AVB：Android Verified Boot
- DTB：Device Tree Binary
- DTS：Device Tree Source
- Fastboot：原为 Android 的一种更新固件方式，现在已被广泛应用于嵌入式领域
- GPT：GUID Partition Table
- MMC：Multi Media Card，包括：eMMC、SD 卡等
- SPL：Secondary Program Loader
- TPL：Tertiary Program Loader
- DTB：设备树 Blob
- DTBO：用于叠加的设备树 Blob
- DTC：设备树编译器
- DTO：设备树叠加操作
- DTS：设备树源文件
- FDT：扁平化设备树
- SCMI：System Control and Management Interface